
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 15.09.2026, 16:50:59
Файлов исходного кода: 111
Суммарный объём кода: 1.97 MB
Глав/билетов в пособии: 36

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. 6. Графы. Представление графов, DFS, BFS [id: graph-basics]
7. 7. Графы. Планарные. Покраска [id: graph-planarity]
8. 7. Графы. Планарные. Покраска [id: planarity-euler]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articulation-points]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cycle]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: shortest-path-accelerations]
22. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Борувка
24. 19. Графы. Остовное дерево. Прима [id: mst-prim]
25. 20. Графы. Остовное дерево. Борувка [id: mst-kruskal]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

Прочее

111. .ai_skills/universal_guide_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot: ')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends imagemagick
          # На Ubuntu политика ImageMagick иногда запрещает конвертировать текст в изображение
          # разрешаем то, что нужно пайплайну (text -> png)
          for policy in /etc/ImageMagick-6/policy.xml /usr/share/ImageMagick-6/policy.xml; do
            if [ -f "$policy" ]; then
              sed -i 's/<policy domain="text" rights="none"/<policy domain="text" rights="write"/g' $policy
            fi
          done
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-export"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
    <title> Дискретка для тупых – Интерактивный гид по дискреткам
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, height=device-height" />
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
     <title> Дискретка для тупых – Интерактивный гид по дискреткам
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { SimulatorHub } from "../components/SimulatorHub
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useStat
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefin
 const next = index < chapters.length - 1 ? chapters[in

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id)
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapter

 return (
 <div className="min-h-screen bg-slate-950 text-slate

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
....

```



```

 className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
 activeTab === 'compiler'
 ? 'bg-emerald-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 }}
 >
 <Terminal className="w-4 h-4" /> Python
</button>
<a
 id="download-pdf-btn"
 href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
 title="Скачать полный PDF сборник всех стра
 >
 <FileDown className="w-4 h-4" /> Скачать PD

<a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
 title="Скачать полный TXT файл со всем кодом
 >
 <FileText className="w-4 h-4" /> TXT

<button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center

```

```

+ <button
+ id="tab-compiler-btn"
+ onClick={() => setActiveTab('compiler')}
+ className={`flex-1 lg:flex-none flex items
duration-200 whitespace-nowrap ${
+ activeTab === 'compiler'
+ ? 'bg-emerald-600 text-white shadow-lg
+ : 'text-slate-400 hover:text-white'
+ }`}
+ >
+ <Terminal className="w-4 h-4" /> Python
+ </button>
+ <a
+ id="download-pdf-btn"
+ href="/export/full_code_all_pages.pdf"
+ href={`${import.meta.env.BASE_URL}export/full
+ download="full_code_all_pages.pdf"
+ target="_blank"
+ rel="noreferrer"
@@ -73,7 +84,7 @@ export const Navbar: React.FC<NavbarP

 <a
 id="download-txt-btn"
 href="/export/full_code_all_pages.txt"
 href={`${import.meta.env.BASE_URL}export/full
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"


```

## `src/components/PythonCompiler.tsx`

### Полное содержимое после изменений

```

````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play

```

```
export function PythonCompiler() {
  const [code, setCode] = useState(STARTER_CODE);
  const [stdin, setStdin] = useState("");
  const [output, setOutput] = useState("Нажмите «Запустить»");
  const [running, setRunning] = useState(false);
  const [runtimeReady, setRuntimeReady] = useState(false);
  const [runtimeMessage, setRuntimeMessage] = useState("");

  const runCode = useCallback(async () => {
    if (running) return;

    setRunning(true);
    setOutput("");
    setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

    const stdout: string[] = [];
    const stderr: string[] = [];
    const inputLines = stdin.split(/\r?\n/);

    try {
      const runtime = await getPythonRuntime();
      setRuntimeReady(true);
      setRuntimeMessage("Python готов — код выполняется");

      runtime.setStdout({ batched: (message) => stdout.push(message) });
      runtime.setStderr({ batched: (message) => stderr.push(message) });
      runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
      await runtime.runPythonAsync(code);

      const text = [...stdout, ...stderr].join("");
      setOutput(text || "Готово: программа ничего не выполнила");
    } catch (error) {
      const captured = [...stdout, ...stderr].join("");
      setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
      setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
    } finally {
      setRunning(false);
    }
  }, [code, stdin, runtimeReady]);
```

```

        <span className="text-[11px] text-slate
</div>
<div className="flex items-center gap-2">
  <button
    type="button"
    onClick={reset}
    className="inline-flex items-center gap-2 text-slate-800 transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" />
  </button>
  <button
    type="button"
    onClick={() => void runCode()}
    disabled={running}
    className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
  >
    {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запустить"}
  </button>
</div>
</div>

<textarea
  value={code}
  onChange={(event) => setCode(event.target.value)}
  onKeyDown={(event) => {
    if ((event.ctrlKey || event.metaKey) && event.key === "Enter") {
      event.preventDefault();
      void runCode();
    }
  }}
  spellCheck={false}
  aria-label="Код Python"
  className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-500"
  placeholder="Напишите Python-код..."

```

```
-----
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
      <Info className="w-4 h-4 shrink-0 text-indigo-600" >
        <span>Ctrl или Cmd + Enter запускает код. Введите код в поле ввода.
      </span>
    </div>
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
      <Info className="w-4 h-4 shrink-0 text-amber-600" >
        <span>Первый запуск скачивает примерно несколько килобайт кода.
      </span>
    </div>
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
      <Info className="w-4 h-4 shrink-0 text-emerald-600" >
        <span>Код выполняется в браузере и не отправляется на сервер.
      </span>
    </div>
  </div>
</div>
</main>
);
}
....
```

Patch относительно `main`

```
````diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и установит библиотеки
+name = "алгоритмы"
+
+def main():
+ print("Привет, мир!")
+ return "Ваш код выполнен успешно!"
+
+main()
+
+`;
```



```

+
+ const runCode = useCallback(async () => {
+ if (running) return;
+
+ setRunning(true);
+ setOutput("");
+ setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+ const stdout: string[] = [];
+ const stderr: string[] = [];
+ const inputLines = stdin.split(/\r?\n/);
+
+ try {
+ const runtime = await getPythonRuntime();
+ setRuntimeReady(true);
+ setRuntimeMessage("Python готов – код выполняется");
+
+ runtime.setStdout({ batched: (message) => stdout
+ runtime.setStderr({ batched: (message) => stderr
+ runtime.setStdin({ stdin: () => (inputLines.length
+ await runtime.runPythonAsync(code);
+
+ const text = [...stdout, ...stderr].join("");
+ setOutput(text || "Готово: программа ничего не выв
+ } catch (error) {
+ const captured = [...stdout, ...stderr].join("")
+ setOutput(`${captured}${captured && !captured.en
+ setRuntimeMessage("Не удалось выполнить код – пр
+ } finally {
+ setRunning(false);
+ }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу.
+ };

```

```

late-800 transition-colors"
+
+ >
+ <RotateCcw className="w-3.5 h-3.5" />
+ </button>
+ <button
+ type="button"
+ onClick={() => void runCode()}
+ disabled={running}
+ className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+ >
+ {running ? <Loader2 className="w-3.5
+ {running ? "Запускаю..." : "Запустить"}
+ </button>
+ </div>
+ </div>
+
+ <textarea
+ value={code}
+ onChange={(event) => setCode(event.target
+ onKeyDown={(event) => {
+ if ((event.ctrlKey || event.metaKey) &
+ event.preventDefault();
+ void runCode();
+ }
+ }}
+ spellCheck={false}
+ aria-label="Код Python"
+ className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+ placeholder="Напишите Python-код..."
+ />
+
+ <div className="border-t border-slate-800
+ <label className="block px-4 pt-3 text-[
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea

```

```

+ </div>
+ <div className="flex gap-2 rounded-xl border
+ <Info className="w-4 h-4 shrink-0 text-eme
+ Код выполняется в браузере и не отпра
+ </div>
+ </div>
+ </div>
+ </main>
+);
+}
+...

```

## `vite.config.ts`

### Полное содержимое после изменений

```
```typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algv0/, локальн
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
},

```

Универсальное пособие • полный исходный код

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7

AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

****CRITICAL DIRECTIVE****:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML tags.

ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2

Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).
При запросе пользователя создать или сгенерировать новую страницу, ты ****строго**** придерживаешься готовых шаблонов разметки, стилей и компонентов.

Правила для малых страниц (Билетов / Глав)

- **Единый стиль:**** Всегда используй Tailwind CSS и HTML-шаблоны.
- **Структура JSON/TS:**** Каждая новая малая страница должна быть описана в формате `{ "id": "...", "content": ... }` для вставки в ``src/data/content.json``.
- **Строгая структура контента:****
 - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
 - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``).
 - Обязательная сетка аналогий (2 колонки: неформальный язык / строгий язык).

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

```
{
  "name": "algv0-exam-guide",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "audit:content": "node scripts/audit-coverage.mjs",
    "audit:terms": "node scripts/audit-terms.mjs",
    "export:txt": "node scripts/export/collect-code.mjs",
    "export:png": "node scripts/export/render-pages.mjs",
    "export:pdf": "node scripts/export/build-pdf.mjs",
    "export": "npm run export:txt && npm run export:png",
    "export:guide": "node scripts/export/build-guide-pdf.mjs",
    "export:shots": "node scripts/export/capture-viz-shots.mjs",
    "verify:unit": "node scripts/verify/live-matrix.test.mjs",
    "verify:code": "node scripts/verify/check-reference.mjs",
    "verify:links": "node scripts/verify/check-links.mjs",
    "verify:dom": "node scripts/verify/run-browser-check.mjs",
    "verify:probe": "node scripts/verify/run-probe.mjs",
    "verify": "npm run verify:unit && npm run verify:links",
  },
  "dependencies": {
    "clsx": "2.1.1",
    "lucide-react": "^1.21.0",
    "motion": "^12.40.0",
    "pdfkit": "^0.20.1",
    "react": "19.2.6",
    "react-dom": "19.2.6",
  }
}
```

Универсальное пособие • полный исходный код

```
-----  
npm run dev          # предпросмотр на http://0.0.0.0:5  
npm run build        # сборка в dist/ (single-file)  
````
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в о

| Команда              | Шаг       | Результат                               |
|----------------------|-----------|-----------------------------------------|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/`                     |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export`     | всё сразу | txt + png + pdf                         |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core  
````
```

Настройки через переменные окружения:

- \* `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- \* `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

### Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается **\*\*на каж**  
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док  
и коммитит обновлённые `public/export/full\_code\_all\_pages`  
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

## Аудит контента

```
```bash
```

Заголовки

- * Заголовки секций: Используют стандартный тег `<h2>` и
- * Подзаголовки: Используют тег `<h3>`.

Текст и параграфы

- * Обычный текст содержится в тегах `<p>`.
- * Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

Математические формулы

В приложении используется MathJax.

- * В самом исходном коде страницы (и внутри тегов) формулы.
- * Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- * Они помещены внутрь оберток с классом `<div class="img">`
- * Все отрисовки находятся внутри векторного тега `<svg>`
- * Подпись к картинке, если она есть, идет сразу после `<img>`

Списки

- * Стандартные списки оформлены через `<ul class="list-n">`

Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```
-----
* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья отрезков**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**
---
```

Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

Глава	id	Комментарий
15. Планарность: K5, K3,3 и формула Эйлера	`planarity-euler`	т «на пальцах».
16. Флойд (Флойд–Уоршелл)	`floyd`	Есть матричный алгоритм
17. Алгоритм Джонсона	`johnson-algo`	Перевзвешивание
Обзор: Кратчайшие пути и Графы	`intro`	Вводная страница
Эйлер: Путь ≠ Цикл	`euler-path-vs-cycle`	Ключевая концепция

Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сложно»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока закрыто

Глава	id	Чего не хватает
3. Декартово дерево (Treap)	`treap`	Нет ни визуализации, ни интерактива
8. Графы. Компоненты связности	`graph-components`	Нет визуализации
24. Классы сложности, сведение задач	`complexity-classes`	Нет визуализации

Сводная таблица по всем главам

#	Глава	Аналогий	2D-виз.	SVG	Картинка
---	---	---	---	---	---

ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "types": ["node"],

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",

    /* Path mapping */
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"]
    },

    /* Linting */
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src", "vite.config.ts"]
}
```

```
    },  
  });  
}
```

РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";  
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";  
import { chapters } from "../data/content";  
import { Navbar } from "../components/Navbar";  
import { Sidebar } from "../components/Sidebar";  
import { MobileToc } from "../components/MobileToc";  
import { ChapterView } from "../components/ChapterView";  
import { ChapterNav } from "../components/ChapterNav";  
import { SimulatorModal } from "../components/hints/SimulatorModal";  
import { LiteGuide } from "../components/LiteGuide";  
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {  
  if (typeof window === "undefined") return { lite: false };  
  const q = new URLSearchParams(window.location.search);  
  const lite = q.has("lite") || q.get("view") === "lite";  
  return { lite, topic: q.get("topic") ?? undefined };  
}
```

```
export default function App() {  
  const liteMode = useMemo(readLiteMode, []);
```

```

    setSidebarMode((m) => {
      const next = m === "wide" ? "compact" : m === "co
      window.localStorage.setItem("sidebarMode", next);
      return next;
    });
  }, []);
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
  const apply = () => {
    const el = contentRef.current;
    if (!el) return;
    el.style.paddingRight = compilerOpen && window.in
  };
  apply();
  window.addEventListener("resize", apply);
  return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
  0,
  chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[i

const goTo = useCallback((id: string) => {
  setActiveChapterId(id);
  window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
  setCompilerOpen(true);
  requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
  {/* Сайдбар только на десктопе — на мобильном
  <div
    id="sidebar-root"
    className={`hidden lg:block shrink-0 border
      -all duration-200 print:hidden ${sidebarCol
    " : "lg:w-56"}`}
  >
    {!sidebarCollapsed && (
      <Sidebar selectedChapterId={activeChapterId}
    )}
  </div>
  {/* Вкладка-переключатель содержания у левого
  <button
    type="button"
    onClick={toggleSidebar}
    className="hidden lg:flex items-center gap-1
    border-slate-700 bg-slate-900/90 px-1 py-3
    aria-label={sidebarMode === "wide" ? "Сжать
    ание"}
    title={sidebarMode === "wide" ? "Сжать содер
    }
  >
    {sidebarCollapsed ? <PanelLeftOpen className="
    <span className="text-[10px] font-bold [wri
  </button>

  <main id="main-content" className="flex-1 min
    {/* Шапка главы с быстрыми переходами */}
    <div className="flex items-center justify-b
      <div className="min-w-0">
        <div className="text-[10px] uppercase t
          {activeChapter.category || "Универсальн
        </div>
        <h2 className="text-base sm:text-xl font
          {activeChapter.title}
        </h2>
      </div>
    </div>

```

```
        onHeightChange={setCompilerHeight}
        onClose={() => setCompilerOpen(false)}
      />
    ))
  </div>
);
}
```

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 129 | РАЗМЕР: 5.1 K

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

/** Числовые константы TreeWalker: глобального `NodeFile`
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/g, "\\$1");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
        const surface = m[1].toLowerCase();
        const id = labelToId.get(surface);
        if (!id) continue;

        const usedTerm = perTerm.get(id) ?? 0;
        const usedSurface = perSurface.get(surface) ?? 0;
        if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
        perTerm.set(id, usedTerm + 1);
        perSurface.set(surface, usedSurface + 1);

        if (m.index > last) frag.appendChild(document.createElement("span"));
        const span = document.createElement("span");
        span.className = "term-hint";
        span.dataset.termId = id;
        span.tabIndex = 0;
        span.setAttribute("role", "button");
        span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
        span.textContent = m[1];
        frag.appendChild(span);
        last = m.index + m[1].length;
    }

    if (last === 0) continue;
    if (last < text.length) frag.appendChild(document.createElement("span"));
    textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
  box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
  box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
  box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
  box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}
```

```
    color: #fff;
    outline: none;
}
@media (hover: none) {
    .term-hint {
        border-bottom-style: solid;
        background: rgba(99, 102, 241, 0.12);
    }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
    color: #a5b4fc;
    border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
    text-decoration: none;
    border-radius: 3px;
    padding: 0 1px;
    cursor: pointer;
    transition:
        background-color 0.15s ease,
        color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
    background: rgba(99, 102, 241, 0.25);
    color: #fff;
    outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}
```



```
    line-height: 1.3 !important;
  }
  .chapter-body :where(h3) {
    font-size: 1.05rem !important;
    line-height: 1.35 !important;
  }
  .chapter-body :where(.p-6, .p-8) {
    padding: 0.9rem !important;
  }
  .chapter-body :where(.text-3xl) {
    font-size: 1.4rem !important;
  }
  .chapter-body :where(.text-2xl) {
    font-size: 1.2rem !important;
  }
  .chapter-body :where(.text-xl) {
    font-size: 1.05rem !important;
  }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #0f172a;
}
::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
```

```
main {
  padding: 0 !important;
  max-width: none !important;
}

/* текст темы – чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
  color: #111827 !important;
  background-color: transparent !important;
  background-image: none !important;
  box-shadow: none !important;
  text-shadow: none !important;
  border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
  background-color: #f3f4f6 !important;
  color: #111827 !important;
  font-size: 9pt;
}

article.chapter-body .prose table {
  border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
  border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
  page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

```
export interface Chapter {
  id: string;
  title: string;
  type: "html" | "markdown";
  content: string;
  description?: string;
  category?: string;
}
```

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
```

```
<div class="space-y-8">
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-blue-400">
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
      <p class="text-xl font-mono text-white">
      <p class="text-sm font-mono text-slate-300">
    ] [k] </p>
  </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячей
лины <span class="font-mono text-emerald-300">
">i</span>. Любой отрезок накрывается <b>дв
перация <b>идемпотентна</b>: <span class="font-mono text-emerald-300">min</span>, <span class="font-mono text-emerald-300">
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-indigo-300 italic">
  <p class="text-xl font-mono text-white">
  <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, накрыва
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двум
по столбцам. Отсюда и цена: уровней станов
s="font-mono text-emerald-300">log N</span>
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
```

```

        <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
            Аналогия 2: Торт-пирамидка
        </div>
        <p class="text-slate-300 text-sm mb-4">
            ия, зато красивый, как пирамидка: каждый из
            сех блоков  $2^{k_x} \times 2^{k_y}$ ), и ответ на запрос
            ой операции нет: крем обратно в торт не запы
             $n(x, x) = x$ ), и цена  $O(1)$  – сто коржей вмес
            <div class="rounded-lg bg-slate-950">
                <div><span class="text-slate-500">
                     $n \ O(1)$ </span></div>
                <div><span class="text-slate-500">
                    = 100 копий матрицы</span></div>
            </div>
            <p class="text-[12px] text-slate-400">
                шает). Менять элементы тоже нельзя. Сумму ч
            </div>
        </div>
        <p class="text-slate-400 text-sm mt-4">Поря
            ерению) → <b>2D: построение</b> (уровни <span
            ос</b> (прямоугольник раскладывается на чет
        </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-rose-400">
    <p class="text-slate-300 text-sm mb-4">Эти
        за <span class="font-mono text-emerald-300">
        они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
    <p class="text-sm font-bold text-white">
    <p class="text-slate-300 text-sm mb-3">
        цы, у разреженной таблицы – размер, умножен
        <table class="w-full text-xs text-left">
            <thead>
                <tr class="text-slate-500 border">
                    <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 5px; display: inline-block; width: 100px; height: 100px; background-color: #f0f0f0; border-radius: 5px; text-align: center; vertical-align: middle;">P[i][j]</p>
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-4">Вот почему вы не можете вычислить логарифм на его части не вычислишь. Приходится хранить матрицу в памяти. log N × log M</p>
xt-emerald-300">log N × log M. На ка

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод: **лишние** – перекрывающиеся блоки посчитают дважды. **лишние** – у минимума нет обратной операции. Стрелки

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">

<h3 class="text-xl font-bold text-amber-400">Проблема разреженной матрицы

<div class="bg-rose-950/30 p-5 rounded-lg border">

<ul class="list-disc pl-5 space-y-2 text-white">

Сумма через разреженную таблицу. Префиксные (билет 5).

Память. N = M = 1000 даём матрицу, считайте ячейки, потом пишите код.

Один log вместо двух. В матрице st[i][j] st[i][j]
span class="font-mono text-emerald-300">st[i][j]

Выход за границу в запросе. st[i][j] st[i][j] и st[i][j] st[i][j]

Нужны обновления. Разреженная матрица (билет 1), а не она.


```

        else
            st[r][c][kx][ky] = min(
    
```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 &lt;&lt; ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 &lt;&lt; kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);
    
```

Четыре запроса к четырем углам запроса и вместе с ним образуют минимальную сумму.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему это так много? Потому что для каждого запроса нужно вычислить сумму на нескольких сотен на сторону), а запросов очень много.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему это так много? Потому что для каждого запроса нужно вычислить сумму на нескольких сотен на сторону), а запросов очень много.

```
<div class="text-center my-6">
  <p class="text-slate-400 text-xs uppercase">
    <p class="my-1 font-bold leading-none">
      <span class="text-2xl text-slate-300">И</span>
    </p>
  <p class="text-slate-300 font-mono text-sm">
  </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия 1: Split – таможня
    </div>
    <p class="text-slate-300 text-sm">С
    х». Спуск от корня: узел, который вместе с
    скрытия</b> – досматривается только одна ветвь
  </div>
```

```
  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
      Аналогия 2: Merge – стыковка
    </div>
    <p class="text-slate-300 text-sm">М
    аг – одно сравнение: чей у у корня больше,
    сохраняет порядок х). Итого  $O(h)$  сравнений
  </div>
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border">
  <p class="text-slate-300 text-sm mb-2">
  <p class="text-slate-400 text-sm mb-1">
    быстрее  $n \cdot \log n$ .</p>
  <p class="text-slate-400 text-sm mb-1">
  <p class="text-slate-400 text-sm">• «У6
```



```

<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
    Код: вставка, merge, erase (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300">
    <pre class="bg-slate-950 p-4 rounded">
def insert(root, key, pri):
    if root is None:
        return Node(key, pri)
    if key <= root.key: # равные x — левый
        root.left = insert(root.left, key, pri)
    else:
        root.right = insert(root.right, key, pri)
    return root

for key, pri in pairs_sorted_by_pri_desc: # порядок э
    root = insert(root, key, pri)</pre>
    <pre class="bg-slate-950 p-4 rounded">
def merge(a, b): # все x в a < все x в b
    if not a or not b: return a or b
    if a.pri > b.pri:
        a.right = merge(a.right, b); return a
    b.left = merge(a, b.left); return b

def erase(root, key): # удалить произвольный элемент
    if key < root.key: root.left = erase(root.left, key)
    elif key > root.key: root.right = erase(root.right, key)
    else: root = merge(root.left, root.right)
    return root</pre>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "splay-tree",
        title: "4. Splay-дерево",
    }

```

по факту

: ничего не хранит, дереву разрё

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
</div>
<p class="text-slate-300 text-sm">С
тировку не ведёшь, но через неделю самое ну
ерх».</p>
```

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
<p class="text-slate-300 text-sm">И,
рно вдвое. Первый проход дорогой, зато опла
ы.</p>
```

```
</div>
</div>
```

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Повор
Среднее поддерево β (между x и p) – единств
x, и под p.</p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">
```

```

      /  \ \      x выше      /  \ \
     x    y      ----->  α    p
    /  \ \      p выше      /  \ \
   α    β      <-----      β    γ</pre>
```

```
<p class="text-slate-300 text-sm mt-3">Обхо
омать дерево поиска поворотами невозможно в
омент «отстёжки», а врезка «Механика одного
```

```
<h4 class="text-lg font-bold text-white mt-3">
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
    амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x):    3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x):   3 × Zig-Zig    + Zig → высота 8
бамбук из 16 узлов:     наивно 16 → 16, Zig-Zig 16 → 10<
```

```

    <p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mo
изации не будет.</p>
</div>
```

```

<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем <b>три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути</b>. Отсюда амортизация: дорогой за
дом потенциала: потенциал  $\Phi$  = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```

<div class="bg-slate-900/40 rounded-lg bord
    <p class="text-slate-300 text-sm mb-2">
    <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
    <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
    <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```

<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```

<p class="text-slate-300 text-sm mt-4">Все
```

```

        rotate_up(x)
    else:                                     # ZIG-ZAG: НИЖНЯЯ
        rotate_up(x)
        rotate_up(x)
    return x

def find(root, key):                         # поиск: спуск + s
    cur, last = root, None
    while cur:
        last = cur
        if key == cur.key: break
        cur = cur.left if key < cur.key else cur.right
    return splay(last) if last else None</pre>
    <p class="text-xs text-slate-400">В
        Одна строка – и вся амортизация.</p>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
        description: "Сумма подматрицы за O(1): 1D и 2D, дв
        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-v
    </div>

    <div class="space-y-8">

        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-emerald-4

```

```

        <td class="py-2 pr-3 font-mono">
      </td>
    </tr>
  </tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
  ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
    <p class="text-[11px] text-slate-500
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
    <p class="text-[11px] text-slate-500
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-rose-300 t
    <p class="text-[11px] text-slate-500
  </div>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-blue-400 m
  <div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800
      <div class="absolute -top-3 left-4 b
      rder border-emerald-500 shadow-md">
        Аналогия 1: Сетка из вафельных
      </div>
      <p class="text-slate-300 text-sm mb
    о накопилось от угла (0,0) до меня». Хочешь
    у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
  Тонкая: одна матрица
</div>
  <p class="text-slate-300 text-sm mb
одной матрицы (плюс рамка из нулей). Никаки
прямоугольника от угла. Всё остальное полу
</div>
  <div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
  Не умеет: всё, что нельзя вычес
</div>
  <p class="text-slate-300 text-sm mb
максимума или gcd вычитания не существует:
е запросы – это <a class="ticket-link" data
платит она за это памятью <span class="fon
</div>
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
  <p class="text-sm font-bold text-white"
  <table class="w-full text-xs text-left"
    <thead>
      <tr class="text-slate-500 borde
        <th class="py-1.5 pr-2 font
        <th class="py-1.5 pr-2 font
        <th class="py-1.5 pr-2 font
      </tr>
    </thead>
    <tbody>
      <tr class="border-b border-slate
        <td class="py-2 pr-3 font-b
        <td class="py-2 pr-3 font-m
        <td class="py-2 pr-3 font-m
      </tr>
      <tr class="border-b border-slate

```

```

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">
    Код: 1D (тот же, что выполняет пане
  </summary>
  <div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounde
      = [3, 1, 4, 1, 5, 9, 2, 6]  # исходный ма
n = len(a)  # префиксов будет n + 1
P = [0]  # P[0] = 0 – пустой преф
for i in range(1, n + 1):
    P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
    <p class="text-slate-300 text-sm">П
  - часть демонстрации: лента подсвечивает, из
    </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">
    Код: 2D – построение и запрос прямо
  </summary>
  <div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounde
      = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = []  # рамка из нуле
for rr in range(n + 1):
    S.append([0] + [0] * m)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2  # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

    },
    {
      id: "graph-planar-colors",
      title: "7. Графы. Планарные. Покраска",
      type: "html",
      description: "Формула Эйлера и теорема о 4 красках.",
      category: "Графы. Теория",
      content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">7
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Графы. Планарные. Покраска
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
        <p class="text-xl font-mono text-white">
          1. Если граф планарный, то он можно вложить в плоскость так, чтобы никакие две
          ребра не пересекались.
          2. Если граф можно вложить в плоскость, то он планарный.
          3. Если граф можно раскрасить 4 цветами, то он планарный.
          4. Если граф планарный, то он можно раскрасить 4 цветами.
        </p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Карта мира
          </div>
          <p class="text-slate-300 text-sm mb-4">
            . Границы не пересекаются в одной точке по-
            ы не сливались цветами, Всегда можно всего 4
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
    },
    {
      id: "top-sort",

```



```

    </div>
</section>`,
  },
  {
    id: "scc-kosaraju",
    title: "10. Графы. Компоненты сильной связности (SCC)",
    type: "html",
    description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju",
    category: "Графы. Продвинутое",
    content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Макро-вершины — это вершины орграфа, которые принадлежат одной и той же компоненте сильной связности (SCC).
        <p class="text-xl font-mono text-white">Макро-вершины являются вершинами графа, полученного из исходного графа путём инвертирования в порядке top-sort.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Улицы с односторонним движением
          </div>
          <p class="text-slate-300 text-sm mb-4">Представьте себе город, где все улицы имеют одностороннее движение. Если вы можете пройти от улицы A к улице B, то A и B принадлежат к одной и той же компоненте сильной связности.
          <p class="text-slate-300 text-sm mb-4">ВЫХ направлениях по односторонним улицам. Как это выглядит на карте.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },

```

```

        description: "Вершины, удаление которых убивает свя
        category: "Графы. Продвинутые",
        content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
          теи > 1).</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            Аналогия 1: Центральный роутер
          </div>
          <p class="text-slate-300 text-sm mb-4">
            правильный роутер - и два сегмента офиса б
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-blue-300 italic"
      <p class="text-xl font-mono text-white"
d-длина.</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия: Копаем туннель под .
        </div>
        <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-emerald-4

```

```

        которое торчит из посещенных в непосещенных
    </div>
    <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                ✦ Аналогия: Заражение вирусом
            </div>
            <p class="text-slate-300 text-sm mb-2">
                ли вирус). Мы стартуем из одного города, и
                Сеть растет только из одного связного куска
            </p>
        </div>
    </div>
</div>
</section>`,
    },
    {
        id: "mst-boruvka",
        title: "20. Графы. Остовное дерево. Борувка",
        type: "html",
        description: "",
        category: "Графы. Остовные",
        content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
                </div>
            <div class="space-y-8">
                <div class="bg-slate-700/50 p-6 rounded-xl border">
                    <h3 class="text-xl font-bold text-emerald-400">
                        <div class="bg-slate-900 p-4 rounded-lg mb-4">
                            <p class="text-lg text-emerald-300 italic">
                                <p class="text-xl font-mono text-white">
                                    ается с соседом.</p>
                                </div>
                            <div class="grid grid-cols-1 gap-6">

```

```

        rder border-emerald-500 shadow-md">
            Аналогия 1: Бумеранг (Префикс
        </div>
        <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "string-z-func",
        title: "22. Z-функция",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

    </div>
</section>`,
  },
  {
    id: "complexity-classes",
    title: "24. Классы сложности, сведение задач",
    type: "html",
    description: "",
    category: "Теория сложности",
    content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1 rounded">24</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-purple-400">Сведение задач</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">Пример: класс P</p>
        <p class="text-base sm:text-lg font-mono">
          <b>P</b>: класс decision-задач, решаемых за полиномиальное время.
          <b>NP</b>: класс decision-задач с сертификатом.
        </p>
      </div>
      <p>
        <b>Сведение (Reduction) A ≤<sub>p</sub> B</b>: задача A сводится к задаче B.
      </p>
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          Аналогия 1: Судоку (P vs NP)
        </div>
        <p class="text-slate-300 text-sm mb-4">Пример сортировка чисел). Класс <b>NP</b> – задачи, для которых существует быструю сетку (ответ), он БЫСТРО (за класс P) решить.</p>
      </div>
    </div>
  </div>
</section>`
  }
]

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"

  <div class="bg-slate-900 p-4 rounded-lg mb-4"
    <p class="text-lg text-blue-300 italic"
    <div class="text-left font-mono text-sm"
      <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
      <p><strong class="text-white">Эйлер
      <div class="p-3 bg-slate-950 rounded"
        <p class="text-sm text-blue-300"
        <p><strong class="text-white">П
        <p><strong class="text-white">Ц
      </div>
    </div>
  </div>
</div>

<p class="text-slate-300 text-sm">
  <strong>Билет 13 (Различие пути и цикла
  <br>• <span class="text-green-400 font-
шел-вошёл-вышел).
  <br>• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
  <div class="bg-slate-800 p-6 rounded-lg bor"
    <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
      Аналогия 1: Путь – из дома в бар,
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Ты вышел из дома <strong>(нечётная)
      талый пришёл в бар <strong>(вторая нечётная
      Домой вернуться не смог, потому что
    </p>

```

```

    </div>
</section>`,
  },
  {
    id: "bridges-code",
    title: "11. Графы. Мосты",
    type: "html",
    description: "Мосты через DFS Тарьяна: tin/low, усл
    category: "Графы. Продвинутые",
    content: `<section id="bridges-code" class="mb-20 s
    <div class="flex items-center mb-6">
      <span class="bg-emerald-600 text-white px-4 py-
      <h2 class="text-3xl font-bold text-white">Мосты
    </div>

    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl bord
        <h3 class="text-xl font-bold text-emerald-400">
          Мосты через DFS Тарьяна: tin/low, усл

        <div class="bg-slate-900 p-4 rounded-lg mb-4">
          <p class="text-lg text-emerald-300 ital">
            <div class="text-left font-mono text-sm">
              <p><strong class="text-white">tin[v
              <p><strong class="text-white">low[v
              <div class="p-3 bg-slate-950 rounde
                <span class="text-rose-400 font
                <p class="text-xl font-bold tex
                <p class="text-xs text-slate-400
              </div>
            </div>
          </div>
        </div>
      </div>
      <div class="bg-slate-800 p-4 rounded-lg mb-4">
        <p class="text-slate-300 text-sm">
          Если из сына u и всех его потомков <strong>
          Единственная ниточка. Порвётся – граф р
        </p>
      </div>
    </div>
  `
  }
]

```



```
-----
<pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n;                // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;           // устанавливаем

    for (int to : adj[v]) {
        if (to == p) continue;         // не идём назад

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v);                 // рекурсивный в
            low[v] = min(low[v], low[to]);

            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
}
```

```

        <p><span class="text-emerald-400 font-
        <p><span class="text-rose-400 font-l
    </div>
</div>

<p class="text-slate-300 text-sm">
    <strong>Следствие:</strong> Для планарн
ode>.
    Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
    <div class="bg-slate-800 p-6 rounded-lg bord
        <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
            K5: Полный граф на 5 вершинах
        </div>
        <p class="text-slate-300 text-sm mb-4">
            У K5: <code class="text-white">V=5,
            -white">10 &gt; 9</code> – БАХ, непланарен!
            Все 5 вершин соединены со всеми. На
            Теорема Куратовского: K5 и K3,3 – д
            Если внутри графа спрятано <b>подра
            – плоской раскладки не будет.
        </p>
    </div>

<div class="bg-slate-900 p-6 rounded-lg bord
    <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
        K3,3: 3 дома и 3 колодца
    </div>
    <p class="text-slate-300 text-sm mb-4">
        Три дома хотят соединиться с тремя
        Нарисовать без пересечений невозможно
        У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в

```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Расписание экзамен

```
</div>
```

```
<p class="text-slate-300 text-sm">В
слот времени. Раскрасить граф = составить р
«берём предметы в каком-то порядке и ставим
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border border-fuchsia-500 shadow-md">
```

Аналогия 2: Частоты вышек

```
</div>
```

```
<p class="text-slate-300 text-sm">С
цвет, интерференция = ребро. Диапазон частот
регистров в компиляторе: «живые» одновременно
```

```
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left l
```

```
<thead>
```

```
<tr class="text-slate-500 borde
```

```
<th class="py-2 pr-3 font-b
```

```
<th class="py-2 pr-3 font-b
```

```
<th class="py-2 font-bold">l
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate
```

```
><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$ <
а свободен</td></tr>
```

```
<tr class="border-b border-slate
```

```
d><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
/td></tr>
```

```

        веркой цветов соседей; проверка двудольности
border border-slate-800">0(V + E)</code>; т
-300 border border-slate-800">NP-трудно</co

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код: жадная раскраска и проверка
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounded
я раскраска: не больше  $\Delta + 1$  цвета
color = [-1] * n                # -1 = ещё не покрашен
for v in order:                  # порядок влияет на чи
    used = set()
    for to in graph[v]:
        if color[to] != -1:
            used.add(color[to]) # цвета уже покрашены
    c = 0
    while c in used:              # минимальный свободны
        c += 1
    color[v] = c
chi = max(color) + 1             #  $\chi$ , который выдал жад

# Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
    if col[start] != -1:
        continue
    col[start] = 0
    dq = deque([start])
    while dq:
        v = dq.popleft()
        for to in graph[v]:
            if col[to] == -1:
                col[to] = col[v] ^ 1 # второй цвет

```

```
<p class="text-slate-300 text-sm">
  <strong>Важно:</strong> Это работает то
  епятся к точкам сочленения. То есть, <strong>
  ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-500">
    <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
      Аналогия: Главный перекресток
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь город, где два района соединены
      её перекроют (удалят вершину) — районы будут
      <strong>хрупкость</strong> системы.
    </p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border border-slate-500">
    <div class="absolute -top-3 left-4 bg-slate-900 border border-emerald-500 shadow-md">
      Телеком: Центральный свитч
    </div>
    <p class="text-slate-300 text-sm mb-4">
      У тебя несколько компьютеров в одной
      абелем (мостом). Сами свитчи — это точки со
    </p>
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-500">
  <summary class="p-4 font-bold text-slate-400">
    Душный мод: Код и корень DFS (Скрыто)
  </summary>
```

```

        <ul class="list-disc list-inside text-sm text-rose-400">
            <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
            <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
            <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
        </ul>
    </div>
</div>
</section>`,
    },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 145 | РАЗМ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { ticket1Segtree } from "../tickets/ticket1_segtr
import { tickets18to20Mst } from "../tickets/ticket18_20
import { chapters as newChapters } from "../content_temp

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает
 * версия главы. `newChapters` (content temp) перетирае

```

```
merged.forEach((c) => {
  if (!toRemove.includes(c.id)) {
    dedup.set(c.id, c);
  }
});

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
  newChapters.forEach((c) => {
    if (!toRemove.includes(c.id)) dedup.set(c.id, c);
  });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
  // Главы без номера билета (введение) стоят ДО билетов
  const numOf = (t: string) => {
    const m = t.match(/(\d+)/);
    return m ? parseInt(m[0], 10) : -1;
  };
  return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 20
 */
export const chapterTopics: Record<string, number[]> = {
  "segment-trees": [1],
  "sparse-table": [2],
  treap: [3],
  "splay-tree": [4],
  "prefix-sums-2d": [5],
  "graph-dfs-bfs": [6],
  "planarity-euler-formula": [7],

```

```

-----
const tickets = chapterTopics[chapterId] ?? [];
// Глав без билетов (введение) относим к первому разд
if (!tickets.length) return SECTIONS[0];
const first = Math.min(...tickets);
return SECTIONS.find((sec) => first >= sec.from && fi
}

export const activeVizIds = [
  "euler-path-vs-cycle", "bridges-code", "planarity-euler
  "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
  "dijkstra", "bellman-ford", "floyd", "aho-corasick",
  "mst", "string-kmp", "string-z-func", "segment-trees", "s
  "dynamic-programming", "johnson-algo", "graph-articulat
  "complexity-classes", "treap", "splay-tree", "prefix-sum
];

```

ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 207 | РАЗМ

```

-----
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

export type DebugValue = null | boolean | number | string

```

```

export interface DebugStep {
  /** Номер строки в коде пользователя (1-based). */
  line: number;
  /** Имя функции-фрейма ("<module>" – верхний уровень)
  func: string;
  /** Локальные переменные: имя → герг (усечённый), для
  locals: Record<string, string>;
  /**

```



```

__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
    pass

def __safe(v):
    try:
        r = repr(v)
    except Exception:
        r = "<...>"
    return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
    """Небольшой JSON-снимок: UI использует его без парсера"""
    if v is None or isinstance(v, (bool, int, str)):
        return v
    if isinstance(v, float):
        return v if v == v and v not in (float("inf"), float("-inf")) else "inf/-inf"
    if depth >= 4:
        return __safe(v)
    limit = 80 if depth < 2 else 32
    if isinstance(v, (list, tuple)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, (set, frozenset)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, dict):
        # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
        # предел 80 незаметно обрезал уровни 4x4 и 8x8
        dict_limit = 192 if depth == 0 else limit
        return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
    # deque, heap-подобные и другие итерируемые учебные
    if type(v).__name__ == "deque":
        return [__snapshot(x, depth + 1) for x in list(v)]
    return __safe(v)

```

```

-----
    # PEP 709: с 3.12 компрекеншны встроены во фрейм
    # на своей строке каждую итерацию. Склеиваем по
    if __steps and __steps[-1]["line"] == frame.f_lineno:
        __steps[-1]["locals"] = loc
        __steps[-1]["values"] = values
    else:
        __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
    if len(__steps) >= __CAP:
        # Нельзя просто отключить trace: бесконечный цикл
        # тогда продолжит работать вечно. Мягко прерываем
        __was_truncated = True
        raise __TraceLimit()
    # Финальные локалы программы (после последней строки)
    if event == "return" and frame.f_code.co_filename == __FILE__:
        loc, values = __capture(frame)
        __final.update(loc)
        __final_values.update(values)
    return __tracer

__err = ""
try:
    __sys.settrace(__tracer)
    exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
    pass
except Exception:
    import traceback as __tb
    __err = __tb.format_exc(limit=3)
finally:
    __sys.settrace(None)

__OUT = __json.dumps({
    "steps": __steps,
    "finalLocals": __final,
    "finalValues": __final_values,
    "error": __err,
    "truncated": __was_truncated,
})

```

ФАЙЛ 21/111: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 639 | РАЗМ

```
import type { DemoKind } from "../components/hints/Mini"
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
  id: string;
```

```
  /** Варианты написания в тексте (регистр не важен). */
```

```
  labels: string[];
```

```
  title: string;
```

```
  /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
  short: string;
```

```
  /** Строгая формулировка / что важно не забыть. */
```

```
  formal?: string;
```

```
  complexity?: string;
```

```
  /** Какая мини-анимация показывается в карточке. */
```

```
  demo?: DemoKind;
```

```
  /** id полного симулятора (см. vizRegistry) – кнопка
```

```
  simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
  {
```

```
    id: "bfs",
```

```
    labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
    title: "BFS – обход в ширину",
```

```
    short:
```

"Не отсортированный список, а «мешок, из которого
formal:

"Полное бинарное дерево в массиве: дети узла i ле

complexity: "вставка/извлечение $O(\log n)$, «посмотре

demo: "heap",

simulator: "heap-beam-search",

},

{

id: "stack",

labels: ["стек", "стека", "стеке", "стеком", "LIFO"],

title: "Стек (LIFO)",

short: "Стопка тарелок: кладёшь сверху и берёшь свер

formal: "push / pop / top за $O(1)$. На стеке живёт р

demo: "stack",

simulator: "stack-dfs",

},

{

id: "queue",

labels: ["очередь", "очереди", "очередью", "FIFO"],

title: "Очередь (FIFO)",

short: "Очередь в столовой: кто первым встал, тот п

formal: "push_back / pop_front за $O(1)$. Основа BFS

demo: "queue",

simulator: "queue-bfs",

},

{

id: "dsu",

labels: ["DSU", "СНМ", "система непересекающихся мн

title: "DSU — система непересекающихся множеств",

short:

"«Кто твой староста?» Каждый элемент знает своего

formal:

"find со сжатием путей + union по рангу/размеру. I

complexity: "почти $O(1)$ — обратная функция Аккерман

demo: "dsu",

simulator: "mst",

},

{

```

    short:
        "«Куда откатиться, если следующая буква не подошла",
        строки.",
    formal: "link(v) = go(link(parent(v)), ch). Считает",
    demo: "suffix-link",
    simulator: "aho-corasick",
},
{
    id: "toposort",
    labels: ["топологическая сортировка", "топсорт", "топ",
    title: "Топологическая сортировка",
    short:
        "Порядок «сначала штаны, потом ботинки»: каждая с",
        циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л",
    complexity: "O(V + E)",
    demo: "toposort",
    simulator: "top-sort",
},
{
    id: "relaxation",
    labels: ["релаксация", "релаксацию", "релаксации",
    title: "Релаксация ребра",
    short:
        "«А через соседа не быстрее?» Если известный путь",
        ,
    formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
        ан и Флойд.",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "prefix-sum",
    labels: ["префиксные суммы", "префиксная сумма", "пре",
    title: "Префиксные суммы",
    short:
        "Заранее посчитанные «сколько всего набежало от н",
    formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r] - P[l-1]"

```

```

    complexity: "O(V + E) одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "то"],
    title: "Точка сочленения",
    short:
        "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
    formal: "Для не-корня: v — точка сочленения  Э ребро",
    demo: "articulation",
    simulator: "graph-articulation",
},
{
    id: "mst",
    labels: [
        "остовное дерево", "остовного дерева", "остовное",
        "минимальное остовное дерево", "MST",
    ],
    title: "Минимальное остовное дерево (MST)",
    short:
        "Связать все точки проводами так, чтобы суммарная",
    formal: "V-1 ребро, никаких циклов. Краскал (жадно и",
        "но).",
    demo: "mst",
    simulator: "mst",
},
{
    id: "amortized",
    labels: [
        "амортизированная", "амортизированный", "амортизир",
        "амортизированное", "амортизированного", "амортизир",
    ],
    title: "Амортизированная сложность",
    short:
        "Средняя цена операции «по абонементу»: одна опер",
    formal: "Метод потенциалов: дорогая операция оплачи
```

```

{
  id: "prefix-function",
  labels: [
    "префикс-функция", "префикс-функции", "префикс-фу",
    "префикс", "префикса", "суффиксом", "суффикс", "с",
    "подстроки", "подстроку", "подстрока",
  ],
  title: "Префикс-функция  $\pi$  (КМП)",
  short:
    "Для каждой позиции запоминаем: какой кусок начался",
    "от кусок.",
  formal:
    " $\pi[i]$  — длина наибольшего собственного префикса  $s$ ",
    "возвращая указатель по тексту.",
  complexity: " $O(n + m)$ ",
  demo: "prefix-function",
  simulator: "string-kmp",
},
{
  id: "z-function",
  labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
  title: "Z-функция",
  short:
    "Для каждой позиции: сколько символов подряд, начи",
    "аждой букве.",
  formal:
    " $z[i]$  = длина наибольшего общего префикса  $s$  и  $s[i$ ",
  complexity: " $O(n)$ ",
  demo: "prefix-function",
  simulator: "string-z-func",
},
{
  id: "dp",
  labels: [
    "динамическое программирование", "динамики", "дин",
    "мемоизация", "мемоизацию", "подзадач", "подзадач",
  ],
  title: "Динамическое программирование",

```

```

    short:
        "На каждом шаге хватаем то, что лучше прямо сейчас"
    formal:
        "Корректность обычно доказывается «леммой о разрезе»"
    demo: "mst",
    simulator: "mst",
},
{
    id: "euler",
    labels: ["эйлеров цикл", "эйлеров путь", "эйлерова теорема"],
    title: "Эйлеров путь и цикл",
    short:
        "Пройти по каждому ребру ровно один раз, не отрываясь"
    formal:
        "Связный граф: эйлеров цикл    все степени чётные; эйлеров путь    все степени нечётные"
    simulator: "euler-path-vs-cycle",
},
{
    id: "planarity",
    labels: ["планарный", "планарность", "планарного", "планаризация"],
    title: "Планарность",
    short:
        "Граф планарен, если его можно нарисовать на листе бумаги"
    formal:
        "V - E + F = 2 для связного планарного графа. Курсивом выделены вершины, рёбра и области"
    simulator: "planarity-euler-formula",
},
{
    id: "splay",
    labels: [
        "splay", "splay-дерево", "AVL", "AVL-дерево", "всесторонний поиск"
    ],
    title: "Splay и повороты",
    short:
        "Каждый раз, когда к узлу обратились, его «вытягивают» к корню"
    formal:

```



```

        "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
        complexity: "O(V · E)",
        demo: "relax",
        simulator: "bellman-ford",
    },
    {
        id: "floyd",
        labels: [
            "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
            "промежуточная вершина", "промежуточную вершину",
        ],
        title: "Флойд-Уоршелл: разрешаем вершину k",
        short:
            "Внешний цикл — это не «шаг», а «какие пересадки сделать, чтобы пересадку в k?",
        formal:
            "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок вершин произвольный.",
        complexity: "O(V³) времени, O(V²) памяти",
        demo: "floyd",
        simulator: "floyd",
    },
    {
        id: "prim",
        labels: ["Прима", "Прим", "Prim"],
        title: "Алгоритм Прима",
        short:
            "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, не создавая циклов.",
        formal:
            "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создавая циклов.",
        complexity: "O(E log V)",
        demo: "mst",
        simulator: "mst",
    },
    {
        id: "boruvka",

```

```
{
  id: "graph",
  labels: [
    "граф", "графа", "графе", "графы", "графов", "граф",
    "вершины", "вершина", "вершин", "ребра с весом",
  ],
  title: "Граф",
  short:
    "Точки (вершины) и связи между ними (рёбра). Города и дороги.",
  formal:
    " $G = (V, E)$ . Ребро бывает ориентированным (улицы с односторонним движением) и неориентированным (двусторонняя улица).",
  demo: "graph",
  simulator: "graph-dfs-bfs",
},
{
  id: "coord-compress",
  labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
  title: "Сжатие координат",
  short:
    "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде целых чисел. Сжимаем координаты в один порядковый номер.",
  formal:
    "sort + unique + binary search: значение → индекс",
  complexity: " $O(n \log n)$ ",
  demo: "coord-compress",
},
{
  id: "treap",
  labels: ["Treap", "декартово дерево", "декартова д-во"],
  title: "Treap = Tree + Heap",
  short:
    "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
  formal:
    "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить). Ожидаемая высота  $O(\log n)$ .",
  complexity: "ожидаемо  $O(\log n)$ ",
}
```

```

    demo: "np",
  },
  {
    id: "range",
    labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
    title: "Запрос на отрезке",
    short:
      "Спрашиваем не про один элемент, а про кусок массива  

      предподсчёты.",
    formal:
      "Суммы — префиксные суммы  $O(1)$ . Минимум без изменений.",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
  },
  {
    id: "string",
    labels: ["строки", "строке", "строку", "строка", "строкам"],
    title: "Строка как массив символов",
    short:
      "Строковые алгоритмы держатся на трёх вопросах: где  

      (Z-функция) и где встречаются слова словаря (бордер-лист).",
    formal:
      "Наивный поиск подстроки —  $O(N \cdot M)$ . Префикс-функция —  $O(N)$ .",
    demo: "prefix-function",
    simulator: "string-kmp",
  },
  {
    id: "rotation",
    labels: ["поворот", "повороты", "поворота", "поворотом"],
    title: "Поворот (rotation) в дереве",
    short:
      "Локальная перевеска двух узлов: ребёнок встанет на  

      дившемся месту. Порядок ключей при этом не ломается.",
    formal:
      "Правый поворот вокруг p:  $x = p.left; p.left = x$ .  

      g и Zig-Zag.",
    complexity: "O(1)",
    demo: "zig",
  },

```

Универсальное пособие • полный исходный код

```
-----  
/** Быстрый доступ по id. */  
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {  
  
/** Все метки, отсортированные от длинных к коротким (ч  
export const GLOSSARY_LABELS: { label: string; id: string }[] =  
  e.labels.map((label) => ({ label, id: e.id }))  
}).sort((a, b) => b.label.length - a.label.length);  
  
-----
```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 411 | РАЗМЕР: 1.5 КБ

```
-----  
import { Chapter } from "../types";  
  
export const graphChapters: Chapter[] = [  
  {  
    id: "intro",  
    title: "Введение. Базовая база: куча, очередь, бинарное  
    type: "html",  
    category: "Введение",  
    content: `  
      <section id="intro-content" class="mb-12">  
        <div class="flex items-center mb-6 flex-wrap gap-2">  
          <span class="bg-indigo-600 text-white px-4 py-1">Введение</span>  
          <h2 class="text-3xl font-bold text-white">Базовая база</h2>  
        </div>  
  
        <div class="bg-slate-900 border border-indigo-500/40 p-6">  
          <p class="text-slate-200 text-sm leading-relaxed">  
            <b class="text-indigo-300">Базовая база</b> — это  
            дерево как массив, куча, очередь и стек, бинарное  
            Кодим по минимуму: там, где за нас уже написали.  
          </p>  
        </div>  
  
        <div class="space-y-8">
```

```
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heappop(h)
h[0] # 1 – минимум виден без всякого обхода
heapq.heappop(h) # 1, куча починилась за  $O(\log n)$ 
```

<p class="text-slate-300 text-sm">Куча – движатель, который вытаскивает минимальный элемент, и список с элементом, который вытаскивается, и так далее.</p></div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4 border-slate-700"><h3 class="text-xl font-bold text-sky-400 mb-4">Очереди</h3><p class="text-slate-300 text-sm mb-4">Очередь (BFS) отличается от стека тем, что первый вышел) отличаются одной операцией снятия элемента: оба конца работают за $O(1)$. Обычный список сдвигает весь хвост и стоит $O(n)$.</p><pre class="bg-slate-950 border border-slate-700">

```
from collections import deque
q = deque([1, 2])
q.append(3); q.popleft() # 1 – очередь (BFS)
s = [1, 2]; s.append(3); s.pop() # 3 – стек (DFS)</pre><p class="text-slate-300 text-sm">Вся разница между BFS (волнами), стек – DFS (вглубь).</p></div>
```

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4 border-slate-700"><h3 class="text-xl font-bold text-amber-400 mb-4">Сортировка</h3><p class="text-slate-300 text-sm mb-4">Если массив отсортирован – понял, в какой половине жить, – и это 20 сравнений. В Python за это отвечает модуль bisect.</p><pre class="bg-slate-950 border border-slate-700">

```
from bisect import bisect_left
a = [10, 20, 30, 40, 50]
bisect_left(a, 30) # 2 – индекс первого элемента, не меньше 30</pre><p class="text-slate-300 text-sm">Без сортировки – это  $O(n)$ .</p></div>
```

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4 border-slate-700"><h3 class="text-xl font-bold text-rose-400 mb-4">Сортировка</h3><p class="text-slate-300 text-sm mb-4">Писать сортировку – это $O(n^2)$.</p></div>

```

left(x) = z          # z становится левым р
высоты пересчитываются только у z и x — ост
</div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
  <h3 class="text-xl font-bold text-teal-400 mb-4
  <p class="text-slate-300 text-sm mb-4">Оба обхо
    тару». Вся разница в таре: стек уходит вглуби
    ext-emerald-300">0(V + E)</span>: каждая верш
    <pre class="bg-slate-950 border border-slate-700
def go(graph, start, tank):          # tank: список-стек
  seen, tank = {start}, tank([start])
  while tank:
    v = tank.pop() if isinstance(tank, list) else t
    for to in graph[v]:
      if to not in seen:
        seen.add(to); tank.append(to)
  return seen</pre>
  <p class="text-slate-300 text-sm">Дальше эта па
    надстройки над этими восемью строками.</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
  <h3 class="text-xl font-bold text-slate-300 mb-
  <div class="grid grid-cols-1 xl:grid-cols-2 gap
    <div class="bg-slate-800 p-6 rounded-lg bord
      <div class="absolute -top-3 left-4 bg-s
        border-indigo-500 shadow-md">
          Аналогия 1: Схема кинозала
        </div>
        <p class="text-slate-300 text-sm">Дерев
          азателей-«usher'ов» не нужно. Место n, дети
          рху вниз и слева направо.</p>
        </div>
        <div class="bg-slate-800 p-6 rounded-lg bord
          <div class="absolute -top-3 left-4 bg-s

```

```

title: "Билет 14. Дейкстра",
type: "html",
category: "Графы",
content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded
  <h2 class="text-3xl font-bold text-white">Дейкст
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border
    <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Позитивное планирование
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">Аналогия 1: Позитивное планирование
      <p class="text-xl font-mono text-white">Аналогия 1: Позитивное планирование
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          Аналогия 1: Позитивное планирование
        </div>
        <p class="text-slate-300 text-sm mb-4">Аналогия 1: Позитивное планирование
        (нельзя поехать и заработать). Он всегда в
        <div id="slot-chapter-image-dijkstra">
        </div>

      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
          Ограничения
        </div>
        <p class="text-slate-300 text-sm mb-4">Ограничения
        (охождение улицы), Дейкстра сломается, так к

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      ♂ Аналогия 1: Параноик
    </div>
    <p class="text-slate-300 text-sm mb-2">
      еряет ВСЕ возможные дороги V-1 раз подряд.
    <div id="slot-chapter-image-bellman"
    </div>

  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
      Отрицательный цикл
    </div>
    <p class="text-slate-300 text-sm mb-2">
      г станет ЕЩЕ короче – значит мы попали во в
    </div>
  </div>

  <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
  id: "floyd",
  title: "Билет 16. Флойд-Уоршелл",
  type: "html",
  category: "Графы",
  content: `<section id="floyd-content" class="mb-12">
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
  <h2 class="text-3xl font-bold text-white">Флойд

```



```

    },
    {
      id: "aho-corasick",
      title: "Билет 23. Алгоритм Ахо-Корасик",
      type: "html",
      category: "Строки",
      content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1 round
  <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-l
    <h3 class="text-xl font-bold text-blue-400 mb-4">

    <div class="bg-slate-900 p-4 rounded-lg mb-6 text
      <p class="text-sm text-blue-300 italic mb-2">0с
      <p class="text-lg font-mono text-white">Бор (Tr
    </div>
    <p class="text-slate-300 text-sm">Позволяет найти
      -mono text-amber-300 bg-black/30 px-1 rounded">
      y.</p>
    </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      мы двигаемся по веткам. Если нужного продолж
      ("нити страховки") в самый длинный из извест
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2

```

```

while (!q.empty()) {
    int v = q.front(); q.pop();
    for (auto const& [ch, child] : nodes[v].trie) {
        // Суффиксная ссылка ребёнка — это переход
        nodes[child].link = nodes[nodes[v].link].go

        // Если суффиксная ссылка сама является сло
        // Иначе наследуем terminal_link
        if (nodes[nodes[child].link].is_terminal) {
            nodes[child].term_link = nodes[child].l
        } else {
            nodes[child].term_link = nodes[nodes[ch
        }

        q.push(child);
    }
}
}
</div>
</div>
</details>
</div>
</section>`
},
{
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
    <span class="bg-purple-600 text-white px-4 py-1
    <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

    options: [
        "Да: минимум и сумма считаются одним приёмом",
        "Нет: блоки перекрываются, а вычесть перекрытие",
        "Да, если заранее хранить суммы в нулевом уровне",
    ],
    correctIndex: 1,
    explanation: "Два (в 2D – четыре) блока накрывают
        обратная операция, но применить её к перекрытию
        ммы, билет 5."
},
{
    question: "Какие операции подходят разреженной та
    options: [
        "Любые ассоциативные",
        "Идемпотентные: min, max, gcd, побитовые И и ИЛИ",
        "Только сумма и произведение"
    ],
    correctIndex: 1,
    explanation: "Нужно  $op(x, x) = x$ : тогда дважды на
        приём и уходит в префиксные суммы."
},
{
    question: "Между запросами элементы матрицы меняю
    options: [
        "Разреженную таблицу: она перестраивается за  $O(n^2)$ ",
        "Префиксную матрицу: достаточно поправить одну
        "Дерево отрезков: и таблица, и префиксы статичны",
    ],
    correctIndex: 2,
    explanation: "Обе структуры статичны: изменение э
        ксной. С обновлениями работает дерево отрезков,
    }
},
"prefix-sums-2d": [
    {
        question: "Сколько памяти занимает двумерная преф
        options: [
            "N × M – как исходная матрица, плюс рамка из ну

```

```

        ной операции нет. Для min, max и gcd берут разро
    },
    {
        question: "Изменили один элемент матрицы. Что ста
        options: [
            "Поправится одна клетка за  $O(1)$ ",
            "Нужен пересчёт  $O(N \cdot M)$ : от этой клетки зависит
            "Ничего не станет: таблица остаётся верной"
        ],
        correctIndex: 1,
        explanation: "Каждая клетка  $S[i][j]$  включает в се
            я частые, это задача дерева отрезков (билет 1).
    }
],
"euler-path-vs-cycle": [
    {
        question: "В связном графе ровно 2 вершины с нечёт
        options: [
            "Есть эйлеров цикл (вернись домой!)",
            "Есть эйлеров путь (из одной нечётной в другую)
            "Ни пути, ни цикла нет — это полный провал"
        ],
        correctIndex: 1,
        explanation: "Если нечётных вершин ровно 2 — это
            ётной, заканчиваем в другой."
    },
    {
        question: "Почему гамильтонов цикл — это NP-полна
        options: [
            "Потому что Гамильтон — болезнь, и лечить её т
            "Потому что для эйлерова цикла есть простой кри
            "Потому что Эйлер был умнее Гамильтона"
        ],
        correctIndex: 1,
        explanation: "Для эйлерова цикла работает критери
            ого критерия. Это NP-полная задача, которую не
    },
    {

```

```

        "O(2^V) – полный перебор"
    ],
    correctIndex: 1,
    explanation: "DFS обходит каждую вершину и каждое
}
],
"planarity-euler-formula": [
{
    question: "Планарный граф имеет V=6, E=12. Возможн
    options: [
        "Да, если это двудольный граф",
        "Нет, нарушается неравенство  $E \leq 3V - 6$ ",
        "Да, если нет треугольных граней"
    ],
    correctIndex: 1,
    explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$  – на грани пр
        Так что такой граф непланарен."
    },
    {
        question: "Сколько граней у планарного графа с V=
        options: [
            "3 грани",
            "5 граней",
            "10 граней"
        ],
        correctIndex: 1,
        explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$ . Не
    },
    {
        question: "Почему K5 непланарен?",
        options: [
            "Потому что  $10 > 3*5 - 6 = 9$  – нарушение",
            "Потому что Эйлер был против",
            "Потому что 5 вершин – несчастливое число"
        ],
        correctIndex: 0,
        explanation: "У K5: V=5, E=10,  $3V - 6 = 9$ .  $10 > 9$ 
    }
}

```

```

/** Транслировать реальные переменные визуализациям тек
export function emitVizState(
  chapterId: string,
  snapshot: Omit<VizStateEventDetail, "chapterId">
) {
  if (typeof window === "undefined") return;
  const detail: VizStateEventDetail = { chapterId, ...s
  latestByChapter.set(chapterId, detail);
  window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Вернуть демонстрацию в ручной режим при закрытии/ре
export function clearVizState(chapterId: string) {
  if (typeof window === "undefined") return;
  latestByChapter.delete(chapterId);
  window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Текущий снимок компилятора для прямой отрисовки мас
export function useVizRuntime(): VizStateEventDetail | n
  const chapterId = useContext(VizChapterContext);
  const [runtime, setRuntime] = useState<VizStateEventD
  chapterId ? latestByChapter.get(chapterId) ?? null
  );

  useEffect(() => {
    setRuntime(chapterId ? latestByChapter.get(chapterId
    if (!chapterId) return;
    const handler = (e: Event) => {
      const detail = (e as CustomEvent<VizStateEventDet
      if (detail?.chapterId === chapterId) {
        setRuntime(detail.variables || detail.line !==
      }
    };
    window.addEventListener(CHANNEL, handler);
    return () => window.removeEventListener(CHANNEL, ha
  }, [chapterId]);

```

```

    }, [chapterId, currentStep, maxStep, goToStep, selectStep]
  }
}

```

```

/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined) => {
  typeof value === "number" && Number.isFinite(value) ? value : null;
}

```

```

export const vizString = (value: DebugValue | undefined) => {
  typeof value === "string" ? value : null;
}

```

```

export const vizArray = (value: DebugValue | undefined) => {
  Array.isArray(value) ? value : null;
}

```

```

export const vizRecord = (value: DebugValue | undefined) => {
  value !== null && typeof value === "object" && !Array.isArray(value)
    ? (value as Record<string, DebugValue>)
    : null;
}

```

```

/* — Матрицы из компилятора: СВОЯ размерность, а не дефолтная

```

```

/**
 * Нормализованная матрица, пришедшая из выполненного Рунима
 *
 * Размерность берётся из самих данных: если пользователь ввёл
 * 5 вершин, визуализация рисует 5x5, а не подставляет его же
 * 7x7 (раньше ячейки «не той» размерности превращались в 0)
 *
 * Поддерживаются все учебные способы хранения:
 * list[list]      d = [[0, 4], [999, 0]]
 * list[dict]      d = [{0: 0, 1: 4}, {1: 0}]
 * dict[dict]      d = {0: {0: 0, 1: 4}}
 * dict[(i, j)]    d = {(0, 1): 4} — ключ после запятой
 */

```

```

export interface LiveMatrix {
  rows: number;
  cols: number;
  /** Значение ячейки; null — таких данных в коде пользователя нет
  at: (i: number, j: number) => number | null;
}

```

```

    if (n === null) return;
    cells.set(`${i}:${j}`, n);
    rows = Math.max(rows, i + 1);
    cols = Math.max(cols, j + 1);
  };

const asArray = vizArray(value);
if (asArray) {
  asArray.slice(0, MAX_MATRIX_SIDE).forEach((row, i) => {
    const inner = vizArray(row);
    if (inner) {
      inner.slice(0, MAX_MATRIX_SIDE).forEach((cell, j) => {
        return;
      });
    }
    const record = vizRecord(row);
    if (record) {
      for (const [key, cell] of Object.entries(record)) {
        const j = Number(key);
        if (Number.isInteger(j)) put(i, j, cell);
      }
    }
  });
} else {
  const record = vizRecord(value);
  if (!record) return null;
  for (const [key, cell] of Object.entries(record)) {
    const pair = parsePairKey(key);
    if (pair) {
      put(pair[0], pair[1], cell);
      continue;
    }
    const i = Number(key);
    if (!Number.isInteger(i)) continue;
    const inner = vizArray(cell);
    if (inner) {
      inner.slice(0, MAX_MATRIX_SIDE).forEach((v, j) => {
        continue;
      });
    }
  }
}

```

/* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)

```
const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
  chapterId: string;
  /** base-запись страницы – demoId пустой; подписанные */
  demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта */
export function emitVizDemo(chapterId: string, demoId: string) {
  if (typeof window === "undefined") return;
  latestDemoByChapter.set(chapterId, demoId);
  window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>("viz-demo", {
    detail: { chapterId, demoId }
  }));
}

/**
 * Подписка: панель компилятора следует за активной вкладкой
 * Сразу отдаём последнее значение: компилятор часто отстает
 * как пользователь переключился с 1D на 2D.
 */
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
  if (typeof window === "undefined") return () => {};
  const latest = latestDemoByChapter.get(chapterId);
  if (latest !== undefined) cb(latest);
  const handler = (e: Event) => {
    const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
    if (detail && detail.chapterId === chapterId) cb(detail.demoId);
  };
  window.addEventListener(DEMO_CHANNEL, handler);
  return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

Универсальное пособие • полный исходный код

```
-----
tree = [0] * (2 * n)
i = 0                                # лист: tree[n + i]

v, l, r = 1, 0, n - 1               # вершина и её отрезок
m = (l + r) // 2                     # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
    variables: [
        { name: "n", role: "длина исходного массива; листы"},
        { name: "i", role: "индекс элемента массива; его левый и правый листы"},
        { name: "v", role: "текущая вершина дерева отрезка"},
        { name: "l, r", role: "границы отрезка, за которыми лежат левый и правый листы"},
        { name: "m", role: "середина отрезка m = (l + r) // 2"},
    ],
},
treap: {
    vizTitle: "Декартово дерево: build → split → merge",
    stepNote: "Шаг = один спуск в split/merge/erase (перезаписывается целиком.",
    code: `points = [[45, 8], [3, 6], [6, 5], [2, 4], [1, 3]]
n = len(points)          # сколько пар (x; y) – столько точек
mid = 7                  # разрез для split: x ≤ mid < y
key = 9                  # какую точку удаляем в erase
step = 0                 # номер шага – листает кадры анимации
height = 0               # высота построенного дерева
root = None

# --- тело алгоритма ---
class Node:
    def __init__(s, x, y):
        s.x = x; s.y = y; s.l = None; s.r = None

def build(pts):
    """Декартово дерево из отсортированных по x точек:
    pts = sorted(pts, key=lambda p: p[0])
    stack = []
    for x, y in pts:
        cur = Node(x, y)
        last = None
```



```

while x.p:
    p = x.p; g = p.p
    step += 1
    if g is None:
        case = "zig" # родители
        rot(x)
    elif (g.l is p) == (p.l is x):
        case = "zig-zig" # x и p -
        rot(p); rot(x)
    else:
        case = "zig-zag" # змейка
        rot(x); rot(x)
    xk, pk, gk = x.k, p.k, (g.k if g else 0)
    print("шаг", step, "->", case, "| x =", xk, "p = ", pk)
    root = x
    print("    корень теперь", x.k, "| обход:", inorder(root))
    return x

```

1) Zig: родитель x – уже корень (дерево 40 -> 20)
 root = N(40); root.l = N(20); root.l.p = root
 splay(root.l)

2) Zig-Zig: бамбук 60 -> 40 -> 20, оба сына левые
 root = N(60); root.l = N(40); root.l.p = root; root.l.l = N(20)
 splay(root.l.l)

3) Zig-Zag: змейка 60 -> 20 -> 40 (40 – правый сын 20)
 root = N(60); root.l = N(20); root.l.p = root; root.l.r = N(40)
 splay(root.l.r),

```

variables: [
    { name: "case", role: "какой случай сработал: \"zig-zig\", \"zig-zag\" или \"zig\" },
    { name: "step", role: "номер применения случая – от 1 до 3"},
    { name: "xk / x", role: "ключ узла, который поднимали"},
    { name: "pk / p", role: "ключ родителя x перед поворотом"},
    { name: "gk / g", role: "ключ деда – нужен в Zig-Zig"},
    { name: "root", role: "корень дерева после расшейки"},
]

```

```

    stepNote: "Шаг = заполнение очередного  $P[i] = P[i-1]$ 
code: `a = [3, 1, 4, 1, 5, 9, 2, 6]` # исходный массив
n = len(a) # длина: префиксов будет n
i = 0 # текущая граница префикса

P = [0] #  $P[0] = 0$  – пустой префикс
for i in range(1, n + 1):
    P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)`
variables: [
    { name: "a", role: "исходный массив 1D-демонстрация" },
    { name: "n", role: "длина массива a; префиксов будет n" },
    { name: "P", role: "префиксные суммы:  $P[i] = a[0] + \dots + a[i-1]$ " },
    { name: "i", role: "граница префикса: на шаге считаем до i" }
],
},
"dynamic-programming": {
    vizTitle: "Динамическое программирование (мемоизация)",
    stepNote: "Шаг = событие внутри fibМемо(k): вызов,
code: `n = 5`
memo = {}

def fib(k):
    if k in memo:
        return memo[k]
    if k <= 2:
        return 1
    print(f"считаем fib({k-1}) + fib({k-2})")
    r1 = fib(k - 1)
    r2 = fib(k - 2)
    memo[k] = r1 + r2
    return memo[k]

print("итог:", fib(n))`
variables: [
    { name: "n", role: "целевой номер числа Фибоначчи" },
    { name: "k", role: "аргумент текущего рекурсивного вызова" },
    { name: "memo", role: "кэш уже посчитанных значений" }
]
```

```

code: `from collections import deque

q = deque(["A"])          # очередь BFS
u = "B"                  # сосед, найденный из текущ
q.append(u)               # enqueue соседа – в конец
front = q[0]              # голова очереди: её обрабо
v = q.popleft()           # шаг: достали голову и обр
print("front =", front, "| v =", v, "| очередь:", list(
    variables: [
        { name: "front", role: "голова очереди – вершина,
        { name: "v", role: "вершина, извлечённая из очере
        { name: "u", role: "сосед вершины v, которого кла
    ],
},
"everyday-basics": {
    vizTitle: "Бытовой тренажёр: стек, очередь, куча",
    stepNote: "Шаг = одна бытовая операция: тарелка на
    code: `from collections import deque

st, q = [], deque()       # тарелки (стек) и очередь
st.append(" "); top = st.pop()
q.append(" "); q.append(" "); first = q.popleft()

i = 0                     # счётчик выполненных опера
for op in ("push", "pop", "enqueue", "dequeue"):
    i += 1
print("тарелка:", top, "| первый из очереди:", first, "
    variables: [
        { name: "top / front", role: "верх стопки (стек)
        { name: "i", role: "индекс в массиве кучи; родите
    ],
},
"graph-dfs-bfs": {
    vizTitle: "DFS и BFS на графе",
    stepNote: "Шаг = переход из текущей вершины v к неп
    code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
```

Универсальное пособие • полный исходный код

```

-----
v = -1                # вершина, в которой обход начался
stack = []            # фронт обхода (стек для DFS)
i = 0                 # номер шага — двигает по какому-то ребру

# --- тело алгоритма ---
for start in range(n):
    if comp[start] != -1:
        continue
    comps += 1         # непокрашенная вершина => новая компонента
    c = comps
    stack = [start]
    comp[start] = c
    used[start] = True
    v = start
    while stack:
        v = stack.pop() # для BFS: v = stack.pop(0)
        i += 1
        for to in adj[v]:
            if comp[to] == -1:
                comp[to] = c
                used[to] = True
                stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)`
variables: [
    { name: "n", role: "число вершин; в демонстрации — 10" },
    { name: "adj", role: "списки смежности — тот же граф, что в примере" },
    { name: "comp", role: "разметка: номер компоненты", range: "-1 ... comps" },
    { name: "used", role: "булев вариант разметки — для BFS" },
    { name: "comps", role: "счётчик найденных компонент" },
    { name: "c", role: "номер компоненты, которую красим сейчас" },
    { name: "v", role: "вершина, где обход находится" },
    { name: "stack", role: "фронт обхода: стек (DFS) или очередь (BFS)" },
    { name: "i / step", role: "номер шага — перематываем видео" },
],
},
"top-sort": {

```

```
def dfs1(v):
    used[v] = True
    for to in adj[v]:
        if not used[to]:
            dfs1(to)
    order.append(v)          # выходим из вершины — она

for v in adj:
    if not used[v]:
        dfs1(v)
    print("DFS из", v, "закончен | order:", order)

# транспонирование: каждое ребро разворачиваем
adjT = {}
for v in adj:
    adjT[v] = []
for v in adj:
    for to in adj[v]:
        adjT[to].append(v)
print("обратный граф:", adjT)

# фаза 2: идём по order С КОНЦА, DFS по adjT красит одн
comp = {}
c = 0
for i in range(len(order) - 1, -1, -1):
    v = order[i]
    if v in comp:
        continue
    stack = [v]
    comp[v] = c
    while stack:
        u = stack.pop()
        for to in adjT[u]:
            if to not in comp:
                comp[to] = c
                stack.append(to)
    print("SCC", c, "собрана из", v, ":", comp)
```



```
},  
    "bridges-code": {  
        vizTitle: "Мосты: DFS + tin/low",  
        stepNote: "Шаг = строка псевдокода слева; ребро (v,  
            code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1  
timer = 3                                # как на демо  
  
v, to = "B", "G"                        # шаг: возврат из G  
is_bridge = low[to] > tin[v]             # 3 > 2 → мост!  
print(f"ребро {v}-{to}: мост? {is_bridge}")`,  
        variables: [  
            { name: "v", role: "текущая вершина DFS", range:  
            { name: "to", role: "сосед; ребро (v, to) проверяе  
            { name: "tin[v]", role: "время входа в вершину – m  
            { name: "low[v]", role: "минимум tin, куда можно ,  
            { name: "timer", role: "глобальный счётчик времени  
        ],  
    },  
    "euler-path-vs-cycle": {  
        vizTitle: "Эйлеров маршрут: алгоритм Иера",  
        stepNote: "Шаг = переход по неиспользованному ребру  
            идти нельзя.",  
        code: `n = 5                      # вершин: 0...4 (на  
edges = [(0, 1), (0, 2), (1, 3), (2, 4), (1, 2), (3, 4)]  
adj = {v: [] for v in range(n)}  
for i, (u, v) in enumerate(edges):  
    adj[u].append((v, i))  
    adj[v].append((u, i))  
deg = {v: len(adj[v]) for v in range(n)}   # степени ве  
odd = [v for v in range(n) if deg[v] % 2 == 1]  # нечёт  
start = odd[0] if odd else 0                # путь начинаем  
used = [False] * len(edges)                 # какие рёбра :  
path = [start]                              # маршрут: пос.  
last = start                                # где мы сейча  
stack = [start]                             # стек для отк  
v = start  
step = 0
```

```

V = n                                     # вершин для формулы
E = sum(len(a) for a in adj) // 2        # рёбер: в списке смежн
F = 2 - V + E                             # граней: из  $V - E + F = 2$ 

# --- тело алгоритма ---
print("V =", V, "| E =", E, "| F =", F, "|  $V - E + F = 2$ ")
for v in range(n):
    used = set()
    for u in adj[v]:
        if color[u] != 0:
            used.add(color[u])
    c = 1
    while c in used:                       # наименьший свободный цвет
        c += 1
    color[v] = c
    chi = max(chi, c)
    i += 1
    print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
variables: [
    { name: "n", role: "число вершин графа на демонстрации" },
    { name: "adj", role: "списки смежности — тот же граф, что и graph" },
    { name: "color", role: "массив цветов вершин (0 = не покрашено)" },
    { name: "v", role: "вершина, которую красим сейчас" },
    { name: "u", role: "сосед вершины v, чей цвет уже известен" },
    { name: "delta", role: "максимальная степень  $\Delta = \max \deg v$ " },
    { name: "chi", role: "сколько цветов реально понадобится" },
    { name: "i / step", role: "номер шага — перематываем таймер" },
    { name: "V, E, F", role: "для вкладки «Планарность»" },
],
},
dijkstra: {
    vizTitle: "Дейкстра",
    stepNote: "Шаг = старт, извлечение ближайшей вершины",
    code: `import heapq

```

Универсальное пособие • полный исходный код

```
-----
nodes = ["S", "A", "B", "C", "D", "E", "F"]
n = len(nodes)           # 7 вершин => n - 1 = 6 работ
m = len(E)               # 9 рёбер
```

```
dist = {v: INF for v in nodes}
dist["S"] = 0             # старт – База
```

```
# --- тело алгоритма ---
```

```
last_changed = ""
for i in range(1, n):     # итерация i = 1 ... n-1
    last_changed = ""
    for u, v, w in E:     # шаг: релаксация ребра (u, v, w)
        if dist[u] != INF and dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            last_changed = v
    if last_changed == "":
        print(f"итерация {i}: изменений нет – сошлось для всех вершин")
        break
    print(f"итерация {i}: " + ", ".join(f"{k}={dist[k]}:" for k in nodes))
```

```
# контрольный (n-й) проход: улучшение здесь означает отрицательный цикл
cycle_edge = None
for u, v, w in E:
    if dist[u] != INF and dist[u] + w < dist[v]:
        cycle_edge = (u, v, w)
        break
```

```
if cycle_edge:
    print("отрицательный цикл: улучшается ребро", cycle_edge)
else:
```

```
    print("отрицательных циклов нет:", {k: dist[k] for k in nodes})
    variables = [
        { name: "n", role: "число вершин графа – ровно n-1"},
        { name: "m", role: "число рёбер, по которым проходят"},
        { name: "i", role: "номер итерации (после i итераций)"},
        { name: "u, v, w", role: "текущее ребро: откуда, куда, вес"},
        { name: "dist[v]", role: "расстояние; если обновилось, то новое"},
    ],
```

```

        на-Форда" },
        { name: "u, v", role: "ребро (u, v) перевзвешивает",
        { name: "d[v]", role: "расстояние Дейкстры в новом графе"
        },
    ],
},
mst: {
    vizTitle: "Остовное дерево: Краскал и DSU",
    stepNote: "Шаг = следующее ребро из отсортированных",
    code: `n = 9 # вершин: A..I — те, что в графе
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "E"),
        (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "B", "G"),
        (12, "H", "I"), (13, "F", "I")]
edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGH I"} # DSU: представителю
rank = {v: 0 for v in "ABCDEFGH I"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое

# --- тело алгоритма ---
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]] # сжатие путей
        x = parent[x]
    return x

for i, (w, a, b) in enumerate(edges):
    u, v = a, b
    ra, rb = find(a), find(b)
    if ra != rb: # разные компоненты
        parent[ra] = rb
        if rank[ra] == rank[rb]:
            rank[rb] += 1
        mst.append((w, a, b))
        total += w
print("вес остова:", total, "| рёбер:", len(mst))

```

```

vizTitle: "Z-функция",
stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]",
code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0          # правый Z-блок [l, r]
z = [None] * n            # пустые клетки: значения по умолчанию
z[0] = 0

for i in range(1, n):     # шаг: вычисляем z[i]
    if i <= r:
        z[i] = min(r - i + 1, z[i - l]) # из блока
    else:
        z[i] = 0           # свежая клетка: с нуля
    while i + z[i] < n and s[z[i]] == s[i + z[i]]:
        z[i] += 1          # досчёт в лоб
    if i + z[i] - 1 > r:
        l, r = i, i + z[i] - 1
print(z)`
variables: [
    { name: "n", role: "длина строки s", range: "n = len(s)" },
    { name: "i", role: "позиция, для которой считаем z[i]" },
    { name: "l, r", role: "правый крайний Z-блок: отрезок [l, r]" },
    { name: "z[i]", role: "длина наибольшего общего префикса s[i:] и s[0:z[i]]" }
],
},
"aho-corasick": {
    vizTitle: "Ахо-Корасик: бор, fail-ссылки и автомат",
    stepNote: "Шаг = один символ текста: state = go(state, c). Если не найдено, то переходят в корень автомата. Если найдено, то переходят с картинкой «водопада».",
    code: `words = ["HERS", "HIS", "SHE", "HE"] # образцы
text = "USHERS" # текст, в котором ищем
n = len(text) # длина текста
state = 0 # текущее состояние автомата
i = 0 # индекс текущего символа
c = "" # сам символ
nodes = 1 # сколько вершин в автомате
queue = [] # очередь BFS, к которой добавляем новые вершины
fail = {} # fail[v] = вершина, в которую нужно перейти, если в автомате нет ребра из v в c`

```

```

        queue.append(u)
print("вершин в боре:", nodes)
print("fail-ссылки:", fail)

def go(v, ch):
    """Переход автомата: сначала по бору, иначе спускаем
    while v != 0 and (v, ch) not in nxt:
        v = fail[v]
    return nxt.get((v, ch), 0)

for i, c in enumerate(text):
    state = go(state, c)
    v = state
    while v != 0:                                # собираем все образцы
        if v in term:
            matches.append((i, term[v]))
        v = fail[v]
    print(f"i={i} c='{c}' state={state} найдено={matches}")

print("всего вхождений:", len(matches), matches),
variables: [
    { name: "words", role: "набор образцов – по нему",
      },
    { name: "text", role: "сканируемый текст – подста",
    { name: "state", role: "текущее состояние автомата",
    { name: "i / c", role: "индекс и сам символ текста",
    { name: "nxt", role: "переходы бора: nxt[(v, ch)]",
    { name: "fail", role: "fail[v] – вершина наибольшей",
    { name: "term", role: "в каких вершинах заканчивае",
    { name: "queue", role: "очередь BFS, которой натя",
    { name: "matches", role: "найденные вхождения: (и",
    { name: "nodes", role: "сколько вершин в боре (ко",
],
},
"complexity-classes": {
    vizTitle: "Классы сложности: рост по n и ops",
    stepNote: "Шаг = одна точка (n, ops): визуализация",
    code: `n = 1                                # размер входа

```

```

    },
    intro: {
        // мнемодкарточки без шагов алгоритма
        variables: [],
    },

    /* — Дополнительные демонстрации внутри страниц (вкл.)

    "planarity-euler-formula#greedy": {
        vizTitle: "Жадная покраска вершин ( $\chi \leq \Delta + 1$ )",
        stepNote: "Шаг = вершина v получает наименьший цвет",
        code: `n = 8                                # вершин в графе
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj)    # максимальная степень
color = [0] * n                     # 0 = ещё не покрашена
chi = 0                             # сколько цветов понадобилось
v = -1                              # вершина, которую красим сейчас
i = 0                               # номер шага

# --- тело алгоритма ---
for v in range(n):
    used = set()
    for u in adj[v]:
        if color[u] != 0:
            used.add(color[u])
    c = 1
    while c in used:                  # наименьший свободный цвет
        c += 1
    color[v] = c
    chi = max(chi, c)
    i += 1
    print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
variables: [
    { name: "color", role: "цвета вершин (0 = не покрашено)", range: "0 ... n" },
    { name: "v", role: "текущая вершина", range: "0 ... n-1" },
    { name: "delta / chi", role: "максимальная степень / количество цветов", range: "0 ... n" },
    { name: "i / step", role: "номер кадра", range: "0 ... n-1" },
]
        `
    },

```

```

    ],
},
"planarity-euler-formula#edges": {
    vizTitle: "Покраска рёбер (Визинг:  $\Delta \leq \chi' \leq \Delta + 1$ )",
    stepNote: "Шаг = ребро e получает наименьший цвет, <= \Delta",
    code: `n = 8
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 4), (3, 4)]
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj)
ecolor = [0] * len(edges) # цвет каждого ребра
inc = [dict() for _ in range(n)] # inc[v][цвет] = ребро
e = -1 # номер текущего ребра
v = u = -1 # его концы
chi_e = 0

# --- тело алгоритма ---
for e, (a, b) in enumerate(edges):
    v, u = a, b
    used = set(inc[a].values()) | set(inc[b].values())
    c = 1
    while c in used:
        c += 1
    ecolor[e] = c
    inc[a][e] = c
    inc[b][e] = c
    chi_e = max(chi_e, c)
    print("ребро", (a, b), "занято:", sorted(used), "-> ", c)
print("chi' =", chi_e, "| delta =", delta, "| delta <= \Delta")
variables: [
    { name: "ecolor", role: "цвета рёбер – рёбра на графе"},
    { name: "e", role: "номер текущего ребра", range: "0..n-1"},
    { name: "v, u", role: "концы текущего ребра – подмножества V"},
    { name: "inc", role: "для каждой вершины: какие цвета рёбер"},
    { name: "chi_e / delta", role: "\chi' – сколько цветов"},
]
},
"graph-components#dfs": {
    vizTitle: "Компоненты связности: DFS-покраска",

```



```

    },
    "graph-components#dsu": {
        vizTitle: "Компоненты связности: DSU по рёбрам",
        stepNote: "Шаг = одно ребро (u, v): union склеивает",
        code: `n = 10                                # вершин в графе
edges = [(0, 1), (1, 2), (2, 3), (0, 3), (1, 3), (4, 5)]
parent = list(range(n))    # DSU: parent[v] = представи
rank = [0] * n             # высота дерева – по ней реш
comps = n                  # сначала каждая вершина – о
e = -1                     # номер обрабатываемого ребра
u, v = -1, -1              # концы текущего ребра
step = 0                   # номер шага – двигает по ка

# --- тело алгоритма ---
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]    # сжатие путей
        x = parent[x]
    return x

def union(a, b):
    global comps
    ra, rb = find(a), find(b)
    if ra == rb:
        return False                    # ребро внутри
    if rank[ra] < rank[rb]:
        ra, rb = rb, ra
    parent[rb] = ra
    if rank[ra] == rank[rb]:
        rank[ra] += 1
    comps -= 1
    return True

for e, (a, b) in enumerate(edges):
    u, v = a, b
    step = e + 1
    union(a, b)
print("компонент:", comps)

```

```

-----
k = 3                                # размер клики, которую ищем

# --- тело алгоритма ---
# Сведение 3-SAT <=p КЛИКА: вершина = {номер дизъюнкта,
# ребро = два литерала из РАЗНЫХ дизъюнктов, не противоречивые
clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
V = [(i, l) for i, cl in enumerate(clauses) for l in cl]
E = [(a, b) for a in V for b in V if a[0] < b[0] and a[1] != b[1]]
for step in range(7):
    print("кадр", step, "| вершин:", len(V), "| рёбер:", len(E))
print("формула выполнима <=> в графе есть клика размера k")
variables: [
    { name: "step", role: "номер кадра сведения (0..6)" },
    { name: "cls", role: "класс задачи A, которую сводят к КЛИКА" },
    { name: "n", role: "размер входа x; после сведения к КЛИКА" },
    { name: "k", role: "размер клики, которую ищет рекурсия" },
],
},
"complexity-classes#verify": {
    vizTitle: "Верификатор 3-SAT: проверка сертификата",
    stepNote: "Шаг = один дизъюнкт: верификатор подставляет значения в формулу",
    code: `clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
cert = [1, 0, 1, 0]          # сертификат: x1=1, x2=0, x3=1
k = 0                        # номер проверяемого дизъюнкта
ok = True                    # вердикт верификатора

# --- тело алгоритма ---
for k, cl in enumerate(clauses):
    hit = False
    for l in cl:
        val = cert[l - 1] if l > 0 else 1 - cert[-l - 1]
        if val == 1:
            hit = True
    ok = ok and hit
    print("дизъюнкт", k + 1, cl, "выполнен" if hit else "не выполнен")
print("сертификат", "принят" if ok else "отвергнут")`,
    variables: [
        { name: "cert", role: "сертификат – назначение переменных" },
    ],
},

```

```

-----
edges.sort()                # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGHI"} # DSU: представител
rank = {v: 0 for v in "ABCDEFGHI"}   # высота дерева
mst = []                        # рёбра, взятые в остов
total = 0                      # вес остова (на этом графе 51)
i = 0                          # номер текущего ребра
u, v, w = None, None, None     # концы и вес ребра, которое рассматриваем

# --- тело алгоритма ---
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]] # сжатие путей
        x = parent[x]
    return x

for i, (w, a, b) in enumerate(edges):
    u, v = a, b
    ra, rb = find(a), find(b)
    if ra != rb:                # разные компоненты
        parent[ra] = rb
        if rank[ra] == rank[rb]:
            rank[rb] += 1
        mst.append((w, a, b))
        total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)`
    variables: [
        { name: "i", role: "номер текущего ребра – перебираем"},
        { name: "u, v, w", role: "концы и вес рассматриваемого ребра"},
        { name: "parent", role: "DSU-массив: цвета компонентов"},
        { name: "mst", role: "рёбра, уже взятые в остов", "51"},
        { name: "total", role: "вес остова (51 на этом графе)"},
    ],
},
"mst#prim": {
    vizTitle: "Остовное дерево: Прим (рост из вершины E)",
    stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
    code: `import heapq

```

```

-----
vizTitle: "Остовное дерево: Борувка (фазы-«пятiletки»)"
stepNote: "Шаг = одна фаза: каждая компонента ОДНОВЕРХОВА"
code: `n = 9                                # вершин: A...I
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "C", "D"),
         (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "F", "G"),
         (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGH I"}         # DSU: корень компоненты
cheapest = {}                               # корень компоненты -> самое дешёвое ребро
mst = []                                    # рёбра остова
total = 0                                  # вес остова (= 51)
phase = 0                                  # номер фазы («пятiletки») — от 0 до 4
u, v, w = None, None, None                # ребро-кандидат

# --- тело алгоритма ---
def find(x):
    while comp[x] != x:
        comp[x] = comp[comp[x]]
        x = comp[x]
    return x

while len(mst) < n - 1:
    phase += 1
    cheapest = {}
    for (w, a, b) in edges:                # КАЖДАЯ компонента ОДНОВЕРХОВА
        ra, rb = find(a), find(b)
        if ra == rb:
            continue
        u, v = a, b
        if ra not in cheapest or w < cheapest[ra][0]:
            cheapest[ra] = (w, a, b)
        if rb not in cheapest or w < cheapest[rb][0]:
            cheapest[rb] = (w, a, b)
    if not cheapest:
        break                               # граф несвязен
    for (w, a, b) in cheapest.values():
        ra, rb = find(a), find(b)
        if ra == rb:
            continue                        # две компоненты

```

```

        { name: "k", role: "уровень: блоки  $2^k \times 2^k$  (шаг
        { name: "st2[{r,c,k,k}]", role: "минимум квадратн
    },
    ],
},
"sparse-table#2d-query": {
    vizTitle: "Разреженная таблица 2D: запрос",
    stepNote: "Демонстрация кликабельна вручную: выбери
    code: `# st2[{r, c, kx, ky}] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
    variables: [
        { name: "r1, c1, r2, c2", role: "углы запрашиваем
        { name: "kx, ky", role: "уровень самого крупного
    ],
},
"prefix-sums-2d#2d": {
    vizTitle: "Префиксные суммы 2D (прямоугольники)",
    stepNote: "Демонстрация кликабельна вручную: наведи
    code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
    S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
    variables: [
        { name: "i, j", role: "текущая ячейка таблицы S (

```

```
-----
const keep: string[] = [];
for (const line of code.split("\n")) {
  const t = line.trim();
  if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
    keep.push(line);
  }
}
// убрать пустые строки с конца
while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
  keep.pop();
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, sync: PageSync) {
  const sync = getPageSync(chapterId, demoId);

  const head = sync.vizTitle
    ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#`
    : `# «${chapterTitle}»\n# демо на странице нет – св

  if (!sync.code) return head;
  const init = extractInit(sync.code);
  return `${head}\n${init}\n`;
}
-----
```

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

=====

ФАЙЛ 26/111: src/data/tickets/ticket1_segtree.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 217 | РАЗМЕР: 1.5 КБ

import { Chapter } from '../../types';

```

span class="font-mono text-emerald-300">[0, 1
тям <span class="font-mono text-emerald-300
– это одиночные элементы массива. В узле л
чество единиц, И/ИЛИ – любая операция, кото
>).</p>

```

```

<p class="text-slate-300 text-sm mb-4">Отсю
а каждом уровне встречает один из трёх случ
не пересекается – возвращаем нейтральный э
склеиваем. Частичных пересечений на уровне
одного элемента – спуск к листу и пересчёт

```

```

<div class="grid grid-cols-1 sm:grid-cols-3
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2
  <div class="bg-slate-800 p-6 rounded-lg
    <div class="absolute -top-3 left-4 l
  <div class="border border-emerald-500 shadow-md">

```

Аналогия 1: Корпоративная пре

```

  </div>
  <p class="text-slate-300 text-sm">Б
икам департаментов: «внутри всем +5 %». Нач
nt-mono text-emerald-300">lazy[v]</span>)</div>
свой баланс. Это и есть <b>отложенное обнов
</div>

```

```
<p class="text-slate-500 text-xs mt-3">
  кивают в строгом порядке: сначала присваива
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border
  <p class="text-slate-300 text-sm mb-2">
  <p class="text-slate-400 text-sm mb-1">
за  $O(1)$ , но строятся один раз и умирают при
ми.</p>
```

```
  <p class="text-slate-400 text-sm mb-1">
идемпотентные запросы (min, gcd, И) по ста
  <p class="text-slate-400 text-sm mb-1">
n ячеек (не  $2n!$ ) – иначе на  $n = 6$  поймать п
n ...  $2n-1$ ] и  $n$  – степень двойки (или использ
  <p class="text-slate-400 text-sm">• <b>
ть лень во всё дерево – это  $O(n)$  и весь смы
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
-2 py-0.5 rounded text-xs font-mono text-em
ся один раз), запрос на отрезке <code class=
-slate-800"> $O(\log n)$ </code>, одиночное обно
-300 border border-slate-800"> $O(\log n)$ </code>
s font-mono text-emerald-300 border border-
xt-xs font-mono text-emerald-300 border bord
ounded text-xs font-mono text-emerald-300 b
```

```
<p class="text-slate-500 text-xs mt-3 leading
  в редакторах и базах, сжатие координат + «
оить», дерево отрезков по дереву (HLD), пер
</p>
```

```
<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

Код: построение и запрос суммы (С

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```



```

        y[4 * MAXN];           // «обещание»: прибавит

void push(int v, int l, int r) {
    if (lazy[v] == 0) return;
    int m = (l + r) / 2;
    tree[2 * v]      += lazy[v] * (m - l + 1);    // п
    lazy[2 * v]      += lazy[v];
    tree[2 * v + 1] += lazy[v] * (r - m);        // и
    lazy[2 * v + 1] += lazy[v];
    lazy[v] = 0;                                // о
}

void add(int v, int l, int r, int L, int R, int x) { //
    if (r < L || R < l) return;
    if (L <= l && r <= R) {
        tree[v] += x * (r - l + 1);
        lazy[v] += x;                        // д
        return;
    }
    push(v, l, r);                            // о
    int m = (l + r) / 2;
    add(2 * v, l, m, L, R, x);
    add(2 * v + 1, m + 1, r, L, R, x);
    tree[v] = tree[2 * v] + tree[2 * v + 1];
}

```

<p class="text-slate-400 text-xs">Пр
 г накопиться. Забыл push – получил правильн
 </div>
 </details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: итеративное дерево (2n, снизу)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 # для удобств

```
</section>
```

```
`,  
  },  
];
```

ФАЙЛ 27/111: src/data/tickets/ticket14_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТР0К: 1235 | РАЗМ:

```
import { Chapter } from '../..types';
```

```
export const tickets14to20: Chapter[] = [  
  {
```

```
    id: "dijkstra",  
    title: "14. Графы. Поиск кратчайшего пути. Дейкстра",  
    type: "html",  
    category: "Графы. Пути",  
    content: `
```

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">  
  <div class="flex items-center mb-6 flex-wrap gap-3">  
    <span class="bg-blue-600 text-white px-4 py-1 rounded">  
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">  
  </div>
```

```
<div class="space-y-8">  
  <div class="bg-slate-700/50 p-6 rounded-xl border">  
    <h3 class="text-xl font-bold text-emerald-400">  
  
    <div class="bg-slate-900 p-4 rounded-lg mb-4">  
      <p class="text-lg text-emerald-300 italic">  
      <p class="text-xl font-mono text-white">  
    </div>
```

```
<p class="text-slate-300 text-sm mb-4">Дейк  
ершины <span class="font-mono text-emerald-300">  
требование к графу — <b>неотрицательные вес
```

```
ext-emerald-300">w ≥ 0</span>}, зафиксирова
ию расстояния!</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
  <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
```

Ограничения: минус ломает жадно

```
</div>
```

```
<p class="text-slate-300 text-sm mb
lass="font-mono text-emerald-300">s→a = 2</
ono text-emerald-300">b→a = -2</span>. Верн
начала вытащит <span class="font-mono text-
o text-emerald-300">3 - 2 = 1</span> придёт
Джонсона (билет 17).</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-blue-400 m
```

```
<p class="text-slate-300 text-sm mb-4">Демон
D, Дом 2 E, ТЦ F, Метро G) и 11 <b>одностор
ршины <span class="font-mono text-emerald-3
(u → v)</span>: переменные <span class="fon
>, <span class="font-mono text-emerald-300">
аут картинку напрямую.</p>
```

```
<ol class="list-decimal pl-5 space-y-2 text
  <li><b>Инициализация.</b> <span class="
text-emerald-300">∞</span>; в кучу кладём <
emerald-300">prev</span> пуст.</li>
```

```
<li><b>Извлечь минимум.</b> Достаём пар
"font-mono text-emerald-300">du</span>. Если
xt-emerald-300">done</span> – пара устарела
```

```
<li><b>Зафиксировать.</b> <span class="
меняется.</li>
```

```
</div>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg">
  <p class="text-sm font-bold text-white">
    <table class="w-full text-xs text-left">
      <thead>
        <tr class="text-slate-500 border">
          <th class="py-1.5 pr-2 font">
          <th class="py-1.5 pr-2 font">
          <th class="py-1.5 font-bold">
        </tr>
      </thead>
      <tbody class="text-slate-300">
        <tr class="border-b border-slate-
y-2 pr-2">лучший известный путь  $s \rightarrow v$ </td><td>
        <tr class="border-b border-slate-
y-2 pr-2">предок  $v$  на текущем лучшем пути</td><td>
        <tr class="border-b border-slate-
lass="py-2 pr-2">вершины с окончательным  $dist[v]$ </td></tr>
        <tr class="align-top"><td class="py-2">исходная вершина, вершина)</td><td class="py-2">извлечение  $v$  из очереди</td></tr>
      </tbody>
    </table>
  </div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border">
  <h3 class="text-xl font-bold text-indigo-400">Извлечение из кучи

  <p class="text-slate-300 text-sm mb-4"><b>Инициализация</b>
  <span>  $s$  извлекается из кучи, <span class="font-mono text-emerald-300"> $s \rightarrow u$ </span>. Отсюда
  <span class="text-emerald-300"> $dist[v] = \delta(s, v)$ </span> для всех достижимых  $v$ .
  <span class="text-emerald-300"> $dist[v] = \infty$ </span> для недостижимых  $v$ .
```

```
<details class="bg-slate-900/40 rounded-lg">
  <summary class="p-4 font-bold text-slate-300">Алгоритм Дейкстры
```

```

        </div>
    </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-slate-800">
    <h3 class="text-xl font-bold text-emerald-400">Дейкстра на графе
    <div class="overflow-x-auto my-4">
        <table class="w-full text-xs text-left">
            <thead>
                <tr class="text-slate-500 border-b">
                    <th class="py-2 pr-3 font-b">Взвешенный ребро
                    <th class="py-2 pr-3 font-b">Взвешенный узел
                    <th class="py-2 font-bold">Путь
                </tr>
            </thead>
            <tbody class="text-slate-300">
                <tr class="border-b border-slate-600">
                    <td>priority_queue (C++)</td><td class="py-2 pr-3">
                        й выбор</td></tr>
                <tr class="border-b border-slate-600">
                    <td>ease-key</td><td class="py-2 pr-3 font-mono">
                        очереди</td></tr>
                <tr class="border-b border-slate-600">
                    <td>поиск минимума</td><td class="py-2 pr-3 font-mono">
                        </tr>
                <tr class="border-b border-slate-600">
                    <td>><td class="py-2 pr-3 font-mono">0(V + E)</td></tr>
                <tr class="align-top">
                    <td class="font-mono">0(E + V · C)</td><td class="py-2">ц
                    </td></tr>
            </tbody>
        </table>
    </div>

<details class="bg-slate-900/40 rounded-lg border-2 border-slate-800">
    <summary class="p-4 font-bold text-slate-400 border-b border-slate-700/50">» Код: Дейкстра на
    <div class="p-5 text-sm text-slate-300"

```

```

    "E": [{"F", 1}],
    "F": [],
    "G": [{"D", 1}, {"E", 2}],
}
dist, prev = dijkstra(g, "A")
print(dist) # A 0, B 3, C 2, D 6, E 7,
print(restore(prev, "A", "F")) # ['A', 'C', 'G', 'E',
    <p class="text-slate-400 text-xs">П
        же вершины, потому что читает ваши <span c
nt-mono">done</span>.</p>
    </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">> Код: C++ (priority
    <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
    t;
#include <queue>;
using namespace std;
const long long INF = 4e18; // не INT_MAX:

int n;
vector<vector<pair<int,int>>>> g;
vector<long long> d;
vector<int> p;

void dijkstra(int s) {
    d.assign(n, INF); p.assign(n, -1); d[s] = 0;
    priority_queue<pair<long long,int>;,
        vector<pair<long long,int>>&
        greater<pair<long long,int>>
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue; // устарев
        for (auto [v, w] : g[u]) {

```

```
used[u] = True # фиксируем d[u]
for v in range(n): # релаксируем v
```

```
    if d[u] + w[u][v] < d[v]:
        d[v] = d[u] + w[u][v]
```

Т. Память – матрица

```
</div>
</details>
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border>
  <h3 class="text-xl font-bold text-amber-400>
```

```
<div class="bg-rose-950/30 p-5 rounded-lg b>
  <p class="text-sm font-bold text-rose-3>
```

```
  <ul class="list-disc pl-5 space-y-2 tex>
    <li><b>Отрицательные веса.</b> Алгор
    ксация вершины необратима. Нужен Форд–Беллм
    <li><b>Отрицательные циклы.</b> При
    ечное число. Сначала проверьте граф Фордом–
    <li><b>Ленивое удаление.</b> Без пр
    одна вершина обрабатывается несколько раз:
    <li><b>Переполнение INF.</b> <span >
  </span> уходит в отрицательное число и «улу
  "font-mono">u</span>.</li>
```

```
    <li><b>Ориентированность.</b> Демо-
    ом графе каждое ребро добавляется в оба спи
    <li><b>Дейкстра ≠ BFS.</b> BFS – ча
    С разными весами FIFO ломается, нужна куча
    <li><b>«Одна пара».</b> Алгоритм сч
    ий выход при извлечении цели (билет 17), но
  </ul>
```

```
</div>
```

```
<p class="text-slate-300 text-sm mt-4 mb-4">
ов ценой скорости <span class="font-mono te
font-mono text-emerald-300"> $O(V^3)$ </span>; б
после перевзвешивания потенциалов). Предст
```

```

<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия 1: Патруль Бэтмена на м
    </div>
    <div class="my-3 rounded-lg overflow">
      все ули
овно <b>1 выходной в году</b>, когда он отде
-ю ночь Бэтмен делает контрольный проезд: е
о-то района снова уменьшилось – значит, Джо
</div>

```

```

  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
      Отрицательный цикл: временная п
    </div>

```



```

    </table>
    <p class="text-slate-400 text-xs mt-3">
бном» порядке: <span class="font-mono">S→A,
порядок рёбер местами – и понадобится до ше
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
    <p class="text-sm font-bold text-white"
    <p class="text-slate-300 text-sm mb-3">
тава A</span> имеет суммарный вес <span cla
веет» на 2.</p>
    <table class="w-full text-xs text-left"
        <thead>
            <tr class="text-slate-500 borde
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 font-bold
            </tr>
        </thead>
        <tbody class="text-slate-300">
            <tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2 pr-2 fo
td></tr>
            <tr class="border-b border-slate
-2 font-mono">-2</td><td class="py-2 pr-2 f
/td></tr>
            <tr class="border-b border-slate
-2 font-mono">-6</td><td class="py-2 pr-2 f
</td></tr>
            <tr class="border-b border-slate
"py-2 pr-2 font-mono">-10</td><td class="py
nt-mono">-2</td></tr>
            <tr class="align-top"><td class
12</td><td class="py-2 pr-2 font-mono">-9</
ont-bold">всё ещё уменьшается → отрицательны
        </tbody>

```

```

<div class="overflow-x-auto">
  <table class="w-full text-xs text-l
    <thead>
      <tr class="text-slate-500 b
        <th class="py-1.5 pr-2
        <th class="py-1.5 pr-2
        <th class="py-1.5 pr-2
        <th class="py-1.5 font-l
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-
2">одна вершина → все</td><td class="py-2 p
      <tr class="border-b border-
font-mono">0((V+E) log V)</td><td class="p
      <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">нет</td><td c
    </tr>
      <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">молча ошибётся
emerald-300">обнаруживает (d[i][i] &lt; 0)<
      <tr class="border-b border-
ass="py-2 pr-2">список смежности</td><td cl
      <tr class="align-top"><td c
φ, веса ≥ 0</td><td class="py-2 pr-2">минус
    </tbody>
  </table>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-emerald-4

  <div class="bg-slate-900/60 p-4 rounded-lg l
    <p class="text-xs font-bold text-amber-:
    :</p>
  <pre class="bg-slate-950 p-3 rounded text-xs font-mono

```

```

if changed_vertex == -1:
    return dist, prev, None # отрицательны

# changed_vertex улучшен на n-й итерации:
# поднимаемся по prev на n шагов – гарантированно з
y = changed_vertex
for _ in range(n):
    y = prev[y]

cycle = [y]
v = prev[y]
while v != y:
    cycle.append(v)
    v = prev[v]
cycle.append(y)
cycle.reverse()
return dist, prev, cycle

# демо-граф страницы (9 рёбер, режим с отрицательным ци
nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [{"S", "A", 4}, {"S", "B", 5}, {"B", "C", -2}, {"A", "C", 1},
      {"C", "D", 3}, {"C", "E", 4}, {"D", "F", 2}, {"E", "F", 1}]
E = [(idx[u], idx[v], w) for u, v, w in E]

dist, prev, cyc = bellman_ford(len(nodes), E + [(idx["D",
print([dist[i] for i in range(len(nodes))]) # [0, 4,
print(cyc) # None –

dist2, _, cyc2 = bellman_ford(len(nodes), E + [(idx["D",
print([nodes[i] for i in cyc2]) # ['D',
print([dist2[i] for i in range(len(nodes))]) # все
    <p class="text-slate-400 text-xs">П
        имая вершина с <span class="font-mono">INF<
        ля <span class="font-mono">float('inf')</sp
    </div>
</details>

```

```

        vector<int> cycle;
        for (int v = y;; v = p[v]) {
            cycle.push_back(v);
            if (v == y && cycle.size() > 1) break;
        }
        reverse(cycle.begin(), cycle.end());
        return cycle;
    }

```

```

int main() {
    int s = 0;
    int x = bellman_ford(s);
    if (x == -1) cout <<< "отрицательных циклов не  

    else {
        cout <<< "цикл: ";
        for (int v : negative_cycle(x)) cout <<< v  

        cout <<< "\n";
    }
}
</code></pre>

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg

<summary class="p-4 font-bold text-slate

-b border-slate-700/50">» Оптимизации: ранни

<div class="p-5 text-sm text-slate-300

```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
    rt deque

```

```

def spfa(n, adj, s):
    """adj[u] = [(v, w), ...]. Релаксируем только «живые» вершины.
    INF = float("inf")
    dist = [INF] * n
    in_queue = [False] * n
    counter = [0] * n # сколько раз вершина попала в очередь
    dist[s] = 0
    q = deque([s])
    in_queue[s] = True

```

```

отрицательным циклом <span class="font-mono" style="font-family: monospace;">
    <li><b>Переполнение INF.</b> <span class="font-mono" style="font-family: monospace;">
        INT_MAX</span> превращается в отрицательное значение, что приводит к переполнению
        запасом и <span class="font-mono" style="font-family: monospace;">long long</span>
    <li><b>Неориентированный граф с минимальным циклом</b> <span class="font-mono" style="font-family: monospace;">-3</span> – это цикл длины 3, который встречается в отрицательном цикле. Минусы в неориентированном графе.
    <li><b>Цикл есть, но не от старта.</b> <span class="font-mono" style="font-family: monospace;">-3</span> </span>. Нужна проверка всего графа – добавляем ребро (1, 2) и запускаем от неё.</li>
    <li><b>«Ответ  $-\infty$ ».</b> Если отрицательный цикл, то <span class="font-mono" style="font-family: monospace;">dist[v] = -\infty</span>: печатать ответ, если не отрицательный цикл, то ходом в обратном графе.</li>
    <li><b>SPFA как «быстрый Форд–Беллман».</b> В условии <span class="font-mono" style="font-family: monospace;">V, E \le 10^5</span>
  </ul>
</div>

<p class="text-slate-300 text-sm mt-4"><b>Сложные задачи</b>: поиск кратчайшего пути. Потенциалы <span class="font-mono text-emerald-500" style="font-family: monospace;"></span> (15) решаются Дейкстрой. Флойд–Уоршелл (16) решает все задачи на кратчайшие пути. Порядок для DAG – билет 9.</p>
</div>
</div>
</section>`
},
{
  id: "floyd",
  title: "16. Графы. Поиск кратчайшего пути. Флойд",
  type: "html",
  category: "Графы. Пути",
  content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">16. Графы. Поиск кратчайшего пути. Флойд</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">16. Графы. Поиск кратчайшего пути. Флойд</h2>
  </div>
  </section>`
}
]

```

```

        Суть фокуса (по шагам)
    </div>
    <p class="text-slate-300 text-sm mb-4 lg:mb-6">
        Главный герой – внешний цикл
        <span class="font-mono text-emerald-300">k</span> т.е.
        </span>, если в качестве промежуточных
        <span class="font-mono text-emerald-300">{i, ..., k}</span>?». Каждый шаг добавляет в ра
    </p>
    <ul class="text-sm text-slate-300 space-y-4">
        <li><b>Шаг k = -1 (старт):</b> разрешены перестановки
        <li><b>Шаг k = 0:</b> разрешаем перестановки
        <span class="font-mono text-emerald-300">i → 0 → j</span>, чем текущий <span class="font-mono text-emerald-300">k</span>
        <li><b>Шаг k = 1:</b> разрешены перестановки
        <span class="font-mono text-emerald-300">1 → j</span> уже может стоять
        <li><b>Шаг k = V - 1:</b> разрешены перестановки
        <span class="font-mono text-emerald-300">1 → V - 1</span>
    </ul>
</div>

<div class="grid grid-cols-1 sm:grid-cols-3 lg:grid-cols-4">
    <div class="bg-slate-900 border border-emerald-500 shadow-md p-6">
        <p class="text-[10px] uppercase tracking-wider">Шаг k = -1 (старт):</p>
        <p class="font-mono text-emerald-300">{i, ..., k}</p>
    </div>
    <div class="bg-slate-900 border border-emerald-500 shadow-md p-6">
        <p class="text-[10px] uppercase tracking-wider">Шаг k = 0:</p>
        <p class="font-mono text-emerald-300">i → 0 → j</p>
    </div>
    <div class="bg-slate-900 border border-emerald-500 shadow-md p-6">
        <p class="text-[10px] uppercase tracking-wider">Шаг k = 1:</p>
        <p class="font-mono text-emerald-300">1 → j</p>
    </div>
    <div class="bg-slate-900 border border-emerald-500 shadow-md p-6">
        <p class="text-[10px] uppercase tracking-wider">Шаг k = V - 1:</p>
        <p class="font-mono text-emerald-300">1 → V - 1</p>
    </div>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Таблица пересадок в
        </div>
    </div>
    <div class="bg-slate-800 p-6 rounded-lg">
        Аналогия 2: Таблица пересадок в
    </div>
</div>

```

```

                <th class="py-1.5 pr-2 font
                <th class="py-1.5 font-bold
            </tr>
        </thead>
        <tbody class="text-slate-300">
            <tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2">в вершн
            <tr class="border-b border-slate
-2 font-mono">4</td><td class="py-2">открыва
            <tr class="border-b border-slate
-2 font-mono">3</td><td class="py-2">2 – вх
            <tr class="border-b border-slate
-2 font-mono">2</td><td class="py-2">3 – вх
            <tr class="border-b border-slate
-2 font-mono">4</td><td class="py-2">через
            <tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2">из 5 в
            <tr class="align-top"><td class=
300">10</td><td class="py-2">ребро 6 → 1 за
        </tbody>
    </table>
    <p class="text-slate-400 text-xs mt-3">
mono text-emerald-300">0 → {0, 4, 7, 2, 6,
"font-mono">0 → 3 → 4 → 6</span> (2 + 4 + 1
всего в финальной матрице 11 таких пар. Диа
</div>

<div class="bg-slate-800/70 p-5 rounded-lg
    <p class="text-sm font-bold text-white
    <table class="w-full text-xs text-left
        <thead>
            <tr class="text-slate-500 borde
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 font-bold
            </tr>
        </thead>
        <tbody class="text-slate-300">

```

```

- mono">d[k][k] < 0 и рассуждение
</div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">> Эксперимент: пере
  <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
M = [[0,4,INF,2,INF,INF,INF],
      [INF,0,3,INF,INF,3,INF],
      [INF,INF,0,INF,2,INF,INF],
      [INF,INF,INF,0,4,2,INF],
      [INF,INF,INF,INF,0,INF,1],
      [INF,INF,INF,INF,INF,0,5],
      [INF,1,INF,INF,INF,INF,0]]

```

```

def floyd(M, order="kij"):
    n = len(M)
    d = [row[:] for row in M]
    for a in range(n):
        for b in range(n):
            for c in range(n):
                k, i, j = {"kij": (a, b, c),
                           "ijk": (c, a, b),
                           "ikj": (c, a, b)}[order]
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    return d

```

```

good = floyd(M, "kij")
bad = floyd(M, "ijk")
print(good[2][1], bad[2][1])    # 4 и 999 – путь 2→4→6→1
print(sum(1 for i in range(7) for j in range(7) if good
# 4 клетки отличаются; при порядке (i,k,j) их 6
<p class="text-slate-400 text-xs">В
угих «почти работают» (большинство клеток с
</div>

```



```

        return d, nxt, negative

def path(nxt, i, j):
    if nxt[i][j] == -1:
        return None # j недостижим
    p = [i]
    while p[-1] != j:
        p.append(nxt[p[-1]][j])
    return p

nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [{"S", "A", 4}, {"S", "B", 5}, {"B", "C", -2}, {"A", "C", 1},
      {"C", "D", 3}, {"C", "E", 4}, {"D", "F", 2}, {"E", "F", 1}]
E = [(idx[u], idx[v], w) for u, v, w in E]

d, nxt, neg = floyd_warshall(len(nodes), E)
print([d[0][j] for j in range(len(nodes))]) # [0, 4, 5, 3, 2, 1]
print(path(nxt, idx["S"], idx["F"])) # [0, 2, 1]
print(neg) # [] — отрицательного цикла нет

```

</div>
</details>

```

<details class="bg-slate-900/40 rounded-lg p-4">
  <summary class="p-4 font-bold text-slate-300">
    -b border-slate-700/50">» Код: C++ (матрица
    <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
    t;
#include <algorithm>
using namespace std;
const long long INF = 4e18;

int n;
vector<vector<long long>>> d(n, vector<long long>(n, INF));

void floyd() {
    for (int i = 0; i < n; ++i) d[i][i] = 0;

```

```

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">» Та же динамика дл
    <div class="p-5 text-sm text-slate-300"
      <table class="w-full text-xs text-l
        <thead>
          <tr class="text-slate-500 b
            <th class="py-1.5 pr-2
            <th class="py-1.5 pr-2
            <th class="py-1.5 font-l
          </tr>
        </thead>
        <tbody class="text-slate-300">
          <tr class="border-b border-
имость)</td><td class="py-2 pr-2 font-mono">
ь ребро; r[i][i] = true</td></tr>
          <tr class="border-b border-
пути)</td><td class="py-2 pr-2 font-mono">d
= w(i,j); d[i][i] = 0</td></tr>
          <tr class="border-b border-
ой сети</td><td class="py-2 pr-2 font-mono">
и отсутствии положительных циклов</td></tr>
          <tr class="align-top"><td c
o">if d[i][k]+d[k][j] == d[i][j]: cnt[i][j]
а переполнением</td></tr>
        </tbody>
      </table>
      <p class="text-slate-400 text-xs mt
> при <span class="font-mono">r[i][k]</span>
V³)</span>.</p>
    </div>
  </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-amber-400

  <div class="bg-rose-950/30 p-5 rounded-lg b

```

```

title: "17. Графы. Поиск кратчайшего пути. Ускорени
type: "html",
description:
  "Джонсон (перевзвешивание потенциалов), встречный
category: "Графы. Пути",
content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-amber-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-amber-300 italic">
        <p class="text-xl font-mono text-white">
        е посещённых вершин (встречный поиск, ранний
        contraction hierarchies).</p>
      </div>

      <p class="text-slate-300 text-sm mb-4">Дейк
      в лоб. На экзамене спрашивают, <b>что сдела
      строки перевзвешивания» до промышленных у

      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Такси с навигатором
          </div>
          <p class="text-slate-300 text-sm mb-4">
            ы одинаково. А* — тот же таксист, но с нави
            /span> прибавляется оценка остатка пути по
            т туда, где сумма меньше. Навигатор обязан
            /span>): занижать можно, завышать нельзя —
            0">h = 0</span> это снова Дейкстра.</p>
          </div>

```

```

        лько 0 и 1</td></tr>
        <tr class="border-b border-slate-200">
<td class="py-2 pr-3">куча → массив корзинок<td class="py-2 pr-3">C небольшое</td></tr>
        <tr class="border-b border-slate-200">
<td class="py-2 pr-3">decrease-key за  $O(1)$  <td class="py-2 pr-3">теоретически; на практике двоичная/d-ар<td class="py-2 pr-3">
        <tr class="border-b border-slate-200">
<td class="py-2 pr-3">не релаксируем все рё<td class="py-2 pr-3">
, на практике быстро</td><td class="py-2 pr-3">о<td class="py-2 pr-3">
        <tr class="align-top"><td class="py-2 pr-3">3">переносит работу в предобработку</td><td class="py-2 pr-3">
миллионы запросов (карты)</td><td class="py-2 pr-3">
        </tbody>
    </table>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-6">
<div class="bg-slate-900 p-4 rounded-lg mb-6">
    <p class="text-xl font-mono text-white">
</div>
<div class="grid grid-cols-1 gap-6 mb-6">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-amber-500 shadow-md">
            ☞ Суть перевзвешивания
        </div>
        <p class="text-slate-300 text-sm mb-6">
            Дейкстра (билет 14) быстрая, но не оптимальный. Алгоритм Джонсона делает финт: пере-
            ь теми же.
        </p>
        <ol class="text-sm text-slate-300 space-y-2">
            <li>Добавляем <b>фиктивную вершину</b>
            <li>Запускаем от s <b>Форда-Беллмана</b>
            </ol>
            <p class="text-slate-300 text-sm">важный факт:</i> Джонсон НЕ «удаляет» отрицательные
            <p class="text-slate-300 text-sm">кой и прекращает работу, так как при отрицательных весах
            <li>Перевзвешиваем: <span class="text-amber-500 font-mono">

```

оторых расстояние только что уменьшилось

ческого прогона, но асимптотика та же, $O(V \cdot E)$.
пользуют осторожно, с эвристиками SLF/LLL).
) уменьшает высоту и на практике быстрее, к

Навигация по графу

в **предобработку**. Сначала – **сжатие** индексами $0 \dots N$, и уже по ним строят граф и сетевые потоки. **Contraction hierarchies** (так и называются) и запрос идёт двунаправленным поиском вверх и вниз. В вершине заранее приписывают «метки», и запрос

тро, но не кратчайший. Переоценка = неверный ответ.

именимой к отрицательным весам и даёт все возможные пути.

in_f + min_b ≥ μ. Остановка по встрече даёт кратчайший путь.

ок сломается. Там нужен Dial.

практике меньше бесполезных релаксаций.

Алгоритмы поиска кратчайшего пути

м числом раскрытий, 0-1 BFS $O(V + E)$, Дейкстра с кучей $O(E + V \cdot \log V)$, Dial $O(E + V \cdot C)$, где C – максимальный вес ребра.

```

        for to, w in g[v]:
            if d[s][to] > dist + w:
                d[s][to] = dist + w
                heapq.heappush(heap, (d[s][to], to))
    for v in range(n):
        if d[s][v] < INF:
            d[s][v] = d[s][v] - h[s] + h[v]    # откорректировать
    return d

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg border-slate-700/50">

<summary class="p-4 font-bold text-slate-300">

-b border-slate-700/50">

Код: встречный Дейкстра, A*, 0-1

</summary>

<div class="p-5 text-sm text-slate-300">

<pre class="bg-slate-950 p-4 rounded">

встречный Дейкстра: остановка по min_f + min_g

def bidirectional(g, rg, s, t): # rg – граф с перестройкой

INF = float("inf")

df = {s: 0}; db = {t: 0}

qf = [(0, s)]; qb = [(0, t)]

mu = INF

while qf and qb:

if qf[0][0] + qb[0][0] >= mu: # вот правый

return mu

dist, v = heapq.heappop(qf)

if dist > df.get(v, INF): continue

for to, w in g[v]:

nd = dist + w

if nd < df.get(to, INF):

df[to] = nd; heapq.heappush(qf, (nd, to))

if to in db:

mu = min(mu, nd + db[to]) # встреча:

dist, v = heapq.heappop(qb)

if dist > db.get(v, INF): continue

for to, w in rg[v]:

```

        (dq.appendleft(to) if w == 0 else dq.append(to))
    return d</pre>
</div>
</details>

<details class="bg-slate-900/40 rounded-lg border-1
  <summary class="p-4 font-bold text-slate-300
  -b border-slate-700/50">
    Вопросы, которые любят задавать (и не любят)
  </summary>
  <div class="p-5 text-sm text-slate-300">
    <p>• Почему после перевзвешивания весов не
    есть условие отсутствия улучшающей релаксации?
    <p>• Почему кратчайшие пути не меняются?
    иш) и одинаков для всех путей между этой парой вершин?
    <p>• Что лучше для всех пар: Флойд-Уоршелл или
    g V) выигрывает на разреженных.</p>
    <p>• Какая эвристика A* допустима на графах с
    ии скорости можно делить на максимальную скорость?
    <p>• Зачем нужен ранний выход в Дейкстре?
    смысла нет.</p>
    <p>• Почему 0-1 BFS работает без кучи?
    же отсортирована.</p>
  </div>
</details>
</div>
</div>
</section>`
  },
];

```

ФАЙЛ 28/111: src/data/tickets/ticket18_20_mst.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 231 | РАЗМЕР: 1.2 KB

```
import { Chapter } from '../../../types';
```

MST. Алгоритмы отличаются только тем, **к** какой букве принадлежит ребро. Прим – «выросшее дерево против остального графа».

```
<div class="bg-slate-800 p-5 rounded-lg border">
  <h4 class="text-amber-400 font-bold text-sm">Пример
  <p class="text-slate-200 text-sm font-sans">
    <p class="text-slate-300 text-sm mb-3">
      Если удалять буквы, остаётся остов.</p>
      <p class="text-slate-300 text-sm">Остов – это связный
      подграф: минимум костей, чтобы всё держалось.
    </p>
  </div>
```

Второй вариант – «поиск остовов». Он никогда не входит в MST. Именно его использовали в алгоритме Крускала, беря ребра по возрастанию веса.

```
<div class="overflow-x-auto my-6">
  <table class="w-full text-xs text-left">
    <thead>
      <tr class="text-slate-500 border-b">
        <th class="py-2 pr-3 font-bold">Веса
        <th class="py-2 pr-3 font-bold">Рёбра
        <th class="py-2 pr-3 font-bold">Минимум
        <th class="py-2 pr-3 font-bold">Максимум
        <th class="py-2 font-bold">Решение
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-700">
        <td class="py-2 pr-3 font-bold">1
        <td class="py-2 pr-3">Рёбра
        <td class="py-2 pr-3 font-medium">1, 2, 3, 4, 5, 6, 7, 8, 9, 10
        <td class="py-2 pr-3 font-medium">1, 2, 3, 4, 5, 6, 7, 8, 9, 10
        <td class="py-2">рёбра даны
      </tr>
      <tr class="border-b border-slate-700">
        <td class="py-2 pr-3 font-bold">2
        <td class="py-2 pr-3">Растёт
      </tr>
```



```
<div class="absolute -top-3 left-4 l
r border-rose-500 shadow-md">
```

Аналогия к билету 20 • Боровк

```
</div>
```

```
<p class="text-slate-300 text-sm">С
айшему чужому племени: к вечеру компонент в,
<b>log2 V</b> дней образуется единая федер
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
```

```
<p class="text-slate-300 text-sm mb-2">
```

```
<p class="text-slate-400 text-sm mb-1">
```

состояние от одной вершины до каждой; MST ми
куче другой: у Прима – вес ребра к дереву,
тся.</p>

```
<p class="text-slate-400 text-sm mb-1">
```

вных весах деревьев может быть несколько, н

```
<p class="text-slate-400 text-sm mb-1">
```

дом вместо почти-константного find/union.</p>

```
<p class="text-slate-400 text-sm mb-1">
```

и Краскал построил минимальный остовный <i>

```
<p class="text-slate-400 text-sm">• <b>P
```

ин раз, DSU спасает от дубля (find(u) == fi
</div>

```
<p class="text-slate-300 text-sm mt-4"><b>с
py-0.5 rounded text-xs font-mono text-emera
```

бесплатный); Прим с двоичной кучей <code>с
rder-slate-800">O(E log V)</code>, на плотн
text-xs font-mono text-emerald-300 border b
d text-xs font-mono text-emerald-300 border
ex <code>class="bg-slate-950 px-2 py-0.5 ro
</p>

```
<details class="bg-slate-900/40 rounded-lg l
```

```
<summary class="p-4 font-bold text-slate
```

```
-b border-slate-700/50">
```

```

key = [INF] * n          # минимальный вес ребра, ведущ
used = [False] * n
key[0] = 0
heap = [(0, 0)]          # (вес ребра, вершина) ← вот
total = 0

```

```

while heap:
    w, v = heapq.heappop(heap)
    if used[v]:
        continue        # устаревшая запись в куче
    used[v] = True
    total += w
    for to, weight in graph[v]:
        if not used[to] and weight < key[to]:
            key[to] = weight
            heapq.heappush(heap, (weight, to))
print(total)

```

(накапливаем путь от старта), здесь
 а – другая задача.

Код 20: Борувка (Скрыто)

Код 20: Борувка (Скрыто)

```

def find(v):
    while parent[v] != v:
        parent[v] = parent[parent[v]]
        v = parent[v]
    return v

comps = n

```

```
        <p>• Что если граф несвязный? — все  
    <p>  
        <p>• Как найти второе по величине о  
    ребро на образовавшемся цикле (LCA + максим  
    </div>  
    </details>  
  </div>  
</div>  
</section>`,  
  },  
];
```

ФАЙЛ 29/111: src/data/tickets/ticket21_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРӨК: 444 | РАЗМ

```
import { Chapter } from '../types';  
  
export const tickets21to24: Chapter[] = [  
  {  
    id: "string-kmp",  
    title: "21. Алгоритм Кнута-Морриса-Пратта (КМР)",  
    type: "html",  
    category: "Строки",  
    content: `  
<section id="string-kmp" class="mb-12 scroll-mt-10">  
  <div class="flex items-center mb-6 flex-wrap gap-3">  
    <span class="bg-indigo-600 text-white px-4 py-1">  
    <h2 class="text-2xl sm:text-3xl font-bold text-  
  </div>  
  <div class="space-y-8">  
    <div class="bg-slate-700/50 p-6 rounded-xl bord  
      <h3 class="text-xl font-bold text-blue-400 r  
      <div class="bg-slate-900 p-4 rounded-lg mb-  
        <p class="text-lg text-blue-300 italic r  
        <p class="text-xl font-mono text-white">
```

```

<p>Тот же пример: <b>abcabcd</b></p>
<ul class="list-disc pl-5 space-y-2">
  <li><b>a</b>: Префиксов нет. <code>
  <li><b>ab</b>: Из начала ("a")
>
  <li><b>abc</b>: Опять мимо. <code>
  <li><b>abca</b>: 0! Начало ("a"
</code></li>
  <li><b>abcab</b>: Начало ("ab")
/li>
  <li><b>abcabc</b>: Начало ("abc
e></li>
  <li><b>abcabcd</b>: «d» всё исп
последняя буква). Длина 3: <b>abc</b> проти
digo-300">π[6] = 0</code></li>
</ul>
</div>
</details>
<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
  Код C++ (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300
  <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
  int j = pi[i-1];
  while (j > 0 && s[i] != s[j]) j = pi[j-1];
  if (s[i] == s[j]) j++;
  pi[i] = j;
}</pre>
</div>
</details>

<h3 class="text-lg font-bold text-white mt-
<div class="bg-slate-900 p-4 rounded-lg mb-
  <p class="text-xl font-mono text-white">

```



```

        </p>
      </div>
    </div>

```

```

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
    -b border-slate-700/50">

```

Пример вычисления (Скрыто)

```

  </summary>
  <div class="p-5 text-sm text-slate-300
    <p>Тот же пример: <b>abcabcd</b></p>
    <ul class="list-disc pl-5 space-y-2
      <li><code>Z[0]</code> – не счита
      <li><code>Z[1]</code> (<b>bcabcd
[1] = 0</code></li>
      <li><code>Z[2]</code> (<b>cabcd
</li>
      <li><code>Z[3]</code> (<b>abcd<
    а дальше s[6]='d' против s[3]='a' – мимо.
      <li><code>Z[4], Z[5], Z[6]</code>
    </ul>

```

```

  </div>
</details>
<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
    -b border-slate-700/50">

```

Код C++ (Скрыто)

```

  </summary>
  <div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounded
int l = 0, r = 0;
for (int i = 1; i &lt; n; i++) {
  if (i &lt;= r) z[i] = min(r - i + 1, z[i - 1]);
  while (i + z[i] &lt; n && s[z[i]] == s[i + z[i]]) z
  if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
  </div>
</details>

```

```

    <div class="bg-slate-900 p-6 rounded-lg border-2"
      <div class="absolute -top-3 left-4 bg-rose-900
        -500 shadow-md">
        Аналогия 2: Матёшка (Терминальные)
      </div>
      <p class="text-slate-300 text-sm mb-4">Представ
        нашли и "he", потому что оно спрятано внутри
      </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg border b
      <summary class="p-4 font-bold text-slate-400 ou
        r-slate-700/50">
        Код (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300 space-y-
        <pre class="bg-slate-950 p-4 rounded overfl

int get_link(int v) {
  if (t[v].link == -1) {
    if (v == 0 || t[v].p == 0) t[v].link = 0;
    else t[v].link = go(get_link(t[v].p), t[v].pch)
  }
  return t[v].link;
}

int go(int v, char c) {
  if (t[v].go[c] == -1) {
    if (t[v].next[c] != -1) t[v].go[c] = t[v].next[
    else t[v].go[c] = v == 0 ? 0 : go(get_link(v),
  }
  return t[v].go[c];
}</pre>
    </div>
  </details>
</div>
</section>`
},

```

```

    </div>
    <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #4b4b9b; border-radius: 10px; box-shadow: 0 10px 30px #4b4b9b; position: relative; width: 100%;">
      <div class="absolute -top-3 left-4 right-4 bottom-4 border border-purple-500 shadow-md" style="border: 1px solid #4b4b9b; border-radius: 5px; padding: 5px;">
        Аналогия 2: Переводчик (сведение)
      </div>
      <p class="text-slate-300 text-sm">Сведение к полиномиальному времени (сведение за полином) и друг-англичанин. Отсюда главный инструмент: если к В сводится А, то А не стоит.</p>
    </div>
  </div>

```

```

<h3 class="text-lg font-bold text-white mt-3">Иерархия сложности
<p class="text-slate-300 text-sm mb-3">Известно, что сложность в цепочке доказывається теоремой об иерархии сложности.
<div class="bg-slate-950 border border-slate-800 p-6 rounded-lg" style="border: 1px solid #4b4b9b; border-radius: 10px; box-shadow: 0 10px 30px #4b4b9b; position: relative; width: 100%;">
  <p class="font-mono text-sm sm:text-base">
     $P \subseteq NP \subseteq PSPACE \subseteq EXP \subseteq NEXP$   

    <span class="text-fuchsia-300">NP-полнота (не доказано)</span>
  </p>
</div>

```

```

<div class="overflow-x-auto">
  <table class="w-full text-xs text-left" style="width: 100%; border-collapse: collapse;">
    <thead>
      <tr class="text-slate-500 border-bottom" style="border-bottom: 1px solid #4b4b9b;">
        <th class="py-2 pr-3 font-bold">Время
        <th class="py-2 pr-3 font-bold">Пространство
        <th class="py-2 font-bold">Сложность
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-800" style="border-bottom: 1px solid #4b4b9b;">
        <td class="py-2 pr-3">детерминированно за  $O(n^k)$ 
        <td class="py-2 pr-3">удольности, максимальный поток
      </tr>
      <tr class="border-b border-slate-800" style="border-bottom: 1px solid #4b4b9b;">
        <td class="py-2 pr-3">сертификат проверяется за полиномиальное время
      </tr>
    </tbody>
  </table>

```



```

        ke.</li>
        <li><b>Параметризованная сложность
n).</li>
        <li><b>Специальные классы входов</b>
полиномиальными (независимое множество на д
        <li><b>Готовые решатели</b>: SAT/ILP
всего алгоритма.</li>
    </ul>
</div>

```

```

<div class="bg-slate-900/40 rounded-lg border
    <p class="text-slate-300 text-sm mb-2">
    <p class="text-slate-400 text-sm mb-1">
NP: любая задача из P автоматически в NP.
    <p class="text-slate-400 text-sm mb-1">
становки – неразрешима, зато NP-трудна). NP
    <p class="text-slate-400 text-sm mb-1">
стной трудной A <b>к</b> B ( $A \leq_p B$ ), а не н
    <p class="text-slate-400 text-sm mb-1">
    <p class="text-slate-400 text-sm mb-1">
дит по времени на  $n = 40$ . NP-полнота запрещ
    <p class="text-slate-400 text-sm">• <b>
ит RSA. «Раз она в NP, значит NP-полная» –
</div>

```

```

<p class="text-slate-300 text-sm mt-4"><b>с
ass="bg-slate-950 px-2 py-0.5 rounded text-
еребор сертификатов <code class="bg-slate-9
^poly(n))</code>; сведение обязано работать
0 border border-slate-800">0(poly(n))</code>

```

```

<details class="bg-slate-900/40 rounded-lg
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

Код: верификатор сертификата и пр

```

</summary>
<div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounded

```

```

        if col[to] == -1:
            col[to] = col[v] ^ 1
            dq.append(to)
        elif col[to] == col[v]:
            return False
    return True</pre>

```

```

        <p class="text-slate-400 text-xs">0
товый ответ, но не находить его; для задачи
        </div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

```

        Вопросы, которые любят задавать (
    </summary>

```

```

    <div class="p-5 text-sm text-slate-300
        <p>• Дайте определение NP через вер
нтны.</p>
        <p>• Что такое NP-полная задача и к
    </p>

```

```

        <p>• Почему первая NP-полная задача
        <p>• Если  $P = NP$ , что станет с крип
их задач исчезнет граница approximability)<
        <p>• Пример задачи в NP и co-NP, пр
– затем AKS доказал  $P$ ></p>

```

```

        <p>• Чем NP-трудная задача отличаетс
облема остановки)</p>
    </div>
</details>

```

```

    </div>
</div>
</section>`
    }
};

```

```

        <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Аналогия: Списки = Контакты
        </div>
        <p class="text-slate-300 text-sm mb-1">
            т. Оптимально для почти всех задач.</p>
        </div>
    </div>

    { /* DFS vs BFS */ }
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
        <h3 class="text-xl font-bold text-indigo-400">DFS vs BFS</h3>

        <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
            <!-- Аналогия DFS -->
            <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-indigo-500 shadow-md">
                    ♂ DFS (Поиск в глубину) = 0
                </div>
                <p class="text-slate-300 text-sm mb-1">
                    <b>Что делает:</b> Поиск в глубину — это алгоритм, который
                    да попадает в тупик — делает шаг назад по следу.
                </p>
                <ul class="text-xs text-indigo-300">
                    <li><b>Где используется:</b></li>
                    <li>Топологическая сортировка (линейное время)</li>
                    <li>Поиск мостов и точек сочленения</li>
                    <li>Компоненты сильной связности</li>
                    <li>Детектирование циклов (через DFS)</li>
                </ul>
            </div>

            <!-- Аналогия BFS -->
            <div class="bg-slate-900 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">

```

```
vis[start_node] = true;
```

```
while (!q.empty()) {
    int v = q.front(); q.pop();
    for (int u : adj[v]) {
        if (!vis[u]) {
            vis[u] = true;
            q.push(u);
        }
    }
}
```

```
</pre>
</div>
</details>
</div>
</div>
```

```
</section>`,
```

```
{,
```

```
{
    id: "graph-planar-colors",
    title: "7. Графы. Планарные. Покраска",
    type: "html",
    description: "Планарные графы: формула Эйлера, K5 и",
    category: "Графы. Теория",
    content: `
```

```
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">7</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Графы. Планарные. Покраска</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">Планарные графы</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Планарный граф — граф, который можно нарисовать в плоскости так, чтобы никакие две его ребра не пересекались.</p>
        <p class="text-2xl font-mono text-white">Графы, которые можно нарисовать в плоскости так, чтобы никакие две его ребра не пересекались, называются планарными.</p>
        <p class="text-xs text-slate-400 mt-1">Планарные графы имеют много интересных свойств. Например, для любого планарного графа справедлива формула Эйлера:</p>
      </div>
    </div>
  </div>
</div>
```

```

<div class="space-y-3 text-slate-300 text-s
  <p><b>Сложность задачи раскраски:</b></p>
  <ul class="list-disc list-inside space-
    <li><b>k = 2 (2-раскрашиваемость):</b>
  )</b> обычным BFS/DFS обходом.</li>
    <li><b>k ≥ 3 (k-colorability):</b>
  </li>
  </ul>

  <p class="mt-3"><b>Теоремы о раскраске:</b>
  <ul class="list-disc list-inside space-
    <li><b>Жадная раскраска:</b> дает в
  максимальная степень вершины.</li>
    <li><b>Теорема Брукса:</b> <span cl
  ных графов  $K_n$  и нечётных циклов  $C_{2k+1}$ .</li>
    <li><b>Теорема о 4 красках:</b> Люб
  no text-emerald-300"> $\chi(G) \leq 4$ </span>.</li>
  </ul>
</div>
</div>
</div>
</section>`,
  },
  {
    id: "graph-components",
    title: "8. Графы. Компоненты связности",
    type: "html",
    description:
      "Компоненты связности: обходами DFS/BFS за  $O(V + E)$ ",
    category: "Графы. База",
    content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">8
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">8. Графы. Компоненты связности
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">

```

```

<div class="overflow-x-auto">
  <table class="w-full text-xs text-left">
    <thead>
      <tr class="text-slate-500 border-b">
        <th class="py-2 pr-3 font-b">
        <th class="py-2 pr-3 font-b">
        <th class="py-2 pr-3 font-b">
        <th class="py-2 font-bold">
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-200">
        <td class="py-2 pr-3">забыли про стрелку
        <td class="py-2 font-mono">0(V + E)
      </tr>
      <tr class="border-b border-slate-200">
        <td class="py-2 pr-3">из любой вершины
        <td class="py-2 font-mono">0(V + E)
      </tr>
      <tr class="align-top">
        <td class="py-2 pr-3">до кого дойдёт старт
        <td class="py-2 font-mono">0(V + E)
      </tr>
    </tbody>
  </table>
</div>
<p class="text-slate-400 text-xs mt-3">Классический алгоритм SCC. Обход из s даёт только вершины,
</p>
</div>
</div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Топологическая сортировка ДАГ и детектирование циклов",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

        <br>* <span class="font-mono text-amber-400">
        <br>* <span class="font-mono text-emerald-400">
        <br>Если во время обхода из серы вершины
и <b>гарантированный цикл</b>!
    </p>
</div>

<details class="bg-slate-900/40 rounded-lg border-1
    <summary class="p-4 font-bold text-slate-300
    -b border-slate-700/50">
    Корректный код C++ с детекцией цикла
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounded">
vector<int> adj[MAXN];
int color[MAXN]; // 0=белый, 1=серый (в стеке), 2=чёрный
vector<int> ans;
bool has_cycle = false;

void dfs(int v) {
    color[v] = 1; // Зашли – вершина серая
    for (int to : adj[v]) {
        if (color[to] == 0) {
            dfs(to);
            if (has_cycle) return;
        } else if (color[to] == 1) {
            // Переход в серую вершину = НАЙДЕН ЦИКЛ!
            has_cycle = true;
            return;
        }
    }
    color[v] = 2; // Вышли – вершина чёрная
    ans.push_back(v); // Сохраняем в момент ВЫХОДА (post-order)
}

bool topological_sort(int n) {
    fill(color, color + n + 1, 0);
    has_cycle = false;

```

```

<p class="text-slate-300 text-sm mb-4">
  <strong>«Это просто цикл?» – Нет!</strong>
</p>

<details class="bg-slate-900/40 rounded-lg mb-4" >
  <summary class="p-4 font-bold text-slate-300 border-slate-700/50">
    Код: Алгоритм Косарайю (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300">
    <p class="text-emerald-400 font-bold">
      <ol class="list-decimal list-inside">
        <li>Запускаем DFS на исходном графе
        </li>
        <li>Разворачиваем этот список (и делаем топологическую сортировку самого графа)
        </li>
        <li>Переворачиваем все ребра в графе
        </li>
        <li>Идем по <code>order</code>: делаем DFS по каждому узлу. Все циклы, которые
        </li>
        </ol>
      </div>
    </div>
  </details>
</div>
</div>
</section>`,
  },
  {
    id: "graph-bridges",
    title: "11. Графы. Мосты",
    type: "html",
    description: "Рёбра, удаление которых увеличивает количество компонент связности графа",
    category: "Графы. Продвинутое",
    content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">

```



```

    }
  }</pre>
</div>
</details>
</div>
</div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров путь и цикл",
    type: "html",
    description: "Условия существования и построение Эйлерова",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Графы
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
      <h3 class="text-xl font-bold text-indigo-400">Графы
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Граф — это совокупность
        <p class="text-base sm:text-lg font-monospace">
          ности!</p>
      </div>

      <div class="grid grid-cols-1 md:grid-cols-2 gap-4">
        <!-- Неориентированные графы -->
        <div class="bg-slate-800 p-5 rounded-lg">
          <p class="text-indigo-300 font-bold">Графы
          <ul class="list-disc list-inside space-y-2">
            <li><b>Эйлеров цикл:</b> связан
              <span class="text-emerald-300">deg[v] % 2 == 0</span>.</li>
            <li><b>Эйлеров путь:</b> связан
              <span class="text-emerald-300">о 2 — путь начинается в одной нечётной и за

```

```
    path.push_back(v); // Добавляем в путь на выходе!
  }</pre>
    </div>
  </details>
</div>
</div>
</section>`,
  },
];
```

РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

ФАЙЛ 31/111: src/components/ArticulationPointsViz.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizNumber } from '../data/vizS
import { Play, Pause, RotateCcw, Info } from "lucide-re
```

```
interface Node {
  id: number;
  x: number;
  y: number;
  label: string;
}
```

```
interface Edge {
  u: number;
  v: number;
}
```

```
const nodes: Node[] = [
  { id: 0, x: 250, y: 50, label: "0" },
```

```

-----
export const ArticulationPointsViz: React.FC = () => {
  const [steps, setSteps] = useState<StepInfo[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  useVizStepSync(currentStepIndex, setCurrentStepIndex, () => {
    const v = vizNumber(vars.v);
    if (v === null) return null;
    return steps.map((step, index) => ({ step, index }));
  });
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const newSteps: StepInfo[] = [];
    let timer = 0;
    const visited = new Set<number>();
    const ap = new Set<number>();
    const currentTin = Array(nodes.length).fill(-1);
    const currentLow = Array(nodes.length).fill(-1);

    const recordStep = (desc: string, u: number | null) => {
      newSteps.push({
        desc,
        visited: new Set(visited),
        ap: new Set(ap),
        currentNode: u,
        tin: [...currentTin],
        low: [...currentLow],
      });
    };

    recordStep("Начало DFS (Тарьян) для поиска точек соarticulation", null);

    const dfs = (v: number, p: number = -1) => {
      visited.add(v);
      currentTin[v] = currentLow[v] = timer++;
      let children = 0;

      recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}`, v);
    };
  }, []);
};

```

```

        v,
      });
    }
  };

  for (let i = 0; i < nodes.length; i++) {
    if (!visited.has(i)) {
      dfs(i);
    }
  }

  recordStep("Алгоритм завершен. Найдены все точки со
  setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
  desc: "",
  visited: new Set(),
  ap: new Set(),
  currentNode: null,
  tin: [],
  low: [],
};

useEffect(() => {
  let interval: NodeJS.Timeout;
  if (isPlaying && currentStepIndex < steps.length -
    interval = setInterval(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {

```

```

<div className="grid grid-cols-1 lg:grid-cols-3 g
  <div className="lg:col-span-2 relative h-96 lg:l
    <svg className="w-full h-full" viewBox="0 0 8
      {/* Edges */}
      {edges.map((edge, i) => {
        const u = nodes.find((n) => n.id === edge
        const v = nodes.find((n) => n.id === edge

        const isBridge =
          (edge.u === 3 && edge.v === 4) ||
          (edge.u === 4 && edge.v === 3);

        return (
          <line
            key={"edge-" + i}
            x1={u.x}
            y1={u.y}
            x2={v.x}
            y2={v.y}
            stroke={isBridge ? "#f43f5e" : "#475568"}
            strokeWidth={isBridge ? "4" : "2"}
            strokeDasharray={isBridge ? "8,4" : "1"}
          />
        );
      })}

      {/* Nodes */}
      {nodes.map((node) => {
        const isCurrent = currentStep.currentNode
        const isVisited = currentStep.visited.has
        const isAp = currentStep.ap.has(node.id);

        const fill = isCurrent
          ? "#3b82f6"
          : isAp
            ? "#e11d48"
            : isVisited
              ? "#10b981"

```

```

        y={node.y - r - 15}
        textAnchor="middle"
        fill="#94a3b8"
        fontSize="10px"
      >
        tin:{" "}
        {currentStep.tin[node.id] >= 0
          ? currentStep.tin[node.id]
          : "-"}
      </text>
      <text
        x={node.x}
        y={node.y + r + 20}
        textAnchor="middle"
        fill="#94a3b8"
        fontSize="10px"
      >
        low:{" "}
        {currentStep.low[node.id] >= 0
          ? currentStep.low[node.id]
          : "-"}
      </text>
    </>
  </g>
</>
</div>

```

```

<div className="bg-slate-950 p-6 border-t lg:bo
  <h4 className="text-sm font-bold text-slate-4
    Статус алгоритма
  </h4>

```

```

<div className="bg-slate-900 border border-sl
  <p className="text-white font-medium min-h-
    {currentStep.desc}

```

```

                {step.desc}
            </div>
        )}}
    </div>
</div>
<div className="mt-4 shrink-0">
  <div className="w-full bg-slate-800 h-2 rounded">
    <div
      className="bg-rose-500 h-full transition"
      style={{
        width: `${(currentStepIndex / Math.max(
          />
        </div>
      </div>
    </div>
  </div>
</div>
);
};

```

ФАЙЛ 32/111: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord, vizString } from 'viz';
import { LiveDistTable } from './LiveDistTable';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  iteration: number;
  checkingEdge: { u: string; v: string; w: number } | null;
  distances: { [key: string]: number };
}

```



```

    if (cycle) {
      return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
    } else {
      return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
    }
  };

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
  const list: { [key: string]: { v: string; w: number } } = {};
  nodes.forEach(n => (list[n.id] = []));
  ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
  return list;
};

const adjList = getAdjList(edges);

const codeLines = [
  { line: 1, text: 'function bellman_ford(graph, start)' },
  { line: 2, text: '  dist = {v: ∞}; dist[start] = 0;' },
  { line: 3, text: '  for i from 1 to V - 1: // |V| - 1 iterations' },
  { line: 4, text: '    for (u, v, weight) in E:' },
  { line: 5, text: '      if dist[u] + weight < dist[v]' },
  { line: 6, text: '        dist[v] = dist[u] + weight;' },
  { line: 7, text: '  for (u, v, weight) in E: // Проверка на отрицательный цикл' },
  { line: 8, text: '    if dist[u] + weight < dist[v]' },
  { line: 9, text: '      return "Обнаружен отрицательный цикл"' },
  { line: 10, text: '  return dist' },
];

useEffect(() => {
  const simulationSteps: Step[] = [];
  let dist: { [key: string]: number } = { S: 0, A: 99 };
  let prev: { [key: string]: string | null } = { S: null };

  simulationSteps.push({
    iteration: 0, checkingEdge: null,
    distances: { ...dist }, previous: { ...prev },
  });

```

```

    if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
      let cycleFound = false;
      for (const edge of edges) {
        if (dist[edge.u] !== 999 && dist[edge.u] + edge.weight < dist[edge.v]) {
          cycleFound = true;
          simulationSteps.push({
            iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
            distances: { ...dist }, previous: { ...prevDist },
            log: `    ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v}`,
            activeCodeLine: 9,
          });
          break;
        }
      }
    }

    if (!cycleFound) {
      simulationSteps.push({
        iteration: vCount, checkingEdge: null,
        distances: { ...dist }, previous: { ...prevDist },
        log: `    Проверка завершена. Ни одно ребро не попало в цикл`,
        activeCodeLine: 10,
      });
    }
  }

  setSteps(simulationSteps);
  setCurrentStepIndex(0);
  setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
}, [isPlaying, currentStepIndex, steps.length]);

```



```

    {nodes.map((node) => {
      const dist = currentStep.distances[node.id]
      const isCheckingNode = currentStep.checkingNode ===
        node.id);

      return (
        <g key={node.id} className="transition-
          <circle cx={node.x} cy={node.y} r={is
            ckingNode ? '#7dd3fc' : '#64748b'} strokeWid
          <text x={node.x} y={node.y + 5} fill=
          <text x={node.x} y={node.y - 28} fill=
            ight="bold" textAnchor="middle" className="
          <text x={node.x} y={node.y + 35} fill=
            split(' ')[0]}</text>
          </g>
        );
      })}
    </svg>

    <div className="w-full bg-slate-950/80 p-4 ro
      <p className="font-semibold text-amber-400
      <p className="min-h-[40px] flex items-cente
    </div>
  </div>

  {/* Список смежности */}
  <div className="w-full lg:w-1/3 bg-emerald-950/
    <h4 className="text-lg font-bold text-emerald
    <div className="space-y-2 text-sm font-mono t
      {Object.keys(adjList).map(u => {
        <div key={u} className="flex flex-wrap g
        -colors">
          <span className="font-bold text-white
            {adjList[u].map((edge, idx) => {
              const isChk = currentStep.checkingE
              return (
                <span key={idx} className={`px-2

```

```
      {codeLines.map(item => (  
        <div key={item.line} className={`py-1.5  
e === item.line ? 'bg-blue-900/80 text-amber-500' : 'text-slate-600 w-5 s  
        <span className="text-slate-600 w-5 s  
        <span className="whitespace-pre flex-  
      </div>  
    ))}  
  </div>  
</div>  
</div>  
</div>  
</div>  
  );  
}
```

ФАЙЛ 33/111: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect, useCallback } from 'react'  
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react'
```

```
const NODES = [  
  { id: 0, label: '0', x: 300, y: 50 },  
  { id: 1, label: '1', x: 150, y: 150 },  
  { id: 2, label: '2', x: 450, y: 150 },  
  { id: 3, label: '3', x: 75, y: 250 },  
  { id: 4, label: '4', x: 225, y: 250 },  
  { id: 5, label: '5', x: 375, y: 250 },  
  { id: 6, label: '6', x: 525, y: 250 },  
];
```

```
const EDGES = [  
  { from: 0, to: 1 }, { from: 0, to: 2 },  
  { from: 1, to: 3 }, { from: 1, to: 4 },  
  { from: 2, to: 5 }, { from: 2, to: 6 }  
];
```

```
while (queue.length > 0) {
  steps.push({
    activeLine: 4,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: null,
    logs: `Очередь не пуста, элементов: ${queue.length}
  });
```

```
const curr = queue.shift()!;
steps.push({
  activeLine: 5,
  queue: [...queue],
  visited: Array.from(visited),
  currentNode: curr,
  logs: `Достаем из очереди (${curr}).`
});
```

```
const neighbors = ADJ[curr];
if (neighbors.length > 0) {
  for (const n of neighbors) {
    if (!visited.has(n)) {
      steps.push({
        activeLine: 7,
        queue: [...queue],
        visited: Array.from(visited),
        currentNode: curr,
        logs: `Рассматриваем соседа: ${n}. Он еще не
      });
```

```
visited.add(n);
queue.push(n);
```

```
steps.push({
  activeLine: 9,
  queue: [...queue],
  visited: Array.from(visited),
  currentNode: curr,
```

```

// @ts-expect-error зарезервировано под кнопку «назад»
const handlePrev = useCallback(() => {
  setStepIdx(prev => Math.max(prev - 1, 0));
}, []);

useEffect(() => {
  if (!isPlaying) return;
  if (stepIdx >= STEPS.length - 1) {
    setIsPlaying(false);
    return;
  }
  const timer = setTimeout(handleNext, 1200);
  return () => clearTimeout(timer);
}, [isPlaying, stepIdx, handleNext]);

return (
  <div className="bg-slate-900 border-2 border-slate-100" >
    <div className="flex flex-col md:flex-row items-center justify-between" >
      <div>
        <h3 className="text-2xl font-bold text-white" >
          <span className="text-blue-400" > </span> Алгоритм
        </h3>
        <p className="text-slate-400 text-sm mt-1">Визуализация
        </p>
      </div>
      <div className="flex bg-slate-800 p-1.5 rounded" >
        <button
          onClick={() => { setIsPlaying(false); setStepIdx(0); }}
          className="p-2 text-slate-400 hover:text-white"
          title="Сброс"
        >
          <RotateCcw className="w-5 h-5" />
        </button>
        <button
          onClick={() => setIsPlaying(p => !p)}
          className="p-2 text-blue-400 hover:text-blue-300"
          title={isPlaying ? 'Пауза' : 'Воспроизведение'}
        >
          {isPlaying ? <Pause className="w-5 h-5" />

```

```

let fill = '#1e293b'; // slate-800
let stroke = '#475569'; // slate-600
let scale = 1;
void scale;

if (isCurrent) {
  fill = '#3b82f6'; // blue-500
  stroke = '#60a5fa'; // blue-400
  scale = 1.3;
} else if (isInQueue) {
  fill = '#059669'; // emerald-600
  stroke = '#34d399'; // emerald-400
  scale = 1.1;
} else if (isVisited) {
  fill = '#0f172a'; // slate-900
  stroke = '#3b82f6'; // blue-500
}

return (
  <g key={n.id} className="transition-tran
    {isCurrent && {
      <circle cx={n.x} cy={n.y} r="30" fi
    }}
    <circle
      cx={n.x} cy={n.y}
      r="20"
      fill={fill}
      stroke={stroke}
      strokeWidth="3"
      className="transition-colors duration
  />
  <text
    x={n.x} y={n.y}
    textAnchor="middle"
    dy=".3em"
    className="fill-white font-bold tex
  >

```



```

        </div>
      </div>

      <div className="bg-slate-800/80 border border
        <div className="text-sm font-bold text-eme
          <span>Состояние Очереди (Queue):</span>
          <span className="text-xs text-slate-500
        </div>
        <div className="flex items-center gap-2 mi
          {step.queue.length === 0 ? (
            <span className="text-slate-500 text-x
          ) : (
            step.queue.map(({qId, i}) => {
              <span key={`${qId}-${i}`} className=
            enter justify-center rounded-md shadow-lg s
              {qId}
            </span>
          ))
        )}
      </div>
      <div className="mt-2 text-sm text-amber-300
        ms-center justify-center">
        "{step.logs}"
      </div>
    </div>
  </div>
</div>
</div>
);
}

```

ФАЙЛ 34/111: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import dijkstraImg from '../assets/dijkstra-freakazoid.
```

```
    });  
  }  
}
```

ФАЙЛ 35/111: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";  
import { ChevronLeft, ChevronRight } from "lucide-react";  
import type { Chapter } from "../types";
```

```
interface Props {  
  prev?: Chapter;  
  next?: Chapter;  
  index: number;  
  total: number;  
  onGo: (id: string) => void;  
  compact?: boolean;  
}
```

```
/**
```

```
 * Переходы «предыдущая / следующая тема» – как на CodeSandbox
```

```
 * Компактный вариант живёт в шапке главы, крупный – по центру
```

```
 */
```

```
export const ChapterNav: React.FC<Props> = ({ prev, next, onGo, compact }) => {  
  if (compact) {  
    return (  
      <div className="flex items-center gap-2 text-xs">  
        <button  
          disabled={!prev}  
          onClick={() => prev && onGo(prev.id)}  
          title={prev ? prev.title : "Это первая тема"}  
          className="flex items-center gap-1 px-2.5 py-1  
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50">  
          <ChevronLeft className="w-4 h-4" />  
          {prev ? prev.title : "Это первая тема"}  
        </button>  
        <button  
          disabled={!next}  
          onClick={() => next && onGo(next.id)}  
          title={next ? next.title : "Это последняя тема"}  
          className="flex items-center gap-1 px-2.5 py-1  
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50">  
          {next ? next.title : "Это последняя тема"}  
          <ChevronRight className="w-4 h-4" />  
        </button>  
      </div>  
    );  
  }  
  return (  
    <div className="flex items-center gap-2 text-sm">  
      <button  
        disabled={!prev}  
        onClick={() => prev && onGo(prev.id)}  
        title={prev ? prev.title : "Это первая тема"}  
        className="flex items-center gap-1 px-3 py-1.5  
          :bg-slate-700 enabled:hover:text-white disabled:opacity-50">  
        <ChevronLeft className="w-5 h-5" />  
        {prev ? prev.title : "Это первая тема"}  
      </button>  
      <button  
        disabled={!next}  
        onClick={() => next && onGo(next.id)}  
        title={next ? next.title : "Это последняя тема"}  
        className="flex items-center gap-1 px-3 py-1.5  
          :bg-slate-700 enabled:hover:text-white disabled:opacity-50">  
        {next ? next.title : "Это последняя тема"}  
        <ChevronRight className="w-5 h-5" />  
      </button>  
    </div>  
  );  
}
```

```
      <button
        onClick={() => onGo(next.id)}
        className="group text-right p-4 rounded-xl bg-
          transition-all sm:col-start-2"
      >
        <div className="flex items-center justify-end
          text-indigo-400 mb-1">
          Следующая тема <ChevronRight className="w-3
        </div>
        <div className="text-sm font-bold text-slate-1
      </button>
    })
  </nav>
);
};
```

ФАЙЛ 36/111: src/components/ChapterQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИ

```
import React, { useEffect, useState } from "react";
import { ArrowRight, CheckCircle2, RotateCcw, XCircle,
import { quizzes } from "../data/quizzes";
```

```
interface Props {
  chapterId: string;
}
```

```
/**
```

```
 * Блиц по прочитанной теме: 3–5 коротких вопросов с не
 *
```

```
 * Вопросы живут в `src/data/quizzes.ts` (по id главы).
```

```
 * компонент-викторина существовали, но не были подклю
```

```
 * контент был, а добраться до него было нельзя. Теперь
```

```
 * демонстрации: прочитал → посмотрел, как работает → пр
```

```
 *
```

```

const restart = () => {
  setIdx(0);
  setPicked(null);
  setScore(0);
  setFinished(false);
};

return (
  <section aria-label="Блиц по теме" className="mt-8">
    <div className="bg-slate-700/50 p-5 rounded-xl border">
      <div className="flex items-center justify-between">
        <h3 className="flex items-center gap-2 text-white">
          <Zap className="w-4 h-4 text-indigo-400" />
          Блиц: {list.length} вопроса по этой теме
        </h3>
        <span className="text-[11px] font-mono text-white">
          {finished ? `Итог ${score}/${list.length}` : ''}
        </span>
      </div>

      {finished ? (
        <div className="text-center py-3">
          <p className="text-sm text-slate-300 mb-1">
            {score === list.length
              ? "Все верно: тему можно закрывать."
              : score * 2 >= list.length
                ? "Половина и больше верно – стоит пересмотреть."
                : "Стоит вернуться к тексту страницы:"
            }
          </p>
          <p className="text-2xl font-bold text-white">
            {score} / {list.length}
          </p>
          <button
            type="button"
            onClick={restart}
            className="inline-flex items-center gap-1 font-medium text-white font-bold text-indigo-200 hover:bg-indigo-500/20"
          >
            Пересмотреть
          </button>
        </div>
      ) : (
        <div className="text-center py-3">
          <p className="text-sm text-slate-300 mb-1">
            {score === 0
              ? "Начните с первого вопроса."
              : "Вы ответили правильно на {score} из {list.length} вопросов."
            }
          </p>
          <p className="text-2xl font-bold text-white">
            {score} / {list.length}
          </p>
          <button
            type="button"
            onClick={restart}
            className="inline-flex items-center gap-1 font-medium text-white font-bold text-indigo-200 hover:bg-indigo-500/20"
          >
            Пересмотреть
          </button>
        </div>
      )}
    </div>
  </section>
);

```

```
        {correct ? "Верно" : "Неверно"}
      </p>
      <p className="text-[13px] leading-relaxed">
    </div>
  )}

  <div className="mt-3 flex justify-end">
    <button
      type="button"
      onClick={next}
      disabled={!answered}
      className="inline-flex items-center gap-2 font-medium text-indigo-200 transition-colors hover:text-indigo-300"
    >
      {idx + 1 < list.length ? "Следующий вопрос" : "Завершить"}
      <ArrowRight className="w-3.5 h-3.5" />
    </button>
  </div>
</>
  )}
</div>
</section>
);
};

export default ChapterQuiz;
```

ФАЙЛ 37/111: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, FileText, Loader2, Maximize2, Minimize2 } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
```

```

        window.clearTimeout(hideTimer.current);
        hideTimer.current = null;
    }
}, []));

const scheduleHide = useCallback(() => {
    cancelHide();
    hideTimer.current = window.setTimeout(() => setAnchor(
}, [cancelHide]);

useEffect(() => {
    setAnchor(null);
    setSaving("idle");
}, [chapter.id]);

/**
 * Печать/«сохранить в PDF» этой темы: раскрываем все
 * чтобы на бумагу попал весь текст (доказательства,
 * а после печати возвращаем их в исходное состояние.
 */
useEffect(() => {
    const each = () => Array.from(contentRef.current?.q
    const openAll = () => {
        each().forEach((d) => {
            d.dataset.wasOpen = d.open ? "1" : "0";
            d.open = true;
        });
    };
    const restore = () => {
        each().forEach((d) => {
            if (d.dataset.wasOpen === "0") d.open = false;
            delete d.dataset.wasOpen;
        });
    };
    window.addEventListener("beforeprint", openAll);
    window.addEventListener("afterprint", restore);
    return () => {
        window.removeEventListener("beforeprint", openAll

```

```

// ссылка на другой билет: переходим внутри приложения
const link = (e.target as HTMLElement).closest<HTMLAnchorElement>('a');
if (link) {
  e.preventDefault();
  onGo(link.dataset.goto ?? "");
  return;
}
const el = (e.target as HTMLElement).closest<HTMLAnchorElement>('a');
if (!el) return;
e.preventDefault();
if (anchor && anchor.termId === el.dataset.termId)
  else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLAnchorElement>('a');
  if (el) open(el);
};

const viz = getViz(chapter.id);

return (
  <>
    <article className="chapter-body">
      {/* панель загрузки: сохранить именно эту страницу */}
      <div className="flex flex-wrap items-center gap-2">
        <button
          onClick={saveHtml}
          disabled={saving === "work"}
          title="Сохранить эту тему одним автономным файлом"
          className="flex items-center gap-1.5 text-x-small font-medium border-1px solid black padding-2px 10px"
          300 hover:text-white hover:border-emerald-500
        >
          {saving === "work" ? (
            <Loader2 className="w-3.5 h-3.5 animate-spin" />
          ) : saving === "done" ? (
            <Check className="w-3.5 h-3.5 text-emerald-500" />
          ) : (

```

```

ed-lg px-3 py-2 print:hidden">
<MousePointerClick className="w-4 h-4 shrink-0" />
<span>
  Подчёркнутые термины – интерактивные: наведи
  объяснение на пальцах, мини-анимацию и кноп
</span>
</p>

{viz && {
  <section id="chapter-viz" className="mt-8 scroll-mt-8" />
  <div className="flex items-center justify-between" />
    <h3 className="flex items-center gap-2 text-slate-600" />
      <Sparkles className="w-4 h-4 text-indigo-500" />
      Демонстрация: {viz.title}
    </h3>
    <div className="flex items-center gap-2">
      <button
        onClick={() => onOpenSimulator(chapter) }
        className="flex items-center gap-1.5 text-slate-300
        hover:text-white hover:border-indigo-500" />
      <Maximize2 className="w-3.5 h-3.5" />
    </button>
    {onOpenCompiler && {
      <button
        onClick={onOpenCompiler}
        title="Открыть Python: код выполняе
рации"
        className="flex items-center gap-1.5 text-emerald-400
        hover:text-white hover:border-emerald-500" />
      <Terminal className="w-3.5 h-3.5" />
    </button>
    }}
  </div>
</div>
{viz.hint && <p className="text-xs text-slate-500" />
<VizChapterContext.Provider value={chapter} />

```



```

*
* Четыре вкладки:
* 1. «Иерархия» —  $P \subseteq NP \subseteq PSPACE \subseteq EXP$  и NP-полные
* 2. «Сведение» —  $A \leq_p B$  по шагам: преобразовал вход
* 3. «Верификатор» — проверка сертификата для 3-SAT по
* 4. «Рост» — живой график по переменным  $n$  и  $ops$ 
* сколько операций насчитал твой алгоритм
* похоже (полином или экспонента).
*
* Синхронизация с компилятором (PAGE_SYNC["complexity-
* cls — имя класса (подсветит нужный овал), step/i —
* k — номер проверяемого дизъюнкта, cert/assign — сер
* n и ops — точки на графике роста, ok/ans — вердикт
*/

```

```

type Tab = "hierarchy" | "reduce" | "verify" | "growth"

```

```

const TABS: { id: Tab; label: string; icon: React.ReactElement } = [
  { id: "hierarchy", label: "1 · Иерархия классов", icon: <img alt="Hierarchy icon" data-bbox="100 525 150 550"/> },
  { id: "reduce", label: "2 · Сведение  $A \leq_p B$ ", icon: <img alt="Reduce icon" data-bbox="100 555 150 580"/> },
  { id: "verify", label: "3 · Верификатор (NP)", icon: <img alt="Verify icon" data-bbox="100 585 150 610"/> },
  { id: "growth", label: "4 · Рост по  $n$  и  $ops$ ", icon: <img alt="Growth icon" data-bbox="100 615 150 640"/> },
];

```

```

/* ----- Иерархия -----

```

```

const CLASSES = [
  { id: "EXP", label: "EXP", sub: "2^(n^k)", color: "#0000FF", note: "(кванторные формулы)" },
  { id: "PSPACE", label: "PSPACE", sub: "полиномиальная с полной информацией" },
  { id: "NP", label: "NP", sub: "сертификат проверяется за полиномиальное время", note: "AT", "гамильтонов цикл", "раскраска в 3 цвета", "клика"},
  { id: "P", label: "P", sub: "решается за  $O(n^k)$ ", color: "#00FF00", note: "ид", "MST", "2-SAT, двудольность" },
];

```

```

const NPHARD NOTE = "NP-трудные — всё, к чему сводится

```

```

];
/** Сертификат, который проверяем:  $x = (1, 0, 1, 0)$  — ok
const CERT = [1, 0, 1, 0];

const VERIFY_STEPS = CLAUSES.map((cl, i) => ({
  title: `Дизъюнкт  $\{i + 1\}$ :  $\{cl.map(l => (l > 0 ?$ 
  note: `Подставляем сертификат  $x = (\{CERT.join(", ")$ 
    .map(l =>  $(l > 0 ? \neg x\{l\}=\{CERT[l - 1]\}$ ` :  $\neg x\{$ 
    .join(", ")}.  $\{cl.some(l) => (l > 0 ? CERT[l - 1]$ 
      : "Все литералы ложны — дизъюнкт НЕ выполнен.")`
  ok: cl.some(l =>  $(l > 0 ? CERT[l - 1] === 1 : CERT[$ 
})));

/* ----- Пост -----

interface Sample { n: number; ops: number }

const GROWTH_LINES = [
  { id: "n", label: "n", fn: (n: number) => n, color: "blue" },
  { id: "n log n", label: " $n \cdot \log_2 n$ ", fn: (n: number) => n * Math.log2(n), color: "green" },
  { id: "n2", label: " $n^2$ ", fn: (n: number) => n * n, color: "red" },
  { id: "n3", label: " $n^3$ ", fn: (n: number) => n * n * n, color: "purple" },
  { id: "2n", label: " $2^n$ ", fn: (n: number) => Math.pow(2, n), color: "brown" },
];

export const ComplexityViz: React.FC = () => {
  const chapterId = useContext(VizChapterContext);
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const [tab, setTab] = useState<Tab>("hierarchy");
  const [step, setStep] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [samples, setSamples] = useState<Sample[]>([]);

  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, tab);
  }, [chapterId, tab]);

```

```

    setSamples((prev) => {
      const last = prev[prev.length - 1];
      if (last && last.n === liveN && last.ops === liveOps) return prev;
      const withoutN = prev.filter((s) => s.n !== liveN);
      return [...withoutN, { n: liveN, ops: liveOps }];
    });
  }, [liveN, liveOps]);

/** Оценка показателя роста по последним точкам (лог-лог) */
const verdict = useMemo(() => {
  const pts = samples.filter((s) => s.ops > 0 && s.n > 0);
  if (pts.length < 3) return null;
  const xs = pts.map((p) => Math.log(p.n));
  const ys = pts.map((p) => Math.log(p.ops));
  const mx = xs.reduce((a, b) => a + b, 0) / xs.length;
  const my = ys.reduce((a, b) => a + b, 0) / ys.length;
  let num = 0;
  let den = 0;
  for (let i = 0; i < xs.length; i++) {
    num += (xs[i] - mx) * (ys[i] - my);
    den += (xs[i] - mx) ** 2;
  }
  const slope = den === 0 ? 0 : num / den;
  // экспонента на лог-лог растёт неограниченно: сравнение
  const half = Math.floor(pts.length / 2);
  const slopeTail = (() => {
    const q = pts.slice(half);
    if (q.length < 2) return slope;
    const lx = q.map((p) => Math.log(p.n));
    const ly = q.map((p) => Math.log(p.ops));
    const ax = lx.reduce((a, b) => a + b, 0) / lx.length;
    const ay = ly.reduce((a, b) => a + b, 0) / ly.length;
    let nn = 0;
    let dd = 0;
    for (let i = 0; i < lx.length; i++) {
      nn += (lx[i] - ax) * (ly[i] - ay);
      dd += (lx[i] - ax) ** 2;
    }
  })

```

```

    {TABS.map((t) => {
      <button
        key={t.id}
        onClick={() => setTab(t.id)}
        className={"flex items-center gap-1.5 px-3 p-1.5 rounded-lg bg-slate-900 text-white"}
      >
        {t.icon} {t.label}
      </button>
    })}
  </div>

```

```

  {(tab === "reduce" || tab === "verify") && (
    <div className="flex flex-wrap items-center gap-2">
      <button onClick={() => { setStep(0); setPlaying(false); }}>
        <RotateCcw className="w-4 h-4" />
      </button>
      <button onClick={() => { setPlaying(false); setStep(1); }}>
        <RotateCcw className="w-4 h-4" />
      </button>
      <button onClick={() => setPlaying((p) => !p)}>
        {playing ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
      </button>
      <button onClick={() => { setPlaying(false); setStep(2); }}>
        <RotateCcw className="w-4 h-4" />
      </button>
      <span className="text-[11px] text-slate-500 ml-2">
        {tab === "reduce" ? "Reduce" : "Verify"}
      </span>
    </div>
  )}

```

```

  {/* ----- ИЕРАРХИЯ ----- */
    {tab === "hierarchy" && (
      <div className="space-y-4">
        <div className="bg-slate-950 rounded-xl p-3 border border-slate-800">
          <svg viewBox="0 0 500 380" className="w-full h-380">
            {CLASSES.map((c) => {
              const on = liveCls ? liveCls.toUpperCase() : c.name.toUpperCase()
              return (

```

```

        return (
          <React.Fragment key={b + i}>
            <div
              className="rounded-lg border px-3"
              style={{
                borderColor: active ? meta.c :
                background: active ? meta.c + "
                opacity: active ? 1 : 0.45,
                boxShadow: active ? `0 0 18px $
                minWidth: 96,
              }}
            >
              <span className="text-[11px] font
            </div>
            {i < reduceFrame_boxes.length - 1 &
          </React.Fragment>
        );
      }}}
    </div>
  </div>
  <div className="bg-slate-900/70 border border
    <p className="text-sm font-bold text-white
    text-purple-400" />{reduceFrame.title}</p>
    <p className="text-xs text-slate-400 leadin
  </div>
</div>
)}

{/* ----- ВЕРИФИКАТОР -----
{tab === "verify" && (
  <div className="space-y-3">
    <div className="bg-slate-950 rounded-xl p-4 b
      <p className="text-[10px] uppercase trackin
      <div className="flex flex-wrap gap-2 mb-3">
        {CERT.map((v, i) => (
          <span key={i} className={"font-mono tex
            ld-600/50 text-emerald-300" : "bg-slate-900
            x{i + 1} = {liveCert && typeof liveCe

```

```

    <div className="bg-slate-900/70 border border-
      <p className="text-sm font-bold text-white
      <p className="text-xs text-slate-400 leading-
        определение NP: короткий сертификат + быстрый
    </div>
  </div>
)}

{/* ----- ПОСТ ----- */
{tab === "growth" && (
  <div className="space-y-3">
    <div className="bg-slate-950 rounded-xl p-4 b
      <div className="flex flex-wrap items-center
        <span className="text-[10px] uppercase tr
        <span className="font-mono text-xs bg-sla
          ?? "-"></span>
          <span className="font-mono text-xs bg-sla
e0ps ?? "-"></span>
          <button onClick={() => setSamples([])} cl
0 rounded-lg px-2 py-1">очистить график</bu
        </div>
        <svg viewBox="0 0 500 240" className="w-ful
          {/* оси в лог-лог */}
          <line x1={44} y1={200} x2={486} y2={200}
          <line x1={44} y1={14} x2={44} y2={200} st
          <text x={265} y={222} textAnchor="middle"
          <text x={16} y={110} fill="#64748b" fontS
          {GROWTH_LINES.map((g) => {
            const pts: string[] = [];
            for (let i = 0; i <= 20; i++) {
              const n = 1 + i * 0.8;
              const v = g.fn(n);
              const x = 44 + (Math.log(n) / Math.lo
              const y = 200 - (Math.log(Math.max(1,
              pts.push(`${x.toFixed(1)} ${Math.max(
            }
            return <polyline key={g.id} points={pts
          acity={0.55} />;

```

```

    <div className="mt-3 flex flex-wrap items-center" >
      <span className="text-[10px] uppercase tracking
        {vars ? (
          ["n", "ops", "cls", "k", "cert", "ok"].map((n) => {
            vars[name] === undefined ? null : (
              <span key={name} className="text-[11px] font-mono" style={{
                display: inline-block, width: 100px, text-align: left, vertical-align: top;
              }}>
                {name} = {JSON.stringify(vars[name]).slice(0, 100)}
              </span>
            )
          })
        ) : (
          <span className="text-[11px] text-slate-500">
            открой панель Python: переменные <code clas
            e-300">k</code>, <code className="font-mono
            e-300">n</code> управляют этой демонстрацией
          </span>
        )}
      </div>
    </div>
  );
};

export default ComplexityViz;

```

ФАЙЛ 39/111: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from ".";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "lucide-react";

```

```

interface CodeLine {
  line: string;
  highlight: string;
}

```

```

    { id: 0, label: "A", x: 100, y: 120 },
    { id: 1, label: "B", x: 260, y: 120 },
    { id: 2, label: "C", x: 180, y: 200 },
    { id: 3, label: "D", x: 180, y: 300 },
    { id: 4, label: "E", x: 100, y: 370 },
    { id: 5, label: "F", x: 260, y: 370 },
    { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
  { u: 0, v: 1, key: "0-1" },
  { u: 1, v: 2, key: "1-2" },
  { u: 2, v: 0, key: "0-2" },
  { u: 2, v: 3, key: "2-3" },
  { u: 3, v: 4, key: "3-4" },
  { u: 4, v: 5, key: "4-5" },
  { u: 5, v: 3, key: "3-5" },
  { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
  {
    currentNode: 0, visitedIndices: [0],
    tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
    activeEdge: null, backEdge: null,
    log: "Входим в A. tin[A]=1, low[A]=1.",
    activeCodeLine: [0,1,2]
  },
  {
    currentNode: 1, visitedIndices: [0,1],
    tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
    activeEdge: [0,1], backEdge: null,
    log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
    activeCodeLine: [4,5,7,10,11]
  },
  {
    currentNode: 6, visitedIndices: [0,1,6],
    tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6]
  }
];

```



```

    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
        activeEdge: [4,5], backEdge: null,
        log: "E→F. tin[F]=7, low[F]=7.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: [5,3],
        log: "Из F видим D (посещён). Обратное ребро (F,D).",
        activeCodeLine: [7,8,9]
    },
    {
        currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: null,
        log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо",
        activeCodeLine: [11,13,16,17]
    },
    {
        currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: null,
        log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо",
        activeCodeLine: [11,13,16,17]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: null,
        log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – не мо",
        activeCodeLine: [13,14,15,16,17]
    },
    {
        currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],

```

```

    ));
    const [dfsAuto, setDfsAuto] = useState(false);
    const dfsTimer = useRef<NodeJS.Timeout | null>(null);

    useEffect(() => {
      if (dfsAuto) {
        dfsTimer.current = setInterval(() => {
          setDfsIdx(prev => {
            if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
            return prev + 1;
          });
        }, 2500);
      }
      return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
    }, [dfsAuto]);

    const step = DFS_STEPS[dfsIdx];

    return (
      <div className="space-y-4">
        <div className="flex items-center justify-between">
          <h4 className="text-base font-bold text-white">Шаг {step}
          <div className="flex items-center gap-1 bg-slate-800 p-1">
            <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
              disabled={dfsIdx === 0}
              aria-label="Шаг назад" title="Шаг назад"
              className="p-1.5 hover:bg-slate-700 rounded"
            <Undo className="h-3.5 w-3.5" />
            </button>
            <button onClick={() => setDfsAuto(!dfsAuto)}
              className={`px-2.5 py-1 rounded text-[10px] ${
                dfsAuto ? "bg-rose-600 text-white" : "bg-slate-800 text-white"
              }`}
            >{dfsAuto ? "<>" : "Пауза"}
            <Pause className="h-3 w-3" />
            </button>
            <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length - 1); }}
              disabled={dfsIdx === DFS_STEPS.length - 1}
              aria-label="Шаг вперёд" title="Шаг вперёд"

```

```

    {step.currentNode !== null && step.currentNode !== null}
    const n = GRAPH_NODES.find(x => x.id === n);
    if (!n) return null;
    return <circle cx={n.x} cy={n.y} r={18} fill={n.fill} />
  >;
  })()}
  {GRAPH_NODES.map(n => {
    const cur = step.currentNode === n.id;
    const vis = step.visitedIndices.includes(n.id);
    const st = step.stack.includes(n.id);
    return <g key={n.id}>
      <circle cx={n.x} cy={n.y} r={12}
        fill={cur ? "#10b981" : st ? "#064e3b" : "#fff"}
        stroke={cur ? "#fff" : st ? "#34d399" : "#000"}
        strokeWidth={2.5}
        className="transition-all duration-300" />
      <text x={n.x} y={n.y+3} textAnchor="middle">{n.label}</text>
      {vis && (
        <rect x={n.x-18} y={n.y-28} width={36} height={56}
          className="transition-all duration-300" />
        <text x={n.x} y={n.y-20} textAnchor="center">
          {step.low[n.id] || "?"}</text>
        </>
      )}
    </g>
  )}
</svg>
<div className="flex items-center justify-between" >
  <span>Шаг {dfsIdx+1}/{DFS_STEPS.length}</span>
  <div className="flex-1 mx-2 bg-slate-950 h-10" >
    <div className="bg-indigo-600 h-full transition-all" />
  </div>
</div>
</div>

```

```

                </div>
            ))
        }
    </div>
</div>

    { /* Log */ }
    <div className="bg-indigo-950/10 border borde
      <p className="text-[11px] text-slate-300 le
        <div className="mt-2 pt-2 border-t border-s
          <span className="text-slate-400">Мосты:</
          <span className="font-mono font-bold text
            {step.bridges.length > 0 ? step.bridges
          </span>
        </div>
      </div>
    </div>
  </div>
</div>
</div>
  );
}

```

ФАЙЛ 40/111: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,
import { LiveDistTable } from './LiveDistTable';

interface Node { id: string; x: number; y: number; name
interface Edge { u: string; v: string; w: number; }
interface Step {
  currentNode: string | null;
  checkingEdge: { u: string; v: string } | null;
  distances: { [key: string]: number };

```

```

    adjList[e.u].push({ v: e.v, w: e.w });
  });

const codeLines = [
  { line: 1, text: 'function dijkstra(graph, start):' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '    pq = PriorityQueue(); pq.push(start, 0)' },
  { line: 4, text: '    while not pq.empty():' },
  { line: 5, text: '        d, u = pq.pop() // Извлекаем минимальный элемент' },
  { line: 6, text: '        if d > dist[u]: continue' },
  { line: 7, text: '        for v, weight in graph.neighbours(u):' },
  { line: 8, text: '            if dist[u] + weight < dist[v]:' },
  { line: 9, text: '                dist[v] = dist[u] + weight' },
  { line: 10, text: '                pq.push(v, dist[v])' },
  { line: 11, text: '    return dist // Все кратчайшие пути найдены' },
];

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999 };
  const visited: Record<string, boolean> = { A: false, B: false };
  const prev: Record<string, string | null> = { A: null, B: null };

  simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ' Дейкстра готов к доставке. Начинаем из Пикет-Пост-Оффиса',
    activeCodeLine: 2,
  });

  for (let iter = 0; iter < nodes.length; iter++) {
    let minD = 999;
    let u = null;
  }

```

```

        });
    }
}

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: ' Дейкстра всё доставил! Все возможные кратчайшие пути найдены! ',
  activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">Зарядка...</div>;
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(vars?.done);
const compilerLinked = liveU !== null || liveV !== null;
const currentStep: Step = compilerLinked ? baseStep : null;

```

```

        title={compilerLinked ? HINT_TEXT : "Проигра"}
        className={`flex items-center gap-1 px-4 py-2
        disabled:cursor-not-allowed ${
            isPlaying ? 'bg-amber-600 hover:bg-amber-500' : ''
        }`}
    >
        {isPlaying ? 'Пауза' : 'Пуск'}
    </button>
    <button
        onClick={() => { setIsPlaying(false); if (currentStepIndex > 0) {
            disabled={compilerLinked || currentStepIndex === 0}
            title={compilerLinked ? HINT_TEXT : "Следующий шаг"}
            className="flex items-center gap-1 bg-indigo-600 hover:bg-indigo-500
            cursor-not-allowed text-white px-3 py-2 rounded"}
        >
        ⏏
    </button>
</div>
</div>

<LiveDistTable
    dist={liveDist}
    prev={livePrev}
    done={liveDone}
    demoIds={nodes.map((node) => node.id)}
    title="Ваши расстояния из кода (Дейкстра)"
/>

<div className="flex flex-col lg:flex-row gap-6 mt-10">
    {/* Граф */}
    <div className="w-full lg:w-2/3 bg-indigo-950/20 p-4
    justify-center min-h-[400px]">
        <div className="absolute top-3 left-3 bg-indigo-900/50
        font-bold shadow-sm">
            Шаг {currentStepIndex + 1} из {steps.length}
        </div>

        <svg className="w-full h-auto max-h-[500px]"

```



```

        <span>{edge.v}</span><span cl
        <span className={isCheckingTh
    an>
        </span>
    });
    }}}
  </div>
</div>
  );
  }}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div>
      <h4 className="text-lg font-bold text-rose-950">
        <div className="overflow-x-auto">
          <table className="w-full text-left border">
            <thead>
              <tr className="border-b border-rose-950">
                <th className="py-2 px-3">Вершина</th>
                <th className="py-2 px-3">Расстояни
                <th className="py-2 px-3">Из (пред.
              </tr>
            </thead>
            <tbody className="text-sm">
              {nodes.map((n) => {
                const dist = currentStep.distances[n.id]
                const prev = currentStep.previous[n.id]
                const vis = currentStep.visited[n.id]
                const isCurr = currentStep.currentN

                return (
                  <tr key={n.id} className={`border
                    isCurr ? 'bg-indigo-950/60 fo

```

```
        </div>
      </div>
    </div>
  </div>
</div>
);
}
```

ФАЙЛ 41/111: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord } from '...';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from '...';
```

```
interface DPStep {
  id: number;
  n: number;
  action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'end';
  desc: string;
  codeLine: number;
  memoState: { [key: number]: number };
}
```

```
const DP_CODE_LINES = [
  /* 1 */ "function fibMemo(n, memo = {}) {",
  /* 2 */ "  if (n in memo) return memo[n];",
  /* 3 */ "  if (n <= 2) return 1;",
  /* 4 */ "  memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",
  /* 5 */ "  return memo[n];",
  /* 6 */ "}"
];
```

```
export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
```

```

        action: 'base_case',
        desc: `Базовый случай: n <= 2 (n=${n}). Возвр
        codeLine: 3,
        memoState: { ...memoMap }
    });
    return 1;
}

generatedSteps.push({
    id: stepId++,
    n,
    action: 'call',
    desc: `Вычисляем рекурсивно: fibМемо(${n - 1})
    codeLine: 4,
    memoState: { ...memoMap }
});

const res1 = simulateFib(n - 1);
const res2 = simulateFib(n - 2);
memoMap[n] = res1 + res2;

generatedSteps.push({
    id: stepId++,
    n,
    action: 'calc',
    desc: `Сохраняем в кэш: мемо[${n}] = ${res1} +
    codeLine: 5,
    memoState: { ...memoMap }
});

return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
    id: stepId++,
    n: targetN,
```

```

        memoState,
        desc: `Компилятор: k=${liveK ?? "-"}; таблица m
    }
    : baseStep;

return (
  <div className="bg-slate-900 rounded-2xl border border-
    { /* Header & Controls */ }
    <div className="flex flex-col lg:flex-row lg:items
      <div>
        <div className="flex items-center gap-3 mb-2">
          <div className="p-2 bg-emerald-500/10 border
            <RefreshCw className="w-6 h-6" />
          </div>
          <h3 className="text-2xl font-extrabold text
            </div>
          <p className="text-sm text-slate-400">
            Смотри, как таблица DP кэширует результаты
          </p>
        </div>

    <div className="flex flex-wrap items-center gap-
      <div className="flex items-center gap-2 bg-sl
        <span>N:</span>
        <select
          value={targetN}
          onChange={(e) => setTargetN(Number(e.targ
          disabled={isPlaying}
          className="bg-transparent text-white font
        >
          <option value={4}>N = 4</option>
          <option value={5}>N = 5</option>
          <option value={6}>N = 6</option>
          <option value={7}>N = 7</option>
        </select>
      </div>

    <div className="flex items-center bg-slate-95

```

```

    { /* DP Table View */ }
    <div className="mb-8 bg-slate-950 p-4 md:p-6 rounded"
      <h4 className="text-xs font-bold uppercase tracking-wider"
        <div className="flex flex-wrap items-center gap-2"
          {Array.from({ length: targetN }, (_, i) => i + 1).map((num) => {
            const isCached = num in memoState || num <= 2;
            const val = num <= 2 ? 1 : memoState[num];
            const isCurrent = currentStep?.n === num;

            return (
              <div
                key={num}
                className={`flex flex-col items-center justify-center gap-2
                  ${isCurrent ? 'bg-emerald-950 border-emerald-500' : ''}
                  ${isCached ? 'bg-slate-900 border-emerald-500/50' : ''}
                  ${!isCurrent & !isCached ? 'bg-slate-900/50 border-slate-800' : ''}
                `}
              >
                <span className="text-xs text-slate-400">N</span>
                <span className={`text-xl md:text-2xl font-medium text-slate-500`}
                  {isCached ? val : '0'}>
                </span>
                {num <= 2 && <span className="text-[9px] font-mono">1</span>}
              </div>
            );
          })}
        </div>

        {currentStep && (
          <div className="mt-6 pt-4 border-t border-slate-800"
            <div className="text-sm font-medium text-slate-500"
              <span className="inline-block w-2 h-2 rounded-full bg-slate-900" />
              <span>{currentStep.desc}</span>
            </div>
            <div className="text-xs font-mono bg-slate-900 p-2"
              Текущий N: <strong className="text-emerald-500">{currentStep.n}</strong>

```

```

    <p className="text-sm text-slate-400">
      В классической рекурсии для  $N=\{targetN\}$  по
      вычисленное значение в таблице, мы момента.
    </p>
    <div className="p-4 bg-slate-900 rounded-xl text-slate-300">
      <span>Всего шагов симуляции: {steps.length}</span>
      <span className="text-emerald-400 font-bold">
    </div>
  </div>
})

{activeTab === 'code' && (
  <div className="bg-slate-950 rounded-2xl border">
    <div className="bg-slate-900 px-6 py-3 border">
      <span className="text-xs font-bold font-mono">
      <span className="text-xs text-emerald-400">
    </div>
    <pre className="p-6 font-mono text-sm space-x-2">
      {DP_CODE_LINES.map((line, idx) => {
        const lineNum = idx + 1;
        const isActive = currentStep?.codeLine === lineNum;
        return (
          <div
            key={lineNum}
            className={`flex items-center px-4 py-2 ${
              isActive
                ? 'bg-emerald-600/20 border-l-4 border-emerald-500'
                : 'text-slate-300 hover:bg-slate-900'
            }`}
          >
            <span className="w-8 text-xs text-slate-400">{lineNum}</span>
            <span>{line}</span>
          </div>
        );
      })}
    </pre>
    <div className="p-6 bg-slate-900/50 border-l-4 border-emerald-500">

```

```

    { id: 3, label: "D", x: 280, y: 250 },
    { id: 4, label: "E", x: 100, y: 250 }
  ];

const C1_EDGES = [
  { u: 0, v: 1 }, { u: 0, v: 2 },
  { u: 1, v: 3 }, { u: 2, v: 4 },
  { u: 1, v: 2 },
  { u: 3, v: 4 }
];

const ekey = (a: number, b: number) => `${Math.min(a, b)}-${Math.max(a, b)}`;
const DEG = C1_NODES.map((n) => C1_EDGES.filter((e) => ekey(e.u, e.v) === n.id));
const ODD = C1_NODES.map((n) => n.id).filter((id) => DEG[id] % 2 !== 0);

/**
 * Билет 13 – «Графы. Эйлеров цикл».
 *
 * Две части:
 * 1. игра «пройди все рёбра» кликами (Эйлер не прощает ошибок);
 * 2. критерий: степени вершин и число нечётных (0 → цикл).
 *
 * Синхронизация с Python-компилятором (PAGE_SYNC["euler"]);
 * path – маршрут (массив вершин) рисуется поверх графа;
 * пройдены, deg/odd – степени и нечётные вершины, lastStep – номер шага.
 */
export function EulerSimulator() {
  const chapterId = useContext(VizChapterContext);
  const runtime = useVizRuntime();
  const vars = runtime?.variables;

  const [c1Path, setC1Path] = useState<number[]>([]);
  const [c1VisitedEdges, setC1VisitedEdges] = useState<boolean[]>([]);
  const [c1Msg, setC1Msg] = useState("Клики на вершину");
  const [c1Done, setC1Done] = useState(false);

  useEffect(() => {

```

```

    } else {
      setClMsg(`Пройдено рёбер: ${nextEdges.length} / $
    }
  };

/* — что говорит компилятор — */
const live = useMemo(() => {
  if (!vars) return null;
  const rawPath = vizArray(vars.path) ?? vizArray(var
  const path = (rawPath ?? [])
    .map((x) => (typeof x === "number" ? x : vizNumbe
    .filter((x): x is number => x !== null && Number.
  const usedRaw = vizArray(vars.used) ?? vizArray(var
  const usedFlags = (usedRaw ?? []).map((x) => Boolean
  const degRec = vizRecord(vars.deg) ?? vizRecord(var
  const deg = degRec
    ? C1_NODES.map((n) => {
      const val = degRec[String(n.id)] ?? degRec[n.
      const num = typeof val === "number" ? val : v
      return num ?? DEG[n.id];
    })
    : null;
  const oddRaw = vizArray(vars.odd);
  const odd = oddRaw ? oddRaw.map((x) => (typeof x ===
  const last = vizNumber(vars.last) ?? vizNumber(vars
  const step = vizNumber(vars.step) ?? vizNumber(vars
  if (path.length === 0 && usedFlags.length === 0 &&
  // рёбра маршрута: пары соседних вершин
  const keys: string[] = [];
  for (let i = 0; i + 1 < path.length; i++) keys.push
  usedFlags.forEach((f, i) => {
    if (f && C1_EDGES[i]) keys.push(ekey(C1_EDGES[i].
  ));
  return {
    path,
    keys: Array.from(new Set(keys)),
    deg,
    odd,

```



```
</div>
```

```
<div className="bg-slate-950 rounded-xl p-4 border-1 border-gray-800 -hidden">
```

```
<div className="absolute inset-0 bg-[radial-gradient(circle_1px_1px_1px_1px)]">
```

```
<svg className="w-full h-[260px] z-10 relative">
```

```
  {C1_EDGES.map((e, i) => {
```

```
    const u = C1_NODES.find(n => n.id === e.u)!
```

```
    const v = C1_NODES.find(n => n.id === e.v)!
```

```
    const key = ekey(e.u, e.v);
```

```
    const used = visitedKeys.includes(key);
```

```
    // номер этого ребра в маршруте – подписываем
```

```
    const order = path.length > 1 ? (() => {
```

```
      for (let s = 0; s + 1 < path.length; s++)
```

```
        return null;
```

```
    })() : null;
```

```
    const isLastEdge = order !== null && order === path.length - 1;
```

```
    return <g key={i}>
```

```
      <line x1={u.x} y1={u.y} x2={v.x} y2={v.y}
```

```
        stroke={isLastEdge ? "#a5b4fc" : used ? "#6366f1" : "#fff"}
```

```
        strokeWidth={isLastEdge ? 5 : used ? 4 : 2}
```

```
        className="transition-all duration-300"/>
```

```
      {order !== null && (
```

```
        <text x={({u.x + v.x} / 2) y={({u.y + v.y} / 2)}
```

```
      )}</text>
```

```
    )}
```

```
  </g>;
```

```
  )})
```

```
{C1_NODES.map(n => {
```

```
  const isLast = lastId === n.id;
```

```
  const visited = path.includes(n.id);
```

```
  const isOdd = odd.includes(n.id);
```

```
  return <g key={n.id} className="cursor-pointer">
```

```
    {isLast && <circle cx={n.x} cy={n.y} r={11}
```

```
    <circle cx={n.x} cy={n.y} r={11}
```

```
      fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
```

```
      stroke={isLast ? "#fff" : visited ? "#6366f1" : "#fff"}
```

```
      strokeWidth={2.5}
```

```

        {live.path.map((id) => C1_NODES[id].label).
        <span className="text-[11px] font-mono bg-slate-500">
        ойдено: {live.keys.length}/{C1_EDGES.length}
        <span className="text-[11px] font-mono bg-slate-500">
        {odd.length}</span>
        {live.step !== null && <span className="text-[11px] font-mono bg-slate-500">
        x-2 py-1">step = {live.step}</span>}
      </>
    ) : (
      <span className="text-[11px] text-slate-500">
        открой панель Python и запусти код: массив
        том графе, а <code className="font-mono text-slate-500">
        есчитают критерий
      </span>
    )}
  </div>
</div>
);
}

```

ФАЙЛ 43/111: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect } from 'react';
import { liveMatrix, pickLiveVar, useVizRuntime, vizNum

```

```

interface Step {
  k: number; i: number; j: number;
  matrix: (number | null)[][]; updated: boolean;
  log: string; activeCodeLine: number;
}

```

```

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
    ва листать вручную.";

```

```
const codeLines = [
  { line: 1, text: 'function floyd_warshall(matrix, V' },
  { line: 2, text: '    dist = copy(matrix)' },
  { line: 3, text: '    for k from 0 to V - 1: // Пос' },
  { line: 4, text: '        for i from 0 to V - 1:' },
  { line: 5, text: '            for j from 0 to V - 1' },
  { line: 6, text: '                if dist[i][k] + d' },
  { line: 7, text: '                    dist[i][j] = ' },
  { line: 8, text: '    return dist' },
];
```

```
useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist = initialMatrix.map(row => [...row]);

  simulationSteps.push({
    k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
    log: '    Инициализация матрицы прямых путей, ∞ (999)',
    activeCodeLine: 2,
  });
});
```

```
const N = 7;
for (let k = 0; k < N; k++) {
  simulationSteps.push({
    k, i: -1, j: -1, matrix: dist.map(r => [...r]),
    log: `+ Внешний цикл. Разрешаем пути через пром'`,
    activeCodeLine: 3,
  });
};
```

```
for (let i = 0; i < N; i++) {
  for (let j = 0; j < N; j++) {
    if (i === j || i === k || j === k) continue;

    const oldVal = dist[i][j];
    const viaVal = dist[i][k] + dist[k][j];
    if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal)
      dist[i][j] = viaVal;
  }
}
```

```

const liveJ = vizNumber(vars?.j);
/**
 * Матрица из кода пользователя – СВОЕЙ размерности.
 * протискивались в демо-сетку 7×7: при n < 7 появлял
 * при n > 7 компонент падал на nodeNames[i].split(..
 */
const liveSource = pickLiveVar(vars, ["d", "dist", "d
const liveMat = liveMatrix(liveSource?.value);
const compilerLinked = liveK !== null || liveI !== nu

const rows = liveMat?.rows ?? baseStep.matrix.length;
const cols = liveMat?.cols ?? (baseStep.matrix[0]?.le
const shownMatrix: (number | null)[][] = liveMat
  ? Array.from({ length: rows }, (_, i) => Array.from
    : baseStep.matrix;

const demoSize = baseStep.matrix.length;
const sizeMismatch = compilerLinked && liveMat && (li
  ? `Демо-схема графа нарисована для ${demoSize} вершн
    ерности; схема графа остаётся демонстрационной.`
  : null;
const labelOf = (idx: number) => nodeNames[idx]?.spli

const currentStep: Step = compilerLinked
  ? {
    ...baseStep,
    k: liveK ?? -1,
    i: liveI ?? -1,
    j: liveJ ?? -1,
    matrix: shownMatrix,
    updated: !!runtime?.changed?.some(({name}) => nam
    log: `Компилятор: k=${liveK ?? "-"}, i=${liveI
      ame})` : ""}. Подсветка взята из реальных пер
  }
  : baseStep;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl bor

```

```

<path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
  <marker id="arrow-fw-via" markerWidth="6"
th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
</defs>
{Object.keys(adjList).flatMap((uStr) => {
  const u = parseInt(uStr);
  return adjList[u].map((edge) => {
    const uNode = nodesSvg.find((n) => n.id
    const vNode = nodesSvg.find((n) => n.id
    const isTarget = currentStep.i === u &&
    const isVia = (currentStep.i === u && c
    let strokeColor = '#475569'; let marker
    if (isTarget) { strokeColor = '#10b981'
    else if (isVia) { strokeColor = '#818cf8
    return (
      <g key={` ${u} - ${edge.v} `}>
        <line x1={uNode.x} y1={uNode.y} x2=
markerEnd={marker} strokeDasharray={isTarg
        <circle cx={({uNode.x + vNode.x) / 2
isVia ? '#818cf8' : '#64748b'} strokeWidth
        <text x={({uNode.x + vNode.x) / 2} y=
8fafc'} fontSize="10" fontWeight="bold" tex
      </g>
    );
  });
});
})}
{nodesSvg.map((node) => {
  const isK = currentStep.k === node.id;
  const isI = currentStep.i === node.id;
  const isJ = currentStep.j === node.id;
  let fill = '#1e293b'; let stroke = '#64748b';
  if (isK) { fill = '#4f46e5'; stroke = '#a
  else if (isI) { fill = '#b45309'; stroke
  else if (isJ) { fill = '#047857'; stroke
  return (
    <g key={node.id} className="transition-
      <circle cx={node.x} cy={node.y} r={is
sJ ? 4 : 2} className="transition-colors du

```

```

        )
      }}}
    </div>
  );
  }}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Матрица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div className="flex justify-between items-center">
      <h4 className="text-lg font-bold text-rose-950">
    </div>
    {sizeMismatch ? (
      <p className="mb-3 rounded-lg border border-rose-950/40">
        {sizeMismatch}
      </p>
    ) : null}
    <div className="overflow-x-auto">
      <table className="w-full text-center border border-rose-950/40">
        <thead><tr className="bg-rose-950/40 text-rose-950">
          <th className="p-2 border-r border-rose-950/40">
            {Array.from({ length: cols }, (_, idx) => {
              <th key={idx} className={`p-2 border-rose-950/40 ${
                labelOf(idx)} {idx}</th>
            })}
          </tr></thead>
          <tbody className="text-xs font-mono">
            {currentStep.matrix.map((row, i) => (
              <tr key={i} className="border-b border-rose-950/40">
                <td className={`p-2 bg-rose-950/40 ${
                  currentStep.k === i ? 'text-amber-400' : ''}`}>
                  {labelOf(i)} {i}
                </td>
                {row.map((val, j) => {
                  const isUpd = currentStep.i === i & currentStep.j === j

```

ФАЙЛ 44/111: src/components/GraphComponentsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useCallback, useContext, useEffect, useLayoutEffect } from 'react';
import { ChevronLeft, ChevronRight, Play, Pause, RotateCcw } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from './viz-context';
```

```
/**
 * Билет 8 – «Графы. Компоненты связности».
 *
 * Две вкладки:
 * 1. «Обход» – DFS/BFS-покраска: внешний цикл по непокрашенным вершинам.
 *    каждый новый запуск = +1 к счётчику компонент.
 * 2. «DSU» – рёбра приходят по одному, union склеивает компоненты.
 *    comps уменьшается.
 *
 * Синхронизация с Python-компилятором (PAGE_SYNC["graphs"]):
 * - comp / color / used – массив разметки: вершины покрашены / нет
 * - parent / p – массив DSU: рисуем множества
 * - v – текущая вершина (пульсирует)
 * - stack / dq / q – фронт обхода (подсвечивает)
 * - comps / c – счётчик компонент (большая цифра)
 * - i / e / step – номер шага: двигает по кадру
 */
```

```
type Mode = "dfs" | "dsu";
```

```
interface Node {
  id: number;
  label: string;
  x: number;
  y: number;
}
```

```
/** Граф из 10 вершин и трёх компонент – ровно как в те
```

```

/** Покадровая раскладка DFS-обхода: ровно тот порядок,
function buildDfsFrames(): DfsFrame[] {
  const frames: DfsFrame[] = [];
  const comp = new Array(NODES.length).fill(0);
  const push = (cur: number | null, frontier: number[],
    => {
    frames.push({ title, note, comp: [...comp], cur, fr
  });
  let c = 0;
  push(null, [], c, "Старт: все вершины серые", "comp =
    дой непокрашенной.");
  for (let start = 0; start < NODES.length; start++) {
    if (comp[start] !== 0) continue;
    c += 1;
    const stack = [start];
    comp[start] = c;
    push(start, stack, c, `Новый запуск: c = ${c}, стар
      цвет ${c} и кладём в стек.`);
    while (stack.length) {
      const v = stack[stack.length - 1];
      const next = ADJ[v].find((to) => comp[to] === 0);
      if (next === undefined) {
        stack.pop();
        push(v, stack, c, `Из ${v} идти некуда – возвра
          ;
        continue;
      }
      comp[next] = c;
      stack.push(next);
      push(next, stack, c, `${v} → ${next}: красим в цв
        }, кладём ${next} в стек.`, [v, next]);
    }
    push(null, [], c, `Компонента ${c} готова`, `Стек п
      } вершин. Ищем следующую серую вершину.`);
  }
  push(null, [], c, "Готово: компонент = " + c, `Внешни
    вязности.`);

```



```

const DFS_FRAMES = buildDfsFrames();
const DSU_FRAMES = buildDsuFrames();

/** Корень DSU-множества с полным сжатием пути. */
const dsuRoot = (parent: number[], v: number): number => {
  let cur = v;
  const seen = new Set<number>();
  while (parent[cur] !== cur && !seen.has(cur)) {
    seen.add(cur);
    cur = parent[cur];
  }
  return cur;
};

export const GraphComponentsViz: React.FC = () => {
  const chapterId = useContext(VizChapterContext);
  const runtime = useVizRuntime();
  const [mode, setMode] = useState<Mode>("dfs");
  const [step, setStep] = useState(0);
  const [playing, setPlaying] = useState(false);
  const timer = useRef<number | null>(null);

  const frames = mode === "dfs" ? DFS_FRAMES : DSU_FRAMES;
  const maxStep = frames.length - 1;
  const frame = frames[Math.min(step, maxStep)] as DfsFrame;

  /** — вкладка сообщает компилятору, какой код подстав. */
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, mode);
  }, [chapterId, mode]);

  /** — автопроигрывание — */
  useEffect(() => {
    if (!playing) return;
    timer.current = window.setInterval(() => {
      setStep((s) => {
        if (s >= maxStep) {

```

```

useEffect(() => {
  if (liveStep === null || !Number.isFinite(liveStep))
    goTo(liveStep);
}, [liveStep, goTo]);

/* — какая разметка сейчас в силе: код пользователя или код автора?
/** Массив parent, который сейчас рисуем: из кода пользователя
const parentNow = liveParent ?? (mode === "dsu" ? frame.parent : []);
/** Номер множества по parent[]: корни нумеруются в порядке возрастания
const setOf = useCallback(
  (parent: number[], id: number): number => {
    const roots = Array.from(new Set(parent.map((_, i) => i)));
    return roots.indexOf(dsuRoot(parent, id)) + 1;
  },
  []
);

const compOf = (id: number): number => {
  if (liveComp && liveComp[id] !== undefined && liveComp[id] !== 0)
    if (parentNow) return setOf(parentNow, id);
  return frame.comp?.[id] ?? 0;
};

const frontier = liveFrontier ?? (mode === "dfs" ? frame.frontier : []);
const cur = liveCur ?? frame.cur;
const comps = liveComps ?? frame.comps;
const edge = frame.edge;
const title = runtime && vars ? `${frame.title} — Контракт` : "Контракт";

const switchMode = (m: Mode) => {
  setMode(m);
  setStep(0);
  setPlaying(false);
};

return (
  <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl" >
    <div className="flex flex-wrap items-center justify-between" >

```



```

        });
    }}}
    </g>
  </svg>
</div>

```

```

{/* подпись кадра */}
<div className="mt-3 bg-slate-900/70 border border-
  <p className="text-sm font-bold text-white mb-1
    {runtime && vars && <Terminal className="w-3.
    {title}
  </p>
  <p className="text-xs text-slate-400 leading-re
</div>

```

```

{/* живые переменные из компилятора */}
<div className="mt-3 flex flex-wrap items-center
  <span className="text-[10px] uppercase tracking
    {vars ? (
      ["v", "comps", "c", "i", "e", "comp", "parent
        vars[name] === undefined ? null : (
          <span key={name} className="text-[11px] f
            py-1">
              {name} = {JSON.stringify(vars[name]).sl
            </span>
          )
        )
      ) : (
        <span className="text-[11px] text-slate-500">
          открой панель Python – переменные <code cla
            erald-400">parent</code>, <code className="
            comps</code> покрасят этот граф сами
        </span>
      )}
    </div>
  </div>
);
};

```

```

    'F': ['C'],
    'G': ['C'],
};

type Action = {
  type: 'visit' | 'process' | 'backtrack';
  node: string;
  desc: string;
  queueOrStack?: string[];
  visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
  const steps: Action[] = [];
  const vis = new Set<string>();
  const stack: string[] = []; // for visualization, j

  function dfs(v: string) {
    vis.add(v);
    stack.push(v);
    steps.push({ type: 'visit', node: v, desc: `Замечание: ${v} посещено`,
      visitedNodes: Array.from(vis) });

    for (const u of adj[v]) {
      if (!vis.has(u)) {
        steps.push({ type: 'process', node: v, desc: `Процесс: ${v}`,
          queueOrStack: [...stack], visitedNodes: Array.from(vis) });
        dfs(u);
      }
    }

    stack.pop();
    steps.push({ type: 'backtrack', node: v, desc: `Результат: ${v}`,
      queueOrStack: [...stack], visitedNodes: Array.from(vis) });
  }

  dfs(start);
  return steps;
}

```

```

const [stepIdx, setStepIdx] = useState(0);

const steps = useMemo(() => mode === 'dfs' ? genera
useVizStepSync(stepIdx, setStepIdx, steps.length -
  const v = vizString(vars.v);
  const structure = vizArray(vars.stack) ?? vizArra
  const visited = vizArray(vars.visited);
  if (v === null && !structure && !visited) return
  const candidates = steps
    .map(({step, index}) => ({ step, index }))
    .filter(({ step }) =>
      (v === null || step.node === v) &&
      (!structure || step.queueOrStack?.length ===
      (!visited || step.visitedNodes?.length === vi
    );
  return candidates.at(-1)?.index ?? null;
});

useEffect(() => {
  setStepIdx(0);
  setAutoPlay(false);
}, [mode]);

useEffect(() => {
  if (autoPlay) {
    const t = setInterval(() => {
      setStepIdx(p => {
        if (p >= steps.length - 1) {
          setAutoPlay(false);
          return p;
        }
        return p + 1;
      });
    }, 1200);
    return () => clearInterval(t);
  }
}, [autoPlay, steps.length]);

```

```

    { /* Nodes */
    {nodes.map(n => {
      const visited = isVisited(n)
      const current = isCurrent(n)

      let fillColor = "fill-slate"
      let strokeColor = "stroke-slate"
      let textColor = "fill-slate"
      let radius = 18;

      if (visited) {
        fillColor = mode === 'd' ? "fill-slate" : "fill-white"
        strokeColor = mode === 'd' ? "stroke-slate" : "stroke-white"
        textColor = "fill-white"
      }

      if (current && step.type !== 'd') {
        strokeColor = "stroke-amber"
        textColor = "fill-amber"
        radius = 22;
      }

      return (
        <g key={n.id} className={mode === 'd' ? "node-d" : "node-v"}>
          <circle cx={n.x} cy={n.y} radius={radius}
            className={mode === 'd' ? "stroke-slate" : "stroke-white"} />
          <text x={n.x} y={n.y} text={n.label}
            className={mode === 'd' ? "fill-slate" : "fill-white"} />
          { /* Ripple for BFS */
            {current && mode === 'd' ? (
              <circle cx={n.x} cy={n.y} radius={22}
                className="stroke-amber" />
            ) : null}
          )}
        </g>
      )
    })}
  )
}

```

```

        <button onClick={() => {setAutoPlay
="Шаг назад" title="Шаг назад" className="p
0 disabled: hover: bg-transparent">
        <Undo strokeWidth={3} size={16} />
        </button>
        <button onClick={() => setAutoPlay(
justify-center min-w-[80px] ${mode === 'df
>
        {autoPlay ? <><Pause size={16} />
        </button>
        <button onClick={() => {setAutoPlay
= steps.length - 1} aria-label="Шаг вперед"
00 rounded-lg disabled:opacity-30 disabled:
        <SkipForward strokeWidth={3} si
        </button>
        <button onClick={() => {setAutoPlay
ame="p-2 text-slate-500 hover:text-rose-400
        <RefreshCw strokeWidth={3} size
        </button>
    </div>

    {/* Description Log */}
    <div className="flex-1 bg-slate-900/50
ow-hidden">
        <div className="text-[10px] text-sl
{steps.length}]</div>
        <div className="text-sm text-slate-
        {step.desc}
        </div>
    </div>
</div>
</div>
);
}

```



```

    if (l < n && a[l].priority > a[largest].priority) l = l;
    if (r < n && a[r].priority > a[largest].priority) l = r;
    if (largest !== idx) {
      [a[largest], a[idx]] = [a[idx], a[largest]];
      idx = largest;
    } else break;
  }
  return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
  return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
  const level = Math.floor(Math.log2(index + 1));
  const posInLevel = index - (Math.pow(2, level) - 1);
  const count = Math.pow(2, level);
  const x = ((posInLevel + 0.5) / count) * 100;
  const y = 10 + level * 24;
  return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
  { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤' },
  { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤' },
  { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
  { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤' },
  { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤' },
  { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
  let arr: Patient[] = [];
  const start = [
    { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤' },
  ];
  arr.push(...start);
  return arr;
})();

```

```

const [healingPatient, setHealing] = useState<Patient>({
  name: 'Пациент',
  emoji: '👤',
  symptom: 'Пациент',
  priority: 0,
  animState: 'in',
})

const addLog = (msg: string) => setLog(prev => [msg, ...prev])

// --- INSERT: Новый пациент поступает в приемный покой
const insertPatient = useCallback(({name: string, priority: number}) => {
  if (busy || heap.length >= 15) {
    addLog('⚠ Все палаты переполнены! Дождитесь выписки пациента')
    setShake(true)
    setTimeout(() => setShake(false), 400)
    return
  }
  setBusy(true)

  const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)]
  const finalName = name || preset.name.split(' ')[0]
  const finalEmoji = preset.emoji
  const finalSymptom = preset.symptom

  const newPatient: Patient = {
    id: `p${uid++}`,
    name: finalName,
    emoji: finalEmoji,
    symptom: finalSymptom,
    priority,
    animState: 'in',
  }

  const nextHeap = heapInsert(heap, newPatient).map(x => x)
  // Mark specifically this new patient as 'in' in the heap
  const target = nextHeap.find(x => x.priority === priority)
  const marked = nextHeap.map(x => x.id === (target?.id || ''))

  setHeap(marked)
  addLog(` Поступил пациент: ${finalName} с тяжестью ${priority}`)

  setTimeout(() => {
    setHeap(nextHeap)
  }, 1000)
}, [heap, busy, uid])

```



```
</div>
```

```
{/* УПРАВЛЕНИЕ */}
```

```
<div className="flex flex-col lg:flex-row items-s
```

```
  {/* Добавить случайного */}
```

```
  <button
```

```
    onClick={handleRandomInsert}
```

```
    disabled={busy || heap.length >= 15}
```

```
    className="flex-1 min-w-[150px] bg-gradient-to
```

```
      opacity-40 disabled:cursor-not-allowed text
```

```
      ve:scale-95 transition-all cursor-pointer f
```

```
  >
```

```
    <span> Привезти по Скорой (INSERT)</span>
```

```
  </button>
```

```
  {/* Добавить кастомного */}
```

```
  <form onSubmit={handleCustomInsert} className="
```

```
    <input
```

```
      type="text"
```

```
      required
```

```
      value={customName}
```

```
      onChange={e => setCustomName(e.target.value
```

```
      placeholder="Имя пациента"
```

```
      className="flex-1 bg-slate-800 border borde
```

```
      er-emerald-500 transition-colors placeholde
```

```
  />
```

```
  <div className="flex flex-col items-center mi
```

```
    <span className="text-[9px] font-mono text-
```

```
    <input
```

```
      type="range"
```

```
      min="10"
```

```
      max="99"
```

```
      value={customPriority}
```

```
      onChange={e => setCustomPriority(Number(e
```

```
      className="w-20 accent-emerald-500 cursor
```

```
    />
```

```
  </div>
```

```
</div>
```

```
<div className="relative w-full h-full min-h-
  <svg className="absolute inset-0 w-full h-f
    {edges}
  </svg>
```

```
{visualHeap.map((patient, idx) => {
  const pos = nodePos(idx);
  const isRoot = idx === 0;
  const isCompilerNode = liveI === idx;
  return (
```

```
    <div
      key={patient.id}
      className={`
        absolute flex flex-col items-center
        -translate-x-1/2 -translate-y-1/2 t
        ${patient.animState === 'in' ? 'anim
        ${patient.animState === 'winner' ?
        ${patient.animState === 'out' ? 'an
        ${isCompilerNode ? 'ring-4 ring-amb
        ${isRoot ? 'bg-rose-950/80 border-r
      `}
    >
```

```
    style={{
      left: `${pos.x}%`,
      top: `${pos.y}%`,
      width: isRoot ? 60 : 52,
      height: isRoot ? 60 : 52,
    }}
  >
```

```
    <span className="text-2xl leading-non
    <span className="text-[10px] font-mon
      {patient.priority}%
    </span>
```

```
    {/* Tooltip */}
    <div className="absolute bottom-full
      <div className="bg-slate-950 border
```

```

        </div>
      </div>
    ) : (
      <div className="text-center text-slate-600">
        <span className="text-5xl mb-2 block">
          <p className="text-xs font-bold font-mono">
            <p className="text-[10px] mt-1">Ждём са
          </p>
        </div>
      </div>
    )}
  </div>

  <div className="w-full h-3 bg-gradient-to-r f
</div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
  <div className="text-[10px] font-mono text-slate-
    {log.map((entry, idx) => {
      <div
        key={idx}
        className={`text-xs font-mono flex items-ce
      >
        {idx === 0 ? <span className="text-emerald-5
        <span>{entry}</span>
      </div>
    })}
  </div>
</div>
);
};

```

```

    text: "#e2e8f0",
    dim: "#94a3b8",
    active: "#6366f1",
    done: "#10b981",
    warn: "#f59e0b",
    bad: "#f43f5e",
    edge: "#475569",
  };

export const Svg: React.FC<{ children: React.ReactNode;
  const paused = useContext(DemoPauseContext);
  return (
    <div className="w-full">
      <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
        {children}
      </svg>
      {caption && (
        <div className="text-[10px] text-slate-400 text-center">
          {caption}
        </div>
      )}
      <div className="text-[9px] text-slate-600 text-center">
        {paused ? "пауза – курсор на карточке" : "наведи"}
      </div>
    </div>
  );
};

export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
  </g>
);

```

```

const step = useLoop(3, 1200);
// i -> j напрямую 9; через k=1 получаем 3+4=7
const pos: Record<string, [number, number]> = { i: [4, 1], k: [1, 1], j: [1, 4] };
const viaOpen = step >= 1;
const relaxed = step >= 2;
return (
  <Svg
    caption={
      step === 0
        ? "k ещё запрещена: путь i → j = 9"
        : step === 1
          ? "Разрешаем пересадку через k: 3 + 4 = 7"
          : "7 < 9 → D[i][j] = 7"
    }
  >
    <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad : C.good} />
    <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done : C.wait} />
    <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done : C.wait} />

    <text x={130} y={112} textAnchor="middle" fontSize=
      9
    </text>
    <text x={72} y={54} textAnchor="middle" fontSize=
      3
    </text>
    <text x={188} y={54} textAnchor="middle" fontSize=
      4
    </text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C.wait} />
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={C.wait} />
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={C.wait} />
  </Svg>
);
};

```

/** Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {

```



```

        ? "Вершины – объекты {города, слова, состояни
        : step === 1
        ? "Рёбра – связи между ними"
        : "Вес ребра – цена перехода"
    }
  >
  {edges.map(([u, v, w], i) => {
    const a = pos[u];
    const b = pos[v];
    return (
      <g key={i}>
        <Edge a={a} b={b} color={step >= 1 ? C.acti
        {step >= 2 && (
          <text
            x={({a[0] + b[0]) / 2}
            y={({a[1] + b[1]) / 2 - 4}
            textAnchor="middle"
            fontSize={10}
            fontWeight="700"
            fill={C.warn}
          >
            {w}
          </text>
        )}
      </g>
    );
  })}
  </Svg>
);
};

```

```

/** Сжатие координат: разрежённые числа → плотные индексы
export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];

```

```

        {sorted.indexOf(v)}
      </text>
    </g>
  )))
</Svg>
);
};

/* ----- Splay: три случая поворота -----

type Pt = [number, number];

const SplayFrames: React.FC<{
  frames: { pos: Record<string, Pt>; edges: [string, string][];
  captions: string[];
  ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
  const step = useLoop(frames.length, ms);
  const f = frames[step];
  const color = (id: string) =>
    id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f1c40f" : "#2ecc71";

  return (
    <Svg caption={captions[step]}>
      {f.edges.map(([a, b], i) => (
        <line
          key={i}
          x1={f.pos[a][0]}
          y1={f.pos[a][1]}
          x2={f.pos[b][0]}
          y2={f.pos[b][1]}
          stroke={C.edge}
          strokeWidth={1.8}
          style={{ transition: MOVE }}
        />
      ))}
      {Object.entries(f.pos).map(([id, [x, y]]) => (
        <g key={id} style={{ transform: `translate(${x}, ${y})` }}>

```

```

    { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
    { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
    { pos: { x: [130, 30], p: [80, 100], g: [180, 100]
  }}
  />
};

```

ФАЙЛ 49/111: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
  A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
  const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
  const step = useLoop(order.length + 1, 850);
  const visited = new Set(order.slice(0, step).flat());
  const wave = step > 0 && step <= order.length ? order
  return (
    <Svg caption={`Уровень ${Math.max(0, Math.min(step,
      {GRAPH_EDGES.map(([u, v], i) => {
        <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
      }}})
    {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
      <Node key={k} x={x} y={y} label={k}
        fill={wave.includes(k) ? C.active : visited.h
        stroke={wave.includes(k) ? "#a5b4fc" : undefi
    }}})
  )

```

```

    )))
    {nodes.map((n) => {
      const idx = placed.indexOf(n.id);
      return (
        <g key={n.id}>
          <rect x={n.x - 34} y={n.y - 12} width={68} h={12}
            " }} />
          <text x={n.x} y={n.y} textAnchor="middle" dx={
            {idx >= 0 && <circle cx={n.x + 30} cy={n.y - 12} r={12}
              {idx >= 0 && {
                <text x={n.x + 30} y={n.y - 12} textAnchor="middle"
                  x + 1}</text>
              }}
            </g>
          );
        )))
      <text x={224} y={62} textAnchor="middle" fontSize=14>12 > 3 + 4 = 7</text>
      <text x={224} y={74} textAnchor="middle" fontSize=14>12 > 3 + 4 → пишем 7. Путь 12</text>
    </Svg>
  );
};

```

```

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь 12" : "12 > 3 + 4 = 7. Путь 12"}>
      <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />
      <text x={130} y={54} textAnchor="middle" fontSize=14>12 > 3 + 4 = 7</text>
      <Node x={60} y={70} r={20} fill={C.active} label="12" />
      <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="12" />
      <text x={60} y={106} textAnchor="middle" fontSize=14>12 > 3 + 4 = 7</text>
      <text x={200} y={106} textAnchor="middle" fontSize=14>12 > 3 + 4 → пишем 7. Путь 12</text>
      <text x={130} y={22} textAnchor="middle" fontSize=14>12 > 3 + 4 = 7</text>
    </Svg>
  );
};

```

```

        <Node key={k} x={x} y={y} label={k} fill={k} stroke={k} />
      } />
    )
  })
</Svg>
);
};

export const MstDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 100], E: [180, 100]
  };
  const edges = [
    { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
    { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
  ];
  const chosen = ["A-D", "A-B", "B-C", "D-E"];
  const step = useLoop(chosen.length + 1, 800);
  const taken = new Set(chosen.slice(0, step));
  return (
    <Svg caption={`Жадно берём самые лёгкие рёбра без циклов`} >
      {edges.map((e, i) => {
        const on = taken.has(`${e.u}-${e.v}`);
        return (
          <g key={i}>
            <Edge a={pos[e.u]} b={pos[e.v]} color={on ? "red" : "black"} />
            <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y={pos[e.v][1]}
              textAnchor="middle" fontSize={9} fontWeight="bold">{e.u}-{e.v}</text>
          </g>
        );
      })}
      {Object.entries(pos).map(([k, [x, y]]) => (<Node key={k} x={x} y={y} label={k} fill={k} stroke={k} />))}
    </Svg>
  );
};

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 100], E: [180, 100]
  };
  const edges = [
    { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
    { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
  ];
  const chosen = ["A-D", "A-B", "B-C", "D-E"];
  const step = useLoop(chosen.length + 1, 800);
  const taken = new Set(chosen.slice(0, step));
  return (
    <Svg caption={`Жадно берём самые лёгкие рёбра без циклов`} >
      {edges.map((e, i) => {
        const on = taken.has(`${e.u}-${e.v}`);
        return (
          <g key={i}>
            <Edge a={pos[e.u]} b={pos[e.v]} color={on ? "red" : "black"} />
            <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y={pos[e.v][1]}
              textAnchor="middle" fontSize={9} fontWeight="bold">{e.u}-{e.v}</text>
          </g>
        );
      })}
      {Object.entries(pos).map(([k, [x, y]]) => (<Node key={k} x={x} y={y} label={k} fill={k} stroke={k} />))}
    </Svg>
  );
};

```

```

        <Node key={i} x={pos[i][0]} y={pos[i][1]} label={label[i]} />
      )}}
    </Svg>
  );
};

```

```

export const StackDemo: React.FC = () => {
  const states = [["A"], ["A", "B"], ["A", "B", "C"], ["A", "B", "C", "D"]];
  const step = useLoop(states.length, 800);
  const items = states[step];
  const popping = step >= 3;
  return (
    <Svg caption={popping ? "pop: забираем ВЕРХНИЮЮ табулицу" : ""}>
      <rect x={90} y={22} width={80} height={96} rx={6} />
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={94} y={100 - i * 26} width={72} height={26} />
          <rect x={94} y={100 - i * 26} width={72} height={26} fill={i === items.length - 1 ? (popping ? "red" : "blue")} />
          <text x={130} y={111 - i * 26} textAnchor="middle">{v}</text>
        </g>
      ))}
      <text x={130} y={16} textAnchor="middle" fontSize={14}>{popping ? "pop" : "push"}</text>
    </Svg>
  );
};

```

```

export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C", "D"], ["C", "D", "E"]];
  const step = useLoop(states.length, 800);
  const items = states[step];
  return (
    <Svg caption="pop_front слева, push_back справа – к концу" />
      <defs>
        <marker id="mdArrow" markerWidth="6" markerHeight="6" />
        <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
      </defs>
      <text x={12} y={44} fontSize={9} fill={C.dim}>выход</text>
    </Svg>
  );
};

```

```

    { id: 4, x: 102, y: 104, ch: "с", parent: 1 },
    { id: 5, x: 188, y: 104, ch: "с", parent: 2 },
  ];

export const TrieDemo: React.FC = () => {
  const step = useLoop(TRIE_NODES.length + 1, 700);
  return (
    <Svg caption="Путь от корня по буквам = слово; общие  

      {TRIE_NODES.filter((n) => n.parent !== null).map(
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
      ]} }
      {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x}
    </Svg>
  );
};

export const TrieBfsDemo: React.FC = () => {
  const levels = [[0], [1, 2], [3, 4, 5]];
  const step = useLoop(levels.length + 1, 950);
  const doneIds = new Set(levels.slice(0, step).flat());
  const wave = step > 0 && step <= levels.length ? levels[step - 1] : [];
  return (
    <Svg caption={`BFS по бору: уровень ${Math.max(0, M
      {TRIE_NODES.filter((n) => n.parent !== null).map(
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
      ]} }
      {TRIE_NODES.map((n) => (
        <Node key={n.id} x={n.x} y={n.y} label={n.ch}
          fill={wave.includes(n.id) ? C.active : doneId
        )} }
      <text x={6} y={12} fontSize={9} fill={C.dim}>очер
    </Svg>
  );
};

export const SuffixLinkDemo: React.FC = () => {

```

```

        <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={n.h}
          fill={n.lvl === step ? C.active : C.idle} stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }}
        <text x={n.x} y={n.y} textAnchor="middle" domClass="text">{n.val}</text>
      </g>
    )}
  </Svg>
);
};

```

```

export const PrefixSumDemo: React.FC = () => {
  const a = [3, 1, 4, 1, 5, 9];
  const pref = a.reduce<number[]>((acc, v) => [...acc, acc.length > 0 ? acc[acc.length - 1] + v : v], []);
  const step = useLoop(4, 1200);
  const l = 1, r = 3;
  const show = step >= 1;
  return (
    <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] - P[${l}]` : "о префиксные суммы P"}>
      <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
      {a.map((v, i) => (
        <g key={i}>
          <rect x={20 + i * 39} y={24} width={34} height={12}
            fill={show && i >= l && i <= r ? C.active : C.idle} stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }}
          <text x={37 + i * 39} y={36} textAnchor="middle" domClass="text">{v}</text>
        </g>
      ))}
      <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
      {pref.map((v, i) => (
        <g key={i}>
          <rect x={20 + i * 33} y={78} width={29} height={12}
            fill={show && i === l ? C.bad : show && i <= r ? C.active : C.idle}
            stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }}
          <text x={34 + i * 33} y={90} textAnchor="middle" domClass="text">{v}</text>
        </g>
      ))}
    </Svg>
  );
};

```



```
};
```

```
export const NpDemo: React.FC = () => {
  const step = useLoop(2, 1400);
  return (
    <Svg caption={step === 0 ? "Найти решение – долго ("}
      <ellipse cx={130} cy={64} rx={112} ry={48} fill="
      <ellipse cx={92} cy={64} rx={52} ry={32} fill="#3
      <text x={92} y={64} textAnchor="middle" dominantB
      <text x={206} y={34} textAnchor="middle" fontSize
      <circle cx={196} cy={84} r={13} fill={step === 1
      <text x={196} y={84} textAnchor="middle" dominant
        ?"}</text>
      <text x={130} y={124} textAnchor="middle" fontSize
    </Svg>
  );
};
```

```
export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);
  const i = Math.max(0, step - 1);
  const len = step > 0 ? pi[i] : 0;
  return (
    <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс
      {s.split("").map((ch, idx) => (
        <g key={idx}>
          <rect x={18 + idx * 33} y={26} width={28} hei
            fill={step > 0 && (idx < len || (idx > i -
            style={{ transition: "fill .25s" }} />
          <text x={32 + idx * 33} y={40} textAnchor="mi
            ext>
          <text x={32 + idx * 33} y={70} textAnchor="mi
            fill={idx <= i && step > 0 ? C.warn : C.idle
        </g>
      )}}
    <text x={4} y={40} fontSize={8} fill={C.dim}>s</t
```

ФАЙЛ 51/111: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
import {
  AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
  SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
} from "../demosStruct";
import {
  ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
  ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
  | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
  | "suffix-link" | "toposort" | "relax" | "prefix-sum"
  | "segment-tree" | "bridge" | "articulation" | "mst"
  | "prefix-function" | "dp" | "floyd" | "components" |
  | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
  bfs: BfsDemo,
  dfs: DfsDemo,
  heap: HeapDemo,
  stack: StackDemo,
  queue: QueueDemo,
  dsu: DsuDemo,
  trie: TrieDemo,
  "trie-bfs": TrieBfsDemo,
  "suffix-link": SuffixLinkDemo,
  toposort: ToposortDemo,
  relax: RelaxDemo,
  "prefix-sum": PrefixSumDemo,
  "sparse-table": SparseTableDemo,
  "segment-tree": SegmentTreeDemo,
```

ФАЙЛ 52/111: src/components/hints/SimulatorModal.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../../data/vizStepBuilder";
import { VizLiveStatus } from "../VizLiveStatus";
```

```
interface Props {
  vizId: string | null;
  onClose: () => void;
}
```

```
/** Модальное окно с полноценным симулятором – вызывает  
export const SimulatorModal: React.FC<Props> = ({ vizId
```

```
  useEffect(() => {
    if (!vizId) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") onClose();
    };
    window.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
      window.removeEventListener("keydown", onKey);
      document.body.style.overflow = "";
    };
  }, [vizId, onClose]);
```

```
const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;
```

```
return createPortal(
  <div className="fixed inset-0 z-[110] flex items-end
```

```

-----
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";

export interface HintAnchor {
  termId: string;
  rect: DOMRect;
}

interface Props {
  anchor: HintAnchor | null;
  onClose: () => void;
  onOpenSimulator: (vizId: string) => void;
  onCardEnter: () => void;
  onCardLeave: () => void;
}

const CARD_W = 320;
const GAP = 10;

export const TermPopover: React.FC<Props> = ({ anchor,
  const cardRef = useRef<HTMLDivElement>(null);
  const [pos, setPos] = useState<{ top: number; left: number}>({
    top: 0, left: 0,
  });

  useEffect(() => {
    if (!anchor) {
      setPos(null);
      return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300;

    let left = anchor.rect.left + anchor.rect.width / 2;
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
  }, [anchor, cardRef]);

```

```

<div className="bg-slate-900 border border-indigo-500" >
  <div className="flex items-start gap-2 px-3 pt-3" >
    <h4 className="text-sm font-bold text-indigo-500" > Подсказка
    <button
      onClick={onClose}
      className="text-slate-500 hover:text-white"
      aria-label="Заккрыть подсказку"
    >
      <X className="w-4 h-4" />
    </button>
  </div>

  <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
    <p className="text-[13px] leading-relaxed text-slate-300" >
      {entry.demo} && <MiniDemo kind={entry.demo} />
    </p>
    {entry.formal && (
      <p className="text-[11.5px] leading-relaxed text-slate-300" >
        {entry.formal}
      </p>
    )}
    {entry.complexity && (
      <div className="text-[11px] font-mono text-slate-300" >
        Сложность: {entry.complexity}
      </div>
    )}
    {viz && simulatorId && (
      <button
        onClick={() => onOpenSimulator(simulatorId)}
        className="w-full flex items-center justify-center text-slate-300 rounded-lg py-2 transition-colors"
      >
        <Play className="w-3.5 h-3.5" /> Показать
      </button>
    )}
  </div>
</div>

```

```

    { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 1 };

```

```

const isNodeVisible = (id: string) => {
  if (id === 'S' && (step === 0 || step === 7)) return true;
  return false;
};

```

```

const getPotential = (id: string) => {
  if (step >= 2 && step < 7) {
    if (id === 'S') return 0;
    if (id === 'A') return 0;
    if (id === 'B') return -2;
    if (id === 'C') return -1;
  }
  return null;
};

```

```

const getEdgeColorClass = (e: any) => {
  if (e.u === 'S') {
    if (step === 0 || step === 7) return "opacity-0";
    if (step === 1) return "stroke-amber-500 stroke-dasharray-4";
    if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
    return "stroke-slate-700 stroke-dasharray-4";
  }

  if (e.id === 'AB') {
    if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
    if (step > 4) return "stroke-emerald-500";
  }
  if (e.id === 'BC') {
    if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
    if (step > 5) return "stroke-emerald-500";
  }
  if (e.id === 'CA') {
    if (step === 6) return "stroke-amber-400 stroke-dasharray-4";
    if (step > 6) return "stroke-emerald-500";
  }
};

```

```

        if (step < 6) return e.weight;
        if (step === 6) return `${e.weight} + (${ -1
        return e.newWeight;
    }
    return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
    if (e.id === 'AB' && step === 4) return true;
    if (e.id === 'BC' && step === 5) return true;
    if (e.id === 'CA' && step === 6) return true;
    return false;
};

const isEdgeTextEmerald = (e: any) => {
    if (e.id === 'AB' && step > 4) return true;
    if (e.id === 'BC' && step > 5) return true;
    if (e.id === 'CA' && step > 6) return true;
    return false;
};

const getDesc = () => {
    switch (step) {
        case 0: return (
            <div>
                <b>Исходный граф.</b> Имеется ребро
                Если мы запустим быстрый алгоритм Д
            </div>
        );
        case 1: return (
            <div>
                <b>Шаг 1: Точка отсчета.</b><br/>
                Добавляем фиктивную вершину S. Соед
                ми</span>.
                Это наша база для расчета потенциал
            </div>
        );
        case 2: return (

```

```

        <b>Готово!</b> Фиктивную вершину S
        Все новые веса ребер в графе стали
        При этом старые кратчайшие пути оста
шин!
    </div>
  );
  default: return "";
}
};

return (
  <div className="bg-slate-800 p-4 rounded-xl border
    <h4 className="text-sm md:text-base font-bo
      <span className="w-2.5 h-2.5 bg-amber-400
        Визуализация: Перевзвешивание алгоритмом
    </h4>

    <div className="flex flex-col md:flex-row gap-4
      <div className="bg-[#0f172a] flex-1 min-h-120
        nter p-4">
          <svg width="400" height="300" class="
            <defs>
              <marker id="arrow-slate" viewBox="0 0 10 5"
                start-reverse">
                  <path d="M 0 0 L 10 5 L 5 10" />
                </marker>
              <marker id="arrow-slate-dim" viewBox="0 0 10 5"
                uto-start-reverse">
                  <path d="M 0 0 L 10 5 L 5 10" />
                </marker>
              <marker id="arrow-amber" viewBox="0 0 10 5"
                start-reverse">
                  <path d="M 0 0 L 10 5 L 5 10" />
                </marker>
              <marker id="arrow-pink" viewBox="0 0 10 5"
                tart-reverse">
                  <path d="M 0 0 L 10 5 L 5 10" />
                </marker>

```



```

                                {getEdgeContent
                                </text>
                                </g>
                                )
                                }}}

    {/* Nodes */}
    {nodes.map(n => {
      if (!isNodeVisible(n.id)) r
      const pot = getPotential(n.)

      return (
        <g key={n.id} className=
          <circle
            cx={n.x} cy={n.y}
            className={`stroke-
              ${n.id === 'S'
                ? 'fill-[#202020]
                : 'fill-slate-500'
              }
          />
          <text x={n.x} y={n.y}
            -white">
              {n.label}
            </text>

            {pot !== null && (
              <g transform={`
fade-in zoom-in duration-500">
                <rect x="-100" y="-100"
k-500 stroke-1" />
                <text x="0" y="0"
font-bold fill-pink-400">
                  {pot}
                </text>
              </g>
            )}
          </g>
        )
      )
    })
  }
)

```

```
        }}}
      </div>
    </div>
  </div>
</div>
);
}
```

ФАЙЛ 55/111: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...";
import {
  Play,
  Pause,
  RotateCcw,
  Layers,
} from "lucide-react";

interface GNode {
  id: number;
  x: number;
  y: number;
  label: string;
}

interface GEdge {
  u: number;
  v: number;
}

const initialNodes: GNode[] = [
  { id: 0, x: 150, y: 100, label: "0" },
  { id: 1, x: 300, y: 100, label: "1" },
```

```

const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
  // Generate step-by-step Kosaraju steps
  const generatedSteps: AlgoStep[] = [];
  const visited = new Set<number>();
  const order: number[] = [];
  const sccMap: Record<number, number> = {};

  const recordStep = (
    phase: AlgoStep["phase"],
    desc: string,
    currentNode: number | null,
    transposed: boolean,
    currentSccId: number,
  ) => {
    generatedSteps.push({
      phase,
      desc,
      visited: new Set(visited),
      currentNode,
      order: [...order],
      transposed,
      sccMap: { ...sccMap },
      currentSccId,
      activeEdges: transposed
        ? initialEdges.map((e) => ({ u: e.v, v: e.u })
          : [...initialEdges],
    ));
  };

  recordStep(
    "init",
    "Инициализация алгоритма Косарайю. Начинаем первый",
    null,
    false,
    -1,
  );

```

```

const topoOrder = [...order].reverse();
recordStep(
  "transpose",
  `Первый проход завершен! Стек выхода в топологиче
    ебра.`,
  null,
  true,
  -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
  visited.add(u);
  sccMap[u] = currentScc;
  recordStep(
    "dfs2",
    `DFS 2: заходим в ${u} на транспонированном гра
    u,
    true,
    currentScc,
  );

  for (const to of adjT[u]) {
    if (!visited.has(to)) {
      dfs2(to, currentScc);
    }
  }
};

// Run DFS2 using the order (reversed from back to

```

```

    visited: new Set(),
    currentNode: null,
    order: [],
    transposed: false,
    sccMap: {},
    currentSccId: -1,
    activeEdges: initialEdges,
  });

  useEffect(() => {
    let timer: NodeJS.Timeout;
    if (isPlaying && currentStepIdx < steps.length - 1)
      timer = setInterval(() => {
        setCurrentStepIdx((prev) => prev + 1);
      }, 1500);
    } else {
      setIsPlaying(false);
    }
    return () => clearInterval(timer);
  }, [isPlaying, currentStepIdx, steps.length]);

  const togglePlay = () => setIsPlaying(!isPlaying);
  const reset = () => {
    setIsPlaying(false);
    setCurrentStepIdx(0);
  };

  const getSccColor = (id: number) => {
    if (id === 1) return "fill-emerald-600 stroke-emera
    if (id === 2) return "fill-indigo-600 stroke-indigo
    return "fill-slate-800 stroke-slate-500";
  };

  return (
    <div className="bg-slate-900 rounded-xl border bord
      <div className="bg-slate-800 p-4 border-b border-
        <h3 className="font-bold text-white flex items-
          <Layers className="w-5 h-5 text-emerald-400"

```

```

        refX="22"
        refY="5"
        markerWidth="6"
        markerHeight="6"
        orient="auto-start-reverse"
    >
    <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
</marker>
<marker
    id="arrow-active"
    viewBox="0 0 10 10"
    refX="22"
    refY="5"
    markerWidth="6"
    markerHeight="6"
    orient="auto-start-reverse"
>
    <path d="M 0 1 L 10 5 L 0 9 z" fill="#1
</marker>
<marker
    id="arrow-transposed"
    viewBox="0 0 10 10"
    refX="22"
    refY="5"
    markerWidth="6"
    markerHeight="6"
    orient="auto-start-reverse"
>
    <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
</marker>
</defs>

```

```

{ /* Edges */ }
{initialEdges.map((edge, idx) => {
    const u = initialNodes.find((n) => n.id === edge.u)
    const v = initialNodes.find((n) => n.id === edge.v)

    // If transposed, draw reversed coordinates

```

```

        stroke={stroke}
        strokeWidth="2.5"
        markerEnd={`url(#${markerId})`}
        className="transition-all duration-300"
      />
    );
  }}}

{/* Nodes */}
{initialNodes.map((node) => {
  const isCurrent = step.currentNode === node.id;
  const isVisited =
    step.visited.has(node.id) ||
    (step.phase === "dfs1" && step.currentNode === node.id);
  const sccId = step.sccMap[node.id];

  let classes = "fill-slate-800 stroke-slate-800";
  if (sccId) {
    classes = getSccColor(sccId);
  } else if (isCurrent) {
    classes = "fill-amber-600 stroke-amber-600";
  } else if (isVisited && step.phase === "dfs2") {
    classes = "fill-emerald-700 stroke-emerald-700";
  }

  return (
    <g key={node.id}>
      <circle
        cx={node.x}
        cy={node.y}
        r="20"
        className={classes}
        strokeWidth="3"
      />
      <text
        x={node.x}
        y={node.y}
        dy=".3em"

```

Порядок обхода и стек

</h4>

```
<div className="bg-slate-900 border border-slate-800" style={step.desc}
  <p className="text-xs text-white leading-relaxed" style={step.desc}
    {step.desc}
  </p>
</div>
```

```
<div className="flex-1 space-y-4 overflow-y-auto" style={{
  /* Top-Sort output / Order stack representation */
  <div>
    <span className="text-[10px] text-slate-500" style={{
      CТЕК ВЫХОДА (DFS 1 ORDER):
    </span>
    <div className="flex flex-wrap gap-1 bg-slate-900" style={{
      {step.order.length === 0 ? (
        <span className="text-slate-600 text-sm" style={{
        ) : (
          [...step.order].reverse().map(({val, idx}) => {
            <div
              key={idx}
              className="bg-indigo-950 border border-slate-800" style={{
            ex items-center gap-1"
            >
              {val}
            </div>
          })
        )}
      </div>
    </div>
```

```
{/* List of actions log */}
```

```
<div>
  <span className="text-[10px] text-slate-500" style={{
    ЛОГ ОПЕРАЦИЙ:
  </span>
  <div className="space-y-1.5 max-h-[150px]" style={{
```



```
        </div>
      </div>
    );
  };
};
```

ФАЙЛ 56/111: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useCallback, useContext, useEffect, useId } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizStepBus } from '../data/vizStepBus';
```

```
interface Vertex {
  id: number;
  label: string;
  x: number;
  y: number;
  color: string; // 'none', 'red', 'blue', 'purple', 'magenta'
}
```

```
interface Edge {
  id: string;
  u: number;
  v: number;
  weight: number;
  status: 'pending' | 'inspecting' | 'included' | 'rejected';
  color: string;
}
```

```
interface StepLog {
  title: string;
  description: string;
  type: 'info' | 'success' | 'warning' | 'error';
}
```

```

const [logs, setLogs] = useState<StepLog[]>([
  {
    title: 'Введение в раскраску (Краскал)',
    description: '«Краскал – это раскраска?». Предста
      овы начинать!',
    type: 'info',
  },
]);

// --- KRUSKAL STEPS ---
const kruskalSteps = [
  // Step 1: Inspect A-B (2)
  () => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
      title: 'Шаг 1: Берем самое дешевое ребро A-B (в
      description: 'Оно соединяет две бесцветные верши
      type: 'info'
    }, ...prev]));
  },
  // Step 2: Include A-B
  () => {
    setVertices(prev => prev.map(v => v.id === 0 || v
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
      title: 'Успех: Ребро A-B в осто
      description: 'Вершины A and B окрашены в красны
      type: 'success'
    }, ...prev]));
  },
  // Step 3: Inspect D-E (3)
  () => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
      title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
      description: 'Оно соединяет две другие бесцветн
      type: 'info'
    }, ...prev]));
  },

```

```

        type: 'warning'
      }, ...prev]));
    },
    // Step 8: Include A-D (Merge to purple)
    () => {
      setVertices(prev => prev.map(v => v.color === 'blue' ? 'purple' : v.color));
      setEdges(prev => prev.map(e => e.status === 'included' ? 'excluded' : e.status));
      setLogs(prev => [{
        title: 'Слияние кланов: Ребро A-D добавлено',
        description: 'Мы объединили красный и синий кланы',
        type: 'success'
      }, ...prev]));
    },
    // Step 9: Inspect B-E (6) - Cycle!
    () => {
      setEdges(prev => prev.map(e => e.id === '1-4' ? {status: 'excluded'} : e));
      setLogs(prev => [{
        title: 'Шаг 5: Берем ребро B-E (вес 6)',
        description: 'НО: это ребро соединяет вершины B и E, которые уже соединены!',
        type: 'warning'
      }, ...prev]));
    },
    // Step 10: Reject B-E
    () => {
      setEdges(prev => prev.map(e => e.id === '1-4' ? {status: 'excluded'} : e));
      setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Мы выбрасываем ребро B-E. Иначе получится цикл!',
        type: 'error'
      }, ...prev]));
    },
    // Step 11: Inspect E-F (7)
    () => {
      setEdges(prev => prev.map(e => e.id === '4-5' ? {status: 'excluded'} : e));
      setLogs(prev => [{
        title: 'Шаг 6: Берем ребро E-F (вес 7)',
        description: 'Оно соединяет фиолетовую вершину E и синюю вершину F',
        type: 'info'
      }, ...prev]));
    }
  ],
  ...prev
);

```

```

    }, ...prev]));
  },
  // Step 16: Reject C-F
  () => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
      title: 'Отказ: Цикл обнаружен!',
      description: 'Выбрасываем ребро C-F.',
      type: 'error'
    }, ...prev]));
  },
  // Step 17: Inspect G-H (10)
  () => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
      title: 'Шаг 9: Берем ребро G-H (вес 10)',
      description: 'Соединяет фиолетовую G и бесцветную H.',
      type: 'info'
    }, ...prev]));
  },
  // Step 18: Include G-H
  () => {
    setVertices(prev => prev.map(v => v.id === 7 ? {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
      title: 'Успех: Ребро G-H в остове',
      description: 'Вершина H перекрашена в фиолетовую.',
      type: 'success'
    }, ...prev]));
  },
  // Step 19: Inspect E-H (11) - Cycle!
  () => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
      title: 'Шаг 10: Берем ребро E-H (вес 11)',
      description: 'Оба конца E и H уже фиолетовые. Соединение создаст цикл!',
      type: 'warning'
    }, ...prev]));
  },

```

```

    setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5']));
    setLogs(prev => [{
      title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
      description: 'Мы выбрали вершину E как стартовую вершину. Добавляем в очередь (Heap): D-E(3), B-E(6), E-F(7)',
      type: 'info'
    }, ...prev]);
  },
  // Step 2: Pop D-E (3)
  () => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
      id: '3-4', status: 'in_heap'
    } : e));
    setLogs(prev => [{
      title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (3)',
      description: 'Плесень тянется по ребру D-E к вершине D',
      type: 'info'
    }, ...prev]);
  },
  // Step 3: Include D-E, add D's edges to heap
  () => {
    setVertices(prev => prev.map(v => v.id === 3 ? {
      id: 3, status: 'in_heap'
    } : v));
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
      id: '3-4', status: 'in_heap'
    } : e));
    setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-E (3)']);
    setLogs(prev => [{
      title: 'Успех: Плесень захватила вершину D',
      description: 'В кучу добавлены новые пути из D: A-D (5), B-E (6), E-F (7)',
      type: 'success'
    }, ...prev]);
  },
  // Step 4: Pop A-D (5)
  () => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
      id: '0-3', status: 'in_heap'
    } : e));
    setLogs(prev => [{
      title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
      description: 'Плесень расширяется в сторону вершины A',
      type: 'info'
    }, ...prev]);
  },

```

```

        description: 'Плесень тянется к вершине C.',
        type: 'info'
    }, ...prev]);
},
// Step 9: Include B-C, add C's edges
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
        eap' } : e));
    setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
    setLogs(prev => [{
        title: 'Успех: Вершина C захвачена',
        description: 'Ребро C-F(9) добавлено в кучу.',
        type: 'success'
    }, ...prev]);
},
// Step 10: Pop B-E (6) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{
        title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
        description: 'ВНИМАНИЕ! Плесень проверяет вершину E',
        type: 'warning'
    }, ...prev]);
},
// Step 11: Reject B-E
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'B-E (6)']);
    setLogs(prev => [{
        title: 'Отказ: Игнорируем внутреннее ребро',
        description: 'Ребро B-E отброшено. Плесень не р
        type: 'error'
    }, ...prev]);
},
// Step 12: Pop E-F (7)
() => {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {

```

```

    },
    // Step 16: Pop C-F (9) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '2-5' ? {
            setLogs(prev => [{
                title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
                description: 'Вершины C и F уже заражены плесенью',
                type: 'warning'
            }, ...prev]));
    },
    // Step 17: Reject C-F
    () => {
        setEdges(prev => prev.map(e => e.id === '2-5' ? {
            setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
            setLogs(prev => [{
                title: 'Отказ: Игнорируем ребро C-F',
                description: 'Выбрасываем из кучи.',
                type: 'error'
            }, ...prev]));
    },
    // Step 18: Pop G-H (10)
    () => {
        setEdges(prev => prev.map(e => e.id === '6-7' ? {
            setLogs(prev => [{
                title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
                description: 'Плесень ползет к вершине H.',
                type: 'info'
            }, ...prev]));
    },
    // Step 19: Include G-H, add H's edges
    () => {
        setVertices(prev => prev.map(v => v.id === 7 ? {
            setEdges(prev => prev.map(e => e.id === '6-7' ? {
                eap' } : e));
            setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)'])
            setLogs(prev => [{
                title: 'Успех: Вершина H захвачена',
                description: 'В кучу добавлено H-I(12).',

```

```

        description: 'Все 9 вершин поглощены. Итоговый
            иной — мы разрастались из одной точки с помо
        type: 'success'
    }, ...prev]);
},
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
    // Step 1: Five-Year Plan 1 (Inspecting outgoing)
    () => {
        // Highlight the cheapest outgoing edges for EACH
        // Node 0(A) -> A-B(2)
        // Node 1(B) -> A-B(2)
        // Node 2(C) -> B-C(4)
        // Node 3(D) -> D-E(3)
        // Node 4(E) -> D-E(3)
        // Node 5(F) -> E-F(7)
        // Node 6(G) -> D-G(8)
        // Node 7(H) -> G-H(10)
        // Node 8(I) -> H-I(12)
        setEdges(prev => prev.map(e =>
            ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8
        ));
        setLogs(prev => [{
            title: 'Первая пятилетка: Каждая деревня выбира
            description: 'Каждая из 9 деревень одновременно
                А выбирает A-B(2), D выбирает D-E(3), а G вы
                !',
            type: 'info'
        }, ...prev]);
    },
    // Step 2: Concurrently merge into Kolkhozes (Compo
    () => {
        // Concurrently build chosen roads. Merge into:
        // Component 1: {A, B, C} (colored red)
        // Component 2: {D, E, F, G, H, I} (colored gold/
        setVertices(prev => prev.map(v =>

```



```

    setLogs(prev => [{
      title: 'Объединение завершено: Один гигантский ',
      description: 'Оба колхоза объединились по мосту ',
      type: 'success'
    }, ...prev]));
  },
  // Step 5: Check remaining edges & mark as rejected
  () => {
    setEdges(prev => prev.map(e => e.status === 'pend
    setLogs(prev => [{
      title: '    Успех: MST построен Борувкой за 2 шаг
      description: 'Все лишние ребра (В-Е, Е-Н, С-Е,
        вес: 51. Благодаря параллельности, мы потрати
      type: 'success'
    }, ...prev]));
  }
];

const currentSteps =
  activeAlgo === 'kruskal' ? kruskalSteps :
  activeAlgo === 'prim' ? primSteps :
  boruvkaSteps;

const handleNextStep = () => {
  if (stepIndex < currentSteps.length) {
    currentSteps[stepIndex]();
    setStepIndex(stepIndex + 1);
  }
};

const introLog = {algo: 'kruskal' | 'prim' | 'boruvka
  if (algo === 'kruskal') {
    return [{
      title: 'Введение в раскраску (Краскал)',
      description: 'Представь, что все 9 вершин графа
      type: 'info',
    }
  ]
}
```

```

    setVertices(initialVertices);
    setEdges(initialEdges);
    setHeapQueue([]);
    setLogs(introLog(activeAlgo));
    for (let i = 0; i < target; i++) currentSteps[i]();
    setStepIndex(target);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [activeAlgo, currentSteps]);

  /** Номер кадра из переменных кода: i / step / число
const selectByVariables = useCallback<VizStepSelector>(
  const i = vizNumber(v.i) ?? vizNumber(v.step);
  if (i !== null) return i + 1;
  const mst = vizArray(v.mst) ?? vizArray(v.answer) ??
  if (mst) return mst.length;
  const taken = vizArray(v.taken) ?? vizArray(v.used) ??
  if (taken) return Math.max(0, taken.length - 1);
  return null;
}, []);

useVizStepSync(stepIndex, goToStep, currentSteps.length);

/** A..I ↔ id 0..8 */
const idOf = (name: string | number | null | undefined):
  if (name === null || name === undefined) return null;
  if (typeof name === 'number') return Number.isInteger(name) ?
  const c = name.trim().toUpperCase().replace(/[^A-I]/g, '');
  return c ? c.charCodeAt(0) - 65 : null;
};

/** Разбор одного ребра из кода: (2, 'A', 'B') / [2, 'A', 'B']
const parseEdge = (item: unknown): { u: number; v: number;
  if (item === null || item === undefined) return null;
  if (typeof item === 'string') {
    const nums = item.match(/-?\d+(?:\.\d+)?/g);
    const letters = item.match(/[A-Ia-i]/g);
    let u: number | null = null;
    let v: number | null = null;

```

```

    /** То же, но только если метка – вершина графа: по
    const parentIdx = new Map<number, number>();
    const feed = (id: number, val: unknown) => {
        const label = String(typeof val === 'string' ? va
        rawComp.set(id, label);
        const inner = typeof val === 'string' ? val : typ
        const pid = idOf(inner);
        if (pid !== null) parentIdx.set(id, pid);
    };
    if (parentRec) {
        Object.entries(parentRec).forEach(([k, val]) => {
            const id = idOf(k);
            if (id === null) return;
            feed(id, val);
        });
    } else if (parentArr) {
        parentArr.forEach((val, id) => {
            if (id > 8) return;
            feed(id, val);
        });
    }
    /**
    * Ключ множества вершины: если код дал DSU-массив
    * идём по цепочке до корня – иначе вершины одной к
    * разные цвета. Для произвольных меток («comp1») б
    */
    const compOfVertex = new Map<number, string>();
    rawComp.forEach((label, id) => {
        if (parentIdx.size === 0) {
            compOfVertex.set(id, label);
            return;
        }
        let cur = id;
        const seen = new Set<number>([id]);
        for (let hop = 0; hop < 16; hop++) {
            const nxt = parentIdx.get(cur);
            if (nxt === undefined || nxt === cur || seen.has(nxt))
                seen.add(nxt);

```

```

        return { ...e, status: 'pending' as const };
    });
    // eslint-disable-next-line react-hooks/exhaustive-deps
    }, [edges, live]));

const effVertices: Vertex[] = useMemo(() => {
    if (!live) return vertices;
    return vertices.map((v) => {
        const root = live.compOfVertex.get(v.id);
        if (root !== undefined) return { ...v, color: color };
        if (live.takenVertices.includes(v.id)) return { ...v, color: 'none' };
        if (live.curU === v.id || live.curV === v.id) return { ...v, color: 'none' };
        return { ...v, color: 'none' };
    });
    // eslint-disable-next-line react-hooks/exhaustive-deps
    }, [vertices, live]);

const getColorClass = (color: string, id: number) => {
    switch (color) {
        case 'red': return 'bg-red-500 border-red-300 text-white';
        case 'blue': return 'bg-blue-500 border-blue-300 text-white';
        case 'purple': return 'bg-purple-600 border-purple-300 text-white';
        case 'mold': return 'bg-emerald-500 border-emerald-300 text-white';
        case 'gold': return 'bg-amber-400 border-amber-200 text-white';
        default:
            if (activeAlgo === 'prim' && id === 4 && stepIn === 1)
                return 'bg-slate-700 border-emerald-500 text-white';
            return 'bg-slate-700 border-slate-500 text-slate-200';
    }
};

const getEdgeStroke = (status: string, color: string) => {
    if (status === 'inspecting') return '#f59e0b';
    if (status === 'rejected') return '#ef4444';
    if (status === 'in_heap') return '#0284c7';
    if (status === 'included') {
        if (color === 'red') return '#ef4444';
    }
};

```

```

        >
          <GitGraph className="w-3.5 h-3.5" />
          <span>Краскал (Билет 18)</span>
        </button>
        <button
          onClick={() => handleReset('prim')}
          className={
            "flex items-center gap-1.5 px-3 py-2
+
            (activeAlgo === 'prim'
              ? 'bg-emerald-600 text-white shadow
              : 'text-slate-400 hover:text-slate-:
            )
          >
          <Layers className="w-3.5 h-3.5" />
          <span>Прима (Билет 19)</span>
        </button>
        <button
          onClick={() => handleReset('boruvka')}
          className={
            "flex items-center gap-1.5 px-3 py-2
+
            (activeAlgo === 'boruvka'
              ? 'bg-rose-600 text-white shadow-md
              : 'text-slate-400 hover:text-slate-:
            )
          >
          <Users className="w-3.5 h-3.5" />
          <span>Борувка (Билет 20)</span>
        </button>
      </div>

    <button
      onClick={() => handleReset(activeAlgo)}
      className="flex items-center justify-centr
-200 font-semibold text-xs rounded-xl borde
    >
    <RotateCcw className="w-3.5 h-3.5" />

```

```

        <span className="text-[11px] font-mono" style="font-family: monospace;">
            в осто́ве: {live.takenEdges.map((e) =>
        </span>
    ))
    {live.compOfVertex.size > 0 && (
        <span className="text-[11px] font-mono" style="font-family: monospace;">
            компонент: {new Set(Array.from(live.c
        </span>
    ))
    {live.takenVertices.length > 0 && (
        <span className="text-[11px] font-mono" style="font-family: monospace;">
            дерево выросло: {live.takenVertices.m
        </span>
    ))
    {live.heapEdges.length > 0 && (
        <span className="text-[11px] font-mono" style="font-family: monospace;">
            в куче: {live.heapEdges.length}
        </span>
    ))
    {live.cheapestEdges.length > 0 && (
        <span className="text-[11px] font-mono" style="font-family: monospace;">
            cheapest: {live.cheapestEdges.length}
        </span>
    ))
    {live.total !== null && (
        <span className="text-[11px] font-mono" style="font-family: monospace;">
            вес осто́ва = {live.total}
        </span>
    ))
</>
) : (
    <span className="text-[11px] text-slate-500" style="font-family: monospace;">
        открой панель Python и запусти код вкладки
        ка'}»: переменные
        {' '}<code className="font-mono text-purple" style="font-family: monospace;">
e className="font-mono text-purple-300">u</code>
mono text-purple-300">w</code> перекрасят э
    </span>

```

```

        <div className="w-3 h-3 rounded-full" />
      </div>
      <div className="flex items-center gap-2" />
        <div className="w-3 h-3 rounded-full" />
      </div>
      <div className="flex items-center gap-2" />
        <div className="w-3 h-3 rounded-full" />
      </div>
    </>
  )}
</div>

<div className="absolute top-4 right-4 bg-slate-300" />
  {currentSteps.map((step, stepIndex) => {
    <div key={stepIndex} />
  })}
</div>

<div className="absolute inset-0 w-full h-full" />
  {effEdges.map(edge => {
    const u = effVertices[edge.u];
    const v = effVertices[edge.v];
    const strokeColor = getEdgeStroke(edge);
    const isInspecting = edge.status === 'inspect';
    const isRejected = edge.status === 'rejected';
    const isInHeap = edge.status === 'in_heap';

    return (
      <g key={edge.id}>
        <line
          x1={u.x + "%"}
          y1={u.y + "%"}
          x2={v.x + "%"}
          y2={v.y + "%"}
          stroke={strokeColor}
          strokeWidth={isInspecting ? 4.5 : 1}
          strokeDasharray={isRejected ? '4, 4' : ''}
        />
      </g>
    );
  })}
</div>

```

```

</div>

{stepIndex >= currentSteps.length && (
  <div className="absolute inset-x-6 bottom-6
    shadow-xl z-20">
    <p className={"font-bold text-base " +
      'ld-300' : 'text-rose-300'}>
      MST построено с помощью {activeAlgo}
    </p>
    <p className="text-slate-300 text-xs mt-1">
      Общий вес минимального остова: 51. Всего
    </p>
  </div>
)}
</div>

{/* Logs / Heap */}
<div className="bg-slate-950 rounded-2xl border-b
  <div className="p-4 bg-slate-900 border-b
    <h4 className="font-bold text-slate-200">
      Ход выполнения
    </h4>
  </div>

  {activeAlgo === 'prim' && (
    <div className="p-3 bg-slate-900/60 border
      <p className="text-[11px] font-bold text-
        ✂ Приоритетная очередь (Min-Heap):
      </p>
      <div className="flex flex-wrap gap-1.5">
        {heapQueue.length === 0 ? (
          <span className="text-xs text-slate-
        ) : (
          heapQueue.map(({item, idx}) => (
            <span key={idx} className={"px-2
0 shadow-md animate-pulse" : "bg-slate-800
          {item}
        </span>

```



```

<div className="flex flex-col sm:flex-row items
  <div className="flex items-center gap-2.5">
    <BookOpen className="w-5 h-5 text-indigo-400">
    <h4 className="text-xl font-bold text-white">
  </div>

  <div className="flex bg-slate-950 p-1 rounded
    <button
      onClick={() => setActiveCheatTab('kruskal')}
      className={
        "px-3 py-1.5 rounded text-xs font-medium" +
        (activeCheatTab === 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-900 text-white")
      }
    >
      Краскал (Билет 18)
    </button>
    <button
      onClick={() => setActiveCheatTab('prim')}
      className={
        "px-3 py-1.5 rounded text-xs font-medium" +
        (activeCheatTab === 'prim' ? "bg-emerald-600 text-white" : "bg-slate-900 text-white")
      }
    >
      Прима (Билет 19)
    </button>
    <button
      onClick={() => setActiveCheatTab('boruvka')}
      className={
        "px-3 py-1.5 rounded text-xs font-medium" +
        (activeCheatTab === 'boruvka' ? "bg-rose-600 text-white" : "bg-slate-900 text-white")
      }
    >
      Борувка (Билет 20)
    </button>
  </div>
</div>

{activeCheatTab === 'kruskal' && (
  <div className="grid grid-cols-1 lg:grid-cols-2" style={gridStyle}>
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-indigo-400">
        </p>

```

```
edges.sort() # O(E log E)
```

```
mst = []
```

```
for w, u, v in edges:
```

```
    if union(u, v):
```

```
        mst.append((u, v, w))`}
```

```
    </pre>
```

```
    </div>
```

```
    </div>
```

```
    </div>
```

```
}}
```

```
{activeCheatTab === 'prim' && {
```

```
    <div className="grid grid-cols-1 lg:grid-cols-
```

```
    <div className="lg:col-span-1 space-y-4">
```

```
        <div className="bg-slate-950 p-4 rounded-
```

```
            <p className="text-slate-400 text-xs for
```

```
                <Clock className="w-3.5 h-3.5 text-em
```

```
            </p>
```

```
            <p className="text-2xl font-black text-v
```

```
            <p className="text-slate-400 text-xs mt
```

```
        </div>
```

```
        <div className="bg-slate-950 p-4 rounded-
```

```
            <p className="text-slate-400 text-xs for
```

```
                <Award className="w-3.5 h-3.5 text-em
```

```
            </p>
```

```
            <p className="text-2xl font-black text-v
```

```
            <p className="text-slate-400 text-xs mt
```

```
        </div>
```

```
        <div className="bg-slate-950 p-4 rounded-
```

```
            <p className="text-slate-400 text-xs for
```

```
        ложении:</p>
```

```
            <p className="text-xs text-slate-300 le
```

```
        ие чистые вершины через самое дешевое досту
```

```
        </div>
```

```
    </div>
```

```
    <div className="lg:col-span-2">
```

```
        <div className="bg-slate-950 p-4 rounded-
```

```
            <p className="text-slate-400 text-xs for
```

```

<div className="bg-slate-950 p-4 rounded-1
  <p className="text-slate-400 text-xs fo
    <Award className="w-3.5 h-3.5 text-ro
  </p>
  <p className="text-2xl font-black text-r
  <p className="text-slate-400 text-xs mt
</div>
<div className="bg-slate-950 p-4 rounded-1
  <p className="text-slate-400 text-xs fo
ении:</p>
  <p className="text-xs text-slate-300 le
, организовав массовое распараллеленное слияни
  </div>
</div>
<div className="lg:col-span-2">
  <div className="bg-slate-950 p-4 rounded-1
    <p className="text-slate-400 text-xs fo
      <Code className="w-3.5 h-3.5 text-ros
    </p>
    <pre className="text-xs text-emerald-400
      80 flex-1">
{`def boruvka(n, edges):
    parent = list(range(n))
    mst = []
    while len(mst) < n - 1:
        # Для каждого колхоза ищем самый дешёвый мост н
        cheapest = [-1] * n
        for u, v, w in edges:
            ru, rv = find(u), find(v)
            if ru != rv:
                if cheapest[ru] == -1 or cheapest[ru][2]
                if cheapest[rv] == -1 or cheapest[rv][2]
        # Объединяем все колхозы по выбранным мостам
        for bridge in cheapest:
            if bridge != -1 and union(bridge[0], bridge
                mst.append(bridge)`}
    </pre>
  </div>

```

```

body { background: #ffffff !important; color: #111827; }
.lite-wrap { max-width: 46rem; margin: 0 auto; padding: 1.6; }
.lite-wrap h1 { font-size: 1.75rem; font-weight: 700; }
.lite-wrap h2 { font-size: 1.35rem; font-weight: 700; color: #111827; }
.lite-wrap h3 { font-size: 1.1rem; font-weight: 700; }
.lite-wrap h4 { font-size: 1rem; font-weight: 700; margin: 0 0 1.25rem 0; }
.lite-wrap p, .lite-wrap li { color: #1f2937; font-size: 1rem; }
.lite-wrap a { color: #1d4ed8; }
.lite-meta { color: #4b5563; font-size: .85rem; margin: 0 0 1.25rem 0; }
.lite-topics { font-size: .8rem; color: #374151; margin: 0 0 1.25rem 0; }
.lite-viz { margin: 1.25rem 0; padding: .8rem 1rem; border: 1px solid #f3f4f6; }
.lite-viz h4 { margin: 0 0 .35rem 0; font-size: .95rem; }
.lite-viz ul { margin: .35rem 0 0 1.1rem; padding: 0; }
.lite-viz code, .lite-wrap code { font-family: ui-monospace; padding: 0 .25rem; }
.lite-wrap pre { background: #f3f4f6; border: 1px solid #f3f4f6; padding: 1.45; }
.lite-wrap table { border-collapse: collapse; width: 100%; }
.lite-wrap th, .lite-wrap td { border: 1px solid #d1d5db; padding: .75rem 1.5rem; }
.lite-wrap th { background: #f3f4f6; }
.lite-wrap details { margin: .6rem 0; border: 1px solid #d1d5db; padding: .75rem 1.5rem; }
.lite-wrap summary { cursor: default; font-weight: 700; }
.lite-toc { font-size: .92rem; }
.lite-toc ol { margin: .4rem 0 0 1.2rem; }
@media print {
    .lite-wrap { max-width: none; padding: 0; }
    h2 { page-break-before: always; }
    h2:first-of-type { page-break-before: avoid; }
    pre, table, details { page-break-inside: avoid; }
}

```

```

/** Раскрываем все спойлеры и вычищаем служебную разметку
function toPlainTextHtml(html: string): string {
    return html
}

```

```

    <ol>
      {list.map((c) => {
        <li key={c.id}>
          <a href={`?lite=1&topic=${encodeURIComponent(
            chapterTopics[c.id]?.length ? ` – биле
          ""}
          </li>
        )}}
      </ol>
    </nav>
  )}

```

```

{list.map((chapter) => {
  const viz = getViz(chapter.id);
  const topics = chapterTopics[chapter.id] ?? [];
  const syncKeys = Object.keys(PAGE_SYNC).filter(
    return (
      <article key={chapter.id} id={chapter.id}>
        <h2>{chapter.title}</h2>
        <p className="lite-topics">
          {topics.length > 0 && <>Билет{topics.leng
          {chapter.category ? <>Раздел: {chapter.ca
          id страницы: <code>{chapter.id}</code>
        </p>

        <div
          dangerouslySetInnerHTML={{
            __html: linkifyTicketsHtml(toPlainTextH
          }}
        />

```

```

{viz && (
  <div className="lite-viz">
    <h4>Визуализация: {viz.title}</h4>
    {viz.hint && <p>{viz.hint}</p>}
    {syncKeys.map((k) => {
      const sync = PAGE_SYNC[k];
      const demo = k.includes("#") ? k.spli

```

```

        <li key={j}>
          {j === q.correctIndex ? <b>
        </li>
      )}}
    </ul>
    <p className="lite-meta">{q.explan
  </li>
    )}}
  </ol>
</div>
  })
</article>
);
}}
</div>
);
};

export default LiteGuide;

```

ФАЙЛ 58/111: src/components/LiveDistTable.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import type { DebugValue } from "../data/debugRunner";

interface LiveDistTableProps {
  /** dist из кода пользователя: имя вершины → расстояние */
  dist: Record<string, DebugValue> | null;
  /** prev/p — предок на кратчайшем пути (необязательно) */
  prev?: Record<string, DebugValue> | null;
  /** done/visited — зафиксированные вершины (необязательно) */
  done?: DebugValue[] | null;
  /** Идентификаторы вершин, нарисованных на схеме демо */
  demoIds: string[];
  /** Подпись таблицы (по умолчанию «Ваши расстояния»).

```

```

const doneSet = new Set((done ?? []).map((value) => S
const shown = keys.slice(0, 64);

return (
  <div className="mb-6 rounded-xl border border-amber
    <p className="text-[11px] leading-relaxed text-am
      <b>{title}</b> В вашем коде вершины{" "}
      <span className="font-mono">
        {unknown.slice(0, 8).join(", ")}
        {unknown.length > 8 ? " ..." : ""}
      </span>
      , а на схеме демо-графа нарисованы{" "}
      <span className="font-mono">{demoIds.join(", ")}
      показаны отдельной таблицей. Чтобы они управлял
      (кнопка-«глаз» в панели Python подставляет его
    </p>
    <div className="mt-2.5 flex flex-wrap gap-1.5">
      {shown.map((key) => {
        const parent = prev ? fmt(prev[key]) : null;
        const fixed = doneSet.has(key);
        return (
          <span
            key={key}
            className={`inline-flex items-baseline gap-1
              {fixed
                ? "border-emerald-500/40 bg-emerald-500/10"
                : "border-slate-700 bg-slate-900/70"}
              `}
            title={fixed ? "вершина зафиксирована (done
          >
          <b className={fixed ? "text-emerald-300"
          <span className="text-slate-500">=</span>
          <span>{fmt(dist[key])}</span>
          {parent && parent !== "-" ? <span classNa
          {fixed ? <span className="text-emerald-400"
        </span>
      )};

```

```

    alt: 'Бэтмен на машине Форда (Форд-Беллман)',
    title: '    Бэтмен на машине Форда',
    desc: 'Патрулирует n-1 ночей в кабине машины Генри',
    highlight: 'отрицательные циклы',
    highlightColor: 'text-rose-400',
    rest: '(бесконечную кормушку Джокера).',
    complexity: 'O(V · E)',
    border: 'border-rose-500/30 hover:border-rose-400/60',
    titleColor: 'text-rose-400',
    badge: 'text-rose-400/70 bg-rose-950/40',
  },
  {
    img: floydImg,
    alt: 'Fortnite Все против Всех (Флойд-Уоршелл)',
    title: '    Все против Всех / Fortnite (Флойд)',
    desc: 'Матрица N×N + три цикла (',
    highlight: 'k, i, j',
    highlightColor: 'text-emerald-400',
    rest: '). Все против всех – ищет кратчайшие пути от',
    complexity: 'O(V³)',
    border: 'border-emerald-500/30 hover:border-emerald-400/60',
    titleColor: 'text-emerald-400',
    badge: 'text-emerald-400/70 bg-emerald-950/40',
  },
];

```

```

export default function MnemonicCards() {
  return (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      {cards.map((c, i) => (
        <div key={i} className={`bg-slate-800 rounded-2xl p-4">
          <div className="h-48 overflow-hidden">
            <img src={c.img} alt={c.alt}
              className="w-full h-full object-cover group-hover:scale-110"/>
          </div>
          <div className="p-4">
            <h3 className={`text-lg font-bold mb-1 ${c.titleColor}`>{c.title}</h3>
            <p className="text-slate-400 text-sm">{c.desc}</p>
            <p className="text-slate-400 text-sm">{c.complexity}</p>
            <p className="text-slate-400 text-sm">{c.rest}</p>
            <p className="text-slate-400 text-sm">{c.badge}</p>
          </div>
        </div>
      ))}
    </div>
  );
}

```



```

    return () => {
      document.body.style.overflow = "";
    };
  }, [open]));

useEffect(() => {
  const onKey = (e: KeyboardEvent) => e.key === "Escape";
  window.addEventListener("keydown", onKey);
  return () => window.removeEventListener("keydown", onKey);
}, []);

return (
  <>
    <div className="lg:hidden sticky top-[57px] z-40">
      <button
        onClick={() => setOpen(true)}
        className="w-full flex items-center gap-2.5 p-2"
        aria-label="Открыть содержание"
      >
        <span className="p-1.5 rounded-lg bg-indigo-600 text-white">
          <List className="w-4 h-4" />
        </span>
        <span className="min-w-0 flex-1">
          <span className="block text-[10px] uppercase">
            Содержание • {index + 1} из {total}
          </span>
          <span className="block text-[13px] font-bold">
            {currentSection}
          </span>
        </span>
      </button>
    </div>

    {open && (
      <div className="lg:hidden fixed inset-0 z-[90]">
        <div className="absolute inset-0 bg-slate-950">
          <div className="absolute inset-y-0 left-0 w-[100%] h-[100%]
            transition-[opacity] duration-[180ms] ease-out">
            <div className="flex items-center justify-center">
              <span className="font-bold text-white text-[1.2em]">
                Содержание
              </span>
            </div>
          </div>
        </div>
      </div>
    )}
  </>
)

```

```
/**
 * Шапка приложения.
 *
 * Кнопка «Скачать PDF» всегда на самом видном месте во
 */
export const Navbar: React.FC<NavbarProps> = ({ compile
  const [busy, setBusy] = useState(false);

  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };

  return (
    <header className="bg-slate-900 border-b border-sla
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
        gap-2.5 lg:gap-0">
        <div className="flex items-center gap-3 w-full
          <div className="bg-gradient-to-tr from-indigo
            0">
              <Sparkles className="w-5 h-5 lg:w-6 lg:h-6"
            </div>
          <div className="min-w-0 flex-1">
            <h1 className="text-lg lg:text-xl font-extr
              Универсальное пособие
            </h1>
            <p className="text-xs text-indigo-400 font-
              Подготовка к экзаменам: Алгоритмы, Структ
            </p>
          </div>
        </div>
      </div>

      <div className="flex items-center flex-wrap sm:
```

```

    <summary
      className="flex items-center justify-center
      bg-slate-700/60 border border-slate-600 cu
      title="Ещё форматы: PDF с кодом, TXT и
    >
    <span>Ещё</span>
  </summary>
  <div className="absolute right-0 top-full
5 shadow-xl shadow-black/50 min-w-[170px]">
    <a
      id="download-pdf-btn"
      href={` ${import.meta.env.BASE_URL}exp
      download="full_code_all_pages.pdf"
      target="_blank"
      rel="noreferrer"
      className="flex items-center gap-2 px
      rald-600/20 border border-emerald-500/30 tr
      title="Полный PDF-дамп исходного кода
    >
      <FileDown className="w-3.5 h-3.5" />
    </a>
    <a
      id="download-txt-btn"
      href={` ${import.meta.env.BASE_URL}exp
      download="full_code_all_pages.txt"
      target="_blank"
      rel="noreferrer"
      className="flex items-center gap-2 px
/20 border border-sky-500/30 transition-all
      title="Скачать полный TXT файл со все
    >
      <FileText className="w-3.5 h-3.5" />
    </a>
    <button
      id="download-html-btn"
      onClick={saveBook}
      disabled={busy}
      className="flex items-center gap-2 w-

```

```

-----
* Два графа на выбор: «основной» (8 вершин,  $\Delta = 3$ , есть
* и «двудольный» (6 вершин,  $\Delta = 3$ ,  $\chi = 2$ ,  $\chi' = 3$ ).
*
* Синхронизация с Python-компилятором (PAGE_SYNC["planar"]);
* color/col – массив цветов вершин (красит кружки на плоскости);
* ecolord/ec – массив цветов рёбер, v – текущая вершина;
* u – сосед, chi/k – число цветов, delta/D – максимальная степень;
* i/step – номер кадра.
*/

type Tab = "planar" | "greedy" | "bipartite" | "edges";
type GraphKey = "main" | "bip";

interface Graph {
  key: GraphKey;
  label: string;
  n: number;
  edges: [number, number][];
  adj: number[][];
  pos: { x: number; y: number }[];
  names: string[];
}

const PALETTE = ["#6366f1", "#10b981", "#f43f5e", "#f59e0b"];
const colorOf = (c: number) => (c >= 1 && c <= PALETTE.length) ? PALETTE[c - 1] : "#000000";

function makeGraph(key: GraphKey): Graph {
  if (key === "bip") {
    const edges: [number, number][] = [[0, 3], [0, 4], [1, 3], [1, 4], [2, 3], [2, 4], [3, 4]];
    const pos = [
      { x: 80, y: 60 }, { x: 80, y: 150 }, { x: 80, y: 240 },
      { x: 380, y: 60 }, { x: 380, y: 150 }, { x: 380, y: 240 }
    ];
    const g: Graph = { key, label: "Двудольный (6 вершин)", pos, edges };
    g.adj = buildAdj(g);
    return g;
  }
  const edges: [number, number][] = [[0, 1], [0, 2], [1, 2], [1, 3], [1, 4], [1, 5], [2, 3], [2, 4], [2, 5], [3, 4], [3, 5], [4, 5]];
  const pos = [
    { x: 80, y: 60 }, { x: 200, y: 60 }, { x: 320, y: 60 },
    { x: 80, y: 150 }, { x: 200, y: 150 }, { x: 320, y: 150 },
    { x: 80, y: 240 }, { x: 200, y: 240 }, { x: 320, y: 240 }
  ];
  const g: Graph = { key, label: "Основной (8 вершин)", pos, edges };
  g.adj = buildAdj(g);
  return g;
}

```

```

        title: "Старт: ни одна вершина не покрашена",
        note: `color = [0]·n. Порядок обхода – просто по номерам. Гарантированно уложится в  $\Delta + 1 = \{\delta + 1\}$  цветов: у вершины v
        color: [...color], v: null, u: null, e: null,
    });
    for (let v = 0; v < g.n; v++) {
        const used = Array.from(new Set(g.adj[v].filter((x) => x !== v)));
        let c = 1;
        while (used.includes(c)) c += 1;
        color[v] = c;
        const deg = g.adj[v].length;
        frames.push({
            title: `Вершина ${g.names[v]}: берём цвет ${c}`,
            note: used.length
                ? `Соседи уже заняли цвета ${used.join(", ")}
                  всегда хватит ( $\chi \leq \Delta + 1$ ).`
                : `У вершины ${g.names[v]} ещё нет покрашенных соседей`,
            color: [...color], v, u: null, e: null, used,
        });
    }
    const chi = Math.max(...color);
    frames.push({
        title: `Готово: жадная покраска использовала ${chi} цветов`,
        note: ` $\chi_{\text{жадн}} = \{\chi\}$ . Истинное хроматическое число  $\chi$ 
            , а теорема Брукса ( $\chi \leq \Delta$ , если граф не полный и не двудольный)`,
        color: [...color], v: null, u: null, e: null,
    });
    return frames;
}

```

```

/** Проверка двудольности BFS-ом: 2 краски, нет нечётных циклов
function bipartiteFrames(g: Graph): Frame[] {
    const frames: Frame[] = [];
    const color = new Array(g.n).fill(0);
    const queue: number[] = [];
    let conflict: [number, number] | null = null;
    frames.push({
        title: "Старт: пробуем покрасить в 2 цвета",

```

```

frames.push({
  title: conflict ? "Граф НЕ двудольный" : "Граф двудольный",
  note: conflict
    ? `BFS уперся в нечётный цикл (${g.names[conflict]}):
      краски — их и даёт жадный алгоритм на вкладке «Жадная окраска»
    : "Все вершины разбиты на две доли, рёбер внутри долей нет,
      справедлива теорема Кёнига:  $\chi' = \Delta$  (рёберная окраска)",
  color: [...color], v: null, u: null, e: null, bad: null,
});
return frames;
}

```

```

/** Жадная покраска рёбер:  $\Delta \leq \chi' \leq \Delta + 1$  (Визинг), для
function edgeFrames(g: Graph): Frame[] {
  const frames: Frame[] = [];
  const ecol = new Array(g.edges.length).fill(0);
  const inc: Map<number, number>[] = Array.from({ length: g.n }, () => new Map());
  const delta = Math.max(...g.adj.map(a => a.length));
  frames.push({
    title: "Старт: рёбра не покрашены",
    note: `Правильная рёберная покраска: у каждой вершины
      рёбер одной вершины различны, а по теореме Визинга
       $\Delta \leq \chi' \leq \Delta + 1$ `,
    color: new Array(g.n).fill(0), ecol: [...ecol],
  });
  g.edges.forEach(([a, b], i) => {
    const usedA = Array.from(inc[a].values());
    const usedB = Array.from(inc[b].values());
    const used = Array.from(new Set([...usedA, ...usedB]));
    let c = 1;
    while (used.includes(c)) c += 1;
    ecol[i] = c;
    inc[a].set(i, c);
    inc[b].set(i, c);
    frames.push({
      title: `Рёбро (${g.names[a]}, ${g.names[b]}): цвет ${c}`,
      note: `В вершине ${g.names[a]} заняты цвета ${usedA}
        (использованы цвета {${usedA.map(x => x - 1).join(", ")} || "—"}). Наименьший свободный цвет — ${c}.`,
    });
  });
  return frames;
}

```

```

const frames = useMemo(() => {
  if (tab === "greedy") return greedyFrames(graph);
  if (tab === "bipartite") return bipartiteFrames(graph);
  if (tab === "edges") return edgeFrames(graph);
  return [] as Frame[];
}, [tab, graph]);
const maxStep = Math.max(0, frames.length - 1);

useEffect(() => {
  setStep(0);
  setPlaying(false);
}, [tab, graphKey]);

useEffect(() => {
  if (!playing) return;
  const t = window.setInterval(() => {
    setStep((s) => {
      if (s >= maxStep) {
        setPlaying(false);
        return s;
      }
      return s + 1;
    });
  }, 1300);
  return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(0) : null;
useEffect(() => {
  if (liveStep === null || !Number.isFinite(liveStep))
    setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

const liveColor = vars ? vizArray(vars.color) ?? vizArray(0) : null;
const liveEcolor = vars ? vizArray(vars.ecolor) ?? vizArray(0) : null;
const liveV = vars ? vizNumber(vars.v) : null;
const liveU = vars ? vizNumber(vars.u) : null;

```

```

return (
  <div className="bg-slate-800/90 p-5 sm:p-6 rounded-
    <Header />
    <Tabs tab={tab} setTab={setTab} />

    <div className="flex flex-wrap items-center gap-2
      <span className="text-[10px] uppercase tracking
        {[["main", "bip"] as GraphKey[]}.map((k) => (
          <button
            key={k}
            onClick={() => setGraphKey(k)}
            className={"px-3 py-1.5 rounded-lg text-[11px]
              "bg-slate-900/60 text-slate-400 hover:text-
            >
              {makeGraph(k).label}
            </button>
          </div>
        </div>

    <div className="flex flex-wrap items-center gap-2
      <button onClick={() => { setStep(0); setPlaying
        rounded-lg bg-slate-900 border border-slate-
      <button onClick={() => go(-1)} disabled={step =
        00 border border-slate-700 text-slate-300 hove
      <button onClick={() => setPlaying((p) => !p)} c
        ext-xs font-bold hover:bg-indigo-500">
        {playing ? <Pause className="w-4 h-4" /> : <P
      </button>
      <button onClick={() => go(1)} disabled={step >=
        late-900 border border-slate-700 text-slate-3
        on>
      <span className="text-[11px] text-slate-500 ml-
      <span className="ml-auto flex flex-wrap items-c
        <span className="bg-slate-950 border border-s
        {tab === "edges"
          ? <span className="bg-slate-950 border bord
          rs}</span>
          : <span className="bg-slate-950 border bord

```



```

        ames[id]]</text>
        {c > 0 && tab !== "edges" && <text x=
old">цвет {c}</text>}
        {tab === "edges" && <text x={p.x} y={
}</text>}
    </g>
  });
  }}}
</svg>
</div>

<div className="lg:col-span-2 space-y-3">
  <div className="bg-slate-900/70 border border
    <p className="text-sm font-bold text-white
    <p className="text-xs text-slate-400 leadin
  </div>
  <div className="bg-slate-950 border border-sl
    <p className="text-[10px] uppercase trackin
    <div className="flex flex-wrap gap-1.5 font
      {(tab === "edges" ? graph.edges.map( (_, i
        <span key={i} className="px-1.5 py-0.5
        ? colorOf(c) : "#475569", background: c > 0
          {tab === "edges" ? `e${i}` : graph.na
        </span>
      )})}
    </div>
  </div>
  <div className="bg-slate-900/70 border border
    {tab === "greedy" && <>Жадная покраска: <b
    : <b className="text-slate-200"> $\chi \leq \Delta$ </b>,
    в  $\chi \leq 5$  (доказывается через  $E \leq 3V - 6$  и це
    ром).</>}
    {tab === "bipartite" && <>Двудольный граф
    /DFS за  $O(V + E)$ . Распознавание 3-раскрашив
    {tab === "edges" && <>Рёберная покраска: <b
       $\Delta$  (Кёниг). Задача « $\chi' = \Delta$ ?» NP-полна даже
    </div>
</div>

```

```
const Tabs: React.FC<{ tab: Tab; setTab: (t: Tab) => void }> = ({ tab, setTab }) => {
  <div className="flex flex-wrap gap-2 mb-4">
    {TABS.map((t) => (
      <button
        key={t.id}
        onClick={() => setTab(t.id)}
        className={"flex items-center gap-1.5 px-3 py-2" +
          (t === tab ? "bg-slate-900 text-white" : "bg-slate-900/60 text-slate-400")}
      >
        {t.icon} {t.label}
      </button>
    ))}
  </div>
);
```

```
export default PlanarColoringViz;
```

ФАЙЛ 63/111: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
export function PlanarityDemo() {
  return (
    <div className="space-y-4">
      <h4 className="text-base font-bold text-white">K5
      <p className="text-xs text-slate-400">Просто смотрим на K5
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        { /* K5 */ }
        <div className="bg-slate-950 rounded-xl p-4 border border-slate-900">
          <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
          <svg className="w-full h-full absolute inset-0">
            {(() => {
              const pts = [
```

```

        const labels = ["A", "B", "C", "D", "E"];
        for(const p of pts) {
            circles.push(<g key={`c${p.id}`}>
                <circle cx={p.x} cy={p.y} r={8} fill=
                <text x={p.x} y={p.y+3} textAnchor="m
            </g>);
        }
        return <>{lines}{circles}</>;
    })()}
</svg>
<div className="absolute bottom-2 left-2 right
    z-20">
    V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 bor
    <div className="absolute top-2 left-2 bg-slate
        00/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
    <svg className="w-full h-full absolute inset-0
        {(() => {
            const pts = [
                {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
                {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
            ];
            const edges = [
                [0,3],[0,4],[0,5],
                [1,3],[1,4],[1,5],
                [2,3],[2,4],[2,5]
            ];
            function findPt(id:number) { return pts.f
            function intersect(a:number,b:number,c:nu
                if (a===c||a===d||b===c||b===d) return
                const p1=findPt(a),p2=findPt(b),p3=find
                if (!p1||!p2||!p3||!p4) return false;
                function ccw(ax:number,ay:number,bx:num
                    return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax)

```

```
        </div>
    </div>
  );
}
```

ФАЙЛ 64/111: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useCallback, useEffect, useMemo, useRef, useState } from 'react';
import {
  ArrowRightLeft,
  Check,
  CheckCircle2,
  ChevronDown,
  ChevronLeft,
  ChevronRight,
  ChevronUp,
  CircleStop,
  Copy,
  Eye,
  EyeOff,
  HelpCircle,
  Link2,
  Loader2,
  Pause,
  Pencil,
  Play,
  Plus,
  RotateCcw,
  SkipBack,
  SkipForward,
  Terminal,
  Unlink,
  Variable,
  X,
}
```

```
-----
  setStderr: (options: { batched: (message: string) => Promise<string> }) => Promise<string>
  setStdin: (options: { stdin: () => string | number | null }) => Promise<string>
};
```

```
type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>
};
```

```
let runtimePromise: Promise<PyodideRuntime> | null = null;
```

```
/**
 * Подмена интерпретатора для проверок: DOM-хакер (jsdom) и CI-скриншоты кладут сюда готовый
 * на CDN. В браузере у пользователя поле не определено
 */
```

```
type PythonRuntimeProvider = PyodideRuntime | (() => Promise<PyodideRuntime>);
```

```
declare global {
  interface Window {
    __algv0PythonRuntime?: PythonRuntimeProvider;
  }
}
```

```
function getPythonRuntime(): Promise<PyodideRuntime> {
  const injected = typeof window !== "undefined" ? window : undefined;
  if (injected) return typeof injected === "function" ? injected : undefined;
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL)
      .then((module as PyodideModule).loadPyodide({ indexURL: indexURL }));
  }
  return runtimePromise;
}
```

```
function errorText(error: unknown) {
  if (error instanceof Error) return error.message;
  return String(error);
}
```

```

    items: [
        {
            label: "arr.sort() / sorted()",
            tip: "In-place сортировка по возрастанию и новая",
            code: 'arr = [5, 2, 8, 1]\narr.sort()
                ) # копия по убыванию\n',
        },
        {
            label: "sorted(key=...)",
            tip: "Сортировка по ключу: рёбра по весу, пары",
            code: 'edges = [(3, "A", "B"), (1, "B", "C"), (
                es)\n',
        },
    ],
},
{
    group: "Циклы",
    items: [
        {
            label: "for i in range(n)",
            tip: "Вложенные циклы по i и j – как обход матри",
            code: "n, m = 3, 4\nfor i in range(n):",
        },
        {
            label: "while ...",
            tip: "Классический while-счётчик – как n-1 итер",
            code: "i = 1\nwhile i < 5:\n    print(f\"итераци",
        },
        {
            label: "for i, x in enumerate(...)",
            tip: "Индекс и элемент одновременно: i – позици",
            code: "for i, x in enumerate([\n\"a\", \n\"b\", \n\"c",
        },
    ],
},
{
    group: "Структуры",
    items: [

```

```

    },
  ];

/**
 * Переживает скрытие панели: код пользователя не теряе
 *
 * Хранилище — ПО СТРАНИЦАМ. Раньше слот был один, и ко
 * восстанавливался на новой странице: редактор показыва
 * и визуализация жили значениями чужого кода.
 */
const savedEditors = new Map<string, { code: string; ba

export function PythonCompiler({ chapterId, chapterTitle
  ilderProps) {
  /** Активная вкладка демонстрации страницы (у sparse-
  const [demoId, setDemoId] = useState<string | null>(n
  const sync = useMemo(() => getPageSync(chapterId, dem
  const syncVarBases = useMemo(
    () =>
      [...new Set(
        sync.variables.flatMap((v) =>
          v.name
            .split(/\s*(?:,|\s\/)\s*/)
            .map(baseVarName)
            .filter((name) => /^[A-Za-z_][A-Za-z0-9_]*$/
        )
      )],
    [sync]
  );
  const syncVarSet = useMemo(() => new Set(syncVarBases
  /** Полный эталон: глаз сразу вставляет его в обычный
  const template = useMemo(() => buildSyncTemplate(char
  /** Дефолт редактора: только инициализация демо; она
  const initTemplate = useMemo(() => buildInitTemplate(

  const [code, setCode] = useState(() => {
    const saved = savedEditors.get(chapterId);
    if (!saved) return initTemplate;
  });

```

```

/** Активная трасса. input нужен, чтобы изменение std
const [debug, setDebug] = useState<{ result: DebugRes
/** Esc оставляет неизменённый текст в редакторе, пока
const [editHold, setEditHold] = useState<string | null>
/** Автопроход включается пользователем; первый шаг по
const [playing, setPlaying] = useState(false);
const [playbackSpeed, setPlaybackSpeed] = useState<num
  if (typeof window === "undefined") return 1;
  const saved = Number(window.localStorage.getItem(PL
  return PLAYBACK_SPEEDS.includes(saved as (typeof PL
));
const listRef = useRef<HTMLDivElement>(null);

useEffect(() => {
  window.localStorage.setItem(PLAYBACK_SPEED_KEY, Str
}, [playbackSpeed]);

useEffect(() => {
  savedEditors.set(chapterId, { code, base: initTempl
}, [code, initTemplate, chapterId]);

// демонстрация сообщает, какая вкладка открыта – код
useEffect(() => setDemoId(null), [chapterId]);
useEffect(() => onVizDemo(chapterId, (d) => setDemoId
useEffect(() => () => clearVizState(chapterId), [chap

// Смена страницы (или вкладки демо): нетронутый реда
// инициализацию; свой код не затираем – предлагаем с
useEffect(() => {
  const prev = prevTplRef.current;
  if (initTemplate === prev.base && template === prev
  prevTplRef.current = { base: initTemplate, full: te
  setPlaying(false);
  setEditHold(null);
  setDebug(null); // трасса ссылается на строки старо
  setRefOn(false);
  if (code === prev.base || code === prev.full || cod

```



```

-----
* события: после st[i][j]= Python уже мог увеличить
*/
const snapshotAt = useCallback((result: DebugResult,
  const step = result.steps[idx];
  const next = result.steps[idx + 1];
  const afterLocals = next?.locals ?? result.finalLocals;
  const afterValues = next?.values ?? result.finalValues;
  const beforeValues = step?.values ?? {};
  const line = step?.line ? source.split("\n")[step.line - 1] : "";
  // Нас интересуют только простые скалярные цели (`i` или `i +=`). Контейнеры и так всегда берутся из afterValues
  // внутри print(f"i={i}") не будет ошибочно принят `i`
  const assignment = line.match(
    /^\\s*([A-Za-z_][A-Za-z0-9_]*)(?:\\s*=|\\s*\\s*[A-Za-z_][A-Za-z0-9_]*)$/;
  );
  const assigned = new Set(assignment?.[1].split(",") || []);

  const variables = { ...afterValues };
  const locals = { ...afterLocals };
  for (const [name, value] of Object.entries(beforeValues)) {
    const scalar = value === null || typeof value !== "object";
    if (scalar && !assigned.has(name) && name in afterValues) {
      variables[name] = value;
      if (step?.locals[name] !== undefined) locals[name] = value;
    }
  }
  const changed = changedVars(step?.locals, locals);
  return {
    line: step?.line,
    func: step?.func,
    locals,
    variables,
    changed: Object.keys(changed).filter((name) => changed[name]),
  };
}, []);

const publishDebug = useCallback(
  (result: DebugResult, idx: number, source: string) => {

```

```

    []
  );

  const stepTo = useCallback(({idx: number}) => goDebug(idx));

  const stopDebug = useCallback(() => {
    setPlaying(false);
    setEditHold(code);
    setDebug(null);
    clearVizState(chapterId);
  }, [chapterId, code]);

  // Базовый интервал 650 мс; множитель скорости хранится в state
  useEffect(() => {
    if (!playing || !debug) return;
    if (debug.idx >= debug.result.steps.length - 1) {
      setPlaying(false);
      return;
    }
    const t = window.setTimeout(() => stepBy(1), 650 / playbackSpeed);
    return () => window.clearTimeout(t);
  }, [playing, debug, stepBy, playbackSpeed]);

  const debugRun = useCallback(async (autoPlay = false) => {
    if (running) return;

    const source = code;
    const input = stdin;
    const request = traceRequest;
    setRunning(true);
    setPlaying(false);
    setEditHold(null);
    setDebug(null);
    clearVizState(chapterId);
    setOutputOpen(false);

    // Пустой редактор: Pyodide не гоняем, панель в режиме редактирования
    // связь с визуализацией снимаем — демо показывает

```

```

        ? `Ошибка:\n${result.error}`
        : printed || "В коде нет исполняемых строк -
    );
    setRuntimeMessage(
        result.error
        ? "Код не выполнен — редактор остался открытым"
        : "Нет шагов трассы: связь с визуализацией прервана"
    );
} else {
    setDebug({ result, idx: 0, src: source, input,
    setPlaying(autoPlay && result.steps.length > 1)
    // Программа что-то напечатала — значит, вывод
    // результат своего же print() панель не имеет
    if (printed.trim() !== "") setOutputOpen(true);
    setOutput(
        (printed || "Программа ничего не вывела — трасса пуста")
        (result.error ? `\\n\\nОшибка (выполнение прервано)` : "")
    );
    if (result.error) setOutputOpen(true);
    setRuntimeMessage(
        result.truncated
        ? `Выполнение остановлено после ${DEBUG_STEPS} шагов`
        : "Трасса готова автоматически. ← → — шаги,
    );
}
} catch (error) {
    if (!aliveRef.current) return;
    const message = errorText(error);
    setDebug({ result: { steps: [], error: message, truncated: true },
    clearVizState(chapterId);
    setOutputOpen(true);
    setOutput(`Ошибка:\n${message}`);
    setRuntimeMessage("Не удалось выполнить код — произошла ошибка");
} finally {
    setRunning(false);
}
}, [chapterId, code, stdin, traceRequest, running, run

```

```

    }, [debug, stepBy, stopDebug]));

// держим текущую строку листинга в видимой области
/**
 * Панель показывает листинг трассы, только если шаги
 * Пустой код или синтаксическая ошибка больше не пряч
 * раньше на их месте возникал пустой листинг без возм
 * «исчезала» и кнопка, и само поле ввода.
 */
const showTrace = !!debug && debug.result.steps.length
/** Трасса построена по старому тексту: пользователь
const stale = !!debug && debug.src !== code;
/** Листинг трассы можно свернуть, но редактор от это
const [traceOpen, setTraceOpen] = useState(true);
const curDebug = debug ? { step: debug.result.steps[d
const curDebugLine = curDebug?.step?.line;
useEffect(() => {
  if (curDebugLine === undefined || !listRef.current)
    listRef.current
      .querySelector(`[data-line="${curDebugLine}"]`)
      ?.scrollIntoView({ block: "nearest" });
}, [curDebugLine]);

// производные данные шага: что изменилось, в каком п
// Строка, которую назначает текущий шаг, подсвечивае
// локали берём со следующего шага трассы (для послед
// трейсером на возврате из фрейма). Значит на строке
const shownLocals = useMemo(() => {
  if (!debug || !curDebug?.step) return {};
  return snapshotAt(debug.result, debug.idx, debug.src
}, [debug, curDebug, snapshotAt]);
const debugChanged = curDebug?.step ? changedVars(cu
const debugOrdered = curDebug?.step ? orderVars(shown

/** Одна и та же кнопка запуска всегда видна: из реда
 * проигрывает её, в отладчике переключает play/pause
const runOrTogglePlayback = useCallback(() => {
  if (running) return;

```

```

>
  { /* Верхний край меняет высоту на телефоне и десктопе. */
    {onHeightChange && (
      <div
        role="separator"
        aria-orientation="horizontal"
        aria-label="Потяните, чтобы изменить высоту терминала"
        title="Потяните, чтобы изменить высоту терминала"
        className="absolute -top-1.5 left-0 right-0 h-10"
        onPointerDown={(event) => {
          const el = event.currentTarget;
          el.setPointerCapture(event.pointerId);
          const startY = event.clientY;
          const startH = el.parentElement?.getBoundingClientRect().height;
          const onMove = (moveEvent: PointerEvent) => {
            const maxHeight = window.innerHeight - (window.topBarHeight || 0);
            const minHeight = Math.min(260, maxHeight);
            const next = Math.round(startH + startY - moveEvent.clientY);
            onHeightChange(Math.max(minHeight, Math.min(maxHeight, next)));
          };
          const onUp = () => {
            el.removeEventListener("pointermove", onMove);
            el.removeEventListener("pointerup", onUp);
            el.removeEventListener("pointercancel", onUp);
          };
          el.addEventListener("pointermove", onMove);
          el.addEventListener("pointerup", onUp);
          el.addEventListener("pointercancel", onUp);
        }}
      <div className="absolute left-1/2 top-1/2 h-10 w-100px
        -translate-x-50% -translate-y-50% border-1px solid black
        rounded-400 transition-colors" />
    </div>
  )}

  { /* Левый край меняет ширину на десктопе. */
    {onWidthChange && (
      <div

```

```

>
  <button
    type="button"
    onClick={runOrTogglePlayback}
    disabled={running}
    className={`inline-flex items-center gap-0 ${
      playing
        ? "bg-amber-500/15 text-amber-300 hover:"
        : "bg-emerald-500/15 text-emerald-300"
    }`}
    aria-label={running ? "Python выполняется" : "Запустить"}
  >
    {running ? <Loader2 className="h-4 w-4 animate-spin" /> : "▶"}
    <span>{playbackSpeed}x</span>
  </button>
</Tooltip>
<Tooltip side="bottom" content="Скопировать код" >
  <button
    type="button"
    onClick={() => void copyCode()}
    className="p-1.5 rounded-lg text-slate-400"
    aria-label="Скопировать код"
  >
    {copied ? <Check className="w-4 h-4 text-emerald-500" /> : "✂"}
  </button>
</Tooltip>
<Tooltip side="bottom" content="Вернуть комментарий" >
  <button
    type="button"
    onClick={resync}
    className="p-1.5 rounded-lg text-slate-400"
    aria-label="Сбросить к шаблону страницы"
  >
    <RotateCcw className="w-4 h-4" />
  </button>
</Tooltip>

```

```

        refOn
        ? "bg-indigo-500/30 text-white ring-1
        : "text-indigo-300 hover:text-white h
    }` }
    aria-label={refOn ? "Убрать эталонный код
ктор"}
    aria-pressed={refOn}
  >
    {refOn ? <Eye className="w-4 h-4" /> : <E
  </button>
</Tooltip>
<label
  className="ml-1 inline-flex items-center gap
  der border-slate-700 hover:border-emerald-5
  title="Скорость автоматического проигрывани
  >
    {running ? <Loader2 className="w-3.5 h-3.5
    ld-400" />}
    <span className="hidden xl:inline">{running
    <select
      aria-label="Скорость автопрохода"
      value={playbackSpeed}
      onChange={(event) => setPlaybackSpeed(Num
      className="cursor-pointer rounded bg-slate
    >
      {PLAYBACK_SPEEDS.map((speed) => (
        <option key={speed} value={speed}>{speed
      )))}
    </select>
  </label>
</div>
</div>

/* — Мини-вкладки: переменные страницы / шпаргал
<div className="shrink-0 border-b border-slate-800
  <div className="flex items-center gap-1 px-2 pt
    <button
      type="button"

```

```

{stripTab === "vars" ? {
  <div className="px-3 pb-2.5 space-y-2 max-h-300">
    <div className="flex items-center gap-2 min-w-100">
      {onOpenGuide ? {
        <Tooltip side="bottom" content="Открыть руководство" >
          <button
            type="button"
            onClick={onOpenGuide}
            className="inline-flex items-center justify-center gap-2 text-slate-300 hover:text-white hover:bg-slate-700" >
            <span className="truncate">{chapterTitle}</span>
          </button>
        </Tooltip>
      } : {
        <span className="inline-flex items-center justify-center gap-2 text-slate-300 max-w-full">
          <span className="truncate">{chapterTitle}</span>
        </span>
      }}
    </div>
    {desyncedFrom !== null && {
      <Tooltip
        side="bottom"
        content="В редакторе ваш код, а страница — чужой. Если вы не уверены, что не допустили ошибок, нажмите на кнопку ниже, чтобы заменить код на оригинальный." >
        <button
          type="button"
          onClick={resync}
          className="flex items-center gap-1.5 text-amber-400 hover:bg-amber-500/20 transition-colors" >
          <Unlink className="w-3.5 h-3.5 shrink-0" />
          Страница сменилась — подставить её пересинхронизировать
        </button>
      </Tooltip>
    }
  }
}

```



```

        </div>
      )}
    </div>
  ) : (
    <div className="px-3 pb-2.5 max-h-36 overflow
      {SNIPPETS.map(({gItem}) => (
        <div key={gItem.group}>
          <div className="text-[9px] font-bold up
            <div className="flex flex-wrap gap-1.5"
              {gItem.items.map(({s}) => (
                <Tooltip key={s.label} side="bottom
                  <button
                    type="button"
                    onClick={() => insertSnippet(s.
                      className="inline-flex items-ce
                        -[10.5px] text-indigo-300 hover:text-white
                          title=""
                        >
                          <Plus className="w-3 h-3 text-s
                            {s.label}
                          </button>
                        </Tooltip>
                      )}}
                    </div>
                  </div>
                )}}
              <p className="text-[10px] text-slate-600">H
            </div>
          )}
        </div>
      )}
    )}
  </div>

```

```

/* — Редактор (всегда на месте) + трасса + ввод
<div className="flex-1 min-h-0 flex flex-col">
  /* Заголовок редактора: сразу видно, что выполн
  <div className="shrink-0 flex items-center gap-1
    <Pencil className="w-3.5 h-3.5 text-emerald-4
    <span className="text-[10px] font-bold upperc
      Редактор • выполняется этот текст

```

```

<div className={`flex flex-col border-t borde
  /* панель управления шагами */
  <div className="shrink-0 flex flex-wrap ite
    <Tooltip content={playing ? `Пауза автопро
    роиграть трассу заново со скоростью ${playb
    л)`}>
      <button
        type="button"
        onClick={runOrTogglePlayback}
        className={`mx-0.5 inline-flex items-
        r-500/15 hover:bg-amber-500/25" : "text-еме
        aria-label={playing ? `Пауза автопрох
      >
        {playing ? <Pause className="w-4 h-4"
        <span className="text-[9px] font-bold
      </button>
    </Tooltip>
    <Tooltip content="В начало трассы">
      <button type="button" aria-label="В нач
    }} disabled={!curDebug?.step} className="p-
    transition-colors">
        <SkipBack className="w-4 h-4" />
      </button>
    </Tooltip>
    <Tooltip content="Шаг назад (←) – автопро
      <button type="button" aria-label="Шаг на
    bled={!curDebug?.prev} className="p-1 round
    tion-colors">
        <ChevronLeft className="w-4 h-4" />
      </button>
    </Tooltip>
    <Tooltip content="Шаг вперёд (→) – автопро
      <button type="button" aria-label="Шаг в
    abled={!curDebug?.step || debug.idx >= debu
    e hover:bg-slate-800 disabled:opacity-40 tr
      <ChevronRight className="w-4 h-4" />
    </button>
  </Tooltip>

```

```

        и переменные текущего шага."}>
        <button
          type="button"
          aria-label={traceOpen ? "Свернуть лис
          aria-expanded={traceOpen}
          onClick={() => setTraceOpen((v) => !v
          className="ml-auto inline-flex shrink
ext-[10px] font-bold text-slate-300 transit
        >
          {traceOpen ? <ChevronDown className="
        ЛИСТИНГ
        </button>
      </Tooltip>
      <Tooltip content="Остановить прогон (Esc)
на показывает встроенное демо. Автозапуск н
      <button
        type="button"
        aria-label="Остановить прогон трассы"
        onClick={stopDebug}
        className="inline-flex shrink-0 items
ext-[10px] font-bold text-rose-300 transit
      >
        <CircleStop className="w-3.5 h-3.5" />
        стоп
        <kbd className="rounded border border
      </button>
    </Tooltip>
  </div>

```

```

{traceOpen && (
  <>
    /* листинг с подсветкой текущей строки */
    <div ref={listRef} className="flex-1 min-h-1
      {debug.src.split("\n").map((line, i) => {
        const ln = i + 1;
        const active = ln === curDebug?.step?.l
        return (
          <div

```

```

        <span
          key={name}
          className={`inline-flex items-base
            changed
              ? "border-amber-500/50 bg-amber-500/50"
              : synced
              ? "border-emerald-500/40 bg-emerald-500/40"
              : "border-slate-700 bg-slate-700"}
        >
          <span className={changed ? "text-amber-500" : "text-emerald-500"}>
            {name}
          </span>
          <span className="text-slate-500">
            {value}
          </span>
        </span>
      );
    }
  }
</div>
</div>
</>
)}
</div>
)}
{debug && !showTrace && debug.result.error ? (
  <pre className="shrink-0 max-h-28 overflow-auto py-2 font-mono text-[11px] leading-snug text-slate-500">
    {debug.result.error}
  </pre>
) : null}
<div className="shrink-0 border-t border-slate-700">
  <label htmlFor="python-stdin" className="text-slate-500">
    stdin для input()
  </label>
  <input
    id="python-stdin"
    value={stdin}
    onChange={(event) => {

```

```
    });  
  }  
}
```

ФАЙЛ 65/111: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useCallbak } from 'react';  
import { Sparkles } from 'lucide-react';  
import { useVizRuntime, vizArray, vizString } from '../utils';
```

```
interface Customer {  
  id: string;  
  name: string;  
  emoji: string;  
  color: string;  
  bubbleText: string;  
  animState: 'in' | 'idle' | 'eating' | 'out';  
}
```

```
const PRESETS = [  
  { name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue',  
    bubbleText: 'Матана!' },  
  { name: 'Кодер Семён', emoji: '👨‍💻', color: 'from-violet',  
    bubbleText: 'Порцию борща!' },  
  { name: 'Кот Борис', emoji: '🐈', color: 'from-amber',  
    bubbleText: 'Плиз.' },  
  { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose',  
    bubbleText: '' },  
  { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald',  
    bubbleText: 'Пиво для нашего НЛО!' },  
  { name: 'Бизнесмен', emoji: '👔', color: 'from-slate',  
    bubbleText: 'Тартапы.' },  
];
```

```
let counter = 3;
```

```

        setTimeout(() => setShake(false), 400);
        return;
    }

    const preset = PRESETS[counter % PRESETS.length];
    counter++;

    const newGuest: Customer = {
        id: Date.now().toString(),
        ...preset,
        animState: 'in',
    };

    setQueue(prev => [...prev, newGuest]);
    addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

    setTimeout(() => {
        setQueue(prev => prev.map(c => c.id === newGuest.id ? newGuest : c));
    }, 450);
}, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
    if (isServing) return;
    if (queue.length === 0) {
        setShake(true);
        addLog(`⚠ DEQUEUE: В очереди никого нет! Повар скучает`);
        setTimeout(() => setShake(false), 400);
        return;
    }

    setIsServing(true);
    const headGuest = queue[0];

    addLog(`- DEQUEUE: Повар наливает суп первому – ${headGuest.name} ${headGuest.emoji}`);

    // Step 1: Head guest gets eating animation
    setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, animState: 'eating' } : c));

```

```

    <div>
      <h3 className="font-black text-indigo-400 text-3xl">
        <p className="text-xs text-slate-300 mt-1 leading-4">
          В очереди за супом всё честно. Кто первый встал, тот и ест.
          Новые гости встают строго <b>в конец</b> очереди.
          Здесь нет обхода и блата – только строгий порядок.
        </p>
      </div>
    </div>

    { /* УПРАВЛЕНИЕ */ }
    <div className="flex items-center gap-3 flex-wrap">
      <button
        onClick={enqueue}
        disabled={isServing}
        className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-indigo-600
          text-white opacity-40 disabled:cursor-not-allowed text-center font-medium
          active:scale-95 transition-all cursor-pointer"
      >
        <span> Встать в очередь (ENQUEUE)</span>
      </button>

      <button
        onClick={dequeue}
        disabled={isServing || queue.length === 0}
        className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-indigo-600
          text-white opacity-40 disabled:cursor-not-allowed text-center font-medium
          active:scale-95 transition-all cursor-pointer flex-shrink-0"
      >
        <span> Налить суп первому (DEQUEUE)</span>
      </button>

      <button
        onClick={reset}
        className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium
          cursor-pointer border border-slate-700"
      >
        Сброс
      </button>
    </div>
  </div>
</div>

```

```

    <span className="text-[8px] font-mono t
  </div>

  {/* СПИСОК ОЧЕРЕДИ */}
  <div className="flex-1 flex items-end gap
    {displayQueue.length === 0 ? {
      <div className="flex-1 flex flex-col
        <span className="text-4xl mb-1"> </
        <p className="text-xs font-bold fon
        <p className="text-[10px]">Все сыты
      </div>
    } : {
      displayQueue.map(({customer, idx}) => {
        const isHead = idx === 0;
        const isTail = idx === displayQueue
        return (
          <div
            key={customer.id}
            className={`
              flex flex-col items-center ga
              ${customer.animState === 'in'
              ${customer.animState === 'eat
              ${customer.animState === 'out
            `}
          >
            {/* Облачко мыслей */}
            <div className={`
              absolute -top-16 left-1/2 -tr
            8 text-center text-slate-200 leading-tight
              ${isHead ? 'opacity-100 borde
            `}>
              <div className="font-bold tex
                {customer.bubbleText}
              <div className="absolute -bot
            it rotate-45"></div>
            </div>

            {/* Персонаж */}

```



```

        order-emerald-800/80 text-[9px] font-mono">
        <Sparkles className="w-3.5 h-3.5 animate-pu
        <span>СЫТЫ </span>
    </div>

    <div className="flex-1 flex flex-col justify-
        {servedHistory.length === 0 ? (
            <div className="flex-1 flex flex-col item
                <span className="text-5xl mb-2"> </span>
                <p className="text-xs font-bold font-mo
                <p className="text-[10px] mt-1">Здесь 6
            </div>
        ) : (
            <div className="flex flex-col gap-1.5 w-f
                {servedHistory.map((item, idx) => (
                    <div
                        key={idx}
                        className="w-full py-1.5 px-3 round
0 flex items-center gap-2 text-[11px] text-
                    >
                        <span>{item}</span>
                    </div>
                ))}
            </div>
        )}
    </div>

    {/* Пол */}
    <div className="w-full h-3 bg-gradient-to-r f
    </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
    <div className="text-[10px] font-mono text-slate
        {log.map((entry, idx) => (
            <div
                key={idx}

```

```

const [newWord, setNewWord] = useState('');
const [nodes, setNodes] = useState<{ [key: number]: T >({});
const [simulationStep, setSimulationStep] = useState<number>(0);
const [bfsQueue, setBfsQueue] = useState<number[]>([]);
const [currentNode, setCurrentNode] = useState<number>(0);
const [speechText, setSpeechText] = useState<string>('Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и поиграем в эхо-буль!');
const [isTalking, setIsTalking] = useState<boolean>(false);
const [isBlinking, setIsBlinking] = useState<boolean>(false);

/* — связь с Python-компилятором (PAGE_SYNC["aho-compilers"])
   words/patterns – набор образцов: бор ПЕРЕСТРАИВАЕТСЯ
   state – текущая вершина автомата, step – шаг BFS, queue – очередь BFS
const chapterId = useContext(VizChapterContext);
const runtime = useVizRuntime();
const vars = runtime?.variables;

useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, "aho");
}, [chapterId]);

const liveWordsRaw = vars ? vizArray(vars.words) ?? vizArray() : vizArray();
const liveWordsSig = useMemo(() => (liveWordsRaw ? JSON.stringify(liveWordsRaw) : ''));
const liveState = vars ? vizNumber(vars.state) ?? vizNumber(0) : vizNumber(0);
const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(0) : vizNumber(0);
const liveQueue = vars ? vizArray(vars.queue) : null;
const liveNodes = vars ? vizNumber(vars.nodes) : null;
const liveFail = vars ? vizRecord(vars.fail) : null;

// набор образцов из кода перестраивает бор целиком
useEffect(() => {
  if (!liveWordsSig) return;
  const parsed = (JSON.parse(liveWordsSig) as unknown[]).filter((w) => Boolean(w)).map((w) => w.toUpperCase().replace(/[^A-ZА-Я]/g, ''));

```

```

useEffect(() => {
  setIsTalking(true);
  const timer = setTimeout(() => setIsTalking(false),
  return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
  const newNodes: { [key: number]: TrieNode } = {
    0: { id: 0, char: 'ROOT', parent: 0, children: {}
  };
  let count = 0;

  wordList.forEach((word) => {
    let curr = 0;
    for (let i = 0; i < word.length; i++) {
      const c = word[i].toUpperCase();
      if (newNodes[curr].children[c] === undefined) {
        count++;
        newNodes[curr].children[c] = count;
        newNodes[count] = {
          id: count,
          char: c,
          parent: curr,
          children: {},
          fail: 0,
          term_link: 0,
          is_terminal: false,
          words: [],
          depth: newNodes[curr].depth + 1
        };
      }
      curr = newNodes[curr].children[c];
    }
    if (!newNodes[curr].words.includes(word)) {
      newNodes[curr].words.push(word);
      newNodes[curr].is_terminal = true;
    }
  });
}

```

```

useEffect(() => {
  buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
  e.preventDefault();
  const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
  if (!cleanWord) return;
  if (words.includes(cleanWord)) {
    setSpeechText(`Слово «${cleanWord}» уже есть в слове`);
    return;
  }
  const updated = [...words, cleanWord];
  setWords(updated);
  setNewWord('');
  buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
  if (words.length <= 1) {
    setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить');
    return;
  }
  const updated = words.filter((w) => w !== wordToRemove);
  setWords(updated);
  buildInitialTrie(updated);
};

// Занульск BFS
const startBFS = () => {
  const rootChildren = Object.values(nodes[0].children);
  const queue = [...rootChildren];
  const updatedNodes = { ...nodes };
  rootChildren.forEach((childId) => {
    updatedNodes[childId].fail = 0;
  });

  setNodes(updatedNodes);

```

```

        // Рассчитываем терминальную ссылку (term_link)
        if (updatedNodes[failTarget].is_terminal) {
            updatedNodes[u].term_link = failTarget;
        } else {
            updatedNodes[u].term_link = updatedNodes[failTa
        }

        logs += `Для #${u} ('${char}') fail = #${failTarg
    }

    if (childrenIds.length === 0) {
        logs += 'У этой вершины нет детей, просто переход
    }

    setNodes(updatedNodes);
    setBfsQueue(newQueue);
    setCurrentNode(r);
    setSpeechText(logs);
};

return (
    <div className="bg-slate-800/90 rounded-2xl p-6 bor
        {/* Шапка виджета */}
        <div className="flex flex-wrap items-center justifi
            <div className="flex items-center space-x-3">
                <span className="text-3xl"> </span>
                <div>
                    <h4 className="text-2xl font-bold text-white
                        Интерактивный конструктор: Говорящий Эхо-
                    </h4>
                    <p className="text-sm text-slate-400">
                        Построение дерева и расчет суффиксных ссы
                    </p>
                </div>
            </div>
            <div className="flex items-center gap-2">
                <button
                    onClick={() => buildInitialTrie(words)}

```

```

        fill="#f87171"
        className="origin-[120px_115px] animate
    />

    {/* Глаз (с морганием) */}
    <circle cx="145" cy="85" r="12" fill="#ff
    {isBlinking ? (
        <line x1="133" y1="85" x2="157" y2="85"
    ) : (
        <>
            <circle cx="148" cy="85" r="6" fill="
            <circle cx="151" cy="82" r="2" fill="
        </>
    )}

    {/* Рот (анимация разговора) */}
    {isTalking ? (
        <path d="M 165 105 Q 155 120 165 125 Q
    ) : (
        <path d="M 165 110 Q 155 115 168 118" s
    )}

    {/* Улыбка / жабры */}
    <path d="M 130 80 Q 125 100 130 120" stro
    <path d="M 120 85 Q 115 100 120 115" stro
</svg>

    {/* Декоративные пузыри */}
    <div className="absolute -top-1 right-4 tex
    <div className="absolute top-8 right-1 text
</div>
    <span className="mt-3 bg-rose-600/30 border b
        Эхо-Лосось Сеня
    </span>
</div>

    {/* Пузырь с текстом (Баббл) */}
    <div className="md:col-span-2 bg-slate-800 bord

```

```

        <span className="bg-blue-800 px-1.5 p-1">
        </button>
    })
    {simulationStep === 2 && (
        <button
            onClick={() => buildInitialTrie(words)}
            className="bg-indigo-600 hover:bg-indigo-500"
            transition-transform active:scale-95"
        >
            <RotateCcw className="w-4 h-4" /> Пер
        </button>
    })
</div>
</div>
</div>
</div>

```

```

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800" >
    <h5 className="text-white font-bold mb-3 flex items-center">
        <span> </span> Дерево и Суффиксные (fail) ссылки
    </h5>

```

```

    {(() => {
        let maxX = 0;
        let maxY = 0;
        Object.values(nodes).forEach(n => {
            if (n.x && n.x > maxX) maxX = n.x;
            if (n.y && n.y > maxY) maxY = n.y;
        });
        const svgWidth = Math.max(maxX + 80, 400);
        const svgHeight = Math.max(maxY + 60, 200);

        return (
            <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`} >
                <defs>
                    <marker id="arrowhead" markerWidth="6" markerHeight="6"
                        <polygone points="0 0, 6 3, 0 6" fill="white" />
                </defs>
            </svg>
        )
    })}

```

```

    { /* Draw Fail Links */
    {simulationStep > 0 && Object.values(nodes)
      if (node.id === 0 || node.fail === 0) return
    }
    // Wait, all nodes get fail link. Let's draw it
    // To avoid too many lines, let's draw it at step
    // step >= 1 and computed it
    // To avoid too many lines, let's draw it at step
    const target = nodes[node.fail];
    if (!target || !node.x || !node.y || !target.x || !target.y) return

    // Simple bezier curve for fail link
    const isCurrent = currentNode === node.id

    return (
      <path
        key={`fail-${node.id}`}
        d={`M ${node.x} ${node.y - 10} Q ${node.x + 10} ${node.y - 16} ${node.x + 16} ${node.y - 16}`}
        fill="none"
        stroke={isCurrent ? '#f43f5e' : 'rgb(128, 128, 128)'}
        strokeWidth={isCurrent ? 2 : 1.5}
        strokeDasharray="4 4"
        markerEnd="url(#fail-arrow)"
      />
    );
  })}

  { /* Draw Nodes */
  {Object.values(nodes).map(node => {
    if (!node.x || !node.y) return null;
    const isRoot = node.id === 0;
    const isCurrent = currentNode === node.id;
    const isInQueue = bfsQueue.includes(node.id);

    let fill = '#1e293b';
    let stroke = '#475569';

    if (isCurrent) {

```



```

        </text>
      })
    </g>
  );
  }}}
</svg>
);
}}({})
</div>

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
  <div className="bg-slate-900/50 p-5 rounded-xl">
    <div>
      <h5 className="text-white font-bold mb-3">
        <span> </span> Словарь (Паттерны)
      </h5>
      <div className="flex flex-wrap gap-2 mb-4">
        {words.map((w) => {
          <span
            key={w}
            className="bg-slate-800 border border-slate-700
              px-2 py-1 gap-1 group"
          >
            {w}
            <button
              onClick={() => removeWord(w)}
              className="text-slate-500 hover:text-white"
              title="Удалить"
            >
              ✕
            </button>
          </span>
        )}}
      </div>
    </div>

    <form onSubmit={handleAddWord} className="flex

```

```

      <th className="py-2 px-3">term_link</th>
      <th className="py-2 px-3">Дети</th>
      <th className="py-2 px-3">Слова (dict)
    </tr>
  </thead>
  <tbody className="divide-y divide-slate-800">
    {Object.values(nodes).map((node) => {
      const isCurrent = currentNode === node
      const isInQueue = bfsQueue.includes(node)

      return (
        <tr
          key={node.id}
          className={`transition-colors ${
            isCurrent
              ? 'bg-blue-900/40 font-bold text-white'
              : isInQueue
              ? 'bg-slate-800/80'
              : 'hover:bg-slate-800/40'
            }`}
        >
          <td className="py-2.5 px-3 flex items-center">
            <span
              className={`w-2 h-2 rounded-full ${
                node.id === 0
                  ? 'bg-purple-500'
                  : isCurrent
                  ? 'bg-blue-400 animate-pulse'
                  : isInQueue
                  ? 'bg-amber-400'
                  : 'bg-slate-600'
                }`}
            ></span>
            <span>#{node.id}</span>
            {node.id === 0 && <span className="font-size-0.8">root</span>}
          </td>
          <td className="py-2.5 px-3">
            <span className="bg-slate-800 block w-10 h-10 rounded" style="display: inline-block; margin-right: 10px;">

```

```

                </span>
            }}
        </td>
    </tr>
    );
    }}}
</tbody>
</table>
</div>
</div>
</div>

```

```

/* что сейчас сообщил компилятор */
<div className="mt-4 flex flex-wrap items-center" >
    <span className="text-[10px] uppercase tracking
        <Terminal className="w-3.5 h-3.5" /> из компи
    </span>
    {compilerLinked ? (
        <>
            {liveWordsSig && <span className="text-[11px]
            y-1">words = {words.join(", ")}</span>}
            {liveNodes !== null && <span className="tex
            x-2 py-1">вершин в бопе = {liveNodes}</span>}
            {liveState !== null && <span className="tex
            -md px-2 py-1">state = {liveState}</span>}
            {liveStep !== null && <span className="text
            -2 py-1">шаг BFS = {liveStep}</span>}
            {liveQueue && <span className="text-[11px]
            -1">queue = {JSON.stringify(liveQueue).slice
        </>
    ) : (
        <span className="text-[11px] text-slate-500">
            открой панель Python: массив <code classNam
            nt-mono text-cyan-300">state</code>/<code c
        </span>
    )}
</div>
</div>

```

```
}
```

```
// ===== STEP GENERATORS =====
```

```
// 1. Build steps (recursive top-down)
```

```
function genBuildSteps(arr: number[]): Step[] {
```

```
    const n = arr.length;
```

```
    const steps: Step[] = [];
```

```
    let id = 0;
```

```
    const tree: Record<number, number> = {};
```

```
    const CODE = [
```

```
        "build(v, l, r) {", // 1
```

```
        "    if (l === r) {", // 2
```

```
        "        tree[v] = A[l]; return", // 3
```

```
        "    }", // 4
```

```
        "    mid = (l + r) >> 1", // 5
```

```
        "    build(2v, l, mid)", // 6
```

```
        "    build(2v+1, mid+1, r)", // 7
```

```
        "    tree[v] = tree[2v]+tree[2v+1]", // 8
```

```
        "}", // 9
```

```
];
```

```
void CODE;
```

```
function go(v: number, l: number, r: number) {
```

```
    if (l === r) {
```

```
        steps.push({ id: id++, desc: `build(${v}, ${l}, $
```

```
        , highlightNodes: [v], arrayHighlight: [l, r] } }
```

```
        tree[v] = arr[l];
```

```
        steps.push({ id: id++, desc: `tree[${v}] = ${arr[
```

```
        l, r] } });
```

```
        return;
```

```
    }
```

```
    const m = (l + r) >> 1;
```

```
    steps.push({ id: id++, desc: `build(${v}, ${l}, ${r
```

```
        { ...tree }, highlightNodes: [v], arrayHighlight:
```

```
        go(2 * v, l, m);
```

```
        steps.push({ id: id++, desc: `Вернулись в build(${v
```

```

    return res;
  }
  const ans = go(1, 0, n - 1, qL, qR);
  steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`,
    highlightNodes: [1], arrayHighlight: [qL, qR], result: ans });
  return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number): Step[] {

```

  const n = arr.length;
  // Build iterative tree: leaves at [n..2n-1]
  const tree: Record<number, number> = {};
  for (let i = 0; i < n; i++) tree[n + i] = arr[i];
  for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]) / 2;

```

```

  const steps: Step[] = [];
  let id = 0;
  let l = qL + n;
  let r = qR + n + 1; // half-open [l, r)
  let res = 0;

```

```

  steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}`,
    codeLine: 3, treeState: tree, highlightNodes: [l, r] });

```

```

  while (l < r) {
    const highlights: number[] = [];
    if (l & 1) {
      res += tree[l] ?? 0;
      steps.push({ id: id++, desc: `l=${l} нечётный → l`,
        codeLine: 4, treeState: tree, highlightNodes: [l], result: res });
      highlights.push(l);
      l++;
    }
    if (r & 1) {
      r--;
      res += tree[r] ?? 0;
      steps.push({ id: id++, desc: `r=${r + 1} нечётный → r`,
        codeLine: 5, treeState: tree, highlightNodes: [r + 1], result: res });
      highlights.push(r + 1);
    }
    l = (l + 1) / 2;
    r = (r - 1) / 2 + 1;
    treeState = tree;
    highlightNodes = [...highlights];
  }
  return steps;
}

```

```

"query(v, l, r, ql, qr) {",
"  if (ql > r || qr < l) return 0;",
"  // -----",
"  if (ql <= l && r <= qr) return tree[v];",
"  // -----",
"  mid = (l + r) >> 1;",
"  // спускаемся в обоих детей",
"  return query(left) + query(right);",
"}"
];
const QUERY_BU_CODE = [
  "queryBU(ql, qr) {",
  "  res = 0;",
  "  l = ql + n; r = qr + n + 1;",
  "  // -----",
  "  while (l < r) {",
  "    if (l & 1) res += tree[l++];",
  "    // -----",
  "    if (r & 1) res += tree[--r];",
  "    l >>= 1; r >>= 1;",
  "  }",
  "  return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
  const [idx, setIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [speed, setSpeed] = useState(700);

  useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

  useEffect(() => {
    if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
      const t = setTimeout(() => setIdx(i => i + 1), speed);
      return () => clearTimeout(t);
    }
  }, [playing, idx, steps, speed]);

  const step = steps[idx] || steps[0];

```

```

    <span className="text-[9px] font-bold">[{nd.l
    <span className="text-xs font-extrabold font-
</div>
    {(nd.left || nd.right) && (
      <div className="flex flex-col items-center mt
        <div className="w-px h-2 bg-slate-700" />
        <div className="flex justify-around w-full l
          {nd.left && renderNode(nd.left)}
          {nd.right && renderNode(nd.right)}
        </div>
      </div>
    )}
  </div>
);
}, [step, clr]);

return (
  <div className={`rounded-2xl border overflow-hidden
    ? 'border-emerald-500/30' : 'border-amber-500/30'
  }/* Header */>
    <div className={`px-4 py-2.5 flex items-center ju
      -emerald-950/50' : 'bg-amber-950/50'} border-b l
      <h4 className="font-bold text-sm text-white fle
        <span className="text-[10px] font-mono text-sla
      </div>

    {/* Array strip */}
    <div className="px-3 py-2 bg-slate-950 border-b b
      {arr.map((v, i) => {
        const inRange = step.arrayHighlight && i >= s
        return (
          <div key={i} className={`flex flex-col item
            go' ? 'bg-indigo-950 border-indigo-500' : c
            er-500'} -translate-y-0.5` : 'bg-slate-900 l
              <span className="text-[7px] text-slate-50
              <span className="text-[11px] font-bold for
            </div>
          );
        }
      )}
    </div>
  );
);

```

```

-----
    <div className="px-3 py-2.5 bg-slate-950 border-t
      <div className="flex items-center bg-slate-900
        <button onClick={() => { setPlaying(false); s
          unded" title="Сброс"><RotateCcw className="
        <button onClick={() => { setPlaying(false); s
          Шаг назад" className="p-1.5 text-slate-400
          Name="w-3.5 h-3.5" /></button>
        <button onClick={() => setPlaying(!playing)}
          white transition-all ${playing ? 'bg-rose-5
          {playing ? <Pause className="w-3 h-3" /> :
        </button>
        <button onClick={() => { setPlaying(false); s
          р вперёд" title="Шаг вперёд" className="p-1
          <ChevronRight className="w-3.5 h-3.5" /></b
        </div>
      <select value={speed} onChange={e => setSpeed(N
        text-[9px] font-bold rounded px-1.5 py-1 outl
        <option value={1200}>Медл.</option>
        <option value={700}>Норма</option>
        <option value={350}>Быстро</option>
        <option value={120}>Турбо</option>
      </select>
    </div>

    {/* Progress bar */}
    <div className="h-1 bg-slate-950"><div className=
      + 1) / total) * 100 : 0}%` }} /></div>
  </div>
  );
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
  const runtime = useVizRuntime();
  const liveI = vizNumber(runtime?.variables?.i);
  const [arr, setArr] = useState<number[]>([5, 8, 3, 12

```



```

<form onSubmit={handleApply} className="flex-
  <div className="flex items-center justify-b
    <label className="text-[10px] font-bold up
  label>
    <button type="button" onClick={randomize}
  "><RefreshCw className="w-3.5 h-3.5" /></bu
</div>
<div className="flex gap-2">
  <input
    value={customInput}
    onChange={e => setCustomInput(e.target.v
    className="flex-1 bg-slate-900 border b
one focus:border-indigo-500"
    placeholder="5, 8, 3, 12, 7, 2"
  />
  <button type="submit" className="bg-indigo
ition-colors">Применить</button>
</div>
<div className="flex flex-wrap gap-1.5 pt-1
  {arr.map((v, i) => (
    <span key={i} className={`border px-2 p
ext-amber-200 ring-2 ring-amber-400/30` : "l
    <span className={liveI === i ? "text-
  </span>
  </span>
  )))}
  <span className="text-xs text-slate-500 f
</div>
</form>

```

```

{/* Query range */}
<div className="bg-slate-950 p-4 rounded-xl b
  <label className="text-[10px] font-bold upp
  <div className="flex items-center gap-2">
    <div className="flex items-center gap-1">
      <span className="text-xs text-slate-400
    <input
      type="number"
      min={0}

```

```

    </button>
    <button onClick={() => setView('theory')} className=
      n-colors ${view === 'theory' ? 'border-blue-500' : 'border-teal-200'}`>
      <Info className="w-3.5 h-3.5" /> Почему / Сравн
    </button>
  </div>

```

```

{ /* ---- TAB CONTENT ---- */
view === 'build' && (
  <SimPanel
    key={`build-${arr.join('-')}`}
    title="Построение дерева (рекурсия)"
    icon="🌳"
    steps={buildSteps}
    code={BUILD_CODE}
    color="indigo"
    arr={arr}
  />
)}

```

```

view === 'queries' && (
  <div className="grid grid-cols-1 xl:grid-cols-2" >
    <SimPanel
      key={`qtd-${arr.join('-')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Сверху` }
      icon="📈"
      steps={queryTDSteps}
      code={QUERY_TD_CODE}
      color="emerald"
      arr={arr}
    />
    <SimPanel
      key={`qbu-${arr.join('-')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Снизу` }
      icon="📈"
      steps={queryBUSteps}
      code={QUERY_BU_CODE}
    />
  </div>
)

```



```

const filtered = q ? chapters.filter((c) => c.title
return SECTIONS.map((section) => ({
  section,
  items: filtered.filter((c) => sectionOf(c.id).id === c.id)
})).filter((g) => g.items.length > 0);
}, [query]);

return (
  <aside className="w-full bg-slate-900/80 lg:bg-transparent" >
    <div className={compact ? "p-2 pb-1.5 shrink-0" : "p-2 pb-1.5"} >
      {compact && (
        <h3 className="text-[10px] font-bold text-slate-400" >
          <Compass className="w-3.5 h-3.5 text-indigo-400" />
        </h3>
      )}
      <h3 className={`text-xs font-bold text-slate-400`} >
        <Compass className="w-4 h-4 text-indigo-400" />
      </h3>
      {!compact && (
        <div className="relative" >
          <Search className="w-4 h-4 text-slate-500 absolute left-0 top-0" >
            <input
              value={query}
              onChange={(e) => setQuery(e.target.value)}
              placeholder="Поиск по темам.."
              className="w-full bg-slate-800/70 border border-slate-500 focus:outline-none focus:border-indigo-500"
            />
            {query && (
              <button
                onClick={() => setQuery("")}
                className="absolute right-2 top-1/2 -translate-y-1/2 text-slate-500"
                aria-label="Очистить поиск"
              />
              <X className="w-3.5 h-3.5" />
            )}
          </div>
        )}
      )}
    </div>
  </aside>
)

```

```

        >
        <span className="shrink-0 mt-px rounded-b
nt-bold text-indigo-300">
          {num ? num[1] : "*"}
        </span>
        <span className={`font-semibold lea
          {label}
          {hasViz && (
            <span className="ml-1.5 inline-b
ая визуализация" />
          )}
        </span>
        {!compact && (
          <ChevronRight
            className={`w-4 h-4 shrink-0 mt
              isSelected ? "text-indigo-400
            }`}
          />
        )}
      </button>
    );
  }}}
</div>
</div>
)}}
</div>

```

```

<div className={`shrink-0 m-3 mt-0 bg-gradient-to
x] text-slate-300 space-y-1.5 hidden lg:block $
<div className="font-bold text-indigo-400 flex .
  <Sparkles className="w-3.5 h-3.5" /> Как поль.
</div>
<p className="leading-relaxed">
  <b className="text-slate-200">Порядок чтения
  номерам билетов 1 → 24, каждая опирается на пр
  Следующая тема» продолжают этот же порядок.
</p>
<p className="leading-relaxed">

```

```
// Stack state
const [stack, setStack] = useState<Plate[]>([
  { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
  { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
  { id: '3', name: 'Блюдец от торта', icon: ' ' },
]);
const [popMessage, setPopMessage] = useState<string | null>('');

// Queue state
const [queue, setQueue] = useState<Person[]>([
  { id: '1', name: 'Студент Вася', icon: ' ' },
  { id: '2', name: 'Голодный кодер', icon: ' ' },
  { id: '3', name: 'Кот Борис', icon: ' ' },
]);
const [dequeueMessage, setDequeueMessage] = useState<string | null>('');

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
  { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"', icon: ' ' },
  { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"', icon: ' ' },
  { id: '3', text: 'Кандидат: "Привет, я испанский интеллект"', icon: ' ' },
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.5);
const [heapMessage, setHeapMessage] = useState<string | null>('');

// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: ' ' },
    { name: 'Кастрюля от супа', icon: ' ' },
    { name: 'Чашка от кофе', icon: '☑' },
    { name: 'Миска с остатками салата', icon: ' ' },
    { name: 'Тарелка от тако', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: `plate-${Date.now()}`,
    name: randomPreset.name,
    icon: randomPreset.icon,
  };
  setStack((prev) => [...prev, newPlate]);
  setPopMessage('');
};

const popPlate = () => {
  setStack((prev) => prev.slice(0, -1));
  setPopMessage('Плита очищена');
};

const addPerson = () => {
  const presets = [
    { name: 'Студент Вася', icon: ' ' },
    { name: 'Голодный кодер', icon: ' ' },
    { name: 'Кот Борис', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPerson = {
    id: `person-${Date.now()}`,
    name: randomPreset.name,
    icon: randomPreset.icon,
  };
  setQueue((prev) => [...prev, newPerson]);
  setDequeueMessage('');
};

const dequeuePerson = () => {
  setQueue((prev) => prev.slice(0, -1));
  setDequeueMessage('Очередь очищена');
};

const addHypothesis = () => {
  const presets = [
    { text: 'Кандидат: "Привет, я искусственный интеллект"', icon: ' ' },
    { text: 'Кандидат: "Привет, я испек вкусный хлеб"', icon: ' ' },
    { text: 'Кандидат: "Привет, я испанский интеллект"', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newHypothesis = {
    id: `hypothesis-${Date.now()}`,
    text: randomPreset.text,
    icon: randomPreset.icon,
  };
  setHeap((prev) => [...prev, newHypothesis]);
  setHeapMessage('');
};

const removeHypothesis = () => {
  setHeap((prev) => prev.slice(0, -1));
  setHeapMessage('Гипотеза удалена');
};
```

```

        name: randomPreset.name,
        icon: randomPreset.icon
    };
    setQueue(prev => [...prev, newPerson]);
    setDequeueMessage(` Встал в конец очереди: ${newPe
});

const servePerson = () => {
    if (queue.length === 0) {
        setDequeueMessage("⚠ Очередь пуста! Суп ждет нов
        return;
    }
    const firstPerson = queue[0];
    setQueue(prev => prev.slice(1));
    setDequeueMessage(` Выдан суп первому в очереди (F
});

const resetQueue = () => {
    setQueue([
        { id: '1', name: 'Студент Вася', icon: ' ' },
        { id: '2', name: 'Голодный кодер', icon: ' ' },
        { id: '3', name: 'Кот Борис', icon: ' ' },
    ]);
    setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
    e.preventDefault();
    if (!newCustomText.trim()) return;
    const item: Hypothesis = {
        id: Date.now().toString(),
        text: `Кандидат: "${newCustomText}"`,
        probability: Number(newCustomProb)
    };
    setHeap(prev => [...prev, item]);
    setNewCustomText('');
    setHeapMessage(` Добавлена гипотеза с вероятностью

```

```

    <p className="text-slate-400 text-sm mt-2">
      Проверь работу принципов LIFO, FIFO и Приоритет
    </p>
  </div>

  { /* Tabs */ }
  <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
    <button
      onClick={() => setActiveTab('stack')}
      className={`flex items-center justify-between border-1
        activeTab === 'stack'
          ? 'bg-blue-600/20 border-blue-500 text-blue-600'
          : 'bg-slate-800/60 border-slate-700/60 text-slate-400'
      }`>
      >
      <div className="flex items-center gap-3">
        <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'text-blue-600' : 'text-slate-400'}>
          <div className="text-left">
            <div className="font-bold text-sm text-white">LIFO
            <div className="text-xs opacity-80">LIFO
          </div>
        </div>
      </div>
      <span className="text-xs bg-blue-500/20 text-blue-600">
        DFS
      </span>
    </button>

    <button
      onClick={() => setActiveTab('queue')}
      className={`flex items-center justify-between border-1
        activeTab === 'queue'
          ? 'bg-indigo-600/20 border-indigo-500 text-indigo-600'
          : 'bg-slate-800/60 border-slate-700/60 text-slate-400'
      }`>
      >
      <div className="flex items-center gap-3">
        <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-indigo-600' : 'text-slate-400'}>
          <div className="text-left">

```



```

        <span className="text-xs font-mono bg-blue-500 text-white">
        </h3>
        <p className="text-slate-400 text-xs mt-1">
        Новые тарелки ставятся наверх. Мыть можн
        </p>
    </div>
    <div className="flex flex-wrap items-center">
        <button
            onClick={addPlate}
            className="flex-1 md:flex-none flex item
            rounded-xl text-sm font-bold shadow-lg tra
        >
            <Plus className="w-4 h-4" /> Поставить
        </button>
        <button
            onClick={popPlate}
            className="flex-1 md:flex-none flex item
            border-blue-500/40 px-4 py-2.5 rounded-xl
        >
            Помыть верхнюю
        </button>
        <button
            onClick={resetStack}
            title="Сбросить"
            className="p-2.5 bg-slate-800 hover:bg-
            tion-colors cursor-pointer"
        >
            <RotateCcw className="w-5 h-5" />
        </button>
    </div>
</div>

{popMessage && (
    <div className="bg-blue-950/60 border borde
    imate-fade-in">
        <span className="inline-block w-2 h-2 roun
        {popMessage}
    </div>

```

```

        <span className={`text-xs font-mono
            isTop ? 'bg-blue-500 text-white' : ''
        }`>
            {isTop ? 'Готово к помывке' : ''}
        </span>
    </div>
    );
    }
}
</div>
)}
<div className="mt-6 text-xs text-slate-500" >
    ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
</div>
</div>
</div>
)}

{/* QUEUE TAB */}
{activeTab === 'queue' && (
    <div className="space-y-6">
        <div className="bg-slate-800/80 p-5 rounded-xl
            justify-between gap-4">
            <div>
                <h3 className="text-lg font-bold text-white">
                    <span> Симуляция очереди за супом</span>
                    <span className="text-xs font-mono bg-blue-500 text-white"
                    /span>
                </h3>
                <p className="text-slate-400 text-xs mt-1">
                    Кто первым пришел, тот первым получил суп
                </p>
            </div>
            <div className="flex flex-wrap items-center">
                <button
                    onClick={addPerson}
                    className="flex-1 md:flex-none flex items-center
                    justify-center gap-2.5 rounded-xl text-sm font-bold shadow-lg"
                >

```

```

<div className="w-full flex items-center" >
  <div className="flex flex-col items-center" >
    300">
      <span className="text-3xl"> </span>
      <span className="text-xs font-bold font">
    </div>

    <div className="text-indigo-500 flex it">
      <ArrowRight className="w-6 h-6 animat">
    </div>

    {queue.map((person, index) => {
      const isFirst = index === 0;
      return (
        <div
          key={person.id}
          className={`flex flex-col items-c
            isFirst
              ? 'bg-gradient-to-b from-indi
relative'
              : 'bg-slate-900/90 border-sla
          }`}
        >
          {isFirst && (
            <span className="absolute -top-1
se tracking-wider shadow">
              Первый (Head)
            </span>
          )}
          <span className="text-4xl mb-2">{
            <span className={`font-bold text-
              {person.name}
            </span>
            <span className={`text-[10px] fon
              isFirst ? 'bg-indigo-500/30 tex
            }`}>
              {isFirst ? 'Получает сун' : `В
            </span>

```

```

        >
        Взять кандидата с макс. вероятностью
    </button>
    <button
        onClick={resetHeap}
        title="Сбросить"
        className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
    >
        <RotateCcw className="w-5 h-5" />
    </button>
</div>
</div>

{/* Add Form */}
<form onSubmit={addHypothesis} className="bg-
s-12 gap-4 items-end">
    <div className="md:col-span-6">
        <label className="block text-xs font-bold">
            <input
                type="text"
                value={newCustomText}
                onChange={e => setNewCustomText(e.target.value)}
                placeholder='например: "Привет, я лучший!"'
                className="w-full bg-slate-900 border border-
rder-emerald-500 transition-colors"
            />
        </div>
        <div className="md:col-span-3">
            <label className="block text-xs font-bold">
                Вероятность: <span className="text-emerald-500">
            </label>
            <input
                type="range"
                min="0.01"
                max="0.99"
                step="0.01"
                value={newCustomProb}
            />
        </div>
    </div>
</form>

```

```

        <span>ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)</span>
    </div>

    {sortedHeap.map(({item, index}) => {
      const isTop = index === 0;
      const percent = (item.probability * 100).toFixed(
      return {
        <div
          key={item.id}
          className={`flex items-center justify-between
            isTop
              ? 'bg-gradient-to-r from-emerald-500 to-slate-900/10 scale-[1.01]'
              : 'bg-slate-900/80 border-slate-800'
            }`}
        >
          <div className="flex items-center justify-between" >
            <span className={`flex items-center justify-between
              isTop ? 'bg-emerald-500 text-white' : 'text-slate-400'
            }`>
              #{index + 1}
            </span>
            <div>
              <div className={`font-bold text-white`} >
                {item.text}
              </div>
              {isTop && (
                <span className="text-[10px] text-white" >
                  Вершина Кучи (Root) • Будущее
                </span>
              )}
            </div>
          </div>
        </div>

        <div className="flex items-center justify-between" >
          {/* Custom progress bar */}
          <div className="hidden sm:block"

```

```

/**
 * Страница splay – три вкладки по нарастанию сложности
 * 1. «Механика» – покадровый разбор Zig / Zig-Zig / Zig-Zag
 *    (прицел → поворот, призраки старых позиций, обход)
 * 2. «Большое дерево» – те же случаи внутри настоящего
 *    поворот – кадры «прицел → отстёгиваем среднее по
 * 3. «Собери сам» – песочница: подними узел кликами,
 */
type Tab = "cases" | "big" | "play";

export const SplayPlayground: React.FC<{ defaultTab?: Tab }> = ({ defaultTab }) => {
  const [tab, setTab] = useState<Tab>(defaultTab);
  const chapterId = useContext(VizChapterContext);

  // вкладка = подрежим страницы: панель Python подстав
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, tab);
  }, [chapterId, tab]);

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border">
      <div className="flex flex-wrap items-center justify-between">
        <div className="flex items-center gap-3">
          <div className="p-2.5 bg-indigo-600 text-white">
            <span className="text-2xl"> </span>
          </div>
          <div>
            <h3 className="text-2xl font-bold text-white">
              <p className="text-sm text-indigo-300">Один
            </div>
          </div>
        </div>
      </div>

      <div className="flex flex-wrap gap-2 mb-5">
        {[
          { id: "cases" as Tab, label: "1 · Механика: Zig-Zig-Zag" },
          { id: "big" as Tab, label: "2 · Большое дерево" },
          { id: "play" as Tab, label: "3 · Собери сам" }
        ]}
      </div>
    </div>
  );
};

```

* Песочница поворотов.

*

* Здесь ничего не подсказывается: дано настоящее дерево

* поднять отмеченный узел в корень. Клик по узлу = один

* Порядок поворотов выбираешь сам; правильный ли он по

* только после того, как узел окажется наверху (или по

*/

```
export interface TNode {  
  key: number;  
  left: TNode | null;  
  right: TNode | null;  
}
```

```
const leaf = (key: number): TNode => ({ key, left: null
```

```
const clone = (t: TNode | null): TNode | null =>  
  t ? { key: t.key, left: clone(t.left), right: clone(t
```

```
/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
```

```
export const PRESETS: { name: string; hintCase: string;
```

```
{
```

```
  name: "Линия влево",
```

```
  hintCase: "Zig-Zig",
```

```
  target: 1,
```

```
  build: () => {
```

```
    const n1 = leaf(1);
```

```
    const n2: TNode = { key: 2, left: n1, right: leaf
```

```
    const n4: TNode = { key: 4, left: n2, right: leaf
```

```
    return { key: 6, left: n4, right: leaf(7) };
```

```
  },
```

```
},
```

```
{
```

```
  name: "Змейка",
```

```
  hintCase: "Zig-Zag",
```

```
  target: 3,
```

```
  build: () => {
```

```
    const n3 = leaf(3);
```

```

-----
export function rotateUp(root: TNode, key: number): TNode {
  const path = findPath(root, key);
  if (path.length < 2) return root;

  const x = path[path.length - 1];
  const p = path[path.length - 2];
  const gg = path.length >= 3 ? path[path.length - 3] : null;

  if (p.left === x) {
    p.left = x.right;
    x.right = p;
  } else {
    p.right = x.left;
    x.left = p;
  }

  if (!gg) return x;
  if (gg.left === p) gg.left = x;
  else gg.right = x;
  return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number): TNode {
  const path = findPath(root, key);
  if (path.length < 2) return null;
  const x = path[path.length - 1];
  const p = path[path.length - 2];
  const g = path.length >= 3 ? path[path.length - 3] : null;

  if (!g) return { node: x.key, kind: "Zig" };

  const xLeft = p.left === x;
  const pLeft = g.left === p;
  if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
  return { node: x.key, kind: "Zig-Zag (сначала нижняя" };
}

```



```
export const SplayRotationSandbox: React.FC = () => {
  const [presetIdx, setPresetIdx] = useState(0);
  const preset = PRESETS[presetIdx];

  const [tree, setTree] = useState<TNode>(() => preset.root);
  const [history, setHistory] = useState<TNode[]>([]);
  const [moves, setMoves] = useState<{ node: number; correct: boolean }>([]);
  const [showAnswer, setShowAnswer] = useState(false);

  const target = preset.target;
  const solved = tree.key === target;

  const reset = useCallback(
    (idx = presetIdx) => {
      setPresetIdx(idx);
      setTree(PRESETS[idx].build());
      setHistory([]);
      setMoves([]);
      setShowAnswer(false);
    },
    [presetIdx]
  );

  const { nodes, edges, width, height } = useMemo(() => preset.graph, [preset]);
  const path = useMemo(() => new Set(findPath(tree, target)), [tree, target]);
  const expected = useMemo(() => (solved ? null : canonicalNext(tree, target)), [tree, target]);

  const clickNode = (key: number) => {
    if (solved) return;
    const p = findPath(tree, key);
    if (p.length < 2) return; // корень крутить не с кем

    const want = canonicalNext(tree, target);
    setHistory((h) => [...h, clone(tree) as TNode]);
    setMoves((m) => [...m, { node: key, correct: want?.key === key }]);
    setTree((t) => rotateUp(clone(t) as TNode, key));
  };
};
```

```

        >
        {p.name}
      </button>
    )))
  </div>

<div className="bg-slate-900 rounded-xl border bo
  <svg
    viewBox={`0 0 ${width} ${height}`}
    className="h-auto block mx-auto"
    style={{ minWidth: Math.min(width * 1.5, 420)
    role="img"
  >
    {edges.map(([a, b], i) => {
      const na = nodes.find((n) => n.key === a);
      const nb = nodes.find((n) => n.key === b);
      if (!na || !nb) return null;
      const onPath = path.has(a) && path.has(b);
      return {
        <line
          key={i}
          x1={na.x}
          y1={na.y}
          x2={nb.x}
          y2={nb.y}
          stroke={onPath ? "#6366f1" : "#475569"}
          strokeWidth={onPath ? 3 : 1.8}
          style={{ transition: "all 700ms cubic-b
        />
      };
    })}
  )))

  {nodes.map((n) => {
    const isTarget = n.key === target;
    const clickable = !solved && n.depth > 0;
    return {
      <g
        key={n.key}

```

```

    >
      <RotateCcw className="w-3.5 h-3.5" /> Заново
    </button>
    <button
      onClick={() => setShowAnswer((s) => !s)}
      className={`flex items-center gap-1.5 px-3 py-2
        showAnswer
          ? "bg-amber-600/20 border-amber-500 text-amber-500"
          : "bg-slate-800 border-slate-700 text-slate-200"}
    >
      {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
    </button>

    <span className="text-[11px] text-slate-500 ml-2">
      поворотов: {moves.length} • минимум: {minimalMoves}
    </span>
  </div>

  {showAnswer && !solved && expected && (
    <div className="mt-3 text-[12px] bg-amber-950/40 p-3">
      Сейчас splay крутил бы узел{" "}
      <span className="font-mono font-bold">{expected}</span>
    </div>
  )}

  {solved && (
    <div
      className={`mt-3 rounded-lg border px-3 py-2.5
        wrong === 0
          ? "bg-emerald-950/40 border-emerald-700/60 text-emerald-500"
          : "bg-rose-950/30 border-rose-800/60 text-rose-500"}
    >
      <div className="flex items-center gap-2 font-lt">
        {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : <XMark>
      </div>
    >
  )}

```

```
* Что сделано ради понятности:
*   * у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
*   сразу видно, что дерево остаётся деревом поиска;
*   * под деревом печатается обход слева направо: он ОДНОМЕРНЫЙ;
*   это и есть доказательство, что поворот ничего не двигает;
*   * кадр «прицел» ничего не двигает, только подсвечивает;
*   * на кадре «результат» остаются призраки – пунктирные линии
*   местах и стрелки «откуда → куда»;
*   * по умолчанию анимация НЕ крутится сама: пользователь
*   идёт в своём темпе. Автопрокрутка включается кнопкой.
*/
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
  pos: Pos;
  edges: [NodeId, NodeId][];
  kind: "start" | "aim" | "result";
  title: string;
  text: string;
  /** Пара, которую крутим на этом шаге: кадр-«прицел». */
  focus?: [NodeId, NodeId];
  /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
  moved?: NodeId[];
  ms: number;
}
```

```
interface Case {
  key: "zig" | "zig-zig" | "zig-zag";
  label: string;
  when: string;
  rotations: string;
  /** Номера узлов: роль → число. */
  keys: Partial<Record<NodeId, number>>;
  /** Обход слева направо – одинаковый на всех кадрах. */
  inorder: string[];
  /** Что означают поддеревья A..D. */
}
```

```

    },
    {
      pos: { x: [160, 50], A: [80, 130], p: [245, 130],
edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
kind: "result",
title: "Стало: 20 – корень",
text: "40 стало правым сыном двадцатки. Поддерев
      ева от 40 – это одно и то же место.",
moved: [{"x", "p", "B"}],
ms: RESULT_MS,
    },
  ],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
  pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
  pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
  key: "zig-zig",
  label: "Zig-Zig",
  when: "x и p – дети с ОДНОЙ стороны (оба слева или об
  rotations: "2 поворота, первым – верхний",
  keys: { x: 20, p: 40, g: 60 },
  inorder: ["A", "20", "B", "40", "C", "60", "D"],
  ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
  frames: [
    {
      ...ZZ_S0,
      kind: "start",
      title: "Было: прямая линия 60 → 40 → 20",
    }
  ]
}

```

```

        ms: RESULT_MS,
    },
],
};

/* ----- Zig-Zag -----
const ZG_S0 = {
    pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
    edges: [{"g", "p"}, {"g", "D"}, {"p", "A"}, {"p", "x"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
    pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
    edges: [{"g", "x"}, {"g", "D"}, {"x", "p"}, {"x", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
    key: "zig-zag",
    label: "Zig-Zag",
    when: "x и p – дети с РАЗНЫХ сторон (змейка)",
    rotations: "2 поворота, первым – нижний",
    keys: { x: 40, p: 20, g: 60 },
    inorder: ["A", "20", "B", "40", "C", "60", "D"],
    ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
    frames: [
        {
            ...ZG_S0,
            kind: "start",
            title: "Было: 60 → влево на 20 → вправо на 40",
            text: "Узлы идут «змейкой»: сначала шаг влево, по",
            ms: RESULT_MS,
        },
        {
            ...ZG_S0,
            kind: "aim",
            title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
            text: "Дед (60) стоит на месте и в этом повороте

```

```

sub: { fill: "#1e293b", stroke: "#475569" },
};

```

```

const ROLE_RU: Partial<Record<NodeId, string>> = {
  x: "x — ищем его",
  p: "p — родитель",
  g: "g — дед",
};

```

```

const isSubtree = (id: NodeId) => id === "A" || id ===

```

```

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
  const { pos, edges, focus, moved } = frame;
  const dim = (id: NodeId) => (focus ? !focus.includes(
  const ghosts = frame.kind === "result" ? (moved ?? [])

```

```

return (
  <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
    <defs>
      <marker id="splay-arrow" viewBox="0 0 10 10" re
        <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
      </marker>
    </defs>

```

```

{/* призраки: где узел стоял до поворота */}

```

```

{ghosts.map((id) => {
  const from = prev[id]!;
  const to = pos[id]!;
  const r = isSubtree(id) ? 15 : 20;
  const dx = to[0] - from[0];
  const dy = to[1] - from[1];
  const len = Math.hypot(dx, dy) || 1;
  const pad = r + 8;
  return (
    <g key={`ghost-${id}`} opacity={0.55}>
      <circle cx={from[0]} cy={from[1]} r={r} fil
        {len > pad * 2 + 6 && (
          <line

```

```

const c = COLOR[sub ? "sub" : id];
const r = sub ? 15 : 20;
const hot = Boolean(focus && focus.includes(id));
return (
  <g
    key={id}
    style={{
      transform: `translate(${p[0]}px, ${p[1]}px)`,
      transition: MOVE,
      opacity: dim(id) ? 0.28 : 1,
    }}
  >
    {hot && <circle r={r + 7} fill="none" stroke="black" />}
    <circle r={r} fill={c.fill} stroke={c.stroke} />
    <text
      textAnchor="middle"
      dominantBaseline="central"
      fontSize={sub ? 12 : 15}
      fontWeight="700"
      fill={sub ? "#94a3b8" : "#fff"}
    >
      {sub ? id : keys[id]}
    </text>
    {!sub && (
      <text textAnchor="middle" y={-r - 7} font-size={10}>
        {id}
      </text>
    )}
  </g>
);
}}
</svg>
);
};

```

/** Имя случая из кода пользователя → вкладка демонстрации

```

const CASE_BY_NAME: Record<string, number> = {
  zig: 0, "zig-zig": 1, zigzig: 1, "zig zig": 1, "zig-z

```



```

const total = active.frames.length;
const idx = Math.min(frame, total - 1);
const current = active.frames[idx];
const prevPos = active.frames[Math.max(idx - 1, 0)].p;
const duration = useMemo(() => Math.round(current.ms - prevPos), [current, prevPos]);
const last = idx === total - 1;

useEffect(() => {
  setFrame(0);
}, [caseIdx]);

useEffect(() => {
  if (!playing) return;
  const t = setTimeout(() => setFrame((f) => (f + 1) % total), duration);
  return () => clearTimeout(t);
}, [playing, frame, duration, total]);

return (
  <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
    <div className="flex gap-2 mb-4 overflow-x-auto">
      {CASES.map((c, i) => (
        <button
          key={c.key}
          onClick={() => setCaseIdx(i)}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-300
            ${i === caseIdx ? "bg-indigo-600 text-white"}
          `}
        >
          {c.label}
        </button>
      ))}
    </div>

    <div className="mb-3 text-xs sm:text-[13px] text-slate-400">
      <span className="font-bold text-indigo-300">Корпус</span>
      <span className="text-slate-500">({active.rotation}°)</span>
    </div>
  </div>
)

```

```

        <span className="text-emerald-400/80 ml-1">
      </div>
      <p className="text-[11px] text-slate-500 mt-1"
    </div>
  </div>

  { /* индикатор автопрокрутки */ }
  <div className="h-1 w-full bg-slate-800 rounded-f
    <div
      key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `
      className="h-full bg-indigo-500"
      style={
        playing
        ? { animation: `demoProgress ${duration}m
          : { width: "100%", background: "#334155"
        }
      }
    />
  </div>

  <div className="flex flex-wrap items-center gap-2"
    <button
      onClick={() => {
        setPlaying(false);
        setFrame((f) => (f - 1 + total) % total);
      }}
      disabled={idx === 0}
      className="px-3 py-2 rounded-lg bg-slate-800 l
        d:hover:text-slate-300 text-xs font-bold tr
    >
      ← Назад
    </button>
    <button
      onClick={() => {
        setPlaying(false);
        setFrame((f) => (f + 1) % total);
      }}
      className="px-4 py-2 rounded-lg bg-indigo-600
        w-indigo-900/40"

```

```

        </span>
      )
    }}
  </div>
</div>

{/* что сейчас сообщил компилятор */}
<div className="mt-3 pt-3 border-t border-slate-800" >
  <span className="text-[10px] uppercase tracking-wider" >
    <Terminal className="w-3.5 h-3.5" /> из компилятора
  </span>
  {liveCase || liveX !== null || liveStep !== null ? (
    <>
      {liveCase && (
        <span className="text-[11px] font-mono bg-slate-900 text-white" >
          case = {liveCase}
        </span>
      )}
      {liveX !== null && <span className="text-[11px] font-mono" >
        x = {liveX}</span>}
      {liveP !== null && <span className="text-[11px] font-mono" >
        p = {liveP}</span>}
      {liveG !== null && <span className="text-[11px] font-mono" >
        g = {liveG || "нет (Zig)"}</span>}
      {liveStep !== null && <span className="text-[11px] font-mono" >
        step = {liveStep}</span>}
    </>
  ) : (
    <span className="text-[11px] text-slate-500" >
      запусти код из панели Python: переменная <code>case</code>
      случай,
      а <code>liveStep</code>
    </span>
  )}
</div>

<p className="mt-2 text-[11px] text-slate-500" >
  Листай кнопкой «Дальше» в своём темпе – кадры н

```

```

    if (l) {
        n.left = l;
        l.parent = n;
    }
    if (r) {
        n.right = r;
        r.parent = n;
    }
    return n;
}

export function bstInsert(root: SN | null, key: number) {
    if (!root) return { key, left: null, right: null, parent: null };
    const r = cloneWithParents(root)!;
    let cur: SN = r;
    for (;;) {
        if (key < cur.key) {
            if (!cur.left) {
                cur.left = { key, left: null, right: null, parent: cur };
                return r;
            }
            cur = cur.left;
        } else {
            if (!cur.right) {
                cur.right = { key, left: null, right: null, parent: cur };
                return r;
            }
            cur = cur.right;
        }
    }
}

/** Поднять child над его родителем (один атомарный поворот)
export function rotateUp(child: SN): void {
    const p = child.parent;
    if (!p) return;
    const g = p.parent;
    if (p.left === child) {

```

```

    }
    return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "cut" |
export interface SplayStep {
  kind: SplayStepKind;
  root: SN | null;
  aimEdge?: [number, number];
  /** Кадр cut: ребро, по которому отстёгивается среднее */
  cutEdge?: [number, number];
  cutLabel?: string;
  /** Куда β переедет: сторона у верхнего узла. */
  cutDest?: string;
  pathKeys?: number[];
  caseLabel?: string;
  note: string;
  rotCount: number;
}

/** Полный пошаговый прогон splay(key): спуск → (прицел
export function splaySteps(
  rootIn: SN | null,
  key: number,
  isInsert = false,
): { steps: SplayStep[]; found: boolean } {
  let root: SN | null = cloneWithParents(rootIn);
  const steps: SplayStep[] = [];
  if (!root)
    return { steps: [{ kind: "done", root: null, note:

  // --- спуск ---
  let cur: SN | null = root;
  const pathKeys: number[] = [];
  let found = false;
  let last: SN = root;
  while (cur) {
    last = cur;

```

```

    steps.push({
      kind: "cut",
      root: cloneWithParents(root)!,
      cutEdge: [child.key, mid ? mid.key : child.key],
      cutLabel: mid ? `β = поддерезо ${mid.key}` : "β п",
      cutDest: mid ? (p.left === child ? "справа у p" :
      caseLabel: label,
      note: mid
        ? `Отстёгиваем среднее поддерезо β = ${mid.key}
          й у них общий – после поворота переедет ${p.l
          : `Среднего поддереза нет (β пусто): ${child.ke
      rotCount,
    });
  const before = collectEdges(root);
  rotateUp(child);
  fixRoot();
  rotCount += 1;
  const diff = diffEdges(before, collectEdges(root));
  steps.push({
    kind: "rot",
    root: cloneWithParents(root)!,
    caseLabel: label,
    note: `Поворот: ${child.key} встал над ${p.key}.
    rotCount,
  });
};

while (target.parent) {
  const p = target.parent;
  const g = p.parent;
  if (!g) {
    emitRotation(
      target,
      "Zig",
      [p.key, target.key],
      `Zig: у ${target.key} нет деда – хватит одного
    );
    continue;
  }
}

```

```

        caseLabel: "Zig-Zag",
        note: `Второе ребро: ${target.parent.key}—${target.key}
        rotCount,
    ));
    emitRotation(
        target,
        "Zig-Zag",
        [target.parent.key, target.key],
        `Доводим: ${target.key} над ${target.parent.key}
    );
}
}

steps.push({
    kind: "done",
    root: cloneWithParents(root)!,
    note: `    ${target.key} в корне за ${rotCount} ${rotCount}
    rotCount,
});
return { steps, found };
}

/* ----- служебное -----

const depthOf = (t: SN | null, key: number): number =>
    let d = 0;
    let cur = t;
    while (cur) {
        if (key === cur.key) return d;
        cur = key < cur.key ? cur.left : cur.right;
        d += 1;
    }
    return -1;
};

const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [
    t.key, ...inorderKeys(t.left), ...inorderKeys(t.right)

export const INITIAL KEYS = [50, 40, 35, 32, 30, 28, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1];

```

```

x1="30" y1="86" x2="48" y2="126"
stroke={isCut ? "#f59e0b" : "#64748b"}
strokeWidth={isCut ? 3 : 2}
strokeDasharray={isCut ? "5,3" : "none"}
/>
<circle cx="55" cy="46" r="15" fill="#1e293b"
<text x="55" y="50" textAnchor="middle" fill=
<circle cx="30" cy="86" r="15" fill="#78350f"
<text x="30" y="90" textAnchor="middle" fill=
<circle cx="12" cy="126" r="13" fill="#0f172a"
<text x="12" y="130" textAnchor="middle" fill=
<circle
  cx="48" cy="126" r="13"
  fill={isCut ? "#78350f" : "#0f172a"}
  stroke={isCut ? "#f59e0b" : "#64748b"}
  strokeWidth={isCut ? 2.5 : 1.5}
  strokeDasharray={isCut ? "4,3" : "none"}
/>
<text x="48" y="130" textAnchor="middle" fill=
<circle cx="80" cy="126" r="13" fill="#0f172a"
<text x="80" y="130" textAnchor="middle" fill=
<text x="98" y="92" fill={isCut ? "#f59e0b" :
  {isCut ? " β отстёрнута" : "β при x"}}
</text>
<text x="16" y="160" fill="#64748b" fontSize=
</g>
<text x="163" y="96" fill="#10b981" fontSize="2
<g transform="translate(186,6)">
  <text y="10" fill="#94a3b8" fontSize="11" fon
  <line x1="70" y1="46" x2="45" y2="86" stroke=
  <line x1="70" y1="46" x2="95" y2="86" stroke=
  <line x1="45" y1="86" x2="27" y2="126" stroke=
  <line
    x1="45" y1="86" x2="63" y2="126"
    stroke={isRot ? "#10b981" : "#64748b"}
    strokeWidth={isRot ? 3 : 2}
  />
  <circle cx="70" cy="46" r="15" fill="#059669"

```



```

    if (!n) return;
    walk(n.left, d + 1);
    pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d });
    slot += 1;
    walk(n.right, d + 1);
  };
  walk(root, 0);
  return pos;
};

```

```

const SplayWalkViz: React.FC = () => {
  const initial = useMemo(buildInitial, []);
  const [steps, setSteps] = useState<SplayStep[] | null>();
  const [idx, setIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [input, setInput] = useState("");
  const [treeVers, setTreeVers] = useState(0);
  const baseRef = useRef<SN>(initial);
  const timer = useRef<ReturnType<typeof setTimeout> | null>();

```

```

  const runKey = (key: number, isInsert = false) => {
    const { steps: st } = splaySteps(baseRef.current, key);
    setSteps(st);
    setIdx(0);
    setPlaying(true);
  };

```

```

  useEffect(() => {
    if (!playing || !steps) return;
    if (idx >= steps.length - 1) {
      setPlaying(false);
      baseRef.current = steps[steps.length - 1].root ?? null;
      setTreeVers((v) => v + 1);
      return;
    }
    timer.current = setTimeout(() => setIdx((v) => v + 1), 100);
    return () => {
      if (timer.current) clearTimeout(timer.current);
    };
  }, [playing, steps, idx, timer]);

```

```
const ino0 = useMemo(() => (steps ? inorderKeys(steps)
const inoNow = inorderKeys(viewRoot).join(",");
const checkOk = !steps || ino0 === inoNow;
```

```
const doInsert = () => {
  const v = parseInt(input, 10);
  if (Number.isNaN(v)) return;
  const withNew = bstInsert(baseRef.current, v);
  baseRef.current = withNew;
  setTreeVers((t) => t + 1);
  runKey(v, true);
  setInput("");
};
```

```
const stepColor = (k: number): string => {
  if (!step) return "#0f172a";
  if (step.kind === "search" && step.pathKeys?.includes(k))
  if ((step.kind === "aim" || step.kind === "cut") &&
    const e = step.aimEdge ?? step.cutEdge!;
    if (e[0] === k) return "#1e293b";
    if (e[1] === k) return "#78350f";
  }
  if (step.kind === "rot" && step.pathKeys?.includes(k))
  return "#0f172a";
};
```

```
const strokeColor = (k: number): string => {
  if (!step) return "#64748b";
  if (step.kind === "search" && step.pathKeys?.includes(k))
  if ((step.kind === "aim" || step.kind === "cut") &&
    const e = step.aimEdge ?? step.cutEdge!;
    if (e.includes(k)) return "#f59e0b";
  }
  if (step.pathKeys?.includes(k)) return "#38bdf8";
  return "#64748b";
};
```

```
return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border
```

```

        className="px-3 py-2 rounded-lg text-xs font-
    >
        Исходное
    </button>
    <button
        onClick={() => {
            const shuffled = [...INITIAL_KEYS].sort(() =>
            let r: SN | null = null;
            for (const k of shuffled) r = bstInsert(r, k);
            baseRef.current = r!;
            setSteps(null);
            setIdx(0);
            setPlaying(false);
            setTreeVers((v) => v + 1);
        }}
        className="px-3 py-2 rounded-lg text-xs font-
    >
        Перемешать
    </button>
</div>
</div>

{/* Плеер */}
{steps && (
    <div className="flex flex-wrap items-center gap-2">
        <button onClick={() => { setIdx(0); setPlaying(false); }}
            ld bg-slate-700 text-white disabled:opacity-50
            В начало
        </button>
        <button onClick={() => { setPlaying(false); setSteps(null); }}
            -lg text-xs font-bold bg-slate-700 text-white
            ← Шаг
        </button>
        <button onClick={() => setPlaying(!playing)}
            er-600" : "bg-emerald-600"}`}>
            {playing ? " Пауза" : "▶ Пуск"}
        </button>
        <button onClick={() => { setPlaying(false); setSteps(null); }}

```

```

        strokeWidth={isAim ? 4 : isCutEdge ? 2 : 1}
        strokeDasharray={isAim ? "6,4" : isCutEdge ? "5,5" : ""}
        className="transition-all duration-500"
      />
      {isCutEdge && (
        <text x={({a.px + b.px} / 2 + 14)} y={({a.py + b.py} / 2)}
        noSpace=" " className="transition-all duration-500"
        {step?.cutLabel} → {step?.cutDest}
      </text>
      )}
    </g>
  );
}
}
}
{nodes.map((n) => {
  const aimE = step?.aimEdge ?? (step?.kind === "aim" ? step : null);
  const isAimNode = aimE?.includes(n.key);
  return (
    <g
      key={`n-${n.key}-${treeVers}`}
      transform={`translate(${n.px}, ${n.py})`}
      onClick={() => {
        if (playing) return;
        const has = inorderKeys(baseRef.current).includes(n.key);
        if (has) runKey(n.key);
      }}
      className="cursor-pointer"
    >
      <circle
        r={isAimNode ? 24 : 20}
        fill={stepColor(n.key)}
        stroke={strokeColor(n.key)}
        strokeWidth={isAimNode ? 4 : 2}
        className="transition-all duration-300"
      />
      <text y={4} fill="#ffffff" fontSize="13">
        {n.key}
      </text>
    </g>
  );
})}

```

```

<div className="flex flex-col lg:flex-row gap-6 m-0" style={{width: '100%'}}>
  <div className="w-full lg:w-1/2 bg-rose-950/10 p-4" style={{width: '100%'}}>
    <h4 className="text-sm font-bold text-rose-300">Шаги
    {steps ? (
      <table className="w-full text-sm">
        <tbody className="font-mono">
          <tr className="border-b border-slate-800">
            <td className="py-2 text-slate-400">г.
            <td className="py-2 text-right text-slate-400">
            </tr>
          <tr className="border-b border-slate-800">
            <td className="py-2 text-slate-400">в.
            <td className="py-2 text-right text-slate-400">
            </tr>
          <tr>
            <td className="py-2 text-slate-400">н.
            <td className="py-2 text-right text-slate-400">
            </tr>
          </tbody>
        </table>
      ) : (
        <p className="text-xs text-slate-400">Занес
      )}
    <p className="text-xs text-slate-500 mt-3">Од
      щений дешевлеет.</p>
  </div>
  <div className="w-full lg:w-1/2 bg-slate-900/60 p-4" style={{width: '100%'}}>
    <h4 className="text-sm font-bold text-slate-200">Шаги
    <div className="bg-slate-950/80 rounded-lg p-4">
      {CODE_LINES.map((l) => {
        const zig = step?.caseLabel === "Zig" &&
        const zz1 = step?.caseLabel === "Zig-Zig"
        const zz2 = step?.caseLabel === "Zig-Zig"
        const za1 = step?.caseLabel === "Zig-Zag"
        const za2 = step?.caseLabel === "Zig-Zag"
        const active = zig || zz1 || zz2 || za1 |
        return (
          <div key={l.line} className={`py-1 px-2

```

```

-----
  { name: 'Тарелка от тако', emoji: ' ', color: 'from-yellow-500 to-yellow-600 border-2 border-yellow-500' },
  { name: 'Тарелка из-под суши', emoji: ' ', color: 'from-yellow-500 to-yellow-600 border-2 border-yellow-500' },
  { name: 'Сковорода от глазуньи', emoji: ' ', color: 'from-yellow-500 to-yellow-600 border-2 border-yellow-500' },
];

let counter = 0;

export const StackViz: React.FC = () => {
  const [dirtyStack, setDirtyStack] = useState<Plate[]>(
    [
      { id: 'p1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'idle' },
      { id: 'p2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'idle' },
      { id: 'p3', name: 'Блюдец от торта', emoji: '🍰', state: 'idle' },
    ]
  );

  const runtime = useVizRuntime();
  const liveStack = vizArray(runtime?.variables?.st) ?? [];
  const displayStack: Plate[] = liveStack
    ? liveStack.map((value, index) => ({
        id: `python-${index}`,
        name: String(value),
        emoji: " ",
        color: "from-indigo-500/20 to-indigo-600/20 border-2 border-indigo-500",
        isDirty: true,
        animState: "idle",
      }))
    : dirtyStack;

  const [cleanRack, setCleanRack] = useState<Plate[]>(
    []
  );
  const [isWashing, setIsWashing] = useState(false);
  const [showSoap, setShowSoap] = useState(false);
  const [shakeEmpty, setShake] = useState(false);
  const [log, setLog] = useState<string[]>(
    []
  );

  const addLog = (msg: string) => setLog(prev => [msg, ...prev]);

```

```

    setIsWashing(true);
    const topPlate = dirtyStack[dirtyStack.length - 1];

    addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.emoji}`);

    // Step 1: Start washing animation (sponge and soap)
    setDirtyStack(prev => prev.map(p => p.id !== topPlate.id));
    setShowSoap(true);

    setTimeout(() => {
      // Step 2: Wash complete. Plate becomes clean and goes to clean rack
      const cleanPlate: Plate = {
        ...topPlate,
        isDirty: false,
        animState: 'clean',
      };

      setCleanRack(prev => [cleanPlate, ...prev].slice(0, 3));
      setDirtyStack(prev => prev.slice(0, prev.length - 1));
      setShowSoap(false);
      setIsWashing(false);
      addLog(`  Готово! Чистая тарелка ${topPlate.emoji}`);
    }, 850);
  }, [dirtyStack, isWashing]);

const reset = () => {
  counter = 0;
  setDirtyStack([
    { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'in' },
    { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'in' },
    { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', state: 'in' },
  ]);
  setCleanRack([]);
  setIsWashing(false);
  setShowSoap(false);
};

```

```

    >
      <span> Помыть верхнюю (POP)</span>
    </button>

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800
        r-pointer border border-slate-700"
    >
      Сброс
    </button>
  </div>

  {/* ИНТЕРАКТИВНАЯ КУХНЯ */}
  <div className="grid grid-cols-1 md:grid-cols-12" >

    {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
    <div className="md:col-span-7 bg-slate-900 rounded-3xl
      [380px]">
      <div className="absolute top-4 left-4 text-[10px] font-mono">
        Грязная стопка (Стек вызовов / LIFO)
      </div>

      {/* Визуализация раковины */}
      <div className="absolute top-4 right-4 flex items-center justify-center
        r-slate-800/80 text-[10px] font-mono">
        <span className="w-1.5 h-1.5 rounded-full bg-slate-700" />
        <span>РАКОВИНА </span>
      </div>

      {/* Мыльные пузыри при мытье */}
      {showSoap && (
        <div className="absolute inset-0 pointer-events-none">
          <div className="absolute w-6 h-6 rounded-full bg-white
            opacity-50"></div>
          <div className="absolute w-4 h-4 rounded-full bg-white
            opacity-50"></div>
          <div className="absolute w-5 h-5 rounded-full bg-white
            opacity-50"></div>
        </div>
      )}
    </div>
  </div>

```



```

        ${plate.animState === 'in' ? 'anim
        ${plate.animState === 'washing' ?
        ${isTop ? 'ring-4 ring-blue-500/50
    `}
  }
  >
    <span className="text-lg">{plate.em
    <span className="text-slate-800 tex
    {isTop && (
      <span className="text-[9px] bg-bl
      LIFO
    </span>
    )}
  </div>
  );
  }}}
</div>
})

{/* Столешница раковины */}
<div className="w-full h-4 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */}
<div className="md:col-span-5 bg-slate-900 round
  <div className="absolute top-4 left-4 text-[10px]
    Сушилка для чистой посуды
  </div>

  <div className="absolute top-4 right-4 flex i
    order-emerald-800/80 text-[9px] font-mono">
    <Sparkles className="w-3.5 h-3.5 animate-pu
    <span>ЧИСТО </span>
  </div>

  <div className="flex-1 flex flex-col justify-c
    {cleanRack.length === 0 ? {
      <div className="flex-1 flex flex-col item
        <span className="text-5xl mb-2"> </span>

```

```
    });  
};
```

ФАЙЛ 75/111: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useRef, useMemo } from "react";  
import { useVizRuntime, vizArray, vizNumber, vizString } from "viz-runtime";  
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";
```

```
function generateKmpSteps(pattern: string, text: string): any[] {  
  if (!pattern) pattern = " ";  
  const s = pattern + "#" + text;  
  const n = s.length;  
  const pi = new Array(n).fill(0);  
  const steps: any[] = [];
```

```
  steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });
```

```
  for (let i = 1; i < n; i++) {  
    let j = pi[i - 1];
```

```
    steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });  
    ;
```

```
    while (j > 0 && s[i] !== s[j]) {  
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });  
      // Идём назад: откат j = pi[j - 1] = pi[j - 1] - 1;  
      j = pi[j - 1];  
    }  
  }
```

```
  if (s[i] === s[j]) {  
    steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });  
    // Если совпадение, то ска увеличивается.  
    j++;  
  }  
}
```

```

        steps.push({ i, l, r, zTarget: z[i], zSource: z[i], desc: `Шаг ${i} падает! Окно растёт.` });
        z[i]++;
    }

    if (i + z[i] < n) {
        steps.push({ i, l, r, zTarget: z[i], zSource: z[i], desc: `Шаг ${i} падает! Окно растёт.` });
    } else {
        steps.push({ i, l, r, z: [...z], desc: `Упёрся в границу.` });
    }

    steps.push({ i, l, r, z: [...z], desc: `Фиксируем текущее состояние.` });

    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
        steps.push({ i, l, r, z: [...z], desc: `Сдвинули окно.` });
    }

    if (z[i] === pattern.length) {
        steps.push({ i, l, r, z: [...z], found: true, desc: `Найдено совпадение!` });
    }
}

return { s, steps, patternLength: pattern.length };
}

export function StringAlgorithmsViz({ defaultMode = "kmp" }) {
    const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
    const [pattern, setPattern] = useState("aba");
    const [text, setText] = useState("abacaba");

    // Ensure inputs are valid dynamically
    const safePattern = pattern || " ";
    const safeText = text || " ";

    const { s: demoString, steps, patternLength: demoPatternLength } = generateKmpSteps(safePattern, safeText);

    const { s: demoString, steps, patternLength: demoPatternLength } = generateZSteps(safePattern, safeText);

    return {
        mode,
        setMode,
        pattern,
        setPattern,
        text,
        setText,
        demoString,
        demoPatternLength,
        steps,
    };
}

```

```

const hasLiveValues = !!vars && Object.prototype.hasOwnProperty.call(vars, 'i');
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const liveL = vizNumber(vars?.l);
const liveR = vizNumber(vars?.r);
const compilerLinked = liveI !== null || hasLiveValues;
const step = compilerLinked
  ? {
      ...baseStep,
      i: liveI ?? baseStep.i,
      j: liveJ ?? baseStep.j,
      l: liveL ?? baseStep.l,
      r: liveR ?? baseStep.r,
      [mode === "kmp" ? "pi" : "z"]: hasLiveValues,
      desc: `Компилятор: i=${liveI ?? "-"}${mode === "kmp" ? " (π-ФУНКЦИЯ)" : " (Z-ФУНКЦИЯ)"}`
    }
  : baseStep;

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <div className="flex bg-slate-900 border border-slate-800 p-2 gap-2">
        <button onClick={() => setMode("kmp")}
          className={`px-3 py-1.5 text-xs font-medium rounded-md ${mode === "kmp" ? "text-slate-400 hover:text-slate-300" : "text-slate-500 hover:text-slate-400"}`>
          КМП (π-ФУНКЦИЯ)
        </button>
        <button onClick={() => setMode("z")}
          className={`px-3 py-1.5 text-xs font-medium rounded-md ${mode === "z" ? "text-slate-400 hover:text-slate-300" : "text-slate-500 hover:text-slate-400"}`>
          Z-ФУНКЦИЯ
        </button>
      </div>
      <div className="flex items-center gap-2">
        <div className="bg-slate-800 p-1 flex items-center gap-1">
          <span className="text-[10px] text-slate-400">Паттерн:</span>
          <input value={pattern} onChange={setPattern} type="text" className="bg-transparent border-none outline-none text-slate-400 font-[10px]">
        </div>
      </div>
    </div>
  </div>
);

```



```

        </div>
      )
    }
  }
</div>
</div>

<div className="flex flex-col sm:flex-row gap-2"
  {/* Controls */}
  <div className="flex bg-slate-900 border border-slate-800 p-4"
    <button onClick={prevStep} disabled={!steps.length}
      -400 hover:text-white hover:bg-slate-800 rounded-lg border-l
      <Undo strokeWidth={3} size={16} />
    </button>
    <button onClick={playPause} className="flex-1"
      x items-center justify-center min-w-[80px]"
      {autoplay ? <><Pause size={16} /> : <><Play size={16} />}
    </button>
    <button onClick={nextStep} disabled={steps.length === 1}
      e="p-2 text-slate-400 hover:text-white hover:bg-slate-800
      <SkipForward strokeWidth={3} size={16} />
    </button>
    <button onClick={reset} aria-label="Reset"
      :bg-slate-800 rounded-lg ml-2 border-l border-slate-800
      <RefreshCw strokeWidth={3} size={16} />
    </button>
  </div>

  {/* Description Log */}
  <div className="flex-1 bg-slate-900/50 border border-slate-800
    overflow-hidden">
    <div className="absolute top-0 right-0 p-4"
      <span className="font-mono text-sm text-slate-400"
        {steps.length}</span>
    </div>
    <div className="text-[10px] text-slate-400"
      {steps.length}</div>
    <div className="text-sm text-slate-400"
      {step.desc}
    </div>
  </div>

```

```
        </pre>
      </div>
    </div>
  );
}
```

ФАЙЛ 76/111: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from "react";

interface TooltipProps {
  /** Содержимое подсказки (можно JSX). */
  content: React.ReactNode;
  /** Элемент, на который наводят курсор / фокус. */
  children: React.ReactNode;
  /** С какой стороны показывать. По умолчанию – сверху
  side?: "top" | "bottom";
  className?: string;
}

/**
 * Всплывающая подсказка (как title="", только красивая
 * Появляется по наведению курсора и по фокусу с клавиатуры
 */
export const Tooltip: React.FC<TooltipProps> = ({ content, children }) => {
  const [open, setOpen] = useState(false);

  const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

  return (
    <span
      className={`relative inline-flex ${className ?? ""}
        onMouseEnter={() => setOpen(true)}
        onMouseLeave={() => setOpen(false)}
    >
```

```

interface TNode {
    id: number;
    x: number;
    y: number;
    label: string;
    name: string;
}

interface TEdge {
    u: number;
    v: number;
}

const dagNodes: TNode[] = [
    { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
    { id: 1, x: 280, y: 70, label: "1", name: "Дискретка" },
    { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра" },
    { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы" },
    { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение" },
];

const dagEdges: TEdge[] = [
    { u: 0, v: 1 },
    { u: 0, v: 2 },
    { u: 1, v: 3 },
    { u: 2, v: 3 },
    { u: 3, v: 4 },
];

interface TStep {
    desc: string;
    visited: Set<number>;
    fullyProcessed: Set<number>;
    currentNode: number | null;
    topoList: number[];
    dfsStack: number[];
}

```



```
// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
  visited.add(u);
  dfsStack.push(u);
  recordStep(
    `DFS: зашли в вершину ${u} (${dagNodes[u].name})`,
    u,
  );

  for (const to of adj[u]) {
    if (!visited.has(to)) {
      dfs(to);
    } else if (!fullyProcessed.has(to)) {
      // Explaining potential cycle detected (not in topological order)
      recordStep(
        `⚠ Внимание: Рёбра в серую вершину ${to} от ${u}`,
        u,
      );
    }
  }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write in reverse order
recordStep(
  `DFS: закончили обработку '${dagNodes[u].name}'`,
  u,
);
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
  if (!visited.has(i)) {
    dfs(i);
  }
}
```

```

    setIsPlaying(false);
    setCurrentStepIdx(0);
  };

const getTopoListString = () => {
  return [...step.topoList]
    .reverse()
    .map((id) => dagNodes[id].name)
    .join(" → ");
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Clock className="w-5 h-5 text-indigo-400" />
        Топологическая сортировка (Зависимости обучен
      </h3>

    <div className="flex items-center gap-2 bg-slate-
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
        500 text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
      </button>

    <button
      onClick={reset}
      aria-label="Сбросить сортировку"
      title="Сбросить сортировку"
      className="flex items-center justify-center

```

```

const u = dagNodes.find((n) => n.id === e.u)
const v = dagNodes.find((n) => n.id === e.v)

const isActive =
  (step.currentNode === edge.u || step.currentNode === edge.v) ||
  step.visited.has(edge.v);

return (
  <line
    key={`edge-${idx}`}
    x1={u.x}
    y1={u.y}
    x2={v.x}
    y2={v.y}
    stroke={isActive ? "#818cf8" : "#334155"}
    strokeWidth={isActive ? "3" : "2"}
    markerEnd={`url(#${isActive ? "dag-arrow" : "dag-arrow-processed"})`}
    className="transition-all duration-300"
  />
);
}}}

{/* Nodes */}
{dagNodes.map((node) => {
  const isCurrent = step.currentNode === node.id;
  const isVisited = step.visited.has(node.id);
  const isProcessed = step.fullyProcessed.has(node.id);

  let fill = "#1e293b";
  let stroke = "#334155";
  let textColor = "text-slate-400";
  void textColor;

  if (isCurrent) {
    fill = "#312e81";
    stroke = "#818cf8";
    textColor = "text-white font-bold";
  } else if (isProcessed) {

```

```

        fontSize="9px"
        className="pointer-events-none"
      >
        {node.name.split(" ").slice(1).join(" ")}
      </text>
    </g>
  );
}}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-400">
    Статус сортировки
  </h4>

  <div className="bg-slate-900 border border-slate-800 p-4">
    <p className="text-xs text-indigo-300 min-h-[100px]">
      {step.desc}
    </p>
  </div>

  <div className="flex-1 space-y-4 overflow-y-auto">
    {/* Topological List output */}
    <div>
      <span className="text-[10px] text-slate-500">
        РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):
      </span>
      <div className="bg-slate-900/60 p-2.5 rounded pr-1 text-slate-300">
        {step.topoList.length === 0 ? (
          <span className="text-slate-600 text-sm">
            ожидаем обработку узлов...
          </span>
        ) : (
          getTopoListString()
        )}
      </div>
    </div>
  </div>
</div>

```

```

        disabled={currentStepIdx >= steps.length}
        onClick={() => setCurrentStepIdx((prev) => prev + 1)}
        className="bg-slate-900 border border-slate-800 p-2 text-white"
      >
        Вперед
      </button>
    </div>
  </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 78/111: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useContext, useEffect, useMemo, useState } from 'react';
import { Terminal } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from './context';

```

```
/**
```

```
 * Интерактивный Треп — шесть режимов:
```

```
 *   build   — сборка: ВИДИМАЯ сортировка точек (и чем сортировать, так и сортировать)
```

```
 *           координатная плоскость НАВЕРХУ (ассоциативная сортировка)
```

```
 *           чек-лист инвариантов (автопроверка, как у бинарного поиска)
```

```
 *   split   — разрез по x с двумя корзинами (грамотная сортировка)
```

```
 *   merge    — склейка двух деревьев по y.
```

```
 *   erase    — удаление ПРОИЗВОЛЬНОЙ точки: найти по x, удалить по y.
```

```
 *   layers   — миф «дети в массиве по 2k/2k+1»: работает для любого n.
```

```
 *   search   — «отсортирую бинарным поиском»: поиск не сортировкой.
```

```
 *           сортировка платит  $\geq n \cdot \log n$  сравнений, сортировка — нет.
```

```
 *           Канонический набор: все x и все y различны  $\Rightarrow$  дерево — бинарное.
```

```

    { key: 9, grade: 3 },
    { key: 12, grade: 0 },
  ];
  // (3,5) дубль ключа/приоритета – уберём: ввод без повт
  export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p) => !p.key || !p.grade);

  const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((a, b) => a.grade - b.grade);

  /* ----- чистые алгоритмы (экспортируются для тестов) ----- */

  export function insert(root: TNode | null, p: Pair): TNode | null {
    if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade, left: null, right: null };
    if (p.key <= root.key) root.left = insert(root.left, p);
    else root.right = insert(root.right, p);
    return root;
  }

  export function buildTree(ps: Pair[]): TNode | null {
    let root: TNode | null = null;
    for (const p of sortedByY(ps)) root = insert(root, p);
    return root;
  }

  export interface SplitResult {
    l: TNode | null;
    r: TNode | null;
    trace: { atId: NodeId; goLeft: boolean; note: string };
  }

  export function splitTree(root: TNode | null, x: number): SplitResult {
    const trace: SplitResult["trace"] = [];
    const go = (t: TNode | null): { l: TNode | null; r: TNode | null } | null {
      if (!t) return { l: null, r: null };
      if (t.key <= x) {
        const sub = go(t.right);
        trace.push({ atId: t.id, goLeft: true, note: `(${t.key} <= ${x}) о правое поддерево.` });
        t.right = sub.l;
        return { l: t, r: sub.r };
      }
      return go(t.left);
    };
    const { l, r } = go(root);
    return { l, r, trace };
  }

```

```

const trace: { atId: NodeId | null; note: string }[] = [];
const go = (t: TNode | null): TNode | null => {
  if (!t) return null;
  if (key < t.key) {
    trace.push({ atId: t.id, note: `${key} < ${t.key}` });
    t.left = go(t.left);
    return t;
  }
  if (key > t.key) {
    trace.push({ atId: t.id, note: `${key} > ${t.key}` });
    t.right = go(t.right);
    return t;
  }
  trace.push({ atId: t.id, note: `Нашли (${t.key}), ${t.key}` });
  const m = mergeTree(t.left, t.right);
  trace.push(...m.trace.map((s) => ({ atId: s.chosenId, note: s.note })));
  return m.root;
};
return { root: go(root), trace };
}

```

```

const cloneTree = (t: TNode | null): TNode | null =>
  t ? { id: t.id, key: t.key, grade: t.grade, left: cloneTree(t.left), right: cloneTree(t.right) } : null;

```

/* ----- валидация treap (для чек-листа и стресс-теста) ----- */

```

export function checkTreap(root: TNode | null): { bst: boolean; heap: boolean; } {
  let bst = true;
  let heap = true;
  let prev: number | null = null;
  const walk = (n: TNode | null, min: number, max: number, parentGrade: number) => {
    if (!n) return;
    if (n.key <= min || n.key > max) bst = false;
    if (parentGrade !== null && n.grade > parentGrade) heap = false;
    walk(n.left, min, n.key, n.grade);
    if (prev !== null && n.key < prev) bst = false;
    prev = n.key;
    walk(n.right, n.key, max, n.grade);
  };
  walk(root, -Infinity, Infinity, null);
  return { bst, heap };
}

```

```

nodeStroke?: (id: NodeId) => string;
edgeColor?: (from: NodeId, to: NodeId) => string;
edgeDash?: (from: NodeId, to: NodeId) => boolean;
pulseId?: NodeId | null;
subtitle?: string;
}

const TreeView: React.FC<TreeViewProps> = ({ root, width, height, pulseId, subtitle }) => {
  const pos = useMemo(() => layoutTree(root), [root]);
  const nodes = useMemo(() => [...pos.entries()].map(([id, node]) => node), [pos]);
  const edges = useMemo(() => treeEdges(root), [root]);
  const autoW = Math.max(420, ...nodes.map((n) => n.px + n.pw));
  const autoH = Math.max(260, ...nodes.map((n) => n.py + n.ph));
  const w = width ?? autoW;
  const h = height ?? autoH;
  const byId = (id: NodeId) => nodes.find((n) => n.id === id);
  return (
    <div>
      <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full">
        {edges.map((e) => {
          const a = byId(e.from);
          const b = byId(e.to);
          if (!a || !b) return null;
          return (
            <line
              key={`te-${e.from}-${e.to}`}
              x1={a.px} y1={a.py} x2={b.px} y2={b.py}
              stroke={edgeColor ? edgeColor(e.from, e.to) : 'black'}
              strokeWidth={2.5}
              strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
              className="transition-all duration-500"
            />
          );
        })}
        {nodes.map((n) => {
          const p = pairParts(n.id);
          const pulse = pulseId === n.id;
          return (

```



```

-----
const Plane: React.FC<PlaneProps> = ({ points, drawnEdges
  }) => {
  const byPair = (p: Pair) => ({ x: PX(p.key), y: PY(p.key) })
  return (
    <svg
      viewBox={`0 0 ${PL_W} ${PL_H}`}
      className={`w-full ${interactive ? "touch-none select-none" : ""}`}
      onPointerMove={onMove}
      style={interactive ? { cursor: "crosshair" } : undefined}
    >
      <defs>
        <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
          <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1"/>
        </pattern>
      </defs>
      <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="white" stroke="black" stroke-width="1"/>
      <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" stroke-width="1"/>
      <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PL_H - 16} stroke="black" stroke-width="1"/>
      {[0, 10, 20, 30, 40].map((k) => {
        <text key={`tx-${k}`} x={PX(k)} y={PY(0) + 16} fill="black" font-size="10"/>
      })}
      {[{-4, 0, 4, 8].map((g) => {
        <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) + 16} fill="black" font-size="10"/>
      })}
      <text x={PL_W - 14} y={PY(0) - 8} fill="#94a3b8" font-size="10"/>
      <text x={PX(1) - 26} y="14" fill="#94a3b8" font-size="10"/>

      {drawnEdges.map((e, i) => {
        const a = byPair(e.a);
        const b = byPair(e.b);
        const isBad = badEdge && ((badEdge.a.key === e.a.key && badEdge.a.grade === e.b.grade) || (badEdge.a.key === e.b.key && badEdge.a.grade === e.a.grade));
        return <line key={`pe-${i}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y} stroke="black" stroke-width="2" strokeDasharray={isBad ? "4,3" : "none"}/>
      })}
    </svg>
  )
}

{/* временная линия игры */}

```

```

/* ===== ИГРА: соединение мышкой =====

export type GEdge = { a: Pair; b: Pair }; // a – родители, b – ребёнок

export const CANON_GAME: GEdge[] = (() => {
  const es: GEdge[] = [];
  const walk = (n: TNode | null) => {
    if (!n) return;
    if (n.left) { es.push({ a: { key: n.key, grade: n.grade, id: n.id }, b: n.left }); }
    if (n.right) { es.push({ a: { key: n.key, grade: n.grade, id: n.id }, b: n.right }); }
  };
  walk(buildTree(POINTS));
  return es;
})();

const sameP = (p: Pair, q: Pair) => p.key === q.key && p.grade === q.grade;
const gpid = (p: Pair) => `${p.key}:${p.grade}`;

/** Цепочка родителей от точки вверх. */
const parentOf = (edges: GEdge[], id: string): GEdge | null =>
  edges.find((e) => gpid(e.b) === id);
const reaches = (edges: GEdge[], from: Pair, target: Pair): boolean => {
  let cur: Pair | undefined = from;
  const seen = new Set<string>();
  while (cur) {
    if (sameP(cur, target)) return true;
    if (seen.has(gpid(cur))) return false;
    seen.add(gpid(cur));
    cur = parentOf(edges, gpid(cur))?.a;
  }
  return false;
};

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 справа.
export function forestIsBst(edges: GEdge[], all: Pair[]) {
  const kids = new Map<string, Pair[]>();
  for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.a)) || []), e.b]);
  for (const p of all) {
    const left = kids.get(gpid(p));
    const right = kids.get(gpid(p));
    if (left.length > 1 || right.length > 1) return false;
  }
  return true;
}

```

```

-----
  if (!forestIsBst(next, POINTS))
    return { ok: false, reason: "Линия вниз по y допустима только в начале строки" };
  return { ok: true };
}

/* ===== РЕЖИМ BUILD ===== */

type BuildPhase = "sort" | "connect" | "game";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";

const parsePairs = (raw: string): { pairs: Pair[]; errors: string[] } => {
  const pairs: Pair[] = [];
  const errors: string[] = [];
  const seen = new Set<string>();
  const chunks = raw.split(/[,;\n]+/).map((c) => c.trim());
  for (const chunk of chunks) {
    const nums = chunk.split(/\sxx*/).filter(Boolean);
    if (nums.length !== 2 || nums.some((v) => !Number.isInteger(+v))) {
      errors.push(`«${chunk}» – нужно ровно два числа: число и индекс`);
      continue;
    }
    const id = `${nums[0]}:${nums[1]}`;
    if (seen.has(id)) {
      errors.push(`точка (${nums[0]}; ${nums[1]}) дважды`);
      continue;
    }
    seen.add(id);
    pairs.push({ key: nums[0], grade: nums[1] });
  }
  const xs = pairs.map((p) => p.key);
  const ys = pairs.map((p) => p.grade);
  return { pairs, errors, equalX: new Set(xs).size !== 1 };
};

const DEMO_RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0, 1 1, 10 1, 11 1, 13 1, 14 1, 15 1, 16 1, 17 1, 18 1, 19 1, 20 1, 21 1, 22 1, 23 1, 24 1, 25 1, 26 1, 27 1, 28 1, 29 1, 30 1, 31 1, 32 1, 33 1, 34 1, 35 1, 36 1, 37 1, 38 1, 39 1, 40 1, 41 1, 42 1, 43 1, 44 1, 45 1, 46 1, 47 1, 48 1, 49 1, 50 1, 51 1, 52 1, 53 1, 54 1, 55 1, 56 1, 57 1, 58 1, 59 1, 60 1, 61 1, 62 1, 63 1, 64 1, 65 1, 66 1, 67 1, 68 1, 69 1, 70 1, 71 1, 72 1, 73 1, 74 1, 75 1, 76 1, 77 1, 78 1, 79 1, 80 1, 81 1, 82 1, 83 1, 84 1, 85 1, 86 1, 87 1, 88 1, 89 1, 90 1, 91 1, 92 1, 93 1, 94 1, 95 1, 96 1, 97 1, 98 1, 99 1, 100 1, 101 1, 102 1, 103 1, 104 1, 105 1, 106 1, 107 1, 108 1, 109 1, 110 1, 111 1, 112 1, 113 1, 114 1, 115 1, 116 1, 117 1, 118 1, 119 1, 120 1, 121 1, 122 1, 123 1, 124 1, 125 1, 126 1, 127 1, 128 1, 129 1, 130 1, 131 1, 132 1, 133 1, 134 1, 135 1, 136 1, 137 1, 138 1, 139 1, 140 1, 141 1, 142 1, 143 1, 144 1, 145 1, 146 1, 147 1, 148 1, 149 1, 150 1, 151 1, 152 1, 153 1, 154 1, 155 1, 156 1, 157 1, 158 1, 159 1, 160 1, 161 1, 162 1, 163 1, 164 1, 165 1, 166 1, 167 1, 168 1, 169 1, 170 1, 171 1, 172 1, 173 1, 174 1, 175 1, 176 1, 177 1, 178 1, 179 1, 180 1, 181 1, 182 1, 183 1, 184 1, 185 1, 186 1, 187 1, 188 1, 189 1, 190 1, 191 1, 192 1, 193 1, 194 1, 195 1, 196 1, 197 1, 198 1, 199 1, 200 1, 201 1, 202 1, 203 1, 204 1, 205 1, 206 1, 207 1, 208 1, 209 1, 210 1, 211 1, 212 1, 213 1, 214 1, 215 1, 216 1, 217 1, 218 1, 219 1, 220 1, 221 1, 222 1, 223 1, 224 1, 225 1, 226 1, 227 1, 228 1, 229 1, 230 1, 231 1, 232 1, 233 1, 234 1, 235 1, 236 1, 237 1, 238 1, 239 1, 240 1, 241 1, 242 1, 243 1, 244 1, 245 1, 246 1, 247 1, 248 1, 249 1, 250 1, 251 1, 252 1, 253 1, 254 1, 255 1, 256 1, 257 1, 258 1, 259 1, 260 1, 261 1, 262 1, 263 1, 264 1, 265 1, 266 1, 267 1, 268 1, 269 1, 270 1, 271 1, 272 1, 273 1, 274 1, 275 1, 276 1, 277 1, 278 1, 279 1, 280 1, 281 1, 282 1, 283 1, 284 1, 285 1, 286 1, 287 1, 288 1, 289 1, 290 1, 291 1, 292 1, 293 1, 294 1, 295 1, 296 1, 297 1, 298 1, 299 1, 300 1, 301 1, 302 1, 303 1, 304 1, 305 1, 306 1, 307 1, 308 1, 309 1, 310 1, 311 1, 312 1, 313 1, 314 1, 315 1, 316 1, 317 1, 318 1, 319 1, 320 1, 321 1, 322 1, 323 1, 324 1, 325 1, 326 1, 327 1, 328 1, 329 1, 330 1, 331 1, 332 1, 333 1, 334 1, 335 1, 336 1, 337 1, 338 1, 339 1, 340 1, 341 1, 342 1, 343 1, 344 1, 345 1, 346 1, 347 1, 348 1, 349 1, 350 1, 351 1, 352 1, 353 1, 354 1, 355 1, 356 1, 357 1, 358 1, 359 1, 360 1, 361 1, 362 1, 363 1, 364 1, 365 1, 366 1, 367 1, 368 1, 369 1, 370 1, 371 1, 372 1, 373 1, 374 1, 375 1, 376 1, 377 1, 378 1, 379 1, 380 1, 381 1, 382 1, 383 1, 384 1, 385 1, 386 1, 387 1, 388 1, 389 1, 390 1, 391 1, 392 1, 393 1, 394 1, 395 1, 396 1, 397 1, 398 1, 399 1, 400 1, 401 1, 402 1, 403 1, 404 1, 405 1, 406 1, 407 1, 408 1, 409 1, 410 1, 411 1, 412 1, 413 1, 414 1, 415 1, 416 1, 417 1, 418 1, 419 1, 420 1, 421 1, 422 1, 423 1, 424 1, 425 1, 426 1, 427 1, 428 1, 429 1, 430 1, 431 1, 432 1, 433 1, 434 1, 435 1, 436 1, 437 1, 438 1, 439 1, 440 1, 441 1, 442 1, 443 1, 444 1, 445 1, 446 1, 447 1, 448 1, 449 1, 450 1, 451 1, 452 1, 453 1, 454 1, 455 1, 456 1, 457 1, 458 1, 459 1, 460 1, 461 1, 462 1, 463 1, 464 1, 465 1, 466 1, 467 1, 468 1, 469 1, 470 1, 471 1, 472 1, 473 1, 474 1, 475 1, 476 1, 477 1, 478 1, 479 1, 480 1, 481 1, 482 1, 483 1, 484 1, 485 1, 486 1, 487 1, 488 1, 489 1, 490 1, 491 1, 492 1, 493 1, 494 1, 495 1, 496 1, 497 1, 498 1, 499 1, 500 1, 501 1, 502 1, 503 1, 504 1, 505 1, 506 1, 507 1, 508 1, 509 1, 510 1, 511 1, 512 1, 513 1, 514 1, 515 1, 516 1, 517 1, 518 1, 519 1, 520 1, 521 1, 522 1, 523 1, 524 1, 525 1, 526 1, 527 1, 528 1, 529 1, 530 1, 531 1, 532 1, 533 1, 534 1, 535 1, 536 1, 537 1, 538 1, 539 1, 540 1, 541 1, 542 1, 543 1, 544 1, 545 1, 546 1, 547 1, 548 1, 549 1, 550 1, 551 1, 552 1, 553 1, 554 1, 555 1, 556 1, 557 1, 558 1, 559 1, 560 1, 561 1, 562 1, 563 1, 564 1, 565 1, 566 1, 567 1, 568 1, 569 1, 570 1, 571 1, 572 1, 573 1, 574 1, 575 1, 576 1, 577 1, 578 1, 579 1, 580 1, 581 1, 582 1, 583 1, 584 1, 585 1, 586 1, 587 1, 588 1, 589 1, 590 1, 591 1, 592 1, 593 1, 594 1, 595 1, 596 1, 597 1, 598 1, 599 1, 600 1, 601 1, 602 1, 603 1, 604 1, 605 1, 606 1, 607 1, 608 1, 609 1, 610 1, 611 1, 612 1, 613 1, 614 1, 615 1, 616 1, 617 1, 618 1, 619 1, 620 1, 621 1, 622 1, 623 1, 624 1, 625 1, 626 1, 627 1, 628 1, 629 1, 630 1, 631 1, 632 1, 633 1, 634 1, 635 1, 636 1, 637 1, 638 1, 639 1, 640 1, 641 1, 642 1, 643 1, 644 1, 645 1, 646 1, 647 1, 648 1, 649 1, 650 1, 651 1, 652 1, 653 1, 654 1, 655 1, 656 1, 657 1, 658 1, 659 1, 660 1, 661 1, 662 1, 663 1, 664 1, 665 1, 666 1, 667 1, 668 1, 669 1, 670 1, 671 1, 672 1, 673 1, 674 1, 675 1, 676 1, 677 1, 678 1, 679 1, 680 1, 681 1, 682 1, 683 1, 684 1, 685 1, 686 1, 687 1, 688 1, 689 1, 690 1, 691 1, 692 1, 693 1, 694 1, 695 1, 696 1, 697 1, 698 1, 699 1, 700 1, 701 1, 702 1, 703 1, 704 1, 705 1, 706 1, 707 1, 708 1, 709 1, 710 1, 711 1, 712 1, 713 1, 714 1, 715 1, 716 1, 717 1, 718 1, 719 1, 720 1, 721 1, 722 1, 723 1, 724 1, 725 1, 726 1, 727 1, 728 1, 729 1, 730 1, 731 1, 732 1, 733 1, 734 1, 735 1, 736 1, 737 1, 738 1, 739 1, 740 1, 741 1, 742 1, 743 1, 744 1, 745 1, 746 1, 747 1, 748 1, 749 1, 750 1, 751 1, 752 1, 753 1, 754 1, 755 1, 756 1, 757 1, 758 1, 759 1, 760 1, 761 1, 762 1, 763 1, 764 1, 765 1, 766 1, 767 1, 768 1, 769 1, 770 1, 771 1, 772 1, 773 1, 774 1, 775 1, 776 1, 777 1, 778 1, 779 1, 780 1, 781 1, 782 1, 783 1, 784 1, 785 1, 786 1, 787 1, 788 1, 789 1, 790 1, 791 1, 792 1, 793 1, 794 1, 795 1, 796 1, 797 1, 798 1, 799 1, 800 1, 801 1, 802 1, 803 1, 804 1, 805 1, 806 1, 807 1, 808 1, 809 1, 810 1, 811 1, 812 1, 813 1, 814 1, 815 1, 816 1, 817 1, 818 1, 819 1, 820 1, 821 1, 822 1, 823 1, 824 1, 825 1, 826 1, 827 1, 828 1, 829 1, 830 1, 831 1, 832 1, 833 1, 834 1, 835 1, 836 1, 837 1, 838 1, 839 1, 840 1, 841 1, 842 1, 843 1, 844 1, 845 1, 846 1, 847 1, 848 1, 849 1, 850 1, 851 1, 852 1, 853 1, 854 1, 855 1, 856 1, 857 1, 858 1, 859 1, 860 1, 861 1, 862 1, 863 1, 864 1, 865 1, 866 1, 867 1, 868 1, 869 1, 870 1, 871 1, 872 1, 873 1, 874 1, 875 1, 876 1, 877 1, 878 1, 879 1, 880 1, 881 1, 882 1, 883 1, 884 1, 885 1, 886 1, 887 1, 888 1, 889 1, 890 1, 891 1, 892 1, 893 1, 894 1, 895 1, 896 1, 897 1, 898 1, 899 1, 900 1, 901 1, 902 1, 903 1, 904 1, 905 1, 906 1, 907 1, 908 1, 909 1, 910 1, 911 1, 912 1, 913 1, 914 1, 915 1, 916 1, 917 1, 918 1, 919 1, 920 1, 921 1, 922 1, 923 1, 924 1, 925 1, 926 1, 927 1, 928 1, 929 1, 930 1, 931 1, 932 1, 933 1, 934 1, 935 1, 936 1, 937 1, 938 1, 939 1, 940 1, 941 1, 942 1, 943 1, 944 1, 945 1, 946 1, 947 1, 948 1, 949 1, 950 1, 951 1, 952 1, 953 1, 954 1, 955 1, 956 1, 957 1, 958 1, 959 1, 960 1, 961 1, 962 1, 963 1, 964 1, 965 1, 966 1, 967 1, 968 1, 969 1, 970 1, 971 1, 972 1, 973 1, 974 1, 975 1, 976 1, 977 1, 978 1, 979 1, 980 1, 981 1, 982 1, 983 1, 984 1, 985 1, 986 1, 987 1, 988 1, 989 1, 990 1, 991 1, 992 1, 993 1, 994 1, 995 1, 996 1, 997 1, 998 1, 999 1, 1000 1, 1001 1, 1002 1, 1003 1, 1004 1, 1005 1, 1006 1, 1007 1, 1008 1, 1009 1, 1010 1, 1011 1, 1012 1, 1013 1, 1014 1, 1015 1, 1016 1, 1017 1, 1018 1, 1019 1, 1020 1, 1021 1, 1022 1, 1023 1, 1024 1, 1025 1, 1026 1, 1027 1, 1028 1, 1029 1, 1030 1, 1031 1, 1032 1, 1033 1, 1034 1, 1035 1, 1036 1, 1037 1, 1038 1, 1039 1, 1040 1, 1041 1, 1042 1, 1043 1, 1044 1, 1045 1, 1046 1, 1047 1, 1048 1, 1049 1, 1050 1, 1051 1, 1052 1, 1053 1, 1054 1, 1055 1, 1056 1, 1057 1, 1058 1, 1059 1, 1060 1, 1061 1, 1062 1, 1063 1, 1064 1, 1065 1, 1066 1, 1067 1, 1068 1, 1069 1, 1070 1, 1071 1, 1072 1, 1073 1, 1074 1, 1075 1, 1076 1, 1077 1, 1078 1, 1079 1, 1080 1, 1081 1, 1082 1, 1083 1, 1084 1, 1085 1, 1086 1, 1087 1, 1088 1, 1089 1, 1090 1, 1091 1, 1092 1, 1093 1, 1094 1, 1095 1, 1096 1, 1097 1, 1098 1, 1099 1, 1100 1, 1101 1, 1102 1, 1103 1, 1104 1, 1105 1, 1106 1, 1107 1, 1108 1, 1109 1, 1110 1, 1111 1, 1112 1, 1113 1, 1114 1, 1115 1, 1116 1, 1117 1, 1118 1, 1119 1, 1120 1, 1121 1, 1122 1, 1123 1, 1124 1, 1125 1, 1126 1, 1127 1, 1128 1, 1129 1, 1130 1, 1131 1, 1132 1, 1133 1, 1134 1, 1135 1, 1136 1, 1137 1, 1138 1, 1139 1, 1140 1, 1141 1, 1142 1, 1143 1, 1144 1, 1145 1, 1146 1, 1147 1, 1148 1, 1149 1, 1150 1, 1151 1, 1152 1, 1153 1, 1154 1, 1155 1, 1156 1, 1157 1, 1158 1, 1159 1, 1160 1, 1161 1, 1162 1, 1163 1, 1164 1, 1165 1, 1166 1, 1167 1, 1168 1, 1169 1, 1170 1, 1171 1, 1172 1, 1173 1, 1174 1, 1175 1, 1176 1, 1177 1, 1178 1, 1179 1, 1180 1, 1181 1, 1182 1, 1183 1, 1184 1, 1185 1, 1186 1, 1187 1, 1188 1, 1189 1, 1190 1, 1191 1, 1192 1, 1193 1, 1194 1, 1195 1, 1196 1, 1197 1, 1198 1, 1199 1, 1200 1, 1201 1, 1202 1, 1203 1, 1204 1, 1205 1, 1206 1, 1207 1, 1208 1, 1209 1, 1210 1, 1211 1, 1212 1, 1213 1, 1214 1, 1215 1, 1216 1, 1217 1, 1218 1, 1219 1, 1220 1, 1221 1, 1222 1, 1223 1, 1224 1, 1225 1, 1226 1, 1227 1, 1228 1, 1229 1, 1230 1, 1231 1, 1232 1, 1233 1, 1234 1, 1235 1, 1236 1, 1237 1, 1238 1, 1239 1, 1240 1, 1241 1, 1242 1, 1243 1, 1244 1, 1245 1, 1246 1, 1247 1, 1248 1, 1249 1, 1250 1, 1251 1, 1252 1, 1253 1, 1254 1, 1255 1, 1256 1, 1257 1, 1258 1, 1259 1, 1260 1, 1261 1, 1262 1, 1263 1, 1264 1, 1265 1, 1266 1, 1267 1, 1268 1, 1269 1, 1270 1, 1271 1, 1272 1, 1273 1, 1274 1, 1275 1, 1276 1, 1277 1, 1278 1, 1279 1, 1280 1, 1281 1, 1282 1, 1283 1, 1284 1, 1285 1, 1286 1, 1287 1, 1288 1, 1289 1, 1290 1, 1291 1, 1292 1, 1293 1, 1294 1, 1295 1, 1296 1, 1297 1, 1298 1, 1299 1, 1300 1, 1301 1, 1302 1, 1303 1, 1304 1, 1305 1, 1306 1, 1307 1, 1308 1, 1309 1, 1310 1, 1311 1, 1312 1, 1313 1, 1314 1, 1315 1, 1316 1, 1317 1, 1318 1, 1319 1, 1320 1, 1321 1, 1322 1, 1323 1, 1324 1, 1325 1, 1326 1, 1327 1, 1328 1, 1329 1, 1330 1, 1331 1, 1332 1, 1333 1, 1334 1, 1335 1, 1336 1, 1337 1, 1338 1, 1339 1, 1340 1, 1341 1, 1342 1, 1343 1, 1344 1, 1345 1, 1346 1, 1347 1, 1348 1, 1349 1, 1350 1, 1351 1, 1352 1, 1353 1, 1354 1, 1355 1, 1356 1, 1357 1, 1358 1, 1359 1, 1360 1, 1361 1, 1362 1, 1363 1, 1364 1, 1365 1, 1366 1, 1367 1, 1368 1, 1369 1, 1370 1, 1371 1, 1372 1, 1373 1, 1374 1, 1375 1, 1376 1, 1377 1, 1378 1, 1379 1, 1380 1, 1381 1, 1382 1, 1383 1, 1384 1, 1385 1, 1386 1, 1387 1, 1388 1, 1389 1, 1390 1, 1391 1, 1392 1, 1393 1, 1394 1, 1395 1, 1396 1, 1397 1, 1398 1, 1399 1, 1400 1, 1401 1, 1402 1, 1403 1, 1404 1, 1405 1, 1406 1, 1407 1, 1408 1, 1409 1, 1410 1, 1411 1, 1412 1, 1413 1, 1414 1, 1415 1, 1416 1, 1417 1, 1418 1, 1419 1, 1420 1, 1421 1, 1422 1, 1423 1, 1424 1, 1425 1, 1426 1, 1427 1, 1428 1, 1429 1, 1430 1, 1431 1, 1432 1, 1433 1, 1434 1, 1435 1, 1436 1, 1437 1, 1438 1, 1439 1, 1440 1, 1441 1, 1442 1, 1443 1, 1444 1, 1445 1, 1446 1, 1447 1, 1448 1, 1449 1, 1450 1, 1451 1, 1452 1, 1453 1, 1454 1, 1455 1, 1456 1, 1457 1, 1458 1, 1459 1, 1460 1, 1461 1, 1462 1, 1463 1, 1464 1, 1465 1, 1466 1, 1467 1, 1468 1, 1469 1, 1470 1, 1471 1, 1472 1, 1473 1, 1474 1, 1475 1, 1476 1, 1477 1, 1478 1, 1479 1, 1480 1, 1481 1, 1482 1, 1483 1, 1484 1, 1485 1, 1486 1, 1487 1, 1488 1, 1489 1, 1490 1, 1491 1, 1492 1, 1493 1, 1494 1, 1495 1, 1496 1, 1497 1, 1498 1, 1499 1, 1500 1, 1501 1, 1502 1, 1503 1, 1504 1, 1505 1, 1506 1, 1507 1, 1508 1, 1509 1, 1510 1, 1511 1, 1512 1, 1513 1, 1514 1, 1515 1, 1516 1, 1517 1, 1518 1, 1519 1, 1520 1, 1521 1, 1522 1, 1523 1, 1524 1, 1525 1, 1526 1, 1527 1, 1528 1, 1529 1, 1530 1, 1531 1, 1532 1, 1533 1, 1534 1, 1535 1, 1536 1, 1537 1, 1538 1, 1539 1, 1540 1, 1541 1, 1542 1, 1543 1, 1544 1, 1545 1, 1546 1, 1547 1, 1548 1, 1549 1, 1550 1, 1551 1, 1552 1, 1553 1, 1554 1, 1555 1, 1556 1, 1557 1, 1558 1, 1559 1, 1560 1, 1561 1, 1562 1, 1563 1, 1564 1, 1565 1, 1566 1, 1567 1, 1568 1, 1569 1, 1570 1, 1571 1, 1572 1, 1573 1, 1574 1, 1575 1, 1576 1, 1577 1, 1578 1, 1579 1, 1580 1, 1581 1, 1582 1, 1583 1, 1584 1, 1585 1, 1586 1, 1587 1, 1588 1, 1589 1, 1590 1, 1591 1, 1592 1, 1593 1, 1594 1, 1595 1, 1596 1, 1597 1, 1598 1, 1599 1, 1600 1, 1601 1, 1602 1, 1603 1, 1604 1, 1605 1, 1606 1, 1607 1, 1608 1, 1609 1, 1610 1, 1611 1, 1612 1, 1613 1, 1614 1, 1615 1, 1616 1, 1617 1, 1618 1, 1619 1, 1620 1, 1621 1, 1622 1, 1623 1, 1624 1, 1625 1, 1626 1, 16
```

```

    edge: { a: parent, b: child },
    reason:
      x.grade > y.grade ? undefined : "линия в treap во
  };
}

export function checkGameState(
  points: Pair[],
  edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected:
  const n = points.length;
  if (n < 2) return { heapOk: true, singleParent: true,
  const heapOk = edges.every((e) => e.a.grade > e.b.grade);
  const parentCount = new Map<string, number>();
  for (const e of edges) parentCount.set(pid(e.b), (parentCount.get(pid(e.b)) || 0) + 1);
  const singleParent = [...parentCount.values()].every(v => v === 1);
  // связность: компоненты через union-find
  const comp = new Map<string, string>(points.map((p) => {
  const find = (x: string): string => {
    while (comp.get(x) !== x) {
      comp.set(x, comp.get(x)!);
      x = comp.get(x)!;
    }
    return x;
  };
  for (const e of edges) {
    const ra = find(pid(e.a));
    const rb = find(pid(e.b));
    if (ra !== rb) comp.set(ra, rb);
  }
  const comps = new Set(points.map((p) => find(pid(p))));
  const connected = comps.size === 1;
  const countOk = edges.length === n - 1;
  // BST: дети по x, inorder не убывает; корень один; э
  const children = new Map<string, Pair[]>();
  for (const e of edges) children.set(pid(e.a), [...(children.get(pid(e.a)) || []), e.b]);
  const roots = points.filter((p) => !parentCount.has(pid(p)));
  let bstOk = roots.length === 1;

```

```
-----
  return { heapOk, singleParent, connected, countOk, bs
}
```

```
/** Дерево из проведённых линий — для показа результата
export function edgesToTree(edges: GameEdge[]): TNode |
  const parentOf = new Map<string, GameEdge>();
  for (const e of edges) parentOf.set(pid(e.b), e);
  const roots = edges.filter((e) => !parentOf.has(pid(e
  if (!roots.length && edges.length) return null;
  const nodes = new Map<string, TNode>();
  const make = (p: Pair): TNode => ({
    id: pid(p),
    key: p.key,
    grade: p.grade,
    left: null,
    right: null,
  });
  const rootPair = roots.length ? roots[0].a : edges.le
  if (!rootPair) return null;
  const build = (p: Pair): TNode => {
    const node = make(p);
    nodes.set(pid(p), node);
    const kids = edges.filter((e) => pid(e.a) === pid(p
    if (kids.length > 2) return node; // невалидно — по
    kids.sort((u, v) => u.key - v.key);
    const left = kids.filter((k) => k.key <= p.key); //
    const right = kids.filter((k) => k.key > p.key);
    if (left.length > 1 || right.length > 1) return nod
    if (left[0]) node.left = build(left[0]);
    if (right[0]) node.right = build(right[0]);
    return node;
  };
  return build(rootPair);
}
```

```
const BuildMode: React.FC = () => {
  const [raw, setRaw] = useState(DEMO_RAW);
  const live = useTreapLive();
```

```

const leftRightEdges = useMemo(() => [...canonEdges].

useEffect(() => {
  if (!playing || phase === "game") return;
  const t = setTimeout(() => {
    if (phase === "sort") {
      if (algo === "none") { setPlaying(false); return; }
      if (sortIdx < 3) setSortIdx(sortIdx + 1);
      else setPlaying(false);
    } else {
      const total = order === "y-desc" ? canonEdges.l
      if (connIdx < total) setConnIdx(connIdx + 1);
      else setPlaying(false);
    }
  }, 900);
  return () => clearTimeout(t);
}, [playing, phase, sortIdx, connIdx, algo, order, ca

const startConnect = (ord: ConnectOrder) => {
  setOrder(ord);
  setPhase("connect");
  setConnIdx(0);
  setPlaying(true);
};

const comparisons = algo === "quicksort" ? (n > 1 ? M
const connEdgesAll = order === "y-desc" ? canonEdges
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && conn

// --- игра «соедини сам» ---
const [gameEdges, setGameEdges] = useState<GameEdge[]>
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: n
const [gameMsg, setGameMsg] = useState<string | null>
const [badEdge, setBadEdge] = useState<GameEdge | nul
const [hintUsed, setHintUsed] = useState(false);

```

```

    } else {
      attemptConnect(sel, p);
      setSel(null);
    }
  };
const pointUp = (p: Pair) => {
  if (phase !== "game" || !sel) return;
  if (sel.key === p.key && sel.grade === p.grade) return;
  attemptConnect(sel, p);
  setSel(null);
  setCursor(null);
};
const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
  if (!sel) return;
  const rect = (e.currentTarget as SVGSVGElement).getBoundingClientRect();
  setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100 });
};

const undoEdge = () => {
  setGameEdges((prev) => prev.slice(0, -1));
  setGameMsg(null);
};
const showSolution = () => {
  setGameEdges(canonEdges.map((e) => ({ a: e.a, b: e.b })));
  setHintUsed(true);
  setGameMsg("Решение показано – собери сам в следующем наборе");
};

return (
  <div className="space-y-5">
    {/* Произвольный список */}
    <div className="bg-slate-900/60 rounded-xl p-4 border border-slate-700">
      <div className="flex flex-wrap items-center justify-center">
        <h4 className="text-sm font-bold text-slate-300">Демо-набор
        <button onClick={() => { setRaw(DEMO_RAW); setRawIndex(0); }}
          className="py-1 rounded-lg text-xs font-bold bg-slate-700 text-slate-200">
            демо-набор
      </div>
    </div>
  </div>
);

```

```

        disabled={n < 2}
        className={`px-3 py-1.5 rounded-lg te
600 text-white" : "bg-slate-700 text-slate-
    >
        {a === "quicksort" ? "quicksort (ср
    </button>
    )}}
  </>
) : (
  <>
    <span className="text-xs text-slate-400
    <button
      onClick={() => startConnect("y-desc")}
      className={`px-3 py-1.5 rounded-lg te
bg-emerald-600 text-white" : "bg-slate-700
    >
      по y ↓ (алгоритм)
    </button>
    <button
      onClick={() => startConnect("left-right
      className={`px-3 py-1.5 rounded-lg te
"bg-emerald-600 text-white" : "bg-slate-700
    >
      слева направо
    </button>
    <button
      onClick={startGame}
      disabled={n < 2}
      className={`px-3 py-1.5 rounded-lg te
go-600 text-white" : "bg-indigo-500/80 text
    >
      соединить сам
    </button>
  </>
  )}
</div>
</div>
<Plane

```



```

        <> Проведено {connEdges.length} линий в по
        </b>: точки прибиты, дерево одно. А теперь
    ) : (
        <>Линий проведено: {connIdx} из {connEdgesA
    })
</div>
</div>

```

```

{/* Чек-лист инвариантов */}
<div className={`rounded-xl p-4 border transition
er-slate-700 bg-slate-900/40`} `>
  <h4 className="text-sm font-bold text-slate-300">
  <ul className="space-y-1.5 text-sm">
    {phase === "game" ? (
      <>
        <li>{verdict.heapOk ? " " : " "} каждое р
        <li>{verdict.singleParent ? " " : " "} у
        <li>{verdict.connected ? " " : " "} все т
        <li>{verdict.countOk ? " " : " "} линий р
        <li>{verdict.bstOk ? " " : " "} обход сле
      </>
    ) : (
      <>
        <li>{connDone ? " " : " "} каждое ребро в
        <li>{connDone ? " " : " "} обход слева-на
        <li>{connDone ? " " : " "} линий ровно <b
      </>
    )}
  </ul>
  {phase === "game" && (
    <div className="flex flex-wrap gap-2 mt-3">
      <button onClick={undoEdge} disabled={!gameE
      white disabled:opacity-30">↶ Убрать линию</b
      <button onClick={resetGame} disabled={!gameE
      ate-700 text-white disabled:opacity-30"> С
      <button onClick={showSolution} disabled={ga
      te disabled:opacity-30"> Показать решение<
    </div>
  )}

```

```

    if (live.mid === null || !SPLIT_KEYS.includes(live.m
    setX0(live.mid);
    setStepIdx(0);
    // eslint-disable-next-line react-hooks/exhaustive-r
  }, [live.mid]);
  useEffect(() => {
    if (live.step === null) return;
    setStepIdx(Math.max(0, Math.min(steps.length - 1, l
    // eslint-disable-next-line react-hooks/exhaustive-r
  }, [live.step, steps.length]);
  const done = idx >= steps.length - 1;

  const inL = new Set<NodeId>();
  const inR = new Set<NodeId>();
  steps.slice(0, idx + 1).forEach((s) => (s.goLeft ? in

  const lTree = split.l;
  const rTree = split.r;

  return (
    <div className="space-y-4">
      <div className="flex flex-wrap items-center gap-3">
        <span className="text-sm text-slate-300">Разрез
        <select value={x0} onChange={(e) => { setX0(Num
          -700 rounded-lg px-3 py-1.5 text-sm font-mono
          {[2, 3, 6, 7, 9, 12, 45].map((k) => <option k
        </select>
        <button onClick={() => setStepIdx(Math.max(0, i
          bg-slate-700 text-white disabled:opacity-30">
        <button onClick={() => setStepIdx(Math.min(step
          font-bold bg-indigo-600 text-white disabled:op
        <span className="text-xs text-slate-400">мар {i
      </div>

      <div className="bg-slate-900/40 rounded-xl p-4 bo
        <h4 className="text-sm font-bold text-slate-300
          , <span className="text-rose-400">розовые → R
        <TreeView

```

```

const merged = useMemo(() => mergeTree(cloneTree(l),
const steps = merged.trace;

// step из кода листает кадры слияния
useEffect(() => {
  if (live.step === null) return;
  setStepIdx(Math.max(0, Math.min(steps.length - 1, l
  // eslint-disable-next-line react-hooks/exhaustive-
}, [live.step, steps.length]));
const idx = Math.min(stepIdx, steps.length - 1);
const done = idx >= steps.length - 1;
const cur = steps[idx];

return (
  <div className="space-y-4">
    <div className="flex items-center gap-3">
      <button onClick={() => setStepIdx(Math.max(0, i
        bg-slate-700 text-white disabled:opacity-30">
      <button onClick={() => setStepIdx(Math.min(step
        font-bold bg-indigo-600 text-white disabled:op
      <span className="text-xs text-slate-400">mar {i
    </div>

    <div className="grid grid-cols-1 md:grid-cols-2 g
      <div className="bg-sky-950/20 rounded-xl p-3 bo
        <h5 className="text-sm font-bold text-sky-300
          <TreeView root={l} width={480} height={220} n
        </div>
      <div className="bg-rose-950/20 rounded-xl p-3 b
        <h5 className="text-sm font-bold text-rose-300
          <TreeView root={r} width={480} height={220} n
        </div>
      </div>

    <div className="text-xs text-slate-300 bg-slate-9
      {cur?.chosenId ? (
        <>⊗ {cur.note}</>
      ) : (

```

```

    if (live.step === null) return;
    setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [live.step, steps.length]);
const idx = Math.min(stepIdx, steps.length - 1);
const done = idx >= steps.length - 1;

const pathIds = useMemo(() => {
  const ids: NodeId[] = [];
  for (const s of steps.slice(0, idx + 1)) if (s.atId)
    return ids;
}, [steps, idx]);
const pathSet = new Set(pathIds);

return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center gap-3">
      <span className="text-sm text-slate-300">Удалить</span>
      <select value={key} onChange={(e) => { setKey(NodeId[e.target.value]); }}>
        <option value="0">0</option>
        {POINTS.map((p) => <option key={p.key} value={p.key}>{p.key}</option>)}
      </select>
      <button onClick={() => setStepIdx(Math.max(0, idx - 1))}>
        <span>-</span>
      </button>
      <button onClick={() => setStepIdx(Math.min(stepIdx, steps.length - 1))}>
        <span>+</span>
      </button>
      <span className="text-xs text-slate-400">мар {idx + 1}</span>
    </div>

    {!done ? (
      <div className="bg-slate-900/40 rounded-xl p-4">
        <TreeView
          root={fullTree}
          nodeColor={(id) => (pathSet.has(id) ? "#783f0f" : "#f5f5dc")}
          nodeStroke={(id) => (pathSet.has(id) ? "#f5f5dc" : "#f5f5dc")}
          subtitle="Куча сама умеет удалять только вершины"
        />
      </div>
    ) : null}
  </div>

```

```

    };
    walk(full, 1);
    return arr;
  }, [full]));
const nodeCount = slots.filter(Boolean).length;

return (
  <div className="space-y-4">
    <div className="flex gap-2">
      <button onClick={() => setPick("heap")} className="text-white" : "bg-slate-700 text-slate-300"`>
      <button onClick={() => setPick("treap")} className="text-white" : "bg-slate-700 text-slate-300">
    </div>

    {pick === "heap" ? (
      <div className="bg-slate-900/40 rounded-xl p-4">
        <h4 className="text-sm font-bold text-slate-300">
        <div className="flex gap-2 overflow-x-auto mb-4">
          {heapArr.slice(1).map((v, i) => (
            <div key={i} className="px-3 py-2 rounded">
              <div className="text-slate-500 text-[10px]>
              <div className="text-white font-bold">{v}</div>
              <div className="text-emerald-400 text-[10px]>
            </div>
          ))}
        </div>
        <p className="text-xs text-slate-400">Полное дерево
          →  $2i/2i+1$  попадает в узел.</p>
      </div>
    ) : (
      <div className="bg-slate-900/40 rounded-xl p-4">
        <h4 className="text-sm font-bold text-rose-300">
        <TreeView root={full} subtitle="Попробуй «посмотреть»>
        </div>
        <div className="flex gap-2 overflow-x-auto mt-4">

```


няя граница, каждое сравнение даёт 1 бит. Но для маленьких целых у (вкладка «Собрать», кнопка «Задать порядок соединения».

```

    </div>
  </div>
);
};

/* ===== РЕЖИМ GAME ===== */

const GameMode: React.FC = () => {
  const [edges, setEdges] = useState<GEdge[]>([]);
  const [sel, setSel] = useState<Pair | null>(null);
  const [hover, setHover] = useState<Pair | null>(null);
  const [msg, setMsg] = useState<string>("Кликни точку-1");

  const v = gameVerdict(edges, POINTS);

  const click = (p: Pair) => {
    if (!sel) {
      setSel(p);
      setMsg(`Выбрана (${p.key}; ${p.grade}). Теперь кликай на вторую точку`);
      return;
    }
    if (sameP(sel, p)) {
      setSel(null);
      return;
    }
    const res = tryConnect(edges, sel, p);
    if (res.ok) {
      const parent = sel.grade > p.grade ? sel : p;
      const child = sel.grade > p.grade ? p : sel;
      setEdges([...edges, { a: parent, b: child }]);
      setMsg(`Линия (${parent.key}; ${parent.grade}) - (${child.key}; ${child.grade})`);
      setSel(null);
    } else {
      setMsg("x " + res.reason);
    }
  };
};

```

```

    )))

    {POINTS.map((p) => {
      const isSel = sel && sameP(sel, p);
      const isHover = hover && sameP(hover, p) &&
      return (
        <g key={`gp-${p.key}:${p.grade}`} onClick=
        l)) className="cursor-pointer">
          <circle cx={PX(p.key)} cy={PY(p.grade)}
          <circle cx={PX(p.key)} cy={PY(p.grade)}
            "#a5b4fc" : isHover ? "#94a3b8" : "#38bdf8
            transition-all duration-200" />
          <text x={PX(p.key)} y={PY(p.grade) + 3.5
            space">{p.key}</text>
          <text x={PX(p.key)} y={PY(p.grade) + 24
            }</text>
          </g>
        );
      )))
    </svg>
  </div>

  /* Судья – условия, как у Ахо–Корасика */
  <div className={`rounded-xl p-4 border transition
    ate-900/40`} >
    <h4 className="text-sm font-bold text-slate-300
    <ul className="space-y-1.5 text-sm">
      <li>{v.hear ? " " : " "} каждое ребро – вниз
      <li>{v.oneParent ? " " : " "} у каждой точки
      <li>{v.acyclic ? " " : " "} циклов нет</li>
      <li>{v.bst ? " " : " "} обход слева-направо –
      <li>{v.complete ? " " : " "} линий n-1 = {POI
    </ul>
    {win && (
      <p className="mt-3 text-sm font-bold text-еме
        ения просто не существует.</p>
    )}
  </div>

```



```

const mid = vars ? vizNumber(vars.mid) : null;
const key = vars ? vizNumber(vars.key) : null;
const step = vars ? vizNumber(vars.step) ?? vizNumber(1) : null;
const height = vars ? vizNumber(vars.height) ?? vizNumber(1) : null;
const n = vars ? vizNumber(vars.n) : null;
return { vars, pairsText, mid, key, step, height, n }
};

```

```

/* ===== КОРПУС ===== */

```

```

type Tab = "build" | "split" | "merge" | "erase" | "layers"

```

```

const TABS: { id: Tab; label: string }[] = [
  { id: "build", label: "Собрать" },
  { id: "split", label: "Split  $\times$  " },
  { id: "merge", label: "Merge " },
  { id: "erase", label: "Erase " },
  { id: "layers", label: "Миф:  $2k/2k+1$  " },
  { id: "search", label: "Поиск  $\neq$  сортировка" },
  { id: "game", label: "Игра " },
];

```

```

export const TreapBuildViz = () => {
  const [tab, setTab] = useState<Tab>("build");
  const chapterId = useContext(VizChapterContext);
  const live = useTreapLive();

```

```

  // вкладка = подрежим страницы: панель Python подстав.
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, tab);
  }, [chapterId, tab]);

```

```

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border" >
      <div className="flex flex-wrap items-center justify-center" >
        <div className="flex items-center gap-3" >
          <div className="p-2.5 bg-emerald-600 text-white" >
            <span className="text-2xl" > </span>

```

```

        {live.step !== null && <span className="text-
x-2 py-1">step = {live.step}</span>}
      </>
    ) : (
      <span className="text-[11px] text-slate-500">
        открой панель Python: переменные <code clas
merald-300">mid</code>, <code className="font-m
step</code> управляют этой демонстрацией
      </span>
    )}
  </div>

  {tab === "build" && <BuildMode />}
  {tab === "split" && <SplitMode />}
  {tab === "merge" && <MergeMode />}
  {tab === "erase" && <EraseMode />}
  {tab === "layers" && <LayersMode />}
  {tab === "search" && <SearchMode />}
  {tab === "game" && <GameMode />}
</div>
);
};

```

ФАЙЛ 79/111: src/components/VizLiveStatus.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { liveMatrix, pickLiveVar, useVizRuntime } from
import { VizSourceBadge } from "../VizSourceBadge";

interface VizLiveStatusProps {
  /** Название демонстрации – подписывается в режиме вс
  demoTitle?: string;
  className?: string;
}

```

}

ФАЙЛ 80/111: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphComponentsViz } from "./GraphComponentsViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { ComplexityViz } from "./ComplexityViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import { PlanarColoringViz } from "./PlanarColoringViz";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimationWidget";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumVisualizer";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";

```

    Component: SegmentTreeVisualizer,
  },
  "sparse-table": {
    title: "Разреженная таблица (1D и 2D)",
    hint: "Наведите курсор на любую ячейку – подсветится",
    Component: SparseTableViz,
  },
  treap: {
    title: "Treap: плоскость → дерево → Split/Merge/Erge",
    hint: "7 вкладок: Собрать (сортировка + соединение) +  

      собери дерево мышкой, судья-инварианты объясняет к",
    Component: TreapBuildViz,
  },
  "prefix-sums-2d": {
    title: "Префиксные суммы (1D и 2D)",
    hint: "Наведите на число – увидите, из чего оно сло",
    Component: PrefixSumViz,
  },
  "dynamic-programming": {
    title: "Динамическое программирование",
    hint: "Таблица подзадач заполняется на глазах.",
    Component: DPVisualizer,
  },
  "splay-tree": {
    title: "Splay: механика → большое дерево → собери с",
    hint: "Три вкладки: покадровая механика поворотов в  

      песочница «подними узел сам» с проверкой порядка.",
    Component: SplayPlayground,
  },
  "graph-dfs-bfs": { title: "DFS и BFS на графе", Component:
  "top-sort": { title: "Топологическая сортировка", Component:
  "scc-kosaraju": { title: "Компоненты сильной связности", Component:
  "graph-articulation": { title: "Точки сочленения", Component:
  "bridges-code": { title: "Мосты: DFS + tin/low", Component:
  "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component:
  "planarity-euler-formula": {
    title: "Планарность и покраска:  $K_5/K_{3,3}$ ,  $\chi \leq \Delta+1$ , д
    hint: "Четыре вкладки: непланарные  $K_5$  и  $K_{3,3}$ , жадна

```

```
    hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
    Component: () => (
      <div className="space-y-10">
        <WaterfallAnimationWidget />
        <SalmonAutomatonWidget />
      </div>
    ),
  },
  intro: {
    title: "Интерактивный симулятор базовых структур: с",
    hint: "Проверь LIFO, FIFO и приоритетную очередь на",
    Component: Simulator,
  },
  "everyday-basics": {
    title: "Бытовой тренажёр: стек, очередь, куча",
    hint: "Тарелки, очередь в столовой и мешок гипотез",
    Component: Simulator,
  },
};

export const getViz = (id?: string): VizEntry | undefined;
```

ФАЙЛ 81/111: src/components/VizSourceBadge.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { Cpu, Film } from "lucide-react";
import { Tooltip } from "../Tooltip";

interface VizSourceBadgeProps {
  /**
   * true – картинку рисуют переменные из выполненного кода
   * false – компонент показывает собственную встроенную картинку
   */
  linked: boolean;
  /**
```

```

        tabIndex={0}
        className={`cursor-help inline-flex items-center
        linked
        ? "border-emerald-500/50 bg-emerald-500/10
        : awaiting
        ? "border-sky-500/50 bg-sky-500/10 text
        : "border-slate-600 bg-slate-800/80 tex
    }`}
    >
    {linked ? <Cpu className="h-3 w-3" /> : await
    {linked ? "данные из компилятора" : awaiting
        встроенное демо"}
    {linked && detail ? <span className="font-mon
    </span>
    </Tooltip>

    {!linked && !awaiting && demoTitle ? (
        <span className="rounded-full border border-sla
        сценарий: {demoTitle}
        </span>
    ) : null}

    {mismatch ? (
        <Tooltip side="bottom" content={mismatch}>
        <span
            tabIndex={0}
            className="cursor-help rounded-full border l
            0"
        >
            размерность не совпадает
        </span>
        </Tooltip>
    ) : null}
    </div>
    );
}

```

```

    = 2 (HE)
};

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
  0: { H: 1, S: 7 },
  1: { E: 2, I: 5 },
  2: { R: 3 },
  3: { S: 4 },
  4: {},
  5: { S: 6 },
  6: {},
  7: { H: 8 },
  8: { E: 9 },
  9: {},
};

export const WaterfallAnimationWidget: React.FC = () => {
  // Переключатель режимов: 'training' (Полный BFS обход)
  const [activeMode, setActiveMode] = useState<'training'>('training');

  /* — связь с Python-компилятором (PAGE_SYNC["aho-corasick"])
     text — сканируемый текст, state — вершина автомата
     matches — найденные вхождения (индекс конца, слово)
  */
  const chapterId = useContext(VizChapterContext);
  const runtime = useVizRuntime();
  const vars = runtime?.variables;

  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, "aho");
  }, [chapterId]);

  const liveText = vars ? vizString(vars.text) ?? vizString("") : "";
  const liveState = vars ? vizNumber(vars.state) ?? vizNumber(0) : 0;
  const liveI = vars ? vizNumber(vars.i) ?? vizNumber(0) : 0;
  const liveMode = vars ? vizString(vars.mode) : null;
  const liveMatchesRaw = vars ? vizArray(vars.matches) : [];
  const liveMatchesSig = useMemo(() => {liveMatchesRaw.map((m) => m[0] + m[1])})

```

```

    // eslint-disable-next-line react-hooks/exhaustive-
  }, [vars]));

const compilerLinked = Boolean(liveText || (liveState

// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
const [textToScan, setTextToScan] = useState<string>(
const [currentIndex, setCurrentIndex] = useState<number>(
const [currentNode, setCurrentNode] = useState<number>(
const [isSearchDone, setIsSearchDone] = useState<boolean>(
const [searchLog, setSearchLog] = useState<string>(
  'Режим поиска: Введите текст и жмите «Следующий шаг»
);
const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>(
const [foundMatches, setFoundMatches] = useState<{ index: number; text: string }[]>(
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const clean = e.target.value.toUpperCase().replace(/ /g, '');
  setTextToScan(clean);
  resetSearchSimulation();
};

const resetSearchSimulation = () => {
  setCurrentIndex(0);
  setCurrentNode(0);
  setIsSearchDone(false);
  setSearchLog('Симуляция сброшена. Лосось вернулся на базу');
  setJumpPath(null);
  setFoundMatches([]);
  setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {

```



```

    }
    curr = nextNode;
  }

  const currentMatches: { index: number; word: string }[] = [];
  const matchedWordsForLog: string[] = [];

  let temp = curr;
  while (temp !== 0) {
    if (WATERFALL_NODES[temp].is_terminal) {
      WATERFALL_NODES[temp].words.forEach((word) => {
        currentMatches.push({ index: currentIndex, word });
        matchedWordsForLog.push(word);
      });
    }
    temp = WATERFALL_NODES[temp].term_link;
  }

  if (matchedWordsForLog.length > 0) {
    logMsg += `    БИНГО! Найдено слово: ${matchedWordsForLog.join(' ')}\n`;
    setActiveSearchCodeLine(4);
  }

  setCurrentNode(curr);
  setCurrentIndex(currentIndex + 1);
  setSearchLog(logMsg);
  if (currentMatches.length > 0) {
    setFoundMatches((prev) => [...prev, ...currentMatches]);
  }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В ЧИСТОМ ВИДЕ)
const TRAINING_STEPS_INFO = [
  {
    targetNodeId: 0,
    title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
    desc: 'Начало алгоритма BFS. Корень (ROOT) не обрабатывается. Первые вершины сразу инициализируются с fail = ROOT',
  },
];

```

```

    },
    {
        targetNodeId: 5,
        title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
        desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
            H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
        codeLine: 3,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'I',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 8,
        title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТА
        desc: '    ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
            ). Рыба плавает в ROOT и ищет ветку "H". У корня
            суффикс "H"!',
        codeLine: 4,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'H',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 3,
        title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
        desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
            е подходят. fail[HER] = ROOT.',
        codeLine: 3,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'R',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 6,
        title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',

```

```

    { /* Шапка и переключатель режимов */ }
    <div className="flex flex-wrap items-center justify-center">
      <div className="flex items-center space-x-3">
        <span className="text-3xl"> </span>
        <div>
          <h4 className="text-2xl font-bold text-white">
            Анимация водопада: Полное обучение автомата
          </h4>
          <p className="text-sm text-slate-400">
            Визуализация пошагового построения всех фаз
          </p>
        </div>
      </div>
    </div>

    { /* Переключатель режимов */ }
    <div className="bg-slate-900 p-1.5 rounded-xl border">
      <button
        onClick={() => setActiveMode('training')}
        className={`flex items-center space-x-1.5 p-2
          ${activeMode === 'training'
            ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
            : 'text-slate-400 hover:text-white'}
        `}
      >
        <GitBranch className="w-4 h-4" />
        <span>Режим 1: Полное обучение (BFS от корня)</span>
      </button>
      <button
        onClick={() => setActiveMode('search')}
        className={`flex items-center space-x-1.5 p-2
          ${activeMode === 'search'
            ? 'bg-blue-600 text-white shadow-lg shadow-blue-500'
            : 'text-slate-400 hover:text-white'}
        `}
      >
        <Play className="w-4 h-4" />
        <span>Режим 2: Поиск в тексте</span>
      </button>
    </div>
  </div>
</div>

```

```

        onClick={() => setTrainStep((prev) => M
        disabled={trainStep === 0}
        className="flex-1 bg-slate-700 hover:bg
ion-all"
      >
        Назад
      </button>
    <button
      onClick={() => setTrainStep((prev) => M
      disabled={trainStep === TRAINING_STEPS_
      className="flex-1 bg-indigo-600 hover:b
ition-all shadow-lg shadow-indigo-900/50"
    >
      {trainStep === TRAINING_STEPS_INFO.leng
    </button>
  </div>
</div>

{/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
  >
    <div>
      <h5 className="text-white font-bold mb-3
        <span className="flex items-center gap-
          <span>❌</span> Код BFS построения fail
        </span>
        <span className="text-xs bg-indigo-950
          Вершина: #{targetTrainingNode.id} ({t
        </span>
      </h5>
      <div className="bg-slate-950 p-4 rounded-
        <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-500 text-indigo-300 font
          <span>1. let u = trie[r][char]; // Те
            {currentTrainInfo.codeLine === 1 && <
          тут</span>}
        </div>
        <div className={`p-1.5 rounded flex ite

```

```

    </div>
  ) : (
    /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
    <div className="grid grid-cols-1 lg:grid-cols-3" >
      <div className="bg-slate-900/60 p-5 rounded-x" >
        <div>
          <h5 className="text-white font-bold mb-2" >
            <span>» </span> Текст для сканирования
          </h5>
          <p className="text-xs text-slate-400 mb-4" >
            Введи текст, чтобы лосось просканировал
          </p>
          <input
            type="text"
            value={textToScan}
            onChange={handleTextChange}
            maxLength={15}
            placeholder="ВВЕДИТЕ ТЕКСТ..."
            className="w-full bg-slate-800 border b
            cus:outline-none focus:border-blue-500"
          />
        </div>

        <div>
          <div className="mb-4 flex flex-wrap gap-1" >
            <span className="text-xs text-slate-400" >
              {textToScan.split('').map((char, idx) =>
                <span
                  key={idx}
                  className={`w-7 h-8 flex items-cent
                    idx === currentIndex
                      ? 'bg-blue-600 text-white shadow
                      : idx < currentIndex
                      ? 'bg-emerald-950/60 text-emera
                      : 'bg-slate-800 text-slate-500'
                  }`}
                >
                  {char}

```

```

border-rose-500 text-rose-300 font-bold an
    <span>2. while (curr != root && !trie
        {activeSearchCodeLine === 2 && <span>
рыжок по fail!</span>}
    </div>
    <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold` :
    <span>3. curr = trie[curr].get(char)
        {activeSearchCodeLine === 3 && <span>
ем по течению</span>}
    </div>
    <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 fon
    <span>4. if (is_terminal[curr]) report
        {activeSearchCodeLine === 4 && <span>
овпадение!</span>}
    </div>
    </div>
</div>

<div>
    <h5 className="text-slate-300 font-bold m
    <AlertCircle className="w-3.5 h-3.5 tex
    </h5>
    <div className="bg-slate-800 p-3.5 rounde
tems-center">
        {searchLog}
    </div>
</div>
</div>
</div>
))

```

```

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 vi
-hidden shadow-2xl">
    {/* Водопадные фоновые эффекты */}
    <div className="absolute inset-0 opacity-10 bg-

```

```

        y2={level.y}
        stroke="#334155"
        strokeWidth="1"
        strokeDasharray="4,4"
        opacity="0.4"
      />
      <rect x="25" y={level.y - 12} width="14" height="12" />
    />
    <text x="32" y={level.y + 4} fill="#94a0b4" />
      {level.label}
    </text>
  </g>
)}}

```

```

{/* Рендер прямых переходов (потоков воды) */}
Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
  const fromNode = WATERFALL_NODES[Number(fromId)];
  return Object.entries(children).map(([childId, child]) => {
    const toNode = WATERFALL_NODES[toId];

```

```

    // Подсветка в режиме поиска
    let isHighlighted = activeMode === 'search' ||
    type === 'normal';

```

```

    // Подсветка в режиме обучения (когда ищем путь)
    let isTrainingExamined = activeMode === 'training' ||
    let isMatchBranch = isTrainingExamined && childId === toId;

```

```

    return (
      <g key={`edge-${fromNode.id}-${toNode.id}`} />
      {/* Линия течения воды */}
      <line
        x1={fromNode.x}
        y1={fromNode.y}
        x2={toNode.x}
        y2={toNode.y}
        stroke={isHighlighted ? '#3b82f6' : '#334155'}
        strokeWidth={isHighlighted ? 2 : 1}
      />
    );

```

```

// Найдём индекс шага, на котором обработать
const stepIdxForNode = TRAINING_STEPS_INF
if (activeMode === 'training' && trainStep

    let isHighlighted = activeMode === 'search
e === 'fail';
    let isJustEstablished = activeMode === 't
-ссылку

// Вычисляем изогнутую кривую для красоты
const dx = targetNode.x - node.x;
const dy = targetNode.y - node.y;
const cx = node.x + dx / 2 + (node.id % 2
const cy = node.y + dy / 2 - 30;

return (
  <g key={`fail-${node.id}`}>
    <path
      d={`M ${node.x} ${node.y} Q ${cx} $
      fill="none"
      stroke={isHighlighted ? '#f43f5e' :
      strokeWidth={isHighlighted || isJus
      strokeDasharray="6,6"
      className={isHighlighted || isJustE
      opacity={isHighlighted || isJustEst
    />
    {isJustEstablished && (
      <text x={cx} y={cy - 10} fill="#10b
  >
    fail[{node.label}] = {targetNode.
    </text>
  )}
</g>
);
}})

```

/* Рендер вершин (бассейнов водопада) */

Object.values(WATERFALL_NODES).map((node)


```

        {isTargetChild && {
            <g transform={`translate(${node.x},
                <rect x="-35" y="-12" width="70"
                <text x="0" y="2" fontSize="10" f
                    ИЩЕМ FAIL
                </text>
            </g>
        }}

        {/* Индикатор словарных слов */}
        {hasWords && {
            <g transform={`translate(${node.x +
                <circle cx="0" cy="0" r="9" fill=
                <text x="0" y="3" fontSize="10" f
                    *
                </text>
            </g>
        }}
    </g>
    );
}}
}}}

{/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */}
<g>
    {/* Круг-подложка под рыбу */}
    <circle
        cx={activeFishPosNode.x + 15}
        cy={activeFishPosNode.y - 25}
        r="18"
        fill={activeMode === 'training' ? '#8b5
        stroke="#ffffff"
        strokeWidth="2"
        style={{ transition: 'cx 0.8s cubic-bez
        className="shadow-lg"
    />
    {/* Сама рыба */}
    <text
        x={activeFishPosNode.x + 15}

```

```

        }}}
      </div>
    })
  </div>
})

{/* что сейчас сообщил компилятор */}
<div className="mt-6 flex flex-wrap items-center" >
  <span className="text-[10px] uppercase tracking-
    <Terminal className="w-3.5 h-3.5" /> из компи.
  </span>
  {compilerLinked ? (
    <>
      {liveText && <span className="text-[11px] f
      >text = {liveText}</span>}
      {liveI !== null && <span className="text-[1
      py-1">i = {liveI}{liveText ? ` (${liveText.
      {liveState !== null && (
        <span className="text-[11px] font-mono bg
        state = {liveState}{WATERFALL_NODES[liv
        </span>
      )}
      {liveMatchesSig && <span className="text-[1
      d px-2 py-1">matches: {foundMatches.map((m)
      {liveFail !== undefined && <span className=
      ed-md px-2 py-1">fail = {JSON.stringify(liv
    </>
  ) : (
    <span className="text-[11px] text-slate-500">
      открой панель Python и запусти Ахо–Корасик:
      "font-mono text-cyan-300">state</code>, <co
      an-300">matches</code> поведут лосося по эт
    </span>
  )}
</div>
</div>
);
};

```

```

    [4, 5, 6, 7],
  ];

const cellBase =
  "flex items-center justify-center rounded-md font-mono"
  "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
  const [mode, setMode] = useState<"1d" | "2d">("1d");
  const chapterId = useContext(VizChapterContext);

  // Вкладка 2D = своё демо (свой скелет): компилятор по
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, mode === "2d"
  }, [chapterId, mode]);

  return (
    <div className="w-full bg-slate-950 rounded-2xl border"
      <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar"
        {(
          [
            ["1d", "1. Одномерные суммы"],
            ["2d", "2. Двумерные суммы"],
          ] as const
        ).map(([key, label]) => (
          <button
            key={key}
            onClick={() => setMode(key)}
            className={`px-3 py-2 rounded-lg text-xs sm:text-sm
              mode === key ? "bg-indigo-600 text-white" : "text-slate-400"
            }`}
          >
            {label}
          </button>
        ))}
      </div>
      {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
    </div>
  );
};

```

```

<div className="overflow-x-auto pb-1">
  <div className="inline-block min-w-full">
    {/* индексы */}
    <div className="flex gap-1.5 mb-1 pl-[52px]">
      {A1.map((_, i) => (
        <div key={i} className={` ${cellBase} !h-5
          {i}
        </div>
      ))}
    </div>

    {/* массив a */}
    <div className="flex gap-1.5 items-center mb-1">
      <div className="w-[46px] shrink-0 text-right">
        {A1.map((v, i) => {
          const inCover = covered && i >= covered.f
          const inRange = i >= range.l && i <= range.r
          const isDrivenAdd = shownDriven !== null
          return (
            <div
              key={i}
              onMouseEnter={() => setHover({ kind:
              onMouseLeave={() => setHover(null)}
              onClick={() => setRange((r) => (i < r
              className={` ${cellBase} cursor-pointe
              isDrivenAdd
                ? "bg-amber-500 text-black ring-2
                : inCover
                ? "bg-indigo-500 text-white rin
                : inRange
                  ? "bg-indigo-900/70 text-indi
                  : "bg-slate-800 text-slate-300
              `}
              title={`a[${i}] = ${v}`}
            >
              {v}
            </div>
          )
        })}
      </div>
    </div>
  );

```

```

        </div>
    );
    }}}
</div>
</div>
</div>

{ /* Пояснение под курсором */
<div className="min-h-[62px] bg-slate-900 border
    {shownDriven !== null && !hover ? (
        <p className="text-slate-300">
            <b className="text-emerald-400">Компилятор
            <span className="font-mono text-white">i =
            <span className="font-mono text-emerald-300
            {hasLiveP && <> = <span className="font-mono
            <span className="ml-1 text-slate-500">Пустой
        </p>
    ) : hover?.kind === "pref" ? (
        hover.i === 0 ? (
            <p className="text-slate-300">
                <b className="text-emerald-400">P[0] = 0<
                отрезка, начинающегося с нуля.
            </p>
        ) : (
            <p className="text-slate-300">
                <b className="text-emerald-400">
                    P[{hover.i}] = {pref[hover.i]}
                </b>{ " " }
                – это сумма всего, что набежало от начала
            <span className="font-mono text-indigo-300
                a[0..{hover.i - 1}] = {A1.slice(0, hove
            </span>
        </p>
    )
    ) : hover?.kind === "a" ? (
        <p className="text-slate-300">
            <b className="text-indigo-400">
                a[{hover.i}] = {A1[hover.i]}

```

```

const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);

const P = useMemo(() => {
  const p = Array.from({ length: n + 1 }, () => Array<number>().length = m + 1);
  for (let i = 1; i <= n; i++) {
    for (let j = 1; j <= m; j++) {
      p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i][j - 1];
    }
  }
  return p;
}, [n, m]);

const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

/**
 * Матрицы из кода пользователя рисуются В СВОЕЙ размерности.
 *
 * Раньше значения S[i][j] протискивались в демо-сетку,
 * размере матрицы часть ячеек становилась «пустыми»,
 * шаги компилятора не совпадали с картинкой.
 */
const livePrefSource = pickLiveVar(vars, ["S", "P", "g"]);
const liveS = liveMatrix(livePrefSource?.value);
const hasLiveS = liveS !== null;

const liveArrSource = pickLiveVar(vars, ["a", "A", "g"]);
const liveA = liveMatrix(liveArrSource?.value);

const prefRows = liveS?.rows ?? P.length;
const prefCols = liveS?.cols ?? (P[0]?.length ?? 0);
const grid: (number | null)[][] = liveS
  ? Array.from({ length: prefRows }, (_, i) =>
    Array.from({ length: prefCols }, (_, j) => liveS[i][j]))
  : P;

```

```
const Aa = cellAt(r2 + 1, c2 + 1);
const total = Aa !== null && B !== null && Cc !== null;
const fmt = (v: number | null) => (v === null ? "-" : v);

const cornerOf = (i: number, j: number) => {
  if (i === r2 + 1 && j === c2 + 1) return "A";
  if (i === r1 && j === c2 + 1) return "B";
  if (i === r2 + 1 && j === c1) return "C";
  if (i === r1 && j === c1) return "D";
  return null;
};

return (
  <div className="space-y-5">
    {sizeMismatch ? (
      <p className="rounded-lg border border-amber-500">
        {sizeMismatch}
      </p>
    ) : null}
    <div className="grid gap-5 lg:grid-cols-2">
      /* Исходная матрица */
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">
          Исходная матрица {liveA ? `${liveArrSource}` : ""}
          чтобы задать прямоугольник
        </div>
        <div className="inline-block">
          {srcGrid.map((row, i) => (
            <div key={i} className="flex gap-1.5 mb-1">
              {row.map((v, j) => {
                const inRect = i >= r1 && i <= r2 &&
                return (
                  <div
                    key={j}
                    onClick={() =>
                      setRect((r) =>
                        i < r.r1 || j < r.c1
                          ? { r1: i, c1: j, r2: Math.max(r.r2, i), c2: Math.max(r.c2, j)}
                          : r
                    >
                      {v}
                    </div>
                )
              })}
            </div>
          ))}
        </div>
      </div>
    </div>
  </div>
);
```

```

        return {
            <div
                key={j}
                onMouseEnter={() => setHover({ i,
                onMouseLeave={() => setHover(null
                className={`${cellBase} cursor-he
                isCompilerCell
                    ? "bg-emerald-600 text-white
                    : isHover
                        ? "bg-amber-500 text-white
                        : blank
                            ? "bg-slate-950 text-slate-300
                            : cornerColor
                `}
                title={hasLiveS ? `${livePrefSource}
            >
            {blank ? "." : String(shownValue)}
            {corner && !isHover && {
                <span className="absolute -top-
ht">
                {corner}
            </span>
            }}
        </div>
    );
    }}}
</div>
    )))
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border
    {hover ? {
        <p className="text-slate-300">
            <b className="text-amber-400">
                {liveS ? livePrefSource?.name ?? "S" : "P
            </b>{" "}
```



```

    vizRecord,
    VizChapterContext,
} from "../../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
    Play,
    RotateCcw,
    ChevronRight,
    ChevronLeft,
    Lock,
    ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
        st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j - 1)][j - 1]);
    }
}

// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
    [45, 12, 88, 34, 11, 76, 23, 90],
    [67, 19, 44, 55, 33, 21, 65, 87],
    [14, 51, 99, 13, 22, 64, 43, 76],
    [89, 32, 54, 71, 15, 88, 29, 60],
    [25, 41, 16, 92, 9, 17, 56, 31],
    [59, 18, 77, 24, 61, 82, 35, 12],
    [73, 8, 38, 85, 47, 95, 19, 58],
    [39, 81, 62, 28, 51, 42, 85, 14]
];

```

```

const [tab, setTab] = useState<"1d" | "2d_build" | "2d_query">("1d");
const chapterId = useContext(VizChapterContext);

// Каждая вкладка = своё демо с собственным скелетом:
useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, tab === "2d_build" ? "2d_build" : "2d_query", [chapterId, tab]);
}, [chapterId, tab]);

return (
  <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
    <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
      <button
        onClick={() => setTab("1d")}
        className={`px-3 sm:px-4 py-2 rounded-lg text-white bg-slate-800 text-slate-400`}
      >
        1. Сворачивание 1D
      </button>
      <button
        onClick={() => setTab("2d_build")}
        className={`px-3 sm:px-4 py-2 rounded-lg text-white bg-slate-800 text-slate-400`}
      >
        2. Сворачивание 2D (Построение)
      </button>
      <button
        onClick={() => setTab("2d_query")}
        className={`px-3 sm:px-4 py-2 rounded-lg text-white bg-slate-800 text-slate-400`}
      >
        3. Запрос 2D (Формула)
      </button>
    </div>

    <AnimatePresence mode="wait">
      {tab === "1d" ? (
        <Viz1D key="1d" />
      ) : tab === "2d_build" ? (

```

```

: null
: hasLiveSource && liveSource && liveSource.length
? `Демо-массив из ${N} элементов, а в вашем коде
  ся демо-размера ${N}×${LOG}.`
: null;

// Total steps: we animate computing each element for
const computeSteps: { i: number; j: number }[] = [];
for (let j = 1; j < LOG; j++) {
  for (let i = 0; i + (1 << j) <= N; i++) {
    computeSteps.push({ i, j });
  }
}

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i,

return (
  <div className="flex flex-col gap-5">
    <div className="flex items-center justify-between">
      <div>
        <h3 className="text-lg font-bold text-indigo-500">
          Построение Sparse Table (Min)
        </h3>
        <p className="text-sm text-slate-400">
          Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$ 
        </p>
        {dimMismatch ? (
          <p className="mt-2 max-w-[560px] rounded-lg">
            {dimMismatch}
          </p>
        ) : null}
      </div>
      <div className="flex gap-2">
        <button

```

```

        }` }
      >
        {display}
      </div>
      <span className="text-[9px] text-slate-100">
    </div>
  );
  }}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
  {hover && covered ? (
    <p className="text-slate-300">
      <b className="text-sky-400">
        ST[{hover.i}][{hover.j}] = {compilerLin
      j]}
      </b>{ " "}
      – это минимум на отрезке{ " "}
      <span className="font-mono text-sky-300">
        [{covered.from}, {covered.to}]
      </span>{ " "}
      длины  $2^{\text{hover.j}}$  = {1 << hover.j}:{ " "}
      <span className="font-mono text-slate-400">
        min({arr1D.slice(covered.from, covered.to)}
      </span>
    </p>
  ) : (
    <p className="text-slate-500">
      Наведите курсор (или тапните) на любое чи
    </p>
  )}
</div>
</div>

<div className="overflow-x-auto pb-2">
  <div className="inline-block min-w-full bg-slate-100">
    <div className="grid grid-cols-[auto_repeat(4_1fr)]">
      { /* Headers */ }
      <div className="text-slate-500 font-mono text-sm">

```

```

    let isActive = !!liveTarget && liveTarget.i === i
    let isSrc1 = false;
    let isSrc2 = false;

    if (compilerLinked && liveTarget && liveTarget.i === i) {
      isSrc1 = i === liveTarget.i && j === liveTarget.j
      isSrc2 = i === liveTarget.i + (1 << liveTarget.j) && j === liveTarget.j
    } else if (!compilerLinked && step > 0) {
      const currentStep = computeSteps[step]
      isActive = currentStep.i === i && currentStep.j === j
      if (currentStep.j - 1 === j) {
        if (currentStep.i === i) isSrc1 = true
        if (currentStep.i + (1 << (currentStep.j - 1)) === i) isSrc2 = true
      }
    }

    // Готовые вычисленные уровни демо не нужны
    // уровень j=0 всегда берём из уже существующих
    const shownValue = compilerLinked ? liveTarget.value : undefined
    const displayValue =
      shownValue === undefined || shownValue === null
        ? "."
        : typeof shownValue === "object"
          ? JSON.stringify(shownValue)
          : String(shownValue);

    return (
      <div key={j} className="relative flex flex-col items-center justify-center" >
        {!isValid ? (
          <div className="w-[clamp(30px,90px,100px)] h-[clamp(30px,90px,100px)]" >
            <div>
              <motion.div
                onMouseEnter={() => (compilerLinked ? setHover(1) : setHover(0))}
                onMouseLeave={() => setHover(0)}
                onClick={() => (compilerLinked ? setStep(0) : setStep(1))}
                style={{ opacity: 0, scale: 0.5 }}
              />
            </div>
          </div>
        ) : (
          <div>
            <motion.div
              style={{ opacity: 1, scale: 1 }}
            />
          </div>
        )}
      </div>
    )
  )}
}

```

```

        overflow: "visible",
      }}
    >
      <path
        d={`M 0 0 C 30 0, 30 ${isSrc1 ? "100%" : "0%"}
        fill="none"
        stroke={isSrc1 ? "#10b981" : "#f87171"}
        strokeWidth="3"
      />
    </motion.svg>
  ))
</div>
);
}}}
</React.Fragment>
)}}
</div>
</div>
</div>

```

```

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-700 p-4">
  {compilerLinked && liveTarget ? (
    <p className="text-base text-slate-300">
      <span className="text-emerald-400 font-bold">ST</span>
      <span className="font-mono text-white">i =
      – активна ячейка <span className="font-mono text-white">
        {(hasLiveTable || hasLiveSource) && <> = <span>
        ?.[liveTarget.j] ?? (liveTarget.j === 0 ? liveTarget.i : liveTarget.j)
      </span>
    </p>
  ) : step > 0 && step <= maxStep ? (
    (() => {
      const c = computeSteps[step - 1];
      const half = 1 << (c.j - 1);
      return (
        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">
            ST[{c.i}][{c.j}]
          </span>
        </p>
      );
    })
  ) : null
  )}

```

```

/**
 * Шаг, закреплённый кликом по вкладке. Без него live
 * перетирал выбор пользователя и вкладки «Шаг 1/2/3»
 * клик есть, а матрица не меняется. Закрепление живё
 */
const [pinnedK, setPinnedK] = useState<number | null>
const prevLiveK = useRef<number | null>(liveK);
useEffect(() => {
  if (liveK !== prevLiveK.current) {
    prevLiveK.current = liveK;
    setPinnedK(null);
  }
}, [liveK]);
const liveR = vizNumber(vars?.r);
const liveC = vizNumber(vars?.c);
const liveTable = vizRecord(vars?.st2);
const hasLiveSource = !!vars && Object.prototype.hasOwnProperty.call(vars, 'source');
const hasLiveTable = !!vars && Object.prototype.hasOwnProperty.call(vars, 'table');
const liveTableEmpty = hasLiveTable && Object.keys(liveTable).length === 0;
const codeCell = liveR !== null && liveC !== null ? {
  const compilerLinked = codeCell !== null || liveK !== null;
  const shownK = pinnedK !== null
    ? pinnedK
    : liveK !== null
      ? Math.max(0, Math.min(3, Math.trunc(liveK)))
      : hasLiveSource
        ? 0
        : k;

  /** 2D-демонстрация нарисована на фиксированной сетке
  const liveSource2D = liveMatrix(vars?.A);
  const dimMismatch2D = liveSource2D
    ? liveSource2D.rows !== 8 || liveSource2D.cols !== 8
    : `Демо 2D построено на матрице 8x8, а в вашем ко
      не её не отрисовываются, значения вашей матрицы
    : null
    : null;

```



```

        3)] md:scale-[1.08]';
            }
            zIndex = 'z-10';
        }
    }

    return (
      <div
        key={idx}
        onMouseEnter={() => { if(is
        onMouseLeave={() => { if(is
        onClick={() => {
          if(isCurr) {
            if(locked && locked.
            else setLocked({r, c
          }
        }}
        className={`flex items-cent
sCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp
5.2vw,30px)] h-[clamp(20px,5.2vw,30px)] min
        title={hasLiveTable ? `st2[
      >
        {isBlank ? "." : String(shor
      </div>
    );
  }}}
  </div>
</div>
</div>
</React.Fragment>
)
  }}}
</div>
</div>

<div className="flex-1 min-w-0 flex flex-col gap-
  <div className="bg-slate-900 p-4 sm:p-6 rounded
    <h3 className="text-lg font-bold text-slate-3

```

```

        &nbsp;&nbsp;&nbsp;<span className="text-rose
e="text-amber-300">1</span>],          <sp
        &nbsp;&nbsp;&nbsp;<span className="text-blue
ext-amber-300">1</span>][k-<span className=
pan><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-emer
="text-amber-300">1</span>][k-<span classNa
span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-ambe
white">c + pL</span>][k-<span className="te
ame="text-slate-500">// Правый Низ</span><b
        {''}}));
    </div>

    {activeCell ? {
      <div className="bg-slate-950 p-5 round
.2)] mt-2 transition-all">
        <p className="text-slate-300 mb-3 t
          <span>Пример для ячейки:</span>
          {locked && <span className="text
ded border border-rose-500/20"><Lock size={
        </p>
        <p className="text-indigo-300 borde
          <span className="font-bold text-w
        }][{shownK}][{shownK}]</span>
          <span className="text-slate-400">
        </p>

        <div className="space-y-2 pt-3 pl-2
          {{{}} => {
            const prevL = 1 << (shownK -
            return (
              <>
                <div className="flex ite
                  <span className="flex
hadow-[0_0_8px_#f43f5e]"></div> ST[{activeC
                  <span className="text
tiveCell.r)][activeCell.c][shownK-1][shownK-

```

```

    })
  </div>
</div>
</div>
);
};

const Viz2DQuery: React.FC = () => {
  const [r1, setR1] = useState(1);
  const [c1, setC1] = useState(1);
  const [r2, setR2] = useState(5);
  const [c2, setC2] = useState(6);
  const runtime = useVizRuntime();

  // Координаты запроса – те же захардкоженные имена,
  useEffect(() => {
    const vars = runtime?.variables;
    const nr1 = vizNumber(vars?.r1);
    const nc1 = vizNumber(vars?.c1);
    const nr2 = vizNumber(vars?.r2);
    const nc2 = vizNumber(vars?.c2);
    if (nr1 !== null) setR1(Math.max(0, Math.min(7, M
    if (nc1 !== null) setC1(Math.max(0, Math.min(7, M
    if (nr2 !== null) setR2(Math.max(0, Math.min(7, M
    if (nc2 !== null) setC2(Math.max(0, Math.min(7, M
  }, [runtime]));

  const queryVars = runtime?.variables;
  const liveCoords = ["r1", "c1", "r2", "c2"].map((na
  const queryMismatch = liveCoords.some((v) => v !== r
    ? `Координаты запроса в вашем коде выходят за дем
      ] ?? "-", r2=${liveCoords[2] ?? "-"}, c2=${live
    : null;

  const H = Math.max(1, r2 - r1 + 1);
  const W = Math.max(1, c2 - c1 + 1);
  const kx = Math.floor(Math.log2(H));
  const ky = Math.floor(Math.log2(W));

```

```

        style={{
          top: `${(r1 / 8) * 100}%`,
          left: `${(c1 / 8) * 100}%`,
          marginTop: '8px',
          marginLeft: '8px',
          height: 'calc(' + (H/8*100) +
          width: 'calc(' + (W/8*100) +
        }}
      />

      {/* Показываем 4 угла в самой матрице */}
      <div className="absolute pointer-events-none shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(
        <div className="absolute pointer-events-none shadow-[0_0_10px_#3b82f6]" style={{ top: `calc(
          width: `calc(
        <div className="absolute pointer-events-none shadow-[0_0_10px_#10b981]" style={{ top: `calc(
          width: `calc(
        <div className="absolute pointer-events-none shadow-[0_0_10px_#f59e0b]" style={{ top: `calc(
          width: `calc(
        </div>

      <div className="bg-slate-950 p-4 rounded-lg border border-slate-800 shadow-[inset_0_2px_10px_rgba(0,0,0,0.5)]" style={{
        <label className="flex items-center justify-between" style={{
          <span className="text-slate-400" style={{
            <div className="flex gap-2" style={{
              <input type="number" min={0} max={100} value={r1} style={{
                14 bg-slate-900 border border-slate-700 py-2 px-4
              <input type="number" min={0} max={100} value={c1} style={{
                14 bg-slate-900 border border-slate-700 py-2 px-4
            </div>
          </label>
        <label className="flex items-center justify-between" style={{
          <span className="text-slate-400" style={{

```

```

        <span className="text-indigo-400">int<
        &nbsp;&nbsp;&nbsp;<span className="text-slate-400">
В запроса)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-rose-900">
r1</span>][<span className="text-white bg-rose-900">

        &nbsp;&nbsp;&nbsp;<span className="text-slate-400">
во, чтобы правый край уперся в c2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-blue-500">
r1</span>][<span className="text-white bg-blue-500">

        &nbsp;&nbsp;&nbsp;<span className="text-slate-400">
рх, чтобы нижний край уперся в r2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-emerald-500">
nded">r2 - Lx + 1</span>][<span className="text-emerald-500">

        &nbsp;&nbsp;&nbsp;<span className="text-slate-400">
и влево от c2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-amber-500">
">r2 - Lx + 1</span>][<span className="text-amber-500">
        {'}}''));
    </div>

    <div className="w-full flex justify-center items-center">
      <div className="flex flex-col items-center">
        <h4 className="text-slate-400 font-bold">
          Откуда берутся эти 4 числа: матрица ST
border-slate-700 shadow-sm">ST[][][{{kx}}][{{ky}}]
        </h4>
        <div className="grid gap-[2px] p-2.5">
Columns: `repeat(${matCols}, 1fr)`}}>
          {Array.from({length: matRows * matCols}, (v, idx) => {
            const r = Math.floor(idx / matCols);
            const c = idx % matCols;
            let isTL = r === r1 && c === c1;
            let isTR = r === r1 && c === c2;
            let isBL = r === r2 - Lx + 1 && c === c1;
            let isBR = r === r2 - Lx + 1 && c === c2;

```

```
                &nbsp;<span className="text-slate-400">
                  <span className="ml-3 text-emerald-500">
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
                </div>
              </div>
            </div>
          )
        }
      }
```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

ФАЙЛ 85/111: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

```
                </div>
            </main>
        </div>

        <script>
            function setMode(mode) {
                document.getElementById('btn-mode-normal').cla
                    ? 'px-3 py-1 rounded text-sm font-bold l
                    : 'px-3 py-1 rounded text-sm font-bold l

                document.getElementById('btn-mode-wtf').cla
                    ? 'px-3 py-1 rounded text-sm font-bold l
                    : 'px-3 py-1 rounded text-sm font-bold l

                const aside = document.querySelector('aside')
                const headerMain = document.querySelector('h1')
```

```

        t-white transition-colors' : 'px-3 py-1 rounded-
n-colors';
document.getElementById('btn-theme-light').
ext-white transition-colors' : 'px-3 py-1 rounded-
ion-colors';
document.getElementById('btn-theme-notebook
-600 text-white transition-colors' : 'px-3 py-1 rounded-
ransition-colors';
    }

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
    const mobileMenuBtn = document.getElementById('mobile-menu');
    const mobileDrawer = document.getElementById('mobile-drawer');
    const closeDrawerBtn = document.getElementById('close-drawer');
    const drawerOverlay = document.getElementById('drawer-overlay');

    function toggleDrawer() {
        const isClosed = mobileDrawer.classList.contains('closed');
        if (isClosed) {
            mobileDrawer.classList.remove('closed');
            if (drawerOverlay) {
                drawerOverlay.classList.remove('hidden');
                // Timeout allows display:block to take effect
                setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
            }
            document.body.style.overflow = "hidden";
        } else {
            mobileDrawer.classList.add('closed');
            if (drawerOverlay) {
                drawerOverlay.classList.add('hidden');
                setTimeout(() => drawerOverlay.classList.add('hidden'), 10);
            }
            document.body.style.overflow = "";
        }
    }
});

```

```
        });
    }
    });
}, observerOptions);

document.querySelectorAll('section[id^="que
    observer.observe(section);
});
});
</script>
</body>
</html>
```

ФАЙЛ 86/111: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРOK: 1169 | РАЗМЕР

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  <title>Пособие по алгебре: От пиццы к предикатам</t
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://polyfill.io/v3/polyfill.min.js
  <script id="MathJax-script" async src="https://cdn.
  <script>
    window.MathJax = {
      tex: {
        inlineMath: [['$', '$'], ['\\(', '\\)']]
        displayMath: [['$$', '$$'], ['\\[', '\\
      }
    };
  </script>
  <style>
    @import url('https://fonts.googleapis.com/css2?
```



```

        scrollbar-width: none;
        -ms-overflow-style: none;
        -webkit-overflow-scrolling: touch;
    }
    mjx-container[display="true"]::-webkit-scrollbar
        display: none;
    }

    @media (max-width: 640px) {
        html { font-size: 13px; }

        .uno-card { width: 40px; height: 60px; font
        .uno-card::before, .uno-card::after { font-
        .uno-card::before { top: 1px; left: 2px; }
        .uno-card::after { bottom: 1px; right: 2px;

        .uno-card.mini { width: 28px !important; he
        us: 4px; }
        .uno-card.mini::before, .uno-card.mini::aft

        .uno-equation .text-2xl { font-size: 1.2rem
        .uno-equation.gap-4 { gap: 0.5rem !important

        /* Allow horizontal scroll for equations in
        .formula-scroll { flex-wrap: nowrap !importan
        px; }
    }
    .uno-inner {
        position: absolute; width: 110%; height: 75%
        border-radius: 50%; transform: rotate(-30deg
        box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
    }
    .uno-val { position: relative; z-index: 2; }
    .card-red { background-color: #e52521; } .card-
    .card-blue { background-color: #004dcf; } .card
    .card-green { background-color: #339e35; } .car
    .card-yellow { background-color: #ffc400; } .ca
    .card-black { background-color: #222; } .card-b

```

```
        border-color: #cbd5e1 !important;
        box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
    }
    body.theme-notebook .text-slate-200, body.theme-
        ant; }
    body.theme-notebook .text-slate-400 { color: #5
    body.theme-notebook .border-slate-600, body.the
        : #94a3b8 !important; }
    body.theme-notebook .font-mono { font-family: "
    body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
    0%, 95%, 98%, 100% { transform: scaleY(1);
    96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 5.2s infinite;
    animation-delay: 1.5s;
}
.creepy-emoji {
    display: inline-block;
    transform-origin: center;
    animation: creepy-blink 3.8s infinite;
}
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead
    <div class="sticky top-0 z-50 w-full bg-slate-950/90
```

```

<marker id="arrow-indigo" markerWidth="6" m
    <path d="M0,0 L6,3 L0,6 Z" fill="#818cf
</marker>
<marker id="arrow-lime" markerWidth="6" mar
    <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63
</marker>
<marker id="arrow-green" markerWidth="6" ma
    <path d="M0,0 L6,3 L0,6 Z" fill="#4ade8
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
    <rect x="0" y="0" width="40" height="20"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-red">
    <rect x="0" y="0" width="40" height="20"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-rem">
    <rect x="0" y="0" width="40" height="10"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<!-- МЯСОРУБКА -->
<g id="meat-grinder">
    <path d="M 10 40 L 15 20 Q 25 15 35 20
    <rect x="5" y="40" width="40" height="20"
    <!-- Ручка -->
    <path d="M 45 50 Q 60 50 60 30 L 75 30"
    <circle cx="75" cy="30" r="4" fill="#333"
    <!-- Выход -->
    <path d="M 5 45 L -5 45 L -5 55 L 5 55

```

Содержание

```
</h3>
<button id="close-drawer-btn" class="
  <svg xmlns="http://www.w3.org/20
    <path stroke-linecap="round
  </svg>
</button>
</div>
<nav class="space-y-3 text-sm font-medium"
  <details class="group" open>
    <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">
      1. Алгебраические структуры
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-sm">
      <a href="#section-1-1" class="block">
      <a href="#section-1-2" class="block">
      <a href="#section-1-3" class="block">
      <a href="#section-1-4" class="block">
    </div>
  </details>

  <details class="group">
    <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">
      2. Комплексные числа
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-sm">
      <a href="#section-2-1" class="block">
      <a href="#section-2-2" class="block">
      <a href="#section-2-3" class="block">
    </div>
  </details>
```

```
<div class="pl-6 space-y-2 text
  <a href="#q5-def" class="block
  <a href="#q5-group" class="bloc
  <a href="#q5-prim" class="block
  <a href="#q5-crit" class="block
  <a href="#q5-cyclic" class="blo
</div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-6" class=
er-lime-500 pl-3 font-bold text-lime-300">
      6. Делимость, НОД и Евклид
    </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#q6-1" class="block ho
    <a href="#q6-2" class="block ho
    <a href="#q6-3" class="block ho
    <a href="#q6-4" class="block ho
    <a href="#q6-5" class="block ho
    <a href="#q6-6" class="block ho
  </div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-7" class=
er-pink-500 pl-3 font-bold text-pink-300">
      7. Взаимно простые числ
    </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#q7-1" class="bloc
    <a href="#q7-2" class="bloc
    <a href="#q7-3" class="bloc
```



```

        <a href="#q14-4" class="block
ition-colors border border-rose-500/30"> К
    </div>
</details>

<details class="group">
    <summary class="list-none cursor
        <a href="#question-15" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
            15. Производная. Критери
        </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2">
        <a href="#q15-1" class="blo
        <a href="#q15-2" class="blo
        <a href="#q15-3" class="blo
        <a href="#q15-4" class="blo
    </div>
</details>

<details class="group">
    <summary class="list-none cursor
        <a href="#question-16" clas
der-red-500 pl-3 font-bold text-red-400">
            16. Неприводимые многоч
        </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2">
        <a href="#q16-1" class="blo
        <a href="#q16-2" class="blo
        <a href="#q16-3" class="blo
    </div>
</details>

<div class="mt-4 mb-2 text-xs font-l
    <a href="#question-17" class="block
-500 pl-3 text-slate-400">
        <span class="font-bold text-tea
```

```

        });
    });

    // Scroll spy logic
    document.addEventListener('DOMContentLoaded', () => {
        const sections = document.querySelectorAll(
            'h2, h3, h4, h5, h6'
        );
        const q5 = document.querySelector('#q5');
        const q6 = document.querySelector('#q6');
        const q7 = document.querySelector('#q7');
        const q13 = document.querySelector('#q13');
        const q14 = document.querySelector('#q14');
        const q15 = document.querySelector('#q15');

        let isScrolling = false;

        document.querySelectorAll('.group-link').forEach(link => {
            link.addEventListener('click', () => {
                isScrolling = true;
                setTimeout(() => isScrolling = false, 200);
            });
        });

        const observer = new IntersectionObserver((entries) => {
            if (isScrolling) return;

            let visibleId = null;
            entries.forEach(entry => {
                if (entry.isIntersecting) {
                    visibleId = entry.target.id;
                }
            });

            // Highlight active link
            document.querySelectorAll('a').forEach(a => {
                a.style.fontWeight = 'normal';
                if (a.href.endsWith(visibleId)) {
                    a.style.fontWeight = 'bold';
                }
            });

            // Open the parent details
            const correspondingLink = document.querySelector(
                `a[href="#${visibleId}"]`
            );
            if (correspondingLink) {
                const details = correspondingLink.closest('details');
                if (details && details.open) {
                    details.open = false;
                }
            }
        });
    });

```



```
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('h2');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('h3');
subHeader.innerText = 'Из-за особенностей веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#1a1a1a';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
```

```

// Smart markdown parsing based

// Smart markdown parsing based
clone.querySelectorAll('h2').fo
clone.querySelectorAll('h3').fo
clone.querySelectorAll('.analog
clone.querySelectorAll('li').fo
clone.querySelectorAll('svg').fo
    let caption = '';
    if (el.nextElementSibling &
        caption = el.nextElement
    }
    el.textContent = '\n\n[ Илл
});
clone.querySelectorAll('.img-cap

let text = clone.innerText;
text = text.replace(/\n{3,}/g,

if (window !== window.top) {
    fallbackModal(text, 'Markdon
    return;
}
const blob = new Blob([text], {
const url = URL.createObjectURL
const link = document.createEle
link.href = url;
link.download = 'vyisshaya_alge
document.body.appendChild(link)
link.click();
document.body.removeChild(link)
}

function setTheme(theme) {

    const body = document.body;
    body.classList.remove('bg-slate
);

```

```

body.bg-white .analogy-
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuc
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

`
;
} else if (theme === 'terminal'
body.classList.add('bg-blac
document.getElementById('th
style.innerHTML = `

```

```
                <span> </span> Markdown
            </button>
            <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center
                <span> </span> HTML
            </button>
        </div>

        <!-- Смена темы -->
        <div>
            <div class="text-xs text-slate-500">
            <div class="flex items-center gap-2">
                <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
                <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white
">* </button>
                <button id="theme-terminal"
-justify-center justify-center text-green-400 hover:text-green-500
терминала"> </button>
            </div>
        </div>
    </div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
-base min-w-0">
    <header class="text-center mb-12 border-b border-slate-700">
        <h1 class="text-4xl md:text-5xl font-bl
4">
            ВЫСШАЯ АЛГЕБРА
        </h1>
        <p class="text-slate-400 italic text-lg">
```

```

оля и кольца, целые числа</div>
    <div class="bg-orange-900/40 bo
ольцо полиномов</div>
    <div class="bg-cyan-900/40 bord
сные числа</div>

    <!-- Row 1: База -->
    <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
    <div class="text-xs text-yellow
    <div class="font-bold text-slate
    </a>
    <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
    <div class="text-xs text-orange
    <div class="font-bold text-slate
    </a>
    <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
    <div class="text-xs text-cyan-5
    <div class="font-bold text-slate
    </a>

    <!-- Row 2: Базовые операции / Евклид
    <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
    <div class="text-xs text-yellow
    <div class="font-bold text-slate
    </a>
    <a href="#question-10" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange
    <div class="font-bold text-slate
    </a>
    <a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
    <div class="text-xs text-cyan-5
    <div class="font-bold text-slate

```

```

        <div class="font-bold text-slate-500">
    </a>
    <a href="#question-4" onclick="setMainQuestion(4)"
-4 border-cyan-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-cyan-500">
        <div class="font-bold text-slate-500">
    </a>

    <!-- Row 6: Кратность -->
    <div class="hidden md:block"></div>
    <a href="#question-14" onclick="setMainQuestion(14)"
-l-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
        <div class="font-bold text-slate-500">
    </a>
    <div class="hidden md:block"></div>

    <!-- Row 7: Производная -->
    <div class="hidden md:block"></div>
    <a href="#question-15" onclick="setMainQuestion(15)"
-l-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
        <div class="font-bold text-slate-500">
    </a>
    <div class="hidden md:block"></div>

    <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
    <div class="col-span-1 md:col-span-3"
b-2 border-b border-emerald-500/30 pb-2">Кл

        <a href="#question-17" onclick="setMainQuestion(17)"
-l-4 border-teal-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-teal-500">
        <div class="font-bold text-slate-500">
    </a>
    <a href="#question-18" onclick="setMainQuestion(18)"
-l-4 border-blue-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-blue-500">

```

```

    <div class="flex items-center mb-6">
      <div class="bg-red-500/20 p-3 rounded-3xl">
        <svg class="w-8 h-8 text-red-500"
rg/2000/svg">
          <path stroke-linecap="round"
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
          </svg>
        </div>
        <h2 class="text-3xl pr-4 border-r border-gray-200">
</div>

```

```
<div class="prose prose-invert max-w-4xl">
  <p class="text-lg">
    Хватит тупить. Доказательства
    есть для того, чтобы жать на кнопки. Вот жес
  </p>
```

Меметика

```
<!-- Прямое -->
<div class="bg-slate-800/80 p-6">
  <h3 class="text-xl text-blue-400">Прямое
  <p class="text-sm text-slate-400">Прямое
  <ul class="list-disc pl-5">
    <li><span class="text-white">
      k$).</li>
    <li><span class="text-white">
    <li><span class="text-white">
      obo.</li>
  </ul>
</div>
```

```
<!-- От противного -->
<div class="bg-slate-800/80 p-6"
  <h3 class="text-xl text-fuc
  <p class="text-sm text-slat
  <ul class="list-disc pl-5 s
```

```
<h3 class="text-xl text-orange-400">
  <p class="text-sm text-slate-500">
    <ul class="list-disc pl-5 space-y-2">
      <li><span class="text-w"
вверх по уравнениям).</li>
      <li><span class="text-w"
рху вниз, показывая, что  $\Delta \vdots r_i$ 
      <li><span class="text-w"
х. Твой  $d$  больше.</li>
    </ul>
  </div>
```

```
<h2 class="text-2xl font-bold border-bottom border-slate-200 p-2">
  алгоритмы экзекуции)</h2>
```

```
<div class="grid grid-cols-1 md:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-4">
    <h4 class="text-red-400">
      <p class="text-sm text-slate-500">
        <p class="text-xs font-sans">
со следующим. Если в конце 0 – пациент мертв
    </div>
```

```
<div class="bg-slate-800 p-4">
  <h4 class="text-red-400">
    <p class="text-sm text-slate-500">
      <p class="text-xs font-sans">
 $c$ . Пока получается 0 – корень жив. Как то
корня минус один.</p>
  </div>
```

```
<div class="bg-slate-800 p-4">
  <h4 class="text-red-400">
    <p class="text-sm text-slate-500">
      <p class="text-xs font-sans">
ициентов (они делятся на  $p$ ), кроме старше
зложить.</p>
```



```
        <span class="text-2xl">
        <div>
            <h4 class="text-cyan">
            <p class="text-sm">
                есть сложение — есть ноль. Если есть умножение, то
                <code class="text-x">
            </div>
        </li>

        <li class="bg-slate-800/50">
            <span class="text-2xl">
            <div>
                <h4 class="text-cyan">
                <p class="text-sm">
                    авь их драться. Умножение всегда врывается и
                    <code class="text-x">
                </div>
            </li>
        </ul>
    </div>
</section>
</div>

<div id="questions-container">
<!-- ===== ВОПРОС 1 =====>
```

РАЗДЕЛ: УТИЛИТЫ

ФАЙЛ 87/111: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

```
import { clsx, type ClassValue } from "clsx";
```

```

    try {
      const res = await fetch(href);
      if (res.ok) parts.push(await res.text());
    } catch {
      /* чужой домен – пропускаем */
    }
  }
}
return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
  border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid #
body { font-size: 15px; line-height: 1.6; }
.export-shell p, .export-shell li { line-height: 1.65;
.export-shell pre { font-size: 12px; line-height: 1.5;
.export-figs { margin: 0 0 1.6rem; }
.export-figs-title { color: #e2e8f0; font-size: .95rem;
.export-fig { margin: 0 0 .9rem; }
.export-fig img { width: 100%; border: 1px solid #1e293b
.export-fig figcaption { color: #94a3b8; font-size: .75
@media print {

```

```

        a.removeAttribute("data-goto");
    });
} catch {
    /* ссылки – улучшение, а не обязательная часть файла */
}
}

const ids: string[] = [];
host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
    const id = el.dataset.termId;
    if (!id) return;
    if (!ids.includes(id)) ids.push(id);
    const link = document.createElement("a");
    link.className = "term-hint";
    link.href = `#term-${id}`;
    link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
    link.textContent = el.textContent ?? "";
    el.replaceWith(link);
});

// <details> в файле лучше раскрыть: иначе код и разбросан
host.querySelectorAll("details").forEach((d) => d.setOpen());

return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[], book = false) {
    if (!termIds.length) return "";
    const items = termIds
        .map((id) => {
            const e = GLOSSARY_BY_ID.get(id);
            if (!e) return "";
            return `<div class="export-term" id="term-${escapeHtml(id)}">
                <h3>${escapeHtml(e.title)}</h3>
                <p>${escapeHtml(e.short)}</p>
                ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
                ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
            </div>`;
        })
        .join("");
}

```

```
    </article>

    ${opts.gloss}

    <footer class="export-foot">Универсальное пособие по
  </div>
</body>
</html>`;
}

/**
 * Картинки встраиваются в файл data-URL: автономная ко
 * иллюстрации офлайн (раньше src вел на ассет приложен
 * картинки исчезали – оставались только самые первые,
 */
async function inlineImages(html: string): Promise<string> {
  const doc = new DOMParser().parseFromString(`<body>${html}</body>`, 'text/html');
  const imgs = Array.from(doc.querySelectorAll("img[src]"));
  await Promise.all(
    imgs.map(async (img) => {
      const src = img.getAttribute("src") ?? "";
      if (!src || src.startsWith("data:")) return;
      try {
        const res = await fetch(src);
        if (!res.ok) return;
        const blob = await res.blob();
        if (blob.size > 4_000_000) return;
        const data = await new Promise<string>((resolve, reject) => {
          const fr = new FileReader();
          fr.onload = () => resolve(String(fr.result));
          fr.onerror = () => reject(fr.error);
          fr.readAsDataURL(blob);
        });
        img.setAttribute("src", data);
      } catch {
        /* картинка недоступна – глава останется со ссы
      }
    })
  )
}
```

```

    } catch {
      return "";
    }
  }

function triggerDownload(html: string, filename: string) {
  const blob = new Blob([html], { type: "text/html;charset=utf-8" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  a.remove();
  setTimeout(() => URL.revokeObjectURL(url), 2000);
}

/** Имя файла: «04-splay-derevo.html» – без кириллицы и пробелов */
function safeName(title: string, id: string): string {
  const num = title.match(/^s*(\d+)/)?[1];
  const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
  return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;
}

/** Собрать автономный HTML одной главы. */
export async function buildChapterHtml(chapter: Chapter) {
  const css = await collectCss();
  const { body: rawBody, termIds } = prepareBody(chapter);
  const body = (await vizFigures(chapter.id)) + (await vizGlossary(termIds));
  return wrap({
    title: chapter.title,
    subtitle: chapter.category || "Универсальное пособие",
    css,
    body,
    gloss: glossarySection(termIds),
    origin: window.location.origin,
  });
}

```

```
    css,
    body,
    gloss: glossarySection(allTerms, true),
    origin: window.location.origin,
  });
  return html;
}

/** Выгрузить всё пособие одним файлом. */
export async function downloadBookHtml(chapters: ChapterList) {
  triggerDownload(await buildBookHtml(chapters), "posobie.html");
}
```

ФАЙЛ 89/111: src/utils/ticketLinks.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 165 | РАЗМЕР: 7.4 KB

```
import { useEffect } from "react";
import { chapterTopics } from "../data/content";

/**
 * Перекрёстные ссылки между билетами.
 *
 * В тексте страниц десятки упоминаний вида «...берите Формулу
 * или «сердце Краскала (билет 18)». Раньше это был просто номер,
 * но попасть на него не мог — ссылка была «битой» и вела в никуда.
 * Здесь такие упоминания превращаются в настоящие ссылки.
 *
 * • в приложении — ``, который
 *   `ChapterView` и переключает страницу без перезагрузки;
 * • в текстовой версии — ``,
 \*   работающий URL, который понимают парсеры и ридеры;
 \* • в PDF — остаётся текст \(ссылки там всё равно не работают\).
 \*
 \* Номер билета → страница берётся из `chapterTopics`, а номер
 \* «билет 19» корректно ведёт на общую страницу остовных билетов.
```

```

-----
  return lite ? `?lite=1&topic=${encodeURIComponent(id)}
}

/**
 * Строковая версия – для текстовой (`?lite=1`) версии,
 * через `dangerouslySetInnerHTML`, а эффекты разметки
 * HTML режется на «тег» и «текст», меняются только тек
 * вне `pre/code/a`.
 */
export function linkifyTicketsHtml(html: string, opts:
  let inCode = 0;
  let inLink = 0;
  return html
    .split(/(<[>]*>)/g)
    .map((part) => {
      if (!part) return part;
      if (part.startsWith("<")) {
        const closing = part.startsWith("</");
        const tag = part.slice(closing ? 2 : 1).split(/
        if (tag === "pre" || tag === "code" || tag ===
          inCode = Math.max(0, inCode + (closing ? -1 :
        } else if (tag === "a") {
          inLink = Math.max(0, inLink + (closing ? -1 :
        }
        return part;
      }
      if (inCode > 0 || inLink > 0) return part;
      return part.replace(TICKET_RE, (whole, a: string,
        const from = Number(a);
        const to = b ? Number(b) : from;
        const id = chapterForRange(from, to);
        if (!id || id === opts.selfId) return whole;
        return `<a class="ticket-link" data-goto="${id}
      });
    });
  .join("");
}

```

```

        const id = chapterForRange(from, to);
        if (!id || id === opts.selfId) continue;
        if (m.index > last) frag.appendChild(document.createElement("a"));
        const a = document.createElement("a");
        a.className = "ticket-link";
        a.dataset.goto = id;
        a.setAttribute("href", hrefFor(id, opts.lite ?? false));
        a.textContent = m[0];
        frag.appendChild(a);
        last = m.index + m[0].length;
        created += 1;
    }
    if (last === 0) continue;
    if (last < text.length) frag.appendChild(document.createTextNode(textNode.parentNode?.replaceChild(frag, textNode)));
}
return created;
}

/** Размечает ссылки при смене главы и подстраховывается
export function useTicketLinks(
  ref: React.RefObject<HTMLElement | null>,
  deps: unknown[],
  opts: { selfId?: string; lite?: boolean } = {}
) {
  useEffect(() => {
    const el = ref.current;
    if (el) annotateTicketLinks(el, opts);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, deps);
}

```



```

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
    analogies,
    hasAnalogy: analogies > 0,
    inlineSvg: /<svg[\\s>]/i.test(html),
    image: /<img[\\s>]/i.test(html),
    interactive: vizIds.has(c.id),
    chars: html.length,
  };
});

```

```

const has2D = (r) => r.interactive || r.inlineSvg || r.image;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters[id]);

```

```

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));

```

```

const flag = (b) => (b ? " " : "-");

```

```

if (process.argv.includes("--md")) {
  console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
  console.log("| --- | --- | --- | --- | --- |");
  for (const r of rows) {
    console.log(
      `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy ? "да" : "нет"} |`
    );
  }
} else {
  console.log(`Глав в пособии: ${rows.length}\\n`);
}

```

```

-----
const esc = (s) => s.replace(/[\.\*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
  `(?<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
    "giu"
  );
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
  const text = c.content
    .replace(/<(script|style|code|pre)[\s\S]*?<\/\w>/gi
    .replace(/<[^>]+>/g, " ");

  const perTerm = new Map();
  const perSurface = new Map();
  let highlights = 0;
  const ids = new Set();

  for (const m of text.matchAll(re)) {
    const surface = m[1].toLowerCase();
    const id = map.get(surface);
    if (!id) continue;
    const ut = perTerm.get(id) ?? 0;
    const us = perSurface.get(surface) ?? 0;
    if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) con
    perTerm.set(id, ut + 1);
    perSurface.set(surface, us + 1);
    highlights += 1;
    ids.add(id);
    used.add(id);
  }
}

```

```

-----
import { htmlToBlocks } from "../html-to-blocks.mjs";

const GUIDE_PDF_PATH = path.join(OUT_DIR, "guide_lite.pdf");
const SHOTS_DIR = path.join(ROOT, "tmp", "viz-shots");
const ENTRY = path.join(ROOT, "tmp", "guide-entry.ts");

/** Векторные схемы графов и таблицы состояний для PDF
const VECTOR_VISUALIZERS = {
  dijkstra: {
    title: "Векторный граф релаксации (Дейкстра)",
    nodes: [
      { id: "A", label: "A", x: 40, y: 60, dist: "0", s: "s" },
      { id: "B", label: "B", x: 130, y: 30, dist: "3", s: "s" },
      { id: "C", label: "C", x: 130, y: 90, dist: "2", s: "s" },
      { id: "G", label: "G", x: 220, y: 90, dist: "5", s: "s" },
      { id: "D", label: "D", x: 220, y: 30, dist: "6", s: "s" },
      { id: "E", label: "E", x: 310, y: 90, dist: "7", s: "s" },
      { id: "F", label: "F", x: 400, y: 60, dist: "8", s: "s" },
    ],
    edges: [
      { u: "A", v: "B", w: 4, highlighted: true },
      { u: "A", v: "C", w: 2, highlighted: true },
      { u: "C", v: "B", w: 1, highlighted: true },
      { u: "B", v: "D", w: 5 },
      { u: "C", v: "G", w: 3, highlighted: true },
      { u: "C", v: "E", w: 8 },
      { u: "G", v: "D", w: 1, highlighted: true },
      { u: "G", v: "E", w: 2, highlighted: true },
      { u: "D", v: "F", w: 2 },
      { u: "E", v: "F", w: 1, highlighted: true },
    ],
    table: [
      ["Вершина", "Старт", "Кратчайшее dist", "Предшествующая"],
      ["Пиццерия А", "s = A", "0", "-", "зафиксирована"],
      ["Парк С", "из А", "2", "А", "зафиксирована"],
      ["Дом 1 В", "из С", "3", "С", "зафиксирована"],

```

```

floyd: {
  title: "Матрица кратчайших путей (Фloyd–Уоршелл)",
  table: [
    ["из \\ в", "S (0)", "A (1)", "B (2)", "C (3)", "D (4)"],
    ["S (0)", "0", "4", "5", "3", "6"],
    ["A (1)", "∞", "0", "∞", "1", "4"],
    ["B (2)", "∞", "∞", "0", "-2", "1"],
    ["C (3)", "∞", "∞", "∞", "0", "3"],
    ["D (4)", "∞", "∞", "∞", "∞", "0"],
  ],
},
mst: {
  title: "Минимальный остов (Краскал / Прим / Борувка)",
  nodes: [
    { id: "1", label: "1", x: 50, y: 30, status: "done" },
    { id: "2", label: "2", x: 150, y: 30, status: "done" },
    { id: "3", label: "3", x: 250, y: 30, status: "done" },
    { id: "4", label: "4", x: 50, y: 90, status: "done" },
    { id: "5", label: "5", x: 150, y: 90, status: "done" },
    { id: "6", label: "6", x: 250, y: 90, status: "done" },
    { id: "7", label: "7", x: 350, y: 60, status: "done" },
  ],
  edges: [
    { u: "1", v: "2", w: 2, highlighted: true },
    { u: "2", v: "3", w: 3, highlighted: true },
    { u: "1", v: "4", w: 1, highlighted: true },
    { u: "4", v: "5", w: 4, highlighted: true },
    { u: "2", v: "5", w: 5 },
    { u: "3", v: "6", w: 2, highlighted: true },
    { u: "5", v: "6", w: 6 },
    { u: "6", v: "7", w: 3, highlighted: true },
  ],
  table: [
    ["Ребро", "Вес w", "Краскал (DSU)", "Прим (Куча)", "Борувка"],
    ["(1, 4)", "1", "добавлено (comp 1-4)", "добавлено", "добавлено"],
    ["(1, 2)", "2", "добавлено (comp 1-2-4)", "добавлено", "добавлено"],
    ["(3, 6)", "2", "добавлено (comp 3-6)", "добавлено", "добавлено"],
    ["(2, 3)", "3", "добавлено (объединение)", "добавлено", "добавлено"],
  ],
}

```

```

const pick = (list) => list.find((f) => fs.existsSync(f))

/**
 * Убираем символы, которых нет в DeJaVu (эмодзи и пиктограммы)
 * на их месте появляется «пустой» глиф. Стрелки, математические
 * геометрические фигуры DeJaVu знает – их оставляем.
 */
const UNSUPPORTED = /[\u{1F000}-\u{1FAFF}\u{2700}-\u{2700}]/gu;

/** Русское склонение счётных существительных: 1 страница */
const plural = (n, one, few, many) => {
  const mod10 = Math.abs(n) % 10;
  const mod100 = Math.abs(n) % 100;
  if (mod10 === 1 && mod100 !== 11) return one;
  if (mod10 >= 2 && mod10 <= 4 && (mod100 < 10 || mod100 > 10)) return many;
};

const plain = (text) =>
  String(text ?? "")
    .replace(UNSUPPORTED, "")
    .replace(/\s{2,}/g, " ")
    .replace(/\u0000/g, "")
    .trim();

/* ----- данные приложения (через esbuild) ----- */

async function loadAppData() {
  ensureDir(path.dirname(ENTRY));
  fs.writeFileSync(
    ENTRY,
    [
      'export { chapters, chapterTopics } from "../src/data/chapters";',
      'export { PAGE_SYNC } from "../src/data/vizSync";',
      'export { VIZ_REGISTRY } from "../src/components/vizRegistry";',
      'export { quizzes } from "../src/data/quizzes";',
    ].join("\n")
  );
}

```

```

class Renderer {
  constructor(doc, fonts) {
    this.doc = doc;
    this.fonts = fonts;
    this.footerTitle = "";
    this.pages = 0;
    this.cy = TOP;
  }

  /**
   * Колонтитул текущей страницы (печатаем до перехода к следующей)
   *
   * Важно: на время печати нижнее поле страницы уменьшаем,
   * решает, что текст не влез, и молча добавляет пустые строки
   */
  stampFooter() {
    if (this.pages === 0 || !this.doc.page) return;
    const prevBottom = this.doc.page.margins.bottom;
    this.doc.page.margins.bottom = 16;
    const y = PAGE.height - PAGE.margin + 4;
    this.doc.font(this.fonts.regular).fontSize(7.6).fill('black');
    this.doc.text("Универсальное пособие • алгоритмы и структуры данных",
      width: CONTENT_W * 0.62,
      align: "left",
      lineBreak: false,
      ellipsis: true,
    ));
    this.doc.text(`${this.footerTitle} • ${this.pages}`,
      width: CONTENT_W * 0.38,
      align: "right",
      lineBreak: false,
      ellipsis: true,
    ));
    this.doc.page.margins.bottom = prevBottom;
    this.doc.y = this.cy;
  }
}

```

```

    const flush = () => {
      if (current.trim()) chunks.push(current.trim());
      current = "";
    };
    for (const sentence of source.split(/(?<=[.!?...;:])\s/)) {
      const withSentence = current ? `${current} ${sentence}` : sentence;
      if (measure(withSentence) <= max) {
        current = withSentence;
        continue;
      }
      flush();
      if (measure(sentence) <= max) {
        current = sentence;
        continue;
      }
      for (const word of sentence.split(/\s+/)) {
        const withWord = current ? `${current} ${word}` : word;
        if (measure(withWord) <= max) current = withWord;
        else {
          flush();
          current = word;
        }
      }
    }
    flush();
    return chunks.length ? chunks : [source];
  }

  rule(color = "#c7d2fe", width = 1) {
    const y = this.cy;
    this.doc.moveTo(PAGE.margin, y).lineTo(PAGE.margin, y);
    this.cy = y + 8;
  }

  hl(rawText, { size = 20, color = "#111827", rule = true }) {
    const text = plain(rawText);
    if (!text) return;
    const h = this.height(text, { font: this.fonts.bold });
  }

```

```

        this.ensure(h + (last ? gapAfter : 0), 6);
        const y0 = this.cy;
        this.doc.font(font).fontSize(size).fillColor(color);
        this.cy = Math.max(y0 + h, this.doc.y) + (last ? 0 : gapBefore);
        if (this.cy > BOTTOM) this.newPage();
    });
}

```

```

bullet(rawText, { ordinal = null, depth = 0, size = 9 }) {
    const text = plain(rawText);
    if (!text) return;
    const indent = 16 + depth * 14;
    const marker = ordinal ? `${ordinal}.` : "*";
    const width = CONTENT_W - indent - 4;
    const opts = { font: this.fonts.regular, size, width };
    const chunks = this.splitToFit(text, opts, BOTTOM);
    chunks.forEach((chunk, index) => {
        const h = this.height(chunk, opts);
        this.ensure(h + 2.5, 6);
        const y0 = this.cy;
        if (index === 0) {
            this.doc.font(this.fonts.regular).fontSize(size)
                .text(marker, { align: "right", lineGap: 1 });
        }
        this.doc.font(this.fonts.regular).fontSize(size)
            .text(chunk, { align: "left", lineGap: 1 });
        this.cy = Math.max(y0 + h, this.doc.y) + 2.5;
        if (this.cy > BOTTOM) this.newPage();
    });
}

```

```

/** Код: режим на фрагменты по высоте страницы, каждый фрагмент
code(rawText, { size = 8.0 } = {}) {
    const font = this.fonts.mono;
    const inner = CONTENT_W - 22;
    const lines = String(rawText ?? "").replace(/[\n\r]/g, " ");
    // собираем фрагменты, которые влезают в одну рамку
    let i = 0;
    while (i < lines.length) {

```



```

        .font(this.fonts.bold)
        .fontSize(9.5)
        .fillColor("#1e293b")
        .text(title, PAGE.margin + 12, startY + 8);
    }

    // Рёбра графа
    for (const e of edges) {
        const fromNode = nodes.find((n) => n.id === e.u);
        const toNode = nodes.find((n) => n.id === e.v);
        if (!fromNode || !toNode) continue;

        const x1 = PAGE.margin + fromNode.x;
        const y1 = startY + fromNode.y;
        const x2 = PAGE.margin + toNode.x;
        const y2 = startY + toNode.y;

        const strokeColor = e.highlighted ? "#059669" : "#000000";
        const strokeWidth = e.highlighted ? 2 : 1;

        this.doc
            .moveTo(x1, y1)
            .lineTo(x2, y2)
            .lineWidth(strokeWidth)
            .strokeColor(strokeColor)
            .stroke();

        if (e.w !== undefined) {
            const mx = (x1 + x2) / 2;
            const my = (y1 + y2) / 2;
            this.doc
                .circle(mx, my, 7)
                .fillAndStroke("#ffffff", strokeColor);
            this.doc
                .font(this.fonts.bold)
                .fontSize(7)
                .fillColor("#0f172a")
                .text(String(e.w), mx - 7, my - 3.5, { width: 100 });
        }
    }
}

```

```

    if (!rows.length) return;
    const cols = Math.max(...rows.map((r) => r.length));
    const widths = new Array(cols).fill(CONTENT_W / cols);
    const pad = 5;
    const header = rows[0];

    const rowHeight = (cells) => {
      let max = 0;
      for (let i = 0; i < cols; i++) {
        const h = this.height(cells[i] ?? "", { font: this.font, size: this.fontSize });
        max = Math.max(max, h);
      }
      return max + pad * 2;
    };

    const drawRow = (cells, y, isHeader, zebra) => {
      const h = rowHeight(cells);
      this.doc.rect(PAGE.margin, y, CONTENT_W, h).fill({ color: zebra ? "#1f2937" : "white" });
      for (let i = 0; i < cols; i++) {
        const x = PAGE.margin + widths.slice(0, i).reduce((a, b) => a + b, 0);
        this.doc
          .font(isHeader ? this.fonts.bold : this.fonts.regular)
          .fontSize(size)
          .fillColor("#1f2937")
          .text(String(cells[i] ?? ""), x, y + pad, { width: widths[i] });
      }
      this.doc.rect(PAGE.margin, y, CONTENT_W, h).lineWidth(1);
      return h;
    };

    this.ensure(rowHeight(header) + 8, 14);
    let y = this.cy;
    rows.forEach((cells, idx) => {
      const row = [];
      for (let i = 0; i < cols; i++) row.push(cells[i]);
      const h = rowHeight(row);
      if (y + h > BOTTOM) {
        this.cy = y;
      }
    });
  }
}

```

```

const scale = Math.min(maxWidth / img.width, (maxHe
const w = img.width * scale;
const h = img.height * scale;
this.ensure(h + capH + 10, 20);
const y0 = this.cy;
const x0 = PAGE.margin + (CONTENT_W - w) / 2;
this.doc.image(file, x0, y0, { width: w, height: h
this.doc.rect(x0, y0, w, h).lineWidth(0.6).strokeCo
this.cy = y0 + h + 4;
if (caption) {
  this.doc.font(this.fonts.regular).fontSize(8).fil
    enter" });
  this.cy = Math.max(this.cy, this.doc.y) + 6;
}
return true;
}

```

```

/** Бокс с заголовком и строками; длинные режутся на
box(rawTitle, rawLines, { color = "#eef2ff", border =
  const title = plain(rawTitle);
  const lines = rawLines.map((l) => plain(l)).filter(
  const inner = CONTENT_W - 20;
  let rest = lines.slice();
  let first = true;
  while (rest.length || first) {
    first = false;
    const headH = this.height(title, { font: this.fon
    this.ensure(headH + 24, 16);
    const y0 = this.cy;
    // сколько строк влезет на эту страницу
    const avail = BOTTOM - y0 - headH - 16;
    const taken = [];
    let h = 0;
    while (rest.length) {
      const ch = this.height(rest[0], { font: this.fon
      if (taken.length && h + ch > avail) break;
      taken.push(rest.shift());
      h += ch;
    }
  }

```

```
    Author: "algv0 • автоэкспорт",
    Subject: "Текст всех тем + визуализации, без кода",
    CreationDate: new Date(),
  },
});
const stream = fs.createWriteStream(GUIDE_PDF_PATH);
doc.pipe(stream);

const R = new Renderer(doc, fonts);
const today = new Date().toISOString().slice(0, 10);
const totalTopics = new Set(Object.values(chapterTopics));
const vizCount = chapters.filter((c) => VIZ_REGISTRY[c]);

/* — обложка — */
R.firstPage("Обложка");
R.cy = TOP + 90;
R.h1("Универсальное пособие", { size: 26, rule: false });
R.h1("Алгоритмы и структуры данных", { size: 17, color: "#4b5563" });
R.para(
  `Билеты 1-${totalTopics.size} • ${chapters.length} ${plusSign} ${vizCount}, "интерактивная демонстрация", "интерактивные примеры"`,
  { size: 10.5, color: "#4b5563" }
);
R.para(
  "Автоматический компактный сборник: весь текст тем + визуализации. Навигация приложения и код панели Python Pages.",
  { size: 9.6, color: "#374151" }
);
R.para(`Дата сборки: ${today}`, { size: 9, color: "#6b728b" });

const missingViz = chapters.filter((c) => !VIZ_REGISTRY[c]);
R.box("Как устроено пособие", [
  "• Страница = один или несколько билетов: текст, анимация, интерактивные примеры",
  "• У каждой страницы есть интерактивная демонстрация, которую можно посмотреть, кликнув на неё. Ссылки на демонстрации выкладывают её напрямую.",
  "• В этом PDF демонстрации показаны снимками (когда они не работают, то скриншоты не удаётся сделать, поэтому иногда приходится управлять вручную).",
]);
```

```

        else R.para(b.text, { size: 10.2, color: "#334155" });
    } else if (b.type === "summary") {
        R.para("► " + b.text, { size: 9.8, color: "#373737" });
    } else if (b.type === "li") {
        R.bullet(b.text, { ordinal: b.ordinal, depth: b.depth });
    } else if (b.type === "code") {
        R.code(b.text);
    } else if (b.type === "table") {
        R.table(b.rows);
    } else if (b.type === "quote") {
        R.quote(b.text);
    } else if (b.type === "caption") {
        R.para(b.text, { size: 8.3, color: "#6b7280", italic: true });
    } else {
        R.para(b.text);
    }
}

```

```

/* демонстрация: интерактивная схема (векторные графы)
const viz = VIZ_REGISTRY[chapter.id];
const vecViz = VECTOR_VISUALIZERS[chapter.id];
const syncKeys = Object.keys(PAGE_SYNC).filter((k) => k !== "viz");

```

```

if (viz || syncKeys.length || vecViz) {
    R.h2("Демонстрация и интерактивная схема");
    if (viz) {
        R.para(viz.title, { size: 10.2, color: "#1e293b" });
        if (viz.hint) R.para(viz.hint, { size: 9.1, color: "#373737" });
    }
}

```

```

if (vecViz) {
    if (vecViz.nodes && vecViz.edges) {
        R.vectorGraph(vecViz.nodes, vecViz.edges, { height: 300 });
    }
    if (vecViz.table) {
        R.table(vecViz.table);
    }
}
}

```

```
// Самопроверка: страниц в файле должно быть ровно столько, сколько
// рендерер. Расхождение означает, что pdfkit создал файл с лишними
// в файл без нижнего колонтитула и без нумерации.
const pdfLib = await import("pdf-lib");
const physical = (await pdfLib.PDFDocument.load(fs.readFileSync(
console.log(`      страниц: ${R.pages} (рендер) / ${R.pages} (физический)`)
if (R.pages > 100) console.warn(`      ВНИМАНИЕ: ${R.pages} страниц`);
if (physical !== R.pages) {
  console.error(`      ОШИБКА: pdfkit добавил ${physical - R.pages} страниц`);
  process.exitCode = 1;
}
}

main().catch((err) => {
  console.error("Ошибка сборки PDF пособия:", err.message);
  process.exit(1);
});
```

ФАЙЛ 93/111: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/ в
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,
  ensureFile } from "../utils";

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };
```

```
-----
    stream.on("error", reject);
  });

  console.log("[3/3] png -> pdf");
  console.log(`    страниц: ${pages.length}`);
  console.log(`    выход:    ${path.relative(ROOT, PDF)}`);
}

main().catch((err) => {
  console.error("Ошибка сборки PDF:", err.message);
  process.exit(1);
});
-----
```

ФАЙЛ 94/111: scripts/export/capture-viz-shots.mjs
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 122 | РАЗМ: 1.5 КБ

```
-----
/**
 * Снимки визуализаций для PDF-конспекта: tmp/viz-shots.
 *
 * Headless-браузер открывает каждую страницу с демонстрацией
 * блок #chapter-viz. У страниц с вкладками (sparse-tabs)
 * снимается каждая вкладка — файл получает имя `<chapter>-<tab>`.
 * и PDF подписывает его как «режим «вкладка»».
 *
 * npm run export:shots # сервер уже поднят
 * SHOTS_BASE_URL=http://127.0.0.1:4173 npm run export:shots
 *
 * В CI это делает rebuild-pdf.yml: сборка → vite preview →
 * npm run export:guide. Локально без браузера скрипт просто
 * честно пишет, что снимки добавляются при сборке в CI.
 */
import fs from "node:fs";
import path from "node:path";
import { build } from "esbuild";
import puppeteer from "puppeteer";
```

```

const ids = await vizIds();
console.log(`[export:shots] страниц со визуализациями:

const browser = await puppeteer.launch({
  headless: true,
  args: ["--no-sandbox", "--disable-setuid-sandbox", "-
});
const page = await browser.newPage();
await page.setViewport({ width: 1400, height: 1000, dev

let done = 0;
let failed = 0;
for (const id of ids) {
  try {
    await page.goto(`${BASE_URL}/?topic=${id}`, { waitU
    await page.waitForSelector("#chapter-viz", { timeou
    // демо-анимации: даём кадру собраться до осмысленн
    await new Promise((r) => setTimeout(r, 1600));
    const viz = await page.$("#chapter-viz");
    if (!viz) throw new Error("нет #chapter-viz");
    await viz.screenshot({ path: path.join(OUT_DIR, `${
    done += 1;

    for (const tab of TABS[id] ?? []) {
      try {
        const clicked = await page.evaluate((label) =>
          const root = document.querySelector("#chapter
          if (!root) return false;
          const btn = Array.from(root.querySelectorAll(
            (b.textContent ?? "").trim().startsWith(lab
          ));
          if (!btn) return false;
          btn.dispatchEvent(new MouseEvent("click", { b
          return true;
        }, tab);
        if (!clicked) continue;
        await new Promise((r) => setTimeout(r, 1200));

```


Универсальное пособие • полный исходный код

```
-----
*
* Список файлов берётся из git (чтобы ничего не забыть
* с фоллбэком на обход файловой системы, если git недо
*/
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
  [/^src\/data\/$/, "Контент и данные пособия"],
  [/^src\/components\/visualizers\/$/, "Визуализаторы (в
  [/^src\/components\/$/, "Интерактивные компоненты и си
  [/^src\/layout\/$/, "HTML-разметка (layout)"],
  [/^src\/utils\/$/, "Утилиты"],
  [/^src\/$/, "Ядро приложения"],
  [/^scripts\/$/, "Скрипты сборки и экспорта"],
  [/^\.github\/$/, "CI / автоматизация"],
  [/^[^\/]+$/, "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re

/** Порядок разделов в документе. */
const ORDER = [
  "Конфигурация проекта",
```

```

    .filter((rel) => CODE_EXT.has(path.extname(rel)))
    .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
    .sort((a, b) => {
      const ia = ORDER.indexOf(categoryOf(a));
      const ib = ORDER.indexOf(categoryOf(b));
      return ia !== ib ? ia - ib : a.localeCompare(b);
    });
  }
}

```

/** Достаём заголовки глав из данных пособия (без запус

```

function extractChapters(files) {
  const chapters = [];
  const seen = new Set();
  for (const rel of files.filter((f) => f.startsWith("s
    const src = fs.readFileSync(path.join(ROOT, rel), "
    const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"
    let m;
    while ((m = re.exec(src)) !== null) {
      const [, id, title] = m;
      if (seen.has(id)) continue;
      seen.add(id);
      chapters.push({ id, title, file: rel });
    }
  }
  return chapters.sort((a, b) => {
    const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
    const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
    return na - nb;
  ));
}

```

```

function main() {
  const files = listFiles();
  const chapters = extractChapters(files);
  ensureDir(OUT_DIR);

```

```

  const out = [];
  const push = (s = "") => out.push(s);

```

```

current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push(HR);
    push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
    push(HR);
    push();
  }
  const content = fs.readFileSync(path.join(ROOT, rel));
  const lines = content.split("\n");
  push(SUBHR);
  push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
  push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
  push(SUBHR);
  push();
  push(content.replace(/\n+$/, ""));
  push();
  push();
});

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сген.`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`    файлов:  ${files.length}`);
console.log(`    глав:    ${chapters.length}`);
console.log(`    строк:   ${txt.split("\n").length}`);
console.log(`    выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

```

    pointsize: 8,
    /** Максимум символов в строке (далее – жёсткий пере
    cols: 138,
    /** Строк содержимого на странице (+2 служебные: шапка
    rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
  fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
  for (const cmd of ["magick", "convert"]) {
    try {
      execFileSync(cmd, ["-version"], { stdio: "ignore" });
      return cmd;
    } catch {
      /* пробуем следующий */
    }
  }
  throw new Error(
    "ImageMagick не найден. Установите его: sudo apt-get install imagemagick
  );
}

```



```

        out += c.text;
    } else if (SKIP.has(c.tag)) {
        continue;
    } else if (BLOCK.has(c.tag)) {
        continue;
    } else if (c.tag === "br") {
        out += "\n";
    } else {
        out += inlineText(c);
    }
}
return clean(out);
}

/** Весь текст поддерева (для ячеек таблиц и подписей).
function collectText(node) {
    if (node.tag === "#text") return clean(node.text);
    if (SKIP.has(node.tag)) return "";
    if (node.tag === "table") return tableRows(node).map(
    return clean(node.children.map(collectText).join(" "))
}

/** Сырой текст внутри <pre> – переносы строк важны. */
function collectRawPre(node) {
    if (node.tag === "#text") return node.text;
    if (node.tag === "br") return "\n";
    if (SKIP.has(node.tag) && node.tag !== "img") return
    return node.children.map(collectRawPre).join("");
}

/** Таблица → массив строк из массивов ячеек. */
function tableRows(node) {
    const rows = [];
    const visit = (n) => {
        if (n.tag === "tr") {
            const cells = n.children.filter((c) => c.tag ===
            if (cells.some((c) => c.length)) rows.push(cells)
            return;

```

```

const walk = (node, listDepth = 0) => {
  if (node.tag !== "#root" && SKIP.has(node.tag)) return;

  if (node.tag === "table") {
    const rows = tableRows(node);
    if (rows.length) push({ type: "table", rows });
    return;
  }
  if (node.tag === "pre") {
    const text = decodeEntities(collectRawPre(node));
    if (text.trim()) push({ type: "code", text: text });
    return;
  }

  // есть блочные дети: идём по порядку, инлайн-пробе
  if (hasBlockChild(node)) {
    let buf = "";
    const flush = () => {
      const t = clean(buf);
      if (t) push({ type: "p", text: t });
      buf = "";
    };
    for (const c of node.children) {
      if (c.tag === "#text") {
        buf += c.text;
        continue;
      }
      if (SKIP.has(c.tag)) continue;
      if (!BLOCK.has(c.tag)) {
        buf += inlineText(c) + " ";
        continue;
      }
      if (c.tag === "ul" || c.tag === "ol") {
        flush();
        emitList(c, listDepth);
        continue;
      }
    }
  }
}

```

```
-----  
    return lines.join("\n").replace(/\n{3,}/g, "\n\n").trim()  
}
```

ФАЙЛ 98/111: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМ: 1.5 КБ

```
-----  
/**  
 * ШАГ 2: TXT -> PNG  
 *  
 * Разбивает full_code_all_pages.txt на страницы формата  
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)  
 *  
 * Результат: tmp/export-pages/page-0001.png, page-0002.png  
 */  
import fs from "node:fs";  
import path from "node:path";  
import os from "node:os";  
import { execFile } from "node:child_process";  
import { promisify } from "node:util";  
import {  
    ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize  
} from "../common.mjs";  
  
const execFileAsync = promisify(execFile);  
  
/** Раскрываем табы и жёстко переносим длинные строки,  
function wrap(line) {  
    const expanded = line.replace(/\t/g, "    ").replace(/  
    if (expanded.length <= PAGE.cols) return [expanded];  
    const parts = [];  
    const indent = "    ".repeat(Math.min((expanded.match(/  
    let rest = expanded;  
    let first = true;  
    while (rest.length > 0) {  
        const width = first ? PAGE.cols : PAGE.cols - indent
```



```

    });

    const args = (job) => [
      "-density", String(PAGE.density),
      "-page", PAGE.size,
      "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
      "-pointsize", String(PAGE.pointsize),
      `text:${job.txtFile}`,
      "-background", "white",
      "-flatten",
      "-colorspace", "Gray",
      ...(PAGE.monochrome ? ["-monochrome"] : []),
      "-strip",
      "-define", "png:compression-level=9",
      job.pngFile,
    ];

    const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
    let done = 0;
    let cursor = 0;

    async function worker() {
      while (cursor < jobs.length) {
        const job = jobs[cursor++];
        await execFileAsync(im, args(job));
        fs.rmSync(job.txtFile, { force: true });
        done += 1;
        if (done % 25 === 0 || done === total) {
          process.stdout.write(`      отрендерено ${done}\n`);
        }
      }
    }

    await Promise.all(Array.from({ length: concurrency }, worker));
    process.stdout.write("\n");

    const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
    if (missing.length) throw new Error(`Не отрендерены ${missing.length} страниц`);
  }
}

```

```

    group: string;
    name: string;
    ok: boolean;
    ms: number;
    detail: string;
    consoleErrors: string[];
}

const results: TestResult[] = [];
let consoleErrors: string[] = [];
const realConsoleError = console.error;

/* -----
/*  утилиты
/* -----

const sleep = (ms: number) => new Promise((r) => setTime

async function settle(ms = 40) {
    await act(async () => {
        await sleep(ms);
    });
}

function bodyText(): string {
    return (document.body.textContent ?? "").replace(/\s+/g, " ");
}

function allButtons(): string[] {
    return Array.from(document.querySelectorAll("button"))
        .filter((b) => (b.getAttribute("aria-label") || b.getAttribute("title")));
}

/** Кнопки без доступного имени: ни aria-label, ни title
function namelessButtons(scope: ParentNode = document): string[] {
    return Array.from(scope.querySelectorAll("button"))
        .filter((b) => {

```

```
function typeInto(ta: HTMLTextAreaElement, value: string) {
  const setter = Object.getOwnPropertyDescriptor(window, 'innerText')
  if (!setter) throw new Error("нет нативного setter'a")
  act(() => {
    setter.call(ta, value);
    ta.dispatchEvent(new window.Event("input", { bubble: true }));
  });
}
```

```
async function key(k: string, init: KeyboardEventInit = {}) {
  await act(async () => {
    document.dispatchEvent(new window.KeyboardEvent("keydown", init));
    await sleep(20);
  });
}
```

```
/** Ждём появления признака в DOM (текст/предикат), перепроверяем каждые 20мс */
async function waitFor(pred: () => boolean, timeoutMs = 1000) {
  const deadline = Date.now() + timeoutMs;
  while (Date.now() < deadline) {
    if (pred()) return true;
    await settle(step);
  }
  return pred();
}
```

```
const RUN_DONE = [
  "Трасса готова автоматически",
  "Нет шагов трассы",
  "Пустой код",
  "Код не выполнялся",
  "Выполнение остановлено после",
  "Не удалось выполнить код",
  "Ошибка:",
];
```

```
let root: Root | null = null;
```

```

const entry = VIZ_REGISTRY[vizId];
if (!entry) throw new Error(`в реестре нет визуализации`);
const Comp = entry.Component;
root = createRoot(host);
await act(async () => {
  root!.render(
    <VizChapterContext.Provider value={vizId}>
      <VizLiveStatus demoTitle={entry.title} />
      <Comp />
    </VizChapterContext.Provider>
  );
});
await settle(60);
return host;
}

```

```

const BAD_DOM = [
  { re: /\bNaN\b/, why: "NaN в разметке" },
  { re: /\bundefined\b/, why: "undefined в разметке" },
  { re: /\[object Object\]/, why: "[object Object] в разметке" },
];

```

/**

* Битые подстановки видны в КОРОТКИХ текстовых узлах (листья
 * вершины): «undefined», «NaN», «[object Object]». В длинных
 * псевдокода те же слова законны — их не трогаем.
 */

```

function domDefects(scope: HTMLElement | null): string[] {
  if (!scope) return [];
  const found: string[] = [];
  const walker = document.createTreeWalker(scope, 4 /* NodeFilter.SHOW_TEXT */);
  let node = walker.nextNode() as Text | null;
  while (node) {
    const text = (node.nodeValue ?? "").replace(/\s+/g, " ");
    if (text.length > 0 && text.length < 60) {
      for (const bad of BAD_DOM) {
        if (bad.re.test(text)) found.push(`${bad.why}: ${text}`);
      }
    }
    node = walker.nextNode();
  }
  return found;
}

```



```

    { name: "пустая матрица", vars: { n: 0, k: 0, i: 0,
    { name: "матрица под именем D и k/i/j вне диапазона
      [10, 11, 12, 0]] }, expect: "4x4" },
  ];

for (const c of floydCases) {
  await test("В. Флойд: размерности", c.name, async () => {
    await mountViz("floyd");
    await act(async () => {
      emitVizState("floyd", { line: 3, func: "<module
      await sleep(30);
    });
    const txt = (host?.textContent ?? "").replace(/\\s
    if (!/данные из компилятора/.test(txt)) throw new
    if (!txt.includes(c.expect)) throw new Error(`в р
    const defects = domDefects(host);
    if (defects.length) throw new Error(`артефакты в
    return `бейдж: ${txt.match(/данные из компилятор
  });
}

/* — 3. Префиксные суммы 2D и разреженная таблица —
const gridCases: { viz: string; chapter: string; name
  { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
    9, 10, 11, 12]], S: [[1, 3, 6, 10], [6, 14, 24, 3
    тора/ },
  { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
    pref: [[1, 3], [4, 10]], r1: 0, c1: 0, r2: 1, c2
  { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
    9]], S: [[1, 3, 6], [5, 12, 21], [12, 27, 45]],
  { viz: "sparse-table", chapter: "sparse-table", nam
    [3, 3, 1, 1, 0], [1, 1, 0, 0, 0]] }, expect: /не
  { viz: "sparse-table", chapter: "sparse-table#2d-bu
    0, i: 2, j: 3, st: Array.from({ length: 4 }, () :
    pa/ },
  { viz: "sparse-table", chapter: "sparse-table#2d-qu
    30, j2: 40, res: 2 }, expect: /не совпадает|данны
];

```

```

        throw new Error(`пустая активная клетка нарисована
    }
    if (domDefects(host).length) throw new Error(`артефакт
    return "оригинал из A с первого шага, вкладка «Шаг 1»";
  });

/* — 4. Дейкстра / Форд-Беллман: чужие имена вершин
for (const [viz, vars] of [
  ["dijkstra", { u: 0, v: 3, w: 4, dist: [0, 4, 2, 6, 4] },
  ["bellman-ford", { n: 5, m: 4, i: 1, u: 2, v: 4, w: [4, 4, 4, 4, 4] }],
] as [string, Record<string, unknown>][]) {
  await test("D. графы путей", `${viz}: вершины 0...4 в
    await mountViz(viz);
    const before = allButtons().length;
    const playEnabled = (findButton(/^Пуск|Пуск$/)) as HTMLButt
    await act(async () => {
      emitVizState(viz, { line: 5, func: "<module>",
      await sleep(30);
    });
    const txt = (host?.textContent ?? "").replace(/\\s/g, " ");
    if (!/данные из компилятора/.test(txt)) throw new Error("ошибка
    if (!/ваши расстояния|dist/i.test(txt)) throw new Error("ошибка
    const step = findButton(/^Шаг$|Шаг /) as HTMLButtonElement;
    if (step && !step.disabled) throw new Error("кнопка уже нажата
    if (domDefects(host).length) throw new Error(`артефакт
    await act(async () => {
      clearVizState(viz);
      await sleep(30);
    });
    const step2 = findButton(/^Шаг$|Шаг /) as HTMLButtonElement;
    if (step2 && step2.disabled) throw new Error("кнопка уже нажата
    return `кнопок до/после: ${before}/${allButtons().length}`;
  });
}

```

```

/* — 5. Все референсные коды: настоящий Python → нас
const syncKeys = Object.keys(PAGE_SYNC).filter((k) => k !== "init");
for (const key of syncKeys) {

```

```
}
```

```
/* — 6. Поведение редактора: пустой код, ошибки, гла
const ch = "dijkstra";
```

```
await test("F. редактор", "только комментарии: редакт
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "п
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(ta, "# просто комментариев\n# и ещё один\n"
  const done = await waitFor(() => bodyText().include
  if (!done) throw new Error(`не дождались сообщения
  if (!textarea()) throw new Error("редактор исчез по
  if (!/встроенное демо/.test(bodyText())) throw new
  return "редактор на месте, бейдж «встроенное демо»"
});
```

```
await test("F. редактор", "синтаксическая ошибка: ред
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "п
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(ta, "def broken(:\n    pass\n");
  const done = await waitFor(() => /Код не выполнен
  if (!done) throw new Error(`ошибка синтаксиса не по
  if (!textarea()) throw new Error("редактор исчез на
  if (!/invalid syntax|SyntaxError/i.test(bodyText()))
  return "ошибка показана, редактор остался";
});
```

```
await test("F. редактор", "гигантский print обрезаетс
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "п
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(ta, 'dist = {0: 0}\nprint("x" * 200000)\np
  const ready = await waitFor(() => bodyText().include
```



```

// — глаз ВКЛ: эталон лежит В РЕДАКТОРЕ, и редактор
await click(findButton("Вставить эталонный код"), "
const trace = await waitFor(() => !!findButton(/Оста
if (!trace) throw new Error(`эталон не запустился.
if (!textarea()) throw new Error("редактор исчез во
if (/Ошибка:/.test(panelText())) throw new Error(`э
const ta2 = textarea();
const firstRefLine = ref.split("\n").find((l) => l.
if (!ta2?.value.includes(firstRefLine)) {
  throw new Error(`выполняется не тот текст, что ле
    0, 100)}`);
}
const mustBe = [
  { what: "пуск/автопроход", re: /Запустить код|Про
  { what: "копирование кода", re: /Скопировать код|
  { what: "заккрыть панель", re: /Заккрыть компилятор
  { what: "шаг вперёд", re: /Шаг вперёд|вперёд/i },
  { what: "шаг назад", re: /Шаг назад|назад/i },
  { what: "свернуть листинг", re: /Свернуть листинг
];
const btns = allButtons().join(" | ");
for (const m of mustBe) {
  if (!m.re.test(btns)) throw new Error(`в панели н
}
// На первом шаге переменных ещё нет (отладчик вст
// поэтому связь с демонстрацией проверяем, дойдя д
await click(findButton("В конец трассы"), "в конец
const linked = await waitFor(() => /данные из компи
if (!linked) throw new Error(`эталон выполнен до ко

// — глаз ВЫКЛ: кода нет вовсе, ничего не выполняе
await click(findButton("Убрать эталонный код"), "гл
const cleared = await waitFor(() => bodyText().incl
if (!cleared) throw new Error(`после выключения гла
const ta3 = textarea();
if (!ta3) throw new Error("редактор пропал после вы
if (ta3.value.trim() !== "") throw new Error(`глаз
if (!/встроенное демо/.test(bodyText())) throw new

```

```

    if (!done) throw new Error(`трасса не собралась. Па
    if (!textarea()) throw new Error("во время прогона
    if (!findButton(/Остановить прогон трассы/)) throw
    // Листинг сворачивается (освобождая место редактор
    // и редактор остаются на месте.
    const codeLines = () => panel()?.querySelectorAll("
    if (codeLines() === 0) throw new Error("в раскрытой
    await click(findButton("Свернуть листинг трассы"),
    await settle(80);
    if (!textarea()) throw new Error("после сворачивани
    if (codeLines() !== 0) throw new Error(`листинг не
    if (!findButton("В конец трассы")) throw new Error(
    await click(findButton("Показать листинг трассы"),
    await settle(80);
    if (codeLines() === 0) throw new Error("листинг не
    // Esc: прогон остановлен, трасса убрана, редактор
    await key("Escape");
    await settle(150);
    if (!textarea()) throw new Error("Esc убрал редактор
    if (findButton(/Остановить прогон трассы/)) throw n
    if (!/встроенное демо/.test(bodyText())) throw new
    return "редактор всегда на месте: трасса рядом, сво
  });

```

```

await test("F. редактор", "заккрытие панели во время пр
  await mountApp("floyd"); // 869 шагов: прогон долги
  await click(findButton("Боковая панель Python"), "п
  await waitFor(() => panel() !== null, 4000);
  await click(findButton("Заккрыть компилятор"), "крес
  await settle(150);
  if (panel()) throw new Error("панель не закрылась")
  const backToDemo = await waitFor(() => /встроенное
  if (!backToDemo) throw new Error(`после закрытия па
  await settle(3000); // ждём, пока догоняющий прогон
  if (/данные из компилятора/.test(bodyText())) {
    throw new Error("закрытая панель всё-таки опублик
  }
  if (!/встроенное демо/.test(bodyText())) throw new

```

```

const linked = await waitFor(() => /данные из компи.
if (!linked) throw new Error("в конце трассы визуал
// 4) сняли связь (Esc + пустой код) – снова демо
await key("Escape");
await settle(80);
const ta = textarea();
if (!ta) throw new Error("редактор не вернулся по E
typeInto(ta, "");
const cleared = await waitFor(() => /встроенное дем
if (!cleared) throw new Error("после очистки кода б
if (/переменных пока нет/.test(bodyText())) throw n
return "демо → выполнено/переменных нет → данные из
});

```

/* — 7. Текстовый режим для PDF/парсеров —————

/* — 9. Ссылки между билетами, разделы содержания, б.

```

await test("Н. доступность", "содержание сжимается в
await mountApp("bridges-code");
const root = () => document.querySelector<HTMLElemen
const toggle = () => findButton(/Сжать содержание|C
if (!root()) throw new Error("нет сайдбара");
if (!/lg:w-56/.test(root()!.className)) throw new E
await click(toggle(), "сжать");
if (!/lg:w-44/.test(root()!.className)) throw new E
const items = Array.from(root()!.querySelectorAll("
if (items.length < chapters.length) throw new Error
if (root()!.querySelector("input")) throw new Error
await click(toggle(), "скрыть");
if (!/lg:w-0/.test(root()!.className)) throw new Er
await click(toggle(), "развернуть");
if (!/lg:w-56/.test(root()!.className)) throw new E
return "широкое → узкое (темы на месте, поиска нет)
});

```

await test("J. шапка", "кнопки загрузки не прячутся и

```

await mountApp("segment-trees");

```

```

const menu = document.querySelector("#download-menu

```

```

    return `ссылка на страницу: ${links.length}, переход
  });

```

```

await test("I. ссылки", "содержание: разделы по порядку");
await mountApp("bridges-code");
const sidebar = document.querySelector("#sidebar-content");
if (!sidebar) throw new Error("нет сайдбара содержания");
const headers = Array.from(sidebar.querySelectorAll("h2"))
  .map((el) => (el.textContent ?? "").trim())
  .filter((t) => SECTIONS.some((sec) => sec.title === t));
if (headers.length !== SECTIONS.length) {
  throw new Error(`в содержании ${headers.length} разделов`);
}
const expected = SECTIONS.map((sec) => sec.title).join(" | ");
if (headers.join(" | ") !== expected) throw new Error("неверный порядок");
// 11 «Мосты» и 12 «Точки сочленения» читаются парой
const items = Array.from(sidebar.querySelectorAll("li"));
const iBridges = items.findIndex((t) => t.includes("Мосты"));
const iArtic = items.findIndex((t) => t.includes("Точки сочленения"));
if (iBridges < 0 || iArtic < 0) throw new Error("неверный порядок");
if (Math.abs(iArtic - iBridges) !== 1) throw new Error("неверный порядок");
// 2 «разреженная» и 5 «префиксные» – в одном разделе
if (sectionOf("sparse-table").id !== sectionOf("prefixes")) {
  throw new Error("билеты 2 и 5 попали в разные разделы");
}
if (sectionOf("bridges-code").id !== sectionOf("graphs")) {
  throw new Error("билеты 11 и 12 попали в разные разделы");
}
// порядок чтения: номера билетов сверху вниз не убывающий
const nums = Array.from(sidebar.querySelectorAll("b"))
  .map((b) => (b.querySelector("span")?.textContent ?? ""))
  .filter((t) => /^\\d/.test(t))
  .map((t) => Number.parseInt(t, 10));
const ticketed = chapters.filter((c) => (chapterTopic === c.title));
if (nums.length !== ticketed) {
  throw new Error(`номера билетов видны у ${nums.length} билетов`);
}
for (let i = 1; i < nums.length; i += 1) {

```

```

const shown = () => (section.textContent ?? "").rep
if (!shown().includes(list[0].question.slice(0, 24))
  throw new Error(`первый вопрос не показан: ${list[0].question}`);
}
const wrong = list[0].options.findIndex((_, i) => i !== correctIndex);
await click(ask(list[0].options[wrong].slice(0, 20), "неверно"));
await settle(80);
if (!/Неверно/.test(shown())) throw new Error("на неверном ответе");
if (!shown().includes(list[0].explanation.slice(0, 24)))
  throw new Error("после ответа нет разбора – пользуетесь подсказкой");
}
const nextBtn = ask("Следующий вопрос");
if (!nextBtn) throw new Error("нет кнопки «Следующий вопрос»");
await click(nextBtn, "дальше");
await settle(80);
if (!shown().includes(list[1].question.slice(0, 24)))
  // верный ответ
  await click(ask(list[1].options[list[1].correctIndex].slice(0, 20), "верно"));
  await settle(80);
  if (!/Верно/.test(shown())) throw new Error("на верном ответе");
  if (!/вопрос 2\\./.test(shown())) throw new Error(`с ${list[1].question} не совпадает`);
  return `блиц из ${list.length} вопросов отвечает, продолжайте`;
});

```

```

await test("I. факты", "память: разреженная жирная, префиксная", async () => {
  const memoryRow = async (topic: string) => {
    await mountApp(topic);
    const rows = Array.from(document.querySelectorAll("table tbody tr"));
    const row = rows.find((r) => /^\\s*Память/.test(r.textContent));
    if (!row) throw new Error(`на странице «${topic}» нет таблицы памяти`);
    return Array.from(row.querySelectorAll("td")).map((cell) => cell.textContent);
  };
  const sparse = await memoryRow("sparse-table");
  const prefix = await memoryRow("prefix-sums-2d");
  // билет 2: у префиксной N × M, у разреженной – с логарифмом
  if (!/log/.test(sparse.join(" "))) throw new Error("неверно");
  const prefixCell = sparse.find((c) => !/log/.test(c));

```

```
    return "билет 2: N×M×logN×logM против N×M; билет 5:
  });
```

/* — 10. Сквозной путь: свой код чужой размерности, 1

```
await test("К. свой код", "Флойд 4×4 из редактора вед
  await mountApp("floyd");
  await click(findButton("Боковая панель Python"), "п
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(
    ta,
    [
      "n = 4",
      "INF = 999",
      "d = [[0, 3, INF, 7], [8, 0, 2, INF], [5, INF,
      "k = 0",
      "i = 0",
      "j = 0",
      "for k in range(n):",
      "    for i in range(n):",
      "        for j in range(n):",
      "            if d[i][k] + d[k][j] < d[i][j]:",
      "                d[i][j] = d[i][k] + d[k][j]",
      "print(d[0], d[1][0])",
    ].join("\n")
  );
  const ready = await waitFor(() => bodyText().includ
  if (!ready) throw new Error(`своя программа не выпол
  if (!bodyText().includes("[0, 3, 5, 6]")) {
    throw new Error(`Флойд на своей матрице 4×4 посчи
  }
  if (!textarea()) throw new Error("редактор исчез, п
  await click(findButton("В конец трассы"), "в конец
  const linked = await waitFor(() => /данные из компи
  if (!linked) throw new Error("демонстрация не приня
  const page = bodyText();
  if (!/4×4/.test(page)) throw new Error(`демонстраци
```

```

    await mountApp("scc-kosaraju");
    await click(findButton("Боковая панель Python"), "п
    if (!textarea()) throw new Error("нет редактора");
    await click(findButton("Вставить эталонный код"), "
    const ready = await waitFor(() => bodyText().includ
    if (!ready) throw new Error(`эталон Косарайю не вып
    if (/Ошибка:/.test(panelText())) throw new Error(`э
    const out = panelText();
    if (!/порядок выхода:/.test(out)) throw new Error(`
    // граф на экране: 0→1→2→0, 2→3, 3→4 – ровно две SC
    if (!/компонент: 2/.test(out)) throw new Error(`в гр
    if (!textarea()) throw new Error("редактор исчез, п
    await click(findButton("В конец трассы"), "в конец
    const linked = await waitFor(() => /данные из компи
    if (!linked) throw new Error("демонстрация не приня
    const defects = domDefects(document.querySelector("
    if (defects.length) throw new Error(`артефакты в де
    return "эталон Косарайю: order считается DFS-ом, на
  });

```

```

await test("G. lite-режим", "?lite=1 отдаёт текст без
  if (root) {
    await act(async () => {
      root?.unmount();
    });
    root = null;
  }
  if (host) host.remove();
  host = freshHost();
  window.history.replaceState({}, "", "/?lite=1&topic
  root = createRoot(host);
  await act(async () => {
    root!.render(<App />);
  });
  await settle(120);
  const txt = bodyText();
  if (txt.length < 2000) throw new Error(`lite-режим
  if (/main\.py|Pyodide/.test(txt)) throw new Error("

```

```
-----  
    g[r.ok ? "ok" : "fail"]++;  
    byGroup.set(r.group, g);  
  }  
  return { total: results.length, passed: results.filter  
    .fromEntries(byGroup) };  
}
```

```
/** Инициализация страницы из vizSync – нужна для проверки  
export const initTemplateOf = (id: string) => buildInit
```

ФАЙЛ 100/111: scripts/verify/check-links.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 199 | РАЗМЕР: 1000

```
-----  
/**  
 * Аудит внутренних ссылок пособия: ничего не должно вешаться  
 *  
 * npm run verify:links  
 *  
 * Проверяет (по настоящим модулям приложения, собранным)  
 * 1. билеты 1–24 закрыты страницами, id в chapterTopic  
 * 2. разделы содержания: каждая страница в своём разделе  
 * 3. упоминания «билет N» в тексте ведут на существующие  
 * 4. все data-goto / href="?topic=..." указывают на существующие  
 * 5. якоря href="#..." есть в той же главе;  
 * 6. glossary.simulator существует в реестре и (кроме того)  
 *    имеет свою страницу – иначе модалька не получит данных  
 * 7. glossary.demo – работающая мини-демо;  
 * 8. у каждого ключа PAGE_SYNC есть запись в реестре  
 * 9. у каждой страницы есть визуализация;  
 * 10. страницы связаны перекрёстными ссылками (темы не исключение)  
 */  
import * as esbuild from "esbuild";  
import { mkdirSync, writeFileSync } from "fs";  
  
const OUT = "tmp/verify/links-entry.mjs";
```



```
-----
const chapterIds = new Set(chapters.map((c) => c.id));
const registryIds = Object.keys(VIZ_REGISTRY);

/** Демо без собственной страницы: открываются из подсказки */
const EXTRA_DEMOS = new Set(["stack-dfs", "queue-bfs",

/* 1. билеты 1–24 */
const covered = new Set();
for (const [id, tickets] of Object.entries(chapterTopics)) {
  ok(chapterIds.has(id), `chapterTopics ссылается на не-
  tickets.forEach((n) => covered.add(n));
}
for (let n = 1; n <= 24; n += 1) {
  ok(covered.has(n), `билет ${n} не закрыт ни одной стр.
}

/* 2. разделы содержания */
for (const chapter of chapters) {
  const sec = sectionOf(chapter.id);
  ok(!!sec, `страница «${chapter.id}» не попала ни в од
  const tickets = chapterTopics[chapter.id] ?? [];
  ok(
    tickets.every((n) => n >= sec.from && n <= sec.to),
    `билеты страницы «${chapter.id}» (${tickets.join(",
  });
}
{
  const bySection = SECTIONS.map((s) => ({
    s,
    nums: chapters.flatMap((c) => (sectionOf(c.id).id =
  }));
  for (let i = 1; i < bySection.length; i += 1) {
    const prevMax = Math.max(...bySection[i - 1].nums);
    const curMin = Math.min(...bySection[i].nums);
    ok(curMin > prevMax, `разделы пересекаются: «${bySe
  }
  const all = bySection.flatMap((x) => x.nums);
  for (let n = all[0]; n <= all[all.length - 1]; n += 1
```

```

-----
// 5. якоря
for (const m of html.matchAll(/href="#([^\"]+)/g)) {
  ok(html.includes(`id="${m[1]}"`), `«${chapter.id}»:
}

if (mentionsHere === 0 && !/data-goto=/.test(html)) w
}

/* 6–7. глоссарий */
const demoKinds = new Set(MINI_DEMO_KINDS);
for (const term of GLOSSARY) {
  if (term.simulator) {
    ok(registryIds.includes(term.simulator), `глоссарий
    ok(
      chapterIds.has(term.simulator) || EXTRA_DEMOS.has
      `глоссарий «${term.id}»: simulator "${term.simula
      ые компилятора`
    );
  }
  if (term.demo) ok(demoKinds.has(term.demo), `глоссарий
}

/* 8–9. реестр и синхронизация */
for (const key of Object.keys(PAGE_SYNC)) {
  const base = key.split("#")[0];
  ok(registryIds.includes(base), `PAGE_SYNC["${key}"]:
}
for (const id of registryIds) {
  if (!chapterIds.has(id) && !EXTRA_DEMOS.has(id)) {
    FAILS.push(`реестр: визуализация "${id}" не являетс
  }
}
for (const chapter of chapters) {
  ok(registryIds.includes(chapter.id), `страница «${cha
}

/* 96. файлы, на которые ведут кнопки шапки, действител
{

```

```
-----
* Каждый референсный Python-код из PAGE_SYNC выполняет
* (pyodide из node_modules) под тем же трейсером, что и
* проверяется:
*   1. код не падает и не обрывается по лимиту шагов;
*   2. трасса не пустая (иначе панель показала бы пустую);
*   3. каждая переменная, обещанная странице, реально
*       иначе визуализация не может за ним следовать;
*   4. в снимках нет «мусорных» значений, которые уронят
*/
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";
import { loadPyodide } from "pyodide";

const OUT_DIR = "tmp/verify";
const t0 = Date.now();
mkdirSync(OUT_DIR, { recursive: true });

writeFileSync(
  `${OUT_DIR}/contract-entry.ts`,
  `import { buildDebugRunner } from "../../src/data/debug";
import { PAGE_SYNC } from "../../src/data/vizSync";
import { chapters } from "../../src/data/content";
export const runnerOf = (src: string) => buildDebugRunner(src);
export const sync = Object.fromEntries(Object.entries(PAGE_SYNC));
export const declared = Object.fromEntries(Object.entries(declared));
export const chapterIds = chapters.map((c) => c.id);
`
);

const built = await esbuild.build({
  entryPoints: [`${OUT_DIR}/contract-entry.ts`],
  bundle: true,
  outfile: `${OUT_DIR}/contract-bundle.mjs`,
  format: "esm",
  platform: "node",
  jsx: "automatic",
  sourcemap: false,
  loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
});
```

```

    .filter((a) => a.some(ident))
    .filter((a) => !a.some((x) => ident(x) && seen.has(x)))
    .map((a) => a.join(" / "));
if (missing.length) issues.push(`код не создаёт обеща
/*
 * Обещанная переменная не должна быть пустой на ВСЕХ
 * объявил и не наполнил – визуализации нечего показы
 * «защитый» результат вместо работы алгоритма. Именн
 * Косарайю: order был дан готовым списком, а comp та
 */
const isEmpty = (v) =>
  Array.isArray(v) ? v.length === 0
  : typeof v === "string" ? v.length === 0
  : typeof v === "object" && v !== null ? Object.keys
  : false;
const neverFilled = (mod.declared[key] ?? [])
  .map(alts)
  .filter((a) => a.some(ident))
  .map((a) => a.find((x) => ident(x) && seen.has(x)))
  .filter(Boolean)
  .filter((name) => {
    let sawValue = false;
    let sawNonEmpty = false;
    for (const step of [...result.steps, { values: re
      const v = step.values?.[name];
      if (v === undefined || v === null) continue;
      sawValue = true;
      if (!isEmpty(v)) sawNonEmpty = true;
    }
    return sawValue && !sawNonEmpty;
  }));
if (neverFilled.length) issues.push(`объявлены, но ни

const stderr = printed.filter((p) => p.startsWith("[s
if (stderr.length) issues.push(`предупреждения: ${std

if (issues.length) problems++;
rows.push({ key, steps: result.steps.length, vars: se

```

```

const jsdomErrors = [];
virtualConsole.on("jsdomError", (error) => {
  jsdomErrors.push(error?.message || String(error));
  onError?.(error);
});
// console.error/warn приложения собираем отдельно —
const consoleErrors = [];
virtualConsole.on("error", (...args) => consoleErrors.push(...args));
virtualConsole.on("warn", () => {});

const dom = new JSDOM(
  `<!doctype html><html lang="ru"><head><meta charset="utf-8"><title></title></head><body><div id="root"></div></body></html>`,
  { url, pretendToBeVisual: true, runScripts: "outside-only" },
);
const { window } = dom;

window.ResizeObserver = window.ResizeObserver ?? OBSERVER_API.ResizeObserver;
window.IntersectionObserver = window.IntersectionObserver ?? OBSERVER_API.IntersectionObserver;
window.matchMedia =
  window.matchMedia ??
  ((media) => ({
    matches: false,
    media,
    onchange: null,
    addListener() {},
    removeListener() {},
    addEventListener() {},
    removeEventListener() {},
    dispatchEvent: () => false,
  }));
window.scrollTo = () => {};
window.HTMLElement.prototype.scrollIntoView = () => {};
window.HTMLElement.prototype.focus = window.HTMLElement.prototype.blur = () => {};
window.URL.createObjectURL = window.URL.createObjectURL = () => {};
window.URL.revokeObjectURL = window.URL.revokeObjectURL = () => {};
Object.defineProperty(window, "innerWidth", { value: 1000 });
Object.defineProperty(window, "innerHeight", { value: 1000 });

```

```
    "FileReader",
  ];
  for (const key of copy) {
    try {
      Object.defineProperty(globalThis, key, { value: w
    } catch {
      /* read-only в Node – не критично */
    }
  }
  globalThis.IS_REACT_ACT_ENVIRONMENT = true;

  return { dom, window, document: window.document, cons
}
```

ФАЙЛ 103/111: scripts/verify/live-matrix.test.mjs
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 103 | РАЗМ

```
/**
 * Модульные проверки разбора «живых» данных компилятор
 *
 * npm run verify:unit
 *
 * liveMatrix() – то, благодаря чему визуализации матриц
 * суммы 2D, разреженная таблица) принимают данные ЛЮБОГО
 * пользователя вместо захардкоженной демо-сетки.
 */
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";

const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
writeFileSync(
  `${OUT_DIR}/unit-entry.ts`,
  `export { liveMatrix, pickLiveVar, clampIndex } from
`);
```

```

check("at() с отрицательным индексом → null", m([[1, 2]
check("список словарей с числовыми ключами", cells([{"
check("словарь списков", cells({ 0: [1, 2], 1: [3, 4] }
check("словарь словарей", cells({ 0: { 0: 7, 1: 8 }, 1:
check('ключи-пары "(i, j)" после снапшота tuple', cells
check('ключи-пары без скобок "0, 1"', cells({ "0, 0": 5

```

```

/* — liveMatrix: что матрицей НЕ является
check("плоский список чисел → null", m([1, 2, 3, 4, 5])
check("словарь по именам → null", m({ dist: 1, prev: 2
check("пустой список → null", m([]), null);
check("пустой словарь → null", m({}), null);
check("undefined → null", m(undefined), null);
check("null → null", m(null), null);
check("число → null", m(42), null);
check("строка → null", m("abc"), null);
check("массив строк-мусора → null", m([["a", "b"], ["c"

```

```

/* — liveMatrix: особые значения
check('"inf" → +Infinity', cells([["inf", 1], [2, "-inf
check('числа-строки "42" → 42', cells([["1", "2"], ["3"
check("булевы значения ячейкой не считаются", m([[true,
check("обрывок до 64×64", shape(Array.from({ length: 80
    4", numbers: true }));
check("hasNumbers=false невозможна для живой матрицы",

```

```

/* — pickLiveVar
check("первое найденное имя по приоритету", pickLiveVar
check("берёт следующее имя, если первого нет", pickLive
check("ничего не найдено → null", pickLiveVar({ x: 1 },
check("нет снимка → null", pickLiveVar(undefined, ["d"]
check("ключ со значением undefined всё равно считается"

```

```

/* — clampIndex
check("null остаётся null", clampIndex(null, 5), null);
check("размер 0 → null", clampIndex(2, 0), null);
check("обрезка сверху", clampIndex(99, 5), 4);
check("обрезка снизу", clampIndex(-7, 5), 0);

```

```

-----
const f = divs.find((d) => d.className.includes("text
return (f?.textContent ?? "").replace(/\s+/g, " ").sl
}
function state(tag: string) {
  const p = panel();
  const ed = editor();
  const txt = (p?.textContent ?? "").replace(/\s+/g, "
  const step = txt.match(/шаг[^\s]*\s*\d+\s*\s*\s*\d+/i)
  process.stdout.write(
    ` [${tag.padEnd(9)}] редактор: ${ed ? "ДА" : "нет"
      step ? step[0] : "-" } | футер: ${footer()}\n`
  );
  process.stdout.write(` бейдж страницы: ${
    /встроенное демо/.test(document.body.textContent ??
}
function typeInto(ta: HTMLTextAreaElement, value: string) {
  const setter = Object.getOwnPropertyDescriptor(window
  act(() => {
    setter.call(ta, value);
    ta.dispatchEvent(new window.Event("input", { bubble
  }));
}
async function click(el: Element | null | undefined, label: string) {
  if (!el) throw new Error(`нет кнопки: ${label}`);
  await act(async () => {
    el.dispatchEvent(new window.MouseEvent("click", { bubble
    await sleep(20);
  });
}
function findButton(match: RegExp): HTMLButtonElement | undefined {
  return Array.from(document.querySelectorAll("button"))
    .filter(b => match.test([b.getAttribute("aria-label"), b.getAttribute("value")
  ) as HTMLButtonElement | undefined;
}

export async function probe() {
  const host = document.createElement("div");
  document.body.appendChild(host);

```



```

state("ввели");
for (const t of [200, 800, 2000, 6000]) {
  await settle(t);
  state(`t+${t}`);
}

process.stdout.write("\n=== 4. очищаем редактор полно
const ed4 = editor();
if (ed4) {
  typeInto(ed4, "");
  state("очистили");
  for (const t of [200, 800, 2000]) {
    await settle(t);
    state(`t+${t}`);
  }
} else {
  process.stdout.write(" редактора нет (листинг) – ж
  await act(async () => {
    document.dispatchEvent(new window.KeyboardEvent("
    await sleep(30);
  });
  state("после Esc");
}
}

```

ФАЙЛ 105/111: scripts/verify/README.md

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ

Проверки приложения (`npm run verify`)

Три уровня проверок, которые можно гонять локально и в чужих браузерах. Для локального запуска нужен: DOM даёт `jsdom`, Python – `pyodide` из `node\_modules`. Для проверки в браузере пользователя, только без

```bash

автозапуск инициализации настоящим Python не роняет |  
`[object Object]`; нет кнопок без имени |  
| В. Флойд: размерности (10) | 5×5, 10×10 (больше демо-  
й список, 80×80 (обрезка), пустая матрица, чужое имя  
| С. матричные страницы (6) | префиксные суммы 2D и раз  
| D. графы путей (2) | Дейкстра/Форд-Беллман с вершинами  
гравания гаснут и оживают обратно |  
| Е. эталонный код (46) | прогон кода в CPython → финал  
е показать мусор |  
| F. редактор (6) | код из одних комментариев, синтаксис  
ляет эталон и возвращает инициализацию; код не перете  
| G. lite-режим (1) | `?lite=1` отдаёт текст без навига  
| Н. доступность (2) | в панели подписаны все кнопки в  
очника |

Фильтры: `HAR\_CHAPTERS=floyd,dijkstra`, `HAR\_SKIP=A,E`,

## Как это заведено в приложение

- \* `window.\_\_algv0PythonRuntime` — точка подмены интерпретатора  
`PythonCompiler.tsx`. В браузере пользователя поле не  
грузит Pyodide с CDN как раньше; харнес кладёт туда л
- \* Состояние визуализаций передаётся через `emitVizState`  
(`src/data/vizStepBus.ts`) — те же события, что шлёт  
идёт по боевому пути, а не через внутренние API компо

---

ФАЙЛ 106/111: scripts/verify/run-browser-checks.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 96 | РАЗМ

---

/\*\*

\* DOM-проверки приложения: jsdom + настоящий Pyodide и

\*

\* npm run verify:dom

\*

\* Фильтры:

```

// Vite-специфичные глобалы, которых нет вне dev-серв
define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.MODE": '"development"',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 "import.meta.env.SSR": "false",
},
loader: { ".css": "empty", ".png": "dataurl", ".jpg":
logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/b

process.on("unhandledRejection", (reason) => {
 jsdomErrors.push(`unhandledRejection: ${reason instan
});

log("прогоняю сценарии...\n");
const results = await mod.runSuite({ py, window, document
const sum = mod.summary();

writeFileSync(`${OUT_DIR}/report.json`, JSON.stringify(

const failed = results.filter((r) => !r.ok);
console.log("\n" + "=".repeat(96));
console.log("СВОДКА");
console.log("=".repeat(96));
for (const [group, g] of Object.entries(sum.byGroup)) {
 console.log(` ${group.padEnd(26)} ok: ${String(g.ok)
}
console.log(`\n ВСЕГО ${sum.total}: прошло ${sum.pass
if (failed.length) {
 console.log("\nПРОВАЛЫ:");
 for (const f of failed) console.log(` x [${f.group}]
}
if (jsdomErrors.length) {
 console.log("\nОШИБКИ ОКРУЖЕНИЯ (jsdom/неперехваченны

```

```
log("поднимаю jsdom...");
const { window, document } = createDom();
window.__algv0PythonRuntime = py;

await esbuild.build({
 entryPoints: ["scripts/verify/probe-compiler.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/probe-bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 sourcemap: true,
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
 logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/probe-bundle.mjs`);
log("зондирую панель...");
await mod.probe();
process.exit(0);
```

---

## РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

---

---

ФАЙЛ 108/111: [.github/workflows/deploy-pages.yml](#)  
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9

---

name: Deploy app to GitHub Pages

```
 node-version: 24
 cache: npm

- name: Install dependencies
 run: npm ci --no-audit --no-fund

- name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
 run: npm run build

- name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html

- name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 # Ранняя и понятная ошибка вместо невнятного «Failed to build»
 # когда в репозитории Pages не переключён на сборку
 - name: Preflight – Pages собирается через GitHub Pages
 env:
 GH_TOKEN: ${ secrets.GITHUB_TOKEN }
 run: |
 bt=$(gh api "repos/${ github.repository }/pages" --jq ".build_type")
 echo "Pages build_type: $bt"
 if ["$bt" != "workflow"]; then
 echo "::error title=Pages не настроен::См. комментарий в начале деплоя
 те этот запуск. См. комментарий в начале деплоя"
 exit 1
 fi
```

```
 required: false
 default: "150"

concurrency:
 group: rebuild-pdf-`${{ github.ref }}
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
 # не реагируем на собственный коммит бота
 if: "!contains(github.event.head_commit.message, '[bot]')"

 steps:
 - name: Checkout
 uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: "24"
 cache: npm

 - name: Install ImageMagick + DejaVu fonts
 run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommends \
 # На Ubuntu политика ImageMagick иногда запрещает
 # разрешаем то, что нужно пайплайну (text -> pdf)
 for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml
 if [-f "$policy"]; then
```

```
npm run export:guide
снимки публикуются вместе с приложением: HTML
в автономный файл прямо из /export/viz-shots
mkdir -p public/export/viz-shots
cp tmp/viz-shots/*.png tmp/viz-shots/manifest
```

- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`public/export/full\_code\_all\_pages` | "  
 echo "| \`public/export/full\_code\_all\_pages` | "  
 echo "| \`public/export/guide\_lite.pdf\` | "  
 echo "| Снимков визуализаций | \${ls tmp/viz-shots} | "  
 echo "| PNG-страниц | \${ls tmp/export-pages} | "  
 } >> "\$GITHUB\_STEP\_SUMMARY"
- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7  
with:  
 name: full-code-export  
 path: |  
 public/export/full\_code\_all\_pages.pdf  
 public/export/full\_code\_all\_pages.txt  
 if-no-files-found: error  
 retention-days: 30
- name: Upload artifacts (PNG-страницы)  
uses: actions/upload-artifact@v7  
with:  
 name: full-code-pages-png

```
 - "**"
 paths-ignore:
 - "public/export/**"
 - "**/*.md"
 pull_request:
 workflow_dispatch:

concurrency:
 group: verify-${{ github.ref }}
 cancel-in-progress: true

permissions:
 contents: read

jobs:
 verify:
 name: unit + эталонный код + DOM
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4

 - uses: actions/setup-node@v4
 with:
 node-version: 22
 cache: npm

 # puppeteer здесь не нужен (он для растеризации Р
 # тянет Chromium – пропускаем, чтобы установка не
 - name: Зависимости
 run: npm ci
 env:
 PUPPETEER_SKIP_DOWNLOAD: 1
 PUPPETEER_SKIP_CHROMIUM_DOWNLOAD: 1

 - name: Типы и сборка
 run: |
 npx tsc --noEmit -p tsconfig.json
 npm run build
```



## ## 1. Объект главы

```
```ts
import { Chapter } from "../../types";

export const myChapters: Chapter[] = [
  {
    id: "dijkstra",           // = ключ VIZ_REGISTRY и
    title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
    type: "html",            // всегда "html": страница
    category: "Графы. Пути", // группа в навигации
    content: `...HTML ниже...`,
  },
];
```
```

### Правила:

- `id` – kebab-case, уникальный, совпадает с ключом `VIZ\_REGISTRY`, `id` подставляется автоматически, вручную `div#slot-\*` вставляются только такие заглушки остались в старых главах и считаются мусором.
- `title` начинается с номера билета: сортировка оглавления по номеру цифре заголовка. Одна страница может закрывать несколько билетов (например `mst` → 18, 19, 20) – тогда номер в заголовке один, а номер билета объявляется в `chapterTopics` (`src/data/content.ts`).
- `content` – шаблонная строка. Внутри **никаких** `<div>` и `<div>` экранирования: это сломает сборку.

## ## 2. Каркас страницы

```
```html
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
  <!-- шапка: бейдж билета + заголовков -->
  <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
```html
<p class="text-slate-300 text-sm mb-4">
 Текст с акцентом и инлайн-кодом
 dist[v]
</p>
```
```

Инлайн-«пилюля» для имени переменной/константы:

```
```html
<span class="bg-slate-950 px-2 py-0.5 rounded text-xs f
```
```

3.3 Сетка ключевых чисел (3 ячейки)

```
```html
<div class="grid grid-cols-1 sm:grid-cols-3 gap-3 my-6">
 <div class="bg-slate-900 border border-slate-700 rounded">
 <p class="text-[10px] uppercase tracking-wider text">
 <p class="font-mono text-emerald-300 text-sm">0{(V+H
 </div>
 <!-- 2-я и 3-я ячейки — тот же markup -->
</div>
```
```

3.4 Сетка аналогий — ****обязательна**** на каждой странице

Слева — неформальная аналогия, справа — техническая аль

```
```html
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border border">
 <div class="absolute -top-3 left-4 bg-slate-700 text
 ald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">...</p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg border border">
 <p class="text-slate-300 text-sm mb-4">...</p>
 </div>
</div>
```
```

3.6 Спойлер ``<details>`` – ****обязателен**** для кода,

```
```html
<details class="bg-slate-900/40 rounded-lg border border-
 <summary class="p-4 font-bold text-slate-400 outline-l
 e-700/50">
 ▶ Код: Дейкстра на куче (Python)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <pre class="bg-slate-950 p-4 rounded overflow-x-aut
...</code></pre>
 <p class="text-slate-300 text-sm">Комментарий после
 </div>
</details>
```
```

- Вложенные списки внутри спойлера: ``<ul class="list-di`
- Подзаголовок внутри блока: ``<h4 class="text-lg font-b`
- или ``<p class="text-lg font-bold text-white mt-8 mb-3`

3.7 Ловушки / частые ошибки

```
```html
<div class="bg-rose-950/30 p-5 rounded-lg border border
 <p class="text-sm font-bold text-rose-300 mb-3">⚠ Гд
 <ul class="list-disc pl-5 space-y-2 text-sm text-slat
 Отрицательные веса. Дейкстра их не перев
 Переполнение: <span class="font-mono text-emera

</div>
```
```

4. Порядок разделов на странице

1. Шапка: бейдж билета + ``h2``.
2. «Одной фразой» / «Суть алгоритма» + центрированное о

компилятора и без навигации). Чтобы экспорт был полным:

- заголовки – только `h2/h3/h4`; списки – `ul/ol/li`; т.к. код – `pre`; определения – обычный `p` (см. 3.1);
- ****смысл не должен держаться на эмодзи****: в PDF используется эмодзи-глифы – пиктограммы вырезаются, остаётся « Операции на отрезках» доедет как «Операции на отрезках», а вот « значит да / значит нет» потеряет смысл, пиктограммы не вырезаются;
- спойлеры в PDF раскрываются полностью – не прячьте ту часть, минимум, но и не дублируйте его в основном тексте;
- картинки страниц берутся из `CHAPTER\_IMAGES` (`script` снимки демонстраций – из `tmp/viz-shots/\*.png` (их перенести в `tmp` `npm run export:shots` в CI); локально без браузера в `tmp` описание вкладок и переменных из `PAGE\_SYNC`.

Цель по объёму: 40–60 страниц, жёсткий предел – 100.